

Python Backend, Distributed Systems, and AI Interview Master Guide

Behavioral, coding, design-pattern, delivery, data, and AI
preparation

Reusable engineering knowledge - no candidate or company identifiers

Software Engineering Knowledge Hub

Contents

Professional Profile and Interview Positioning	6
Profile Design Principles	6
Reusable Profile Template	6
Example Backend and AI Profile	6
Sixty-Second Introduction	6
Evidence Inventory	7
Story Record Format	7
Tailoring Without Overclaiming	7
Project Explanation Template	8
Privacy Checklist	8
Behavioral and Technical Interview Playbook	9
How to Use This Playbook	9
STAR Answer Model	9
Twenty-Five Common Behavioral Questions	9
Technical-Behavioral Follow-Ups	12
Honest Scope Language	12
Final Practice Checklist	12
Backend, Data, and AI Project Deep-Dive Guide	14
Python Microservices	14
Bandit From POC to Production	17
Backward Compatibility Matrix	18
Financial and High-Volume Data Workflows	18
LLM and Agent Evaluation	19
Likely Production Scenarios	20
Interview Delivery Checklist	22
Coding Interview Patterns, Data Structures & Progressive Optimization	23
Recommended Study Flow	23
Quick Summary	23
1. Opening a Coding Problem	24
2. Python Complexity Cheat Sheet	25
3. Pattern: Hash Map - Two Sum	26
4. Pattern: Sliding Window - Longest Substring Without Repeating Characters	27
5. Pattern: Top-K - Frequency Map + Heap	27
6. Pattern: Sorting + Scan - Merge Intervals	28
7. Pattern: Prefix Sum + Hash Map - Subarray Sum Equals K	28

8. Pattern: Prefix/Suffix - Product of Array Except Self	29
9. Pattern: Heap or Quickselect - Kth Largest	30
10. Pattern: Intervals + Heap - Meeting Rooms	30
11. Pattern: Graph Traversal - Number of Islands	30
12. Pattern: Topological Sort - Course Schedule	31
13. Pattern: K-Way Merge of Sorted Streams	31
14. Choosing the Data Structure from the Bottleneck	32
15. Edge-Case Testing Checklist	32
16. What to Say When Temporarily Stuck	33
17. Common Coding Interview Mistakes	33
18. High-Probability Follow-Ups	33
19. 60-Minute Coding Round Game Plan	34
20. Quick Revision Checklist	34
21. Sliding-Window Hit Counter	35

Python 37

Recommended Study Flow	37
Quick Summary	37
Core Topics to Master	37
Consolidated Interview Questions & Technical Notes	37
Focused Python Semantics and Debugging Drills	72
Frequency Aggregation, Top-K Selection & Streaming Follow-Ups	76
Loop Progress Invariants in Grouped Simulations	78
Senior Python Concurrency & Thread-Safety Interview Guide	79

REST API & Backend APIs 89

Recommended Study Flow	89
Quick Summary	89
Core Topics to Master	89
Consolidated Interview Questions & Technical Notes	89
Production-Style AI-Enabled Backend Project Walkthrough	116
Create-Endpoint Debugging: Persistence, Nested Resources & Business Identifiers	118
Senior Backend API Interview Addendum	120
FastAPI: Interview-Ready Architecture	123
Mocking External Services	125
Unit Testing FastAPI	126
Request Flow: Explain It Literally	126
API Race Condition Example	127
Backward-Compatible API Evolution	127
API Interview Quick Questions	127

Quick Backend API Revision Card	128
Async FastAPI, SQL, Service Layers, and Test Doubles	129

SQL, Databases, ORM, Caching & Idempotency **136**

Recommended Study Flow	136
Quick Summary	136
Core Topics to Master	136
Consolidated Interview Questions & Technical Notes	136
Relational Versus NoSQL Databases and ACID	154
Backend Persistence & Concurrency Interview Patterns	155
Quick Persistence Revision Card	159

System Design, Microservices & Distributed Systems **160**

Recommended Study Flow	160
Quick Summary	160
Core Topics to Master	160
Consolidated Interview Questions & Technical Notes	160
AI-Enabled Backend Versus Agentic Architecture	170
Monolith Versus Microservices: Decision Framework	171
Async I/O Versus Background Processing	172
Capacity Planning, Load Testing & Scaling Under Unexpected Demand	173
Design Sequencing, Storage Fit & Messaging Resilience	177
Senior 60-Minute System Design Interview Framework	181
Complete Example: Podcast Subscription and Latest-Episode Feed	195
System-Specific Deep-Dive Patterns	199
High-Probability System Design Follow-Up Questions	201
Common System Design Mistakes	203
Quick Interview Revision Card	203
Additional Senior System-Design Deep Dives	203
Reusable Mock-Interview Prompt Template	207

Software Design Patterns for Python and Backend Interviews **208**

Recommended Study Flow	208
Foundations	208
Creational Patterns	209
Structural Patterns	210
Behavioral Patterns	211
Modern Backend Patterns	213
High-Value Comparisons	215
LLD Case Study: Extensible Transaction Processing	215
Common Red Flags	216

60-Second Answer Template	217
Quick Revision Checklist	217
Further Reading	217

GenAI, LLMs, Prompting, Agents, RAG & Evaluation 218

Recommended Study Flow	218
Quick Summary	218
Consolidated Interview Questions & Technical Notes	218
AI Evaluation Work Versus AI Product Engineering	267
Safe Tool-Using Agents: Orchestration, State, Cost, and Evaluation	268
Retrieval Top-K, LangGraph, and MCP Protocol Deep Dive	274

Cloud, DevOps, Docker, Kubernetes & CI/CD 280

Recommended Study Flow	280
Quick Summary	280
Consolidated Interview Questions & Technical Notes	280
Platform Engineering, Golden Paths & Developer Experience	301
Senior CI/CD, Containers & Deployment Interview Addendum	304
Quick DevOps Revision Card	311

Testing, QA, Security, Observability & Reliability 312

Recommended Study Flow	312
Quick Summary	312
Consolidated Interview Questions & Technical Notes	312
Reliability Lessons from an Incident Alert Router	335
Security and Escalation Guardrails for Customer-Support Agents	336
Reliability Lessons from Load-Related Application Crashes	339
Repository Debugging Under Time Pressure	340
Senior Testing, Security & Reliability Interview Addendum	342
Quick Testing/Security/Observability Revision Card	350
Production Scenario Rapid-Fire Answers	350

Professional Profile and Interview Positioning

Source: docs/my_profile.md

Purpose: Build a reusable, evidence-based professional profile without storing a person's name, contact details, or employer-specific history.

Profile Design Principles

A strong profile answers four questions quickly:

1. What kind of engineer are you?
2. What systems have you built?
3. What evidence proves the impact?
4. What type of problem do you want to solve next?

Keep the public profile generic. Store personalized stories, metrics, and employment history in owner/my_profile.md.

Reusable Profile Template

I am a [role] with [experience range] in [two or three core areas].
I have built [representative systems] using [relevant technologies].
My strongest evidence is [measurable result or ownership example].
I am now looking to apply that background to [target problem or role].

Example Backend and AI Profile

I am a Python-focused backend engineer with experience building APIs, microservices, cloud delivery pipelines, and data-intensive applications. I have worked across the full delivery lifecycle, from requirements and design through testing, deployment, monitoring, and production support. I also have experience evaluating LLM and agent workflows for grounding, tool use, structured outputs, and safety. I am most effective in roles that combine sound software engineering with practical, governed AI integration.

Sixty-Second Introduction

Use a four-part structure:

- **Present:** current engineering identity.
- **Past:** one or two relevant systems or responsibilities.
- **Proof:** one quantified or observable result.
- **Future:** why the target role is a logical next step.

Example Introduction

I am a backend engineer focused on Python, FastAPI, distributed services, and production delivery. In recent projects I have designed API contracts, integrated PostgreSQL, automated quality and security checks in CI, and supported containerized services in cloud environments. I have also evaluated retrieval and agent workflows, which taught me to treat grounding, permissions, and observability as product requirements. I am looking for a role where I can combine those backend fundamentals with responsible AI-enabled services.

Evidence Inventory

Create an evidence inventory before writing answers. Do not invent metrics; use an observable outcome when an exact number is unavailable.

Capability	Evidence prompt	Generic example
Ownership	What did you initiate and finish?	Took a security scanner from POC to a CI gate
Reliability	What failure did you prevent or diagnose?	Added timeouts, retries, and service-level alerts
API design	What contract decision did you own?	Preserved older clients with additive schema evolution
Quality	How did you improve confidence?	Raised meaningful branch coverage around critical paths
Collaboration	Whose approval or input was needed?	Aligned product, security, and engineering stakeholders
Scale	What volume or concurrency mattered?	Processed records in bounded, restartable batches
AI safety	What model risk did you control?	Validated tool arguments and required approval for writes

Story Record Format

A machine-readable record makes it easier to reuse a story across interview questions without changing the facts.

```

story_id: security_scanner_rollout
situation: CI did not include a consistent Python static-security check
task: evaluate a scanner and introduce it without blocking delivery unfairly
actions:
  - built a proof of concept on representative repositories
  - triaged findings by severity and confidence
  - documented justified suppressions with owners and expiry dates
  - obtained product, management, and engineering approval
  - added the scanner to pull-request and release pipelines
result: repeatable security feedback became part of normal delivery
follow_ups:
  - false-positive handling
  - release-blocking policy
  - developer adoption

```

Tailoring Without Overclaiming

Map each requirement to one of three evidence levels:

Level	Meaning	Interview language
Production	Personally built or operated it	"I implemented and supported..."
Adjacent	Worked with it or evaluated it	"I worked with the team on..."
Portfolio	Built outside paid production work	"I implemented a portfolio example to learn..."

If direct experience is missing, explain the closest transferable pattern and how you would close the gap. This is stronger than claiming expertise that a technical follow-up will expose.

Project Explanation Template

Use this order for a two-minute project walkthrough:

1. Business or operational motivation.
2. System boundary and users.
3. Architecture and data flow.
4. Personal contribution.
5. Difficult trade-off.
6. Testing, security, deployment, and monitoring.
7. Result and lesson.

Concise Project Example

A Python team needed security findings earlier in delivery, so I evaluated a static-analysis tool against representative services. I built the POC, classified true and false positives, and proposed severity-based enforcement with documented suppressions. After stakeholder review, I integrated it into CI, added developer guidance, and monitored adoption. The project converted an informal review concern into a repeatable engineering control.

Privacy Checklist

Before publishing a profile, search for:

- names, personal email addresses, phone numbers, and street addresses;
- exact interview dates, compensation, recruiter details, and job identifiers;
- private repository owners, customer names, internal hostnames, and secrets;
- confidential production data or unapproved business metrics.

Use the owner/ directory for personalized preparation, but remember that a tracked directory is not private merely because it is named owner.

Behavioral and Technical Interview Playbook

Source: docs/interview_questions.md

Purpose: A company-neutral answer bank for software-engineering interviews. Replace generic examples with truthful evidence from owner/ when practicing.

How to Use This Playbook

- Answer the question directly in the first sentence.
- Use STAR for experience questions: Situation, Task, Action, Result.
- Spend most of the answer on actions you personally took.
- State the trade-off, result, and lesson.
- Keep the first response to 60-90 seconds, then invite follow-up questions.

STAR Answer Model

Situation: Give only the context needed to understand the problem.

Task: State your responsibility and the success condition.

Action: Explain your decisions, implementation, and collaboration.

Result: Give evidence, impact, and what you learned.

Generic STAR Example - Security Scanner Rollout

Our Python delivery pipeline had tests and deployment automation but no consistent static-security gate. I owned an evaluation of Bandit, ran a POC on representative services, triaged findings by severity and confidence, and separated genuine risks from justified false positives. I proposed a staged policy, obtained product and engineering approval, added the check to CI, and documented suppression rules. The result was earlier, repeatable security feedback without turning the scanner into an arbitrary release blocker.

Generic STAR Example - Backward Compatibility

An API and its client library were evolving at different speeds, and pairwise version mappings were becoming brittle. I defined compatibility as a matrix of client versions, API versions, capabilities, and expected behavior, then used that matrix to drive contract tests. We kept changes additive, translated legacy payloads at the boundary, and deprecated behavior over a published window. This reduced special-case logic and made release decisions traceable.

Twenty-Five Common Behavioral Questions

1. Tell me about yourself

I am a Python-focused backend engineer with experience in APIs, microservices, cloud delivery, databases, and production support. I have owned work across design, implementation, tests, CI/CD, and observability, and I have also worked with LLM evaluation and agent safety patterns. I am looking for a role where I can build reliable services and integrate modern AI capabilities responsibly.

2. Why should we hire you

I combine hands-on backend delivery with an ownership mindset. I can design an API, implement it, test it, automate its delivery, and support it in production. I also communicate trade-offs clearly, which matters when reliability, security, and delivery speed compete.

3. What are your strengths and weaknesses

My strengths are structured problem solving, follow-through, and translating operational risks into practical engineering controls. A weakness I have worked on is going too deep before confirming the decision needed; I now time-box investigation, state assumptions, and share an early recommendation.

4. Why do you want to work here

The role matches the problems I want to solve: reliable backend services, secure delivery, and responsible use of AI. I would connect this answer to one current company initiative and explain how my evidence is relevant, without pretending to know internal details.

5. What are your career goals

I want to become a stronger end-to-end engineer who can own a service from architecture through operations and mentor others in reliable delivery. Over time, I want to lead technical decisions while remaining hands-on.

6. Are you overqualified

No. I see the role as a strong match between my current skills and the depth I want to build. I am motivated by the actual work, team, and ownership scope, not by collecting a particular title.

7. What motivates you

I am motivated by turning ambiguous problems into systems that are easier to use and operate. Clear feedback loops - tests, metrics, user feedback, and incident learning - make progress visible and keep me engaged.

8. Who inspires you

I am inspired by engineers who combine technical depth with calm, generous leadership. The important behavior for me is making complex decisions understandable and helping a team improve rather than seeking personal credit.

9. Define success

Success means the intended outcome is delivered and remains reliable after the launch. It includes user value, maintainability, security, observability, and a team that understands how to operate what was built.

10. What was a difficult decision you made

I once had to recommend delaying strict enforcement of a new security check. The high-risk findings needed immediate action, but blocking every historical warning would have stopped unrelated releases. I proposed severity-based gates and a tracked remediation backlog, balancing risk reduction with continuity.

11. How do you handle pressure and stress

I make the problem smaller and visible: establish impact, assign priorities, separate mitigation from root-cause work, and communicate a predictable update cadence. That approach keeps urgency from turning into random activity.

12. Tell me about a failure

I once implemented too much before validating a compatibility assumption with all consumers. The design worked technically, but rework was needed when an older client behavior surfaced. I owned the miss and introduced a traceability matrix and contract-test review before implementation.

13. How do you stay current

I combine primary documentation with small implementations. I read release notes or specifications, build a narrow experiment, record the trade-offs, and only then decide whether the technique belongs in production.

14. Describe your ideal work environment

I do best where expectations are clear, engineers can challenge decisions respectfully, and teams own reliability as well as feature delivery. I value written decisions, useful reviews, and enough autonomy to solve the problem.

15. How do you handle disagreement

I restate the shared goal, identify which assumptions differ, and seek evidence through data, documentation, or a small experiment. Once a decision is made, I support it fully and document why it was chosen.

16. What are your salary expectations

I am looking for compensation aligned with the role's scope, employment model, and local market. I would prefer to understand the complete package and expectations before giving a narrow number, but I am happy to discuss the approved range.

17. Where do you see yourself in five years

I expect to be a trusted senior engineer or technical lead who owns important services, mentors teammates, and makes architecture decisions grounded in operational evidence. I still want implementation work to be part of my role.

18. How do you learn a new skill

I start with the official mental model, build the smallest useful example, and then apply it to a realistic failure case. I write down what I learned and ask for review before using it in a high-risk production path.

19. Describe a challenge you overcame

A shared API framework needed to support clients moving at different release speeds. I replaced growing pairwise exceptions with a capability-oriented compatibility matrix and matrix-driven tests. That made supported combinations explicit and reduced the risk of silent client breakage.

20. How do you prioritize work

I rank work by user or operational impact, urgency, risk reduction, dependency, and effort. I make trade-offs visible, reserve capacity for unplanned work, and revisit priorities when new evidence changes the risk.

21. What questions do you have for us

How is success measured in the first six months? What reliability or delivery problem needs the most attention? How are architecture decisions reviewed, and how does the team learn from production incidents?

22. How do you handle constructive criticism

I clarify the expected behavior, avoid defending my first draft, and turn the feedback into a concrete change. I then follow up with evidence that the change worked and retain the lesson in a checklist or review practice.

23. Describe your teamwork experience

I work best by making ownership and interfaces explicit. On shared platform work, I align contracts early, request focused reviews, document release dependencies, and help with integration and production follow-through.

24. What do you know about our company

I would answer with three verified facts from primary sources: the company's mission, one current product or engineering initiative, and one principle that affects this role. I would then connect those facts to relevant evidence from my background.

25. Why should we hire you over other candidates

I cannot compare myself with people I have not met, but I can be specific about my value: I bring Python backend depth, full-lifecycle delivery, security and reliability judgment, and practical AI evaluation experience. I also explain uncertainty honestly and turn it into a testable plan.

Technical-Behavioral Follow-Ups

Prepare a 30-second and a two-minute version of each answer:

- Describe a service you personally designed or changed.
- Explain a trade-off between speed, quality, and security.
- Tell me about a production issue and the diagnostic evidence you used.
- Explain how you introduced a tool or practice across a team.
- Describe an API change that preserved older consumers.
- Tell me about a time a metric changed your implementation.
- Separate your personal contribution from the team's result.

Honest Scope Language

Use precise verbs:

I designed ...	I implemented ...	I reviewed ...
I operated ...	I evaluated ...	I collaborated on ...
I built a POC ...	I have not used X in production, but ...	

Avoid saying "we" for an entire answer. Establish the team context once, then explain what you personally decided, built, tested, or communicated.

Final Practice Checklist

- Introduction fits in 60 seconds.
- Two main stories have measurable or observable results.
- Every claim can survive a technical follow-up.
- Company facts are current and come from primary sources.
- Salary, availability, and work-arrangement answers are calm and concise.
- Closing questions reveal expectations, architecture, and team practices.

Backend, Data, and AI Project Deep-Dive Guide

Source: docs/project_deep_dive.md

Purpose: Company-neutral answers and implementation examples for resume deep-dives and production scenario interviews.

Python Microservices

Explain the architecture and purpose of the services

A strong answer starts with the system boundary, not the framework:

The platform exposed versioned REST APIs through an ingress or API gateway. Stateless FastAPI services owned separate business capabilities and accessed PostgreSQL through a service and repository layer. Long-running work was sent to a queue so the request path stayed responsive. Services emitted structured logs, metrics, and traces, and were packaged as containers for Kubernetes.

```
flowchart LR
    Client --> Gateway
    Gateway --> API[FastAPI service]
    API --> Service[Service layer]
    Service --> DB[(PostgreSQL)]
    Service --> Queue[Event queue]
    Queue --> Worker
    API --> Telemetry[Logs, metrics, traces]
```

Personal contribution should be explicit: name the endpoint, domain logic, schema, test, deployment control, or production issue you owned.

Why FastAPI

FastAPI is useful when a Python service benefits from typed request validation, OpenAPI generation, dependency injection, and asynchronous I/O. It does not make CPU-bound Python work parallel; that work belongs in a worker process or separate compute service.

```
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI()

class OrderCreate(BaseModel):
    customer_id: int = Field(gt=0)
    amount_cents: int = Field(gt=0)

class OrderRead(OrderCreate):
    id: int
    status: str

class OrderService:
    async def create(self, payload: OrderCreate) -> OrderRead:
        return OrderRead(id=42, status="pending", **payload.model_dump())

def get_order_service() -> OrderService:
    return OrderService()
```

```
@app.post("/orders", response_model=OrderRead, status_code=201)
async def create_order(
    payload: OrderCreate,
    service: Annotated[OrderService, Depends(get_order_service)],
) -> OrderRead:
    try:
        return await service.create(payload)
    except ValueError as exc:
        raise HTTPException(status_code=409, detail=str(exc)) from exc
```

Where asynchronous programming helps

Async helps when a request spends time waiting for independent I/O such as a database, object store, or HTTP service. Use bounded concurrency, deadlines, and cancellation; never create an unbounded task for every item.

```
import asyncio

import httpx

async def fetch_inputs(urls: list[str]) -> list[dict]:
    limits = httpx.Limits(max_connections=20)
    timeout = httpx.Timeout(3.0)
    semaphore = asyncio.Semaphore(10)

    async with httpx.AsyncClient(limits=limits, timeout=timeout) as client:
        async def fetch(url: str) -> dict:
            async with semaphore:
                response = await client.get(url)
                response.raise_for_status()
                return response.json()

        return await asyncio.gather(*(fetch(url) for url in urls))
```

How Pydantic models are used

Use distinct input, domain, and output models. Request models validate the API boundary; domain objects enforce business invariants; response models prevent accidental data leakage. For partial updates, inspect `model_fields_set` so an omitted value is different from an explicit `null`.

```
from pydantic import BaseModel, ConfigDict, EmailStr

class UserCreate(BaseModel):
    email: EmailStr
    display_name: str

class UserRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    email: EmailStr
    display_name: str
```

How services access PostgreSQL

Use a request-scoped session, a small transaction boundary, parameterized SQL or an ORM, and a pooled engine. Keep HTTP concerns outside the repository layer.

```
from collections.abc import AsyncIterator

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine

engine = create_async_engine("postgresql+asyncpg://app:secret@db/app", pool_pre_ping=True)
SessionFactory = async_sessionmaker(engine, expire_on_commit=False)

async def get_session() -> AsyncIterator[AsyncSession]:
    async with SessionFactory() as session:
```

```
try:
    yield session
    await session.commit()
except Exception:
    await session.rollback()
    raise
```

How Docker and Kubernetes are used

Docker creates an immutable runtime image. Kubernetes supplies desired-state deployment, service discovery, health checks, configuration, secrets, rolling updates, and autoscaling. Readiness should indicate whether a pod can receive traffic; liveness should detect a stuck process without depending on every downstream service.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src ./src
USER 10001
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: orders-api
  template:
    metadata:
      labels:
        app: orders-api
    spec:
      containers:
        - name: api
          image: registry.example/orders-api:1.4.2
          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
```

What stages exist in Jenkins

A credible pipeline fails fast and promotes the same artifact:

1. checkout and dependency lock verification;
2. formatting, linting, secret scan, and Bandit;
3. unit and contract tests with coverage thresholds;
4. container build and software-bill-of-materials generation;
5. image and dependency vulnerability scans;
6. publish a content-addressed image;
7. deploy to a non-production environment;
8. smoke and integration tests;
9. approval for a controlled production rollout;
10. post-deployment verification and automatic rollback signals.

```
pipeline {
  agent any
  stages {
```



```
stage('Quality') {
  steps { sh 'ruff check . && bandit -r src -c pyproject.toml' }
}
stage('Test') {
  steps { sh 'pytest --cov=src --cov-fail-under=85' }
}
stage('Build') {
  steps { sh 'docker build -t "$IMAGE_URI:$GIT_COMMIT" .' }
}
stage('Deploy staging') {
  steps { sh 'helm upgrade --install orders deploy/chart --set image.tag="$GIT_COMMIT"' }
}
}
```

How deployment to AWS works

A common path builds the image in CI, scans it, pushes it to ECR, and updates an EKS deployment or ECS service. Runtime identity should use an IAM role rather than static keys. Configuration belongs in Parameter Store or AppConfig and secrets in Secrets Manager. CloudWatch or an OpenTelemetry backend receives logs, metrics, and traces. A canary or rolling deployment is promoted only when health and service-level indicators remain acceptable.

How meaningful coverage is reached

Coverage is a feedback signal, not the goal. Begin with critical service rules, error paths, permission checks, serialization, and repository boundaries. Add tests for every escaped defect, remove nondeterminism with injected clocks and clients, and use branch coverage. A threshold prevents regression, while mutation testing or targeted review checks whether assertions are meaningful.

```
def test_rejects_duplicate_idempotency_key(order_service, existing_order):
    order_service.repository.find_by_key.return_value = existing_order

    result = order_service.create(idempotency_key="request-123")

    assert result == existing_order
    order_service.repository.insert.assert_not_called()
```

How releases are coordinated across products

Use independently versioned components, a compatibility policy, matrix-driven contract tests, release notes, and staged rollout. Publish the shared component first, verify each supported consumer in CI, release a canary consumer, then promote remaining products. Assign an owner and rollback condition for every stage.

Bandit From POC to Production

1. Define the gap and success criteria: earlier Python security feedback with an

acceptable false-positive rate and runtime.

1. Run a POC on representative repositories and preserve a baseline report.

2. Triage by severity and confidence with security and service owners.

3. Fix true positives; document narrow suppressions with reason, owner, and

review date.

1. Propose policy: warn on existing debt, block newly introduced high-confidence

high-severity findings, and track remediation.

1. Obtain approval, add CI configuration, publish developer instructions, and

introduce the gate in stages.

1. Monitor findings, bypasses, runtime, and developer feedback.

```
[tool.bandit]
exclude_dirs = ["tests", ".venv"]
skips = ["B101"]

[tool.ruff.lint]
select = ["E", "F", "I", "B", "S"]
```

If Bandit reports a high-severity issue during a release, confirm the file and data flow, reproduce the risk, and treat a high-confidence true positive as a stop condition. Fix and rerun the pipeline. A time-limited exception requires a risk owner, compensating control, approval, and expiry; a bare `# nosec` is not a resolution.

Backward Compatibility Matrix

A one-to-one map such as `client 2.1 -> API 4.3` grows poorly and hides which capability is incompatible. A matrix expresses support and expected behavior.

Client	API v1	API v2	Capability set	Expected result
1.x	Supported	Supported through adapter	basic	Legacy response shape
2.x	Supported	Supported	basic, bulk	Native response shape
3.x	Not supported	Supported	basic, bulk, audit	Native plus audit metadata

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
class CompatibilityCase:
    client_major: int
    api_major: int
    capabilities: frozenset[str]
    expected_status: int

CASES = [
    CompatibilityCase(1, 2, frozenset({"basic"}), 200),
    CompatibilityCase(2, 2, frozenset({"basic", "bulk"}), 200),
    CompatibilityCase(3, 1, frozenset({"audit"}), 426),
]
```

Generate contract tests from the matrix. Prefer additive fields, stable defaults, tolerant readers, explicit capability negotiation, boundary adapters, and a published deprecation window. Observe old-client traffic before removal.

Financial and High-Volume Data Workflows

What workflows might be supported

Examples include ingestion, validation, reconciliation, exception handling, approval, settlement status, reporting, and audit history. Explain the state transitions, invariants, actors, and failure recovery rather than saying only "transaction processing."

How to process more than one million records

Do not load all rows into memory. Read stable, ordered chunks; validate and transform vectorially; write idempotent batches; checkpoint progress; and send invalid records to a review path. Parallelize only after measuring database and downstream capacity.

```
from collections.abc import Iterator

import pandas as pd

def read_batches(path: str, size: int = 50_000) -> Iterator[pd.DataFrame]:
    yield from pd.read_csv(path, chunksize=size)

for batch in read_batches("transactions.csv"):
    valid = batch.loc[batch["amount"] >= 0].copy()
    valid["amount_cents"] = (valid["amount"] * 100).round().astype("int64")
    upsert_batch(valid) # Uses a stable business key and a transaction.
```

How SQL and processing performance are optimized

Start with evidence: p95 latency, row estimates, wait events, CPU, I/O, locks, and EXPLAIN (ANALYZE, BUFFERS). Remove N+1 queries, select only needed columns, index selective predicates and join keys, use keyset pagination, and precompute only when freshness requirements allow it.

```
CREATE INDEX CONCURRENTLY idx_transactions_account_created
ON transactions (account_id, created_at DESC)
INCLUDE (status, amount_cents);

SELECT id, status, amount_cents, created_at
FROM transactions
WHERE account_id = :account_id
AND (created_at, id) < (:cursor_created_at, :cursor_id)
ORDER BY created_at DESC, id DESC
LIMIT 100;
```

How Flask, databases, and Lambda can interact

Flask can validate synchronous commands and persist durable state. An outbox row written in the same transaction can trigger asynchronous work through a stream or queue. Lambda processes the event idempotently and records completion. Avoid invoking Lambda and updating the database as unrelated operations because one can succeed while the other fails.

How security and data integrity are maintained

Use least privilege, TLS, encryption at rest, secret rotation, input validation, parameterized queries, strong authorization, audit events, idempotency keys, and database constraints. Reconcile totals and counts across stages and make every manual correction attributable and reversible.

LLM and Agent Evaluation

What to evaluate

Evaluate task success, groundedness, citation correctness, instruction following, structured-output validity, tool selection, argument accuracy, recovery behavior, latency, token cost, safety, and policy compliance. Use a versioned evaluation set with normal, adversarial, and boundary cases.

How to identify hallucination and tool failures

- **Hallucination:** a factual claim lacks support or contradicts the retrieved evidence.

- **Grounding failure:** the answer ignores, misquotes, or overgeneralizes the supplied context.

- **Tool-selection failure:** the agent selects the wrong tool, calls a tool when none is needed, or fails to call a required tool.

- **Argument failure:** the right tool receives invalid, unauthorized, or semantically wrong arguments.

Compare each trace with an expected policy and tool sequence, then review the final answer against evidence. Separate retrieval failure, reasoning failure, tool failure, and presentation failure so the fix targets the right component.

```
from typing import Literal

from pydantic import BaseModel, Field

class ToolDecision(BaseModel):
    tool: Literal["search_policy", "get_account", "none"]
    reason: str = Field(min_length=5, max_length=300)
    account_id: str | None = None

def authorize(decision: ToolDecision, allowed_accounts: set[str]) -> None:
    if decision.tool == "get_account" and decision.account_id not in allowed_accounts:
        raise PermissionError("Tool request exceeds the caller's account scope")
```

Professional experience versus portfolio work

State the boundary directly. Describe production work with verbs such as "operated" or "deployed" only when true. Describe a local demonstration as a "portfolio implementation" and explain which production concerns it does not prove, such as scale, incident ownership, or regulated-data handling.

How to productionize an agent safely

Use a constrained state machine, allowlisted tools, schema validation, caller- scoped authorization, read-only defaults, approval for consequential writes, bounded iterations, timeouts, sandboxing, prompt-injection defenses, redaction, durable audit trails, evaluation gates, and a kill switch. The model may propose an action; trusted application code must authorize and execute it.

Likely Production Scenarios

An API suddenly becomes slow

Confirm user impact and compare p50, p95, and p99 latency with a known baseline. Follow a trace across gateway, application, database, cache, and dependencies. Check recent deployments and saturation, then mitigate safely by rollback, traffic reduction, or disabling the affected path. Reproduce the bottleneck and add a regression test after the root cause is fixed.

A downstream service is unavailable

Use short timeouts, capped exponential backoff with jitter, a circuit breaker, bulkheads, bounded queues, and a defined fallback. Retry only safe or idempotent operations and propagate a correlation ID. Load shedding is better than allowing every worker and connection pool to become exhausted.

```
async def call_with_timeout(client, url: str) -> dict:
    response = await client.get(url, timeout=2.0)
    response.raise_for_status()
```

```
return response.json()
```

A Kafka consumer receives an event twice

Assume at-least-once delivery. Use a stable event ID, insert it into a processed- events table under a unique constraint, and apply the business change in the same transaction. Commit the offset only after durable success.

```
BEGIN;
INSERT INTO processed_events (event_id)
VALUES (:event_id)
ON CONFLICT DO NOTHING;
-- Apply the business mutation only when the insert affected one row.
COMMIT;
```

A new API field must not break older clients

Make it optional or give it a stable default, preserve existing field semantics, keep old response parsing valid, and add consumer-driven contract tests. If the meaning changes, introduce a new field or version instead of silently redefining the old one.

Several I/O calls must run concurrently

Use `asyncio.gather` or a task group with a semaphore, per-call deadlines, clear partial-failure semantics, and cancellation. Protect the downstream service with connection limits; concurrency without a bound merely moves the outage.

A query slows as data grows

Capture the real query and parameters, inspect the execution plan and buffer use, compare estimated and actual row counts, and check locks and table statistics. Then test the smallest change - query shape, index, partition pruning, or data model - against production-like volume.

A deployment introduces errors

Stop promotion and roll back or shift traffic to the last known-good immutable artifact. Preserve logs, traces, metrics, configuration, and deployment metadata. Reproduce the failure, identify why pre-production checks missed it, add the test or guard, and redeploy through the normal pipeline.

An agent requests a sensitive tool operation

Do not let the model authorize itself. Validate the structured request, bind it to the caller identity, enforce policy in trusted code, show a preview, require human approval, use a least-privilege credential, and record an audit event.

How to monitor a FastAPI service

Monitor request rate, error rate, duration, and saturation; dependency latency; database pool use; queue depth; worker restarts; and business success metrics. Use structured logs with request IDs, distributed traces, health endpoints, SLOs, and alerts based on user impact rather than every individual exception.

```
from time import perf_counter

from fastapi import Request

@app.middleware("http")
async def record_latency(request: Request, call_next):
    started = perf_counter()
    response = await call_next(request)
    response.headers["Server-Timing"] = f"app;dur={{(perf_counter() - started) * 1000:.1f}}"
    return response
```

Interview Delivery Checklist

Clarify requirements and state assumptions, then begin with a simple design and identify its scaling boundary. Cover failure modes, security, tests, observability, and rollback; distinguish personal implementation from team context; use only accurate metrics; and end with the trade-off and next improvement.

Coding Interview Patterns, Data Structures & Progressive Optimization

Source: docs/coding_interview_patterns.md

Purpose: Reusable senior-level coding interview strategy, algorithm patterns, complexity reasoning, edge-case testing, and progressive problem solving. **Use this file for:** 45-60 minute coding rounds where the interviewer expects a simple correct solution first, followed by optimization, complexity analysis, testing, and follow-up constraints.

Recommended Study Flow

1. Memorize the **problem-solving loop** rather than isolated answers.
2. Review the **Python complexity cheat sheet** until the common operations are automatic.
3. Practice each pattern by explaining the brute-force approach before writing the optimized solution.
4. Run the companion exercises in `coding_questions/progressive_algorithm_patterns.py`.
5. Finish every problem with edge cases, complexity, and one likely follow-up.

Quick Summary

A strong coding interview is not only about producing the optimal algorithm. The interviewer also evaluates how the solution is derived, whether the data structure matches the bottleneck, whether complexity is explained correctly, and whether the implementation remains correct under changed constraints.

Use this progression:

Understand

Clarify constraints

State a simple correct solution

Analyze time and space

Identify the expensive repeated operation

Choose a structure/algorithm that removes that bottleneck

Implement cleanly

Recalculate complexity

Test edge cases

Handle follow-up constraints

A useful interview sentence is:

The straightforward solution is __ and costs __. The bottleneck is __. If I maintain __, I can reduce the runtime to __ at the cost of __ additional memory.

1. Opening a Coding Problem

1.1 Restate the requirement

Before coding, confirm what the function must produce.

Let me restate the problem to make sure I understand it. We need to ___, and the output should ___.

1.2 Ask only constraint-changing questions

Good questions include:

- Can the input be empty?
- Are duplicates allowed?
- Is the input sorted?
- Can I modify the input?
- Do I need original indices or only values?
- What should happen when no solution exists?
- Is output ordering important?
- Is this a one-time batch operation or repeated queries?
- Can the data arrive as a stream?
- What are the approximate input-size limits?

Do not spend several minutes asking questions whose answers do not change the algorithm.

1.3 State the invariant while coding

Do not narrate syntax such as "now I am writing a loop." Explain what state means.

Weak:

I am adding a dictionary here.

Strong:

This dictionary maps each previously seen value to its index, so I can test for the required complement in expected $O(1)$ time.

2. Python Complexity Cheat Sheet

2.1 List

Operation	Typical Complexity	Interview Note
<code>append(x)</code>	amortized $O(1)$	Dynamic array may occasionally resize.
<code>pop()</code>	$O(1)$	Removes from the end.
<code>pop(0)</code>	$O(n)$	Remaining elements shift left.
<code>insert(0, x)</code>	$O(n)$	Existing elements shift right.
<code>x in list</code>	$O(n)$	Linear search.
index access	$O(1)$	Direct array indexing.
slicing	$O(k)$	Creates a new list of k items.
sort	$O(n \log n)$	Python uses Timsort.

2.2 Dictionary / set

Operation	Expected Complexity	Note
lookup	$O(1)$	Hash-based expected case.
insert/update	$O(1)$	Expected case.
delete	$O(1)$	Expected case.
membership	$O(1)$	Expected case.

Worst-case hash-table behavior can degrade toward $O(n)$, but expected $O(1)$ is the normal interview model unless adversarial hashing is part of the question.

2.3 collections.deque

Operation	Complexity
<code>append()</code>	$O(1)$
<code>appendleft()</code>	$O(1)$
<code>pop()</code>	$O(1)$
<code>popleft()</code>	$O(1)$

Use a deque when elements must be added/removed from both ends. A list is a poor FIFO queue when `pop(0)` is required.

2.4 Heap

Operation	Complexity
peek minimum	$O(1)$
push	$O(\log n)$
pop minimum	$O(\log n)$
heapify n items	$O(n)$

A size- k min-heap is common when only the best k items are needed from a much larger set.

2.5 Graphs and search

- BFS: $O(V + E)$
- DFS: $O(V + E)$
- Binary search: $O(\log n)$
- Sorting: $O(n \log n)$

Always define what n , V , E , k , or other variables mean before giving complexity.

3. Pattern: Hash Map - Two Sum

Question

Given an array of integers and a target, return the indices of two values whose sum is the target.

Simple solution

Compare every pair.

```
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] + nums[j] == target:
            return [i, j]
```

- Time: $O(n^2)$
- Auxiliary space: $O(1)$

Optimization reasoning

The expensive operation is repeatedly searching for the complement $\text{target} - \text{value}$.

Maintain:

value $\rightarrow$ index

Then each complement lookup is expected $O(1)$.

```
def two_sum(nums, target):
    seen = {}

    for index, value in enumerate(nums):
        complement = target - value
        if complement in seen:
            return [seen[complement], index]
        seen[value] = index

    return []
```

- Time: $O(n)$ expected
- Space: $O(n)$

Follow-up: Can memory be reduced?

If original indices are not required and mutation is allowed, sort and use two pointers:

- Time: $O(n \log n)$
- Extra space: potentially $O(1)$, depending on sorting/runtime details

The trade-off is losing the simple original-index mapping.

4. Pattern: Sliding Window - Longest Substring Without Repeating Characters

Question

Find the length of the longest substring containing no repeated character.

Optimization insight

Restarting a scan from every index repeats work. Maintain one valid window `[left, right]` and the most recent index of every character.

```
def longest_unique_substring(s):
    last_seen = {}
    left = 0
    best = 0

    for right, char in enumerate(s):
        if char in last_seen and last_seen[char] >= left:
            left = last_seen[char] + 1

        last_seen[char] = right
        best = max(best, right - left + 1)

    return best
```

- Time: $O(n)$
- Space: $O(\min(n, \text{alphabet size}))$

Why does `left` never move backward?

A character occurrence before `left` is no longer part of the active window and cannot invalidate it.

5. Pattern: Top-K - Frequency Map + Heap

Question

Return the k most frequent values.

Baseline

1. Count frequencies in $O(n)$.
2. Sort all unique values in $O(u \log u)$, where u is the number of unique values.

Total:

$O(n + u \log u)$

Improved when $k \ll u$

Keep a min-heap of size k .

count frequencies: $O(n)$
select top k : $O(u \log k)$

Total:

$O(n + u \log k)$

Follow-up: What if k is almost u ?

Sorting can be simpler and similarly efficient because $\log k$ approaches $\log u$. Choose based on actual constraints rather than assuming the heap is always superior.

The repository also contains a deeper batch/streaming variant in `coding_questions/top_k_frequent_paths.py`.

6. Pattern: Sorting + Scan - Merge Intervals

Question

Merge overlapping intervals.

Reasoning

Without ordering, detecting overlaps can require repeated comparison. Sorting by start time ensures that any interval capable of overlapping the current merged interval appears next.

```
def merge_intervals(intervals):
    if not intervals:
        return []

    intervals = sorted(intervals, key=lambda interval: interval[0])
    merged = [list(intervals[0])]

    for start, end in intervals[1:]:
        last_end = merged[-1][1]

        if start <= last_end:
            merged[-1][1] = max(last_end, end)
        else:
            merged.append([start, end])

    return merged
```

- Time: $O(n \log n)$
- Result space: $O(n)$ in the worst case

Edge cases

- Empty input
- One interval
- Nested intervals
- Touching boundaries such as `[1, 3]` and `[3, 5]`
- Already sorted or reverse-sorted input

Clarify whether touching intervals should merge.

7. Pattern: Prefix Sum + Hash Map - Subarray Sum Equals K

Question

Count contiguous subarrays whose sum equals k .

Baseline

Enumerating every start/end pair with a running sum costs $O(n^2)$.

Optimization

Let the current prefix sum be P . If an earlier prefix was $P - k$, then the subarray between them sums to k :

$$P - (P - k) = k$$

```
def subarray_sum_equals_k(nums, k):
    prefix_count = {0: 1}
    prefix = 0
    result = 0

    for value in nums:
        prefix += value
        result += prefix_count.get(prefix - k, 0)
        prefix_count[prefix] = prefix_count.get(prefix, 0) + 1

    return result
```

- Time: $O(n)$
- Space: $O(n)$

The initial $\{0: 1\}$ handles subarrays beginning at index 0.

8. Pattern: Prefix/Suffix - Product of Array Except Self

Question

For each index, return the product of every other value without division.

Simple solution

Multiply every other value per index: $O(n^2)$.

Better approach

Store the product of everything to the left, then multiply by a running suffix product from the right.

```
def product_except_self(nums):
    result = [1] * len(nums)

    prefix = 1
    for i in range(len(nums)):
        result[i] = prefix
        prefix *= nums[i]

    suffix = 1
    for i in range(len(nums) - 1, -1, -1):
        result[i] *= suffix
        suffix *= nums[i]

    return result
```

- Time: $O(n)$
- Auxiliary space: $O(1)$, excluding the required output array

Always state the "excluding output" qualification when claiming $O(1)$ auxiliary space.

9. Pattern: Heap or Quickselect - Kth Largest

Options

Approach	Time	Good Fit
Sort entire array	$O(n \log n)$	Simplicity, one moderate-sized query
Size-k min-heap	$O(n \log k)$	Streaming or small k
Quickselect	$O(n)$ average, $O(n^2)$ worst	One selection query where average linear time matters

Interview answer

I would start with sorting because it is simple and correct. If n is large and k is small, a size- k heap avoids ordering everything. If one batch selection must be optimized further, quickselect gives expected $O(n)$ time, with a more complex implementation and $O(n^2)$ worst case unless stronger pivot strategies are used.

10. Pattern: Intervals + Heap - Meeting Rooms

Question

Find the minimum number of rooms required to schedule all meetings.

Min-heap interpretation

The heap contains end times of rooms currently in use. The smallest end time tells whether the next meeting can reuse a room.

```
import heapq

def min_meeting_rooms(intervals):
    if not intervals:
        return 0

    intervals = sorted(intervals, key=lambda interval: interval[0])
    heap = []

    for start, end in intervals:
        if heap and heap[0] <= start:
            heapq.heappop(heap)
        heapq.heappush(heap, end)

    return len(heap)
```

- Time: $O(n \log n)$
- Space: $O(n)$ worst case

A sorted-start/sorted-end two-pointer approach is another $O(n \log n)$ solution.

11. Pattern: Graph Traversal - Number of Islands

Question

Count connected components of land cells in a grid.

DFS/BFS idea

Every unvisited land cell starts a new island. Traverse its entire connected component and mark it visited.

```
scan cell  
  
new unvisited land?  
  yes  
  increment island count  
  
DFS/BFS all connected land
```

- Time: $O(\text{rows} \times \text{cols})$
- Space: $O(\text{rows} \times \text{cols})$ worst case for recursion/queue/visited state

Follow-up: Cannot mutate input

Maintain a separate visited set or boolean matrix, trading additional memory for preserving the original grid.

For very deep components in Python, iterative DFS/BFS can avoid recursion-depth limits.

12. Pattern: Topological Sort - Course Schedule

Question

Given prerequisite pairs, determine whether all courses can be completed.

Model

This is cycle detection in a directed graph.

Kahn's algorithm maintains indegree counts:

```
nodes with indegree 0  
  
  queue  
  
remove node and decrement neighbors  
  
new indegree 0 nodes enter queue
```

If fewer than V nodes are processed, a directed cycle exists.

- Time: $O(V + E)$
 - Space: $O(V + E)$
-

13. Pattern: K-Way Merge of Sorted Streams

This pattern is useful both in coding interviews and feed/search aggregation designs.

Question

Given P streams, each already sorted newest-to-oldest, return the newest N items across all streams.

Naive option

Concatenate all items and sort them all.

If there are M total candidates:

```
 $O(M \log M)$ 
```

Better option

Put only the current head of every stream into a heap. Each pop reveals the next globally best item; then push the next item from that same stream.

```
P sorted streams
heap of at most P heads
pop best, advance that stream
stop after N results
```

Complexity for returning N items after stream heads are available:

$O(N \log P)$

This is especially valuable when N is small relative to the total history in all streams.

14. Choosing the Data Structure from the Bottleneck

Repeated expensive operation	Likely structure/pattern
Search for previously seen value	Hash map / set
Add/remove from both ends	Deque
Repeated min/max among changing candidates	Heap
Need only top k	Size-k heap / selection
Contiguous range with changing boundary	Sliding window
Repeated range/subarray sum	Prefix sum
Overlap among ranges	Sort + scan / heap
Connectivity	DFS/BFS / Union-Find
Dependency order	Topological sort
Ordered lookup	Binary search / balanced tree
$O(1)$ lookup + $O(1)$ recency updates	Hash map + doubly linked list

The best answer is not "use data structure X." The best answer explains what repeated work X removes.

15. Edge-Case Testing Checklist

Before saying "done," consider:

```
normal case
empty input
single element
minimum/maximum bounds
duplicates
negative/zero values
no-solution case
multiple valid solutions
ordering/tie rules
whether input was accidentally mutated
very large input assumptions
```

For stateful data structures additionally test:

```
insert
update same key
remove
```


capacity boundary
reinsert
repeated operation
expiration boundary
stale metadata entries

16. What to Say When Temporarily Stuck

Do not go silent. Anchor on correctness and identify the bottleneck.

The brute-force version is clear, so I'll anchor correctness there. The expensive operation appears to be _____. I'm thinking about whether I can avoid repeating it by maintaining _____.

Or:

I see two possible directions. One trades memory for faster lookup using a hash structure, while the other imposes ordering first. Let me compare their complexity against the constraints.

This keeps the interviewer inside the reasoning process.

17. Common Coding Interview Mistakes

- Jumping directly to an optimized pattern without explaining why it applies.
 - Giving Big-O without defining the variables.
 - Claiming $O(1)$ space while ignoring an auxiliary map/list that grows with input.
 - Using `list.pop(0)` as an $O(1)$ queue operation in Python.
 - Mutating input without confirming it is allowed.
 - Treating hash-table $O(1)$ as an absolute worst-case guarantee.
 - Forgetting stable tie-breaking requirements in top-k/ranking problems.
 - Writing code silently for several minutes.
 - Saying "done" before testing a normal case and boundaries.
 - Optimizing before a correct baseline is established when the interview specifically values progressive improvement.
-

18. High-Probability Follow-Ups

Be ready for the interviewer to change one constraint:

- What if input arrives as a stream?
- What if memory is limited?
- What if there are repeated queries with different k?
- What if the input is already sorted?
- What if values are extremely high cardinality?
- What if the input cannot be modified?
- What if the operation must be thread-safe?
- What if the state lives across multiple processes/machines?
- What if exactness can be traded for bounded memory?
- What if the query becomes latency-sensitive at 10x scale?

A good response is to explain which assumption changed and therefore why the original data structure may no longer be ideal.

19. 60-Minute Coding Round Game Plan

A realistic one-hour round often looks like:

0-5 min	clarify + baseline approach
5-15 min	implement simple/correct solution or core structure
15-25 min	analyze + optimize
25-40 min	implement optimized version
40-50 min	tests + edge cases
50-58 min	follow-up constraint / concurrency / scaling
58-60 min	summarize complexity + trade-off

The exact pacing varies, but the principle is the same: correctness first, then measured improvement.

20. Quick Revision Checklist

Before a coding interview, be able to explain from memory:

- Hash map and set use cases
- Two pointers
- Sliding window
- Prefix sums
- Stack and monotonic stack basics
- Queue/deque
- Heap/top-k
- Intervals
- Linked-list pointer manipulation
- Trees and BST traversal
- BFS/DFS
- Topological sort
- Binary search
- LRU design
- Time/space complexity of common Python operations
- Edge-case testing
- How the solution changes for streaming or concurrency

Companion runnable implementations are in:

- `coding_questions/progressive_algorithm_patterns.py`
- `coding_questions/concurrent_data_structures.py`
- `coding_questions/expiring_window_map.py`
- `coding_questions/top_k_frequent_paths.py`

21. Sliding-Window Hit Counter

Question

Design a counter that returns how many events occurred during the last W seconds/minutes.

Simple exact deque approach

Store event timestamps in chronological order:

```
hit(t)
    -> append timestamp

count(now)
    -> popleft while timestamp <= now - W
    -> len(deque)
```

With each timestamp inserted and removed once, cleanup is amortized $O(1)$ per event; memory is $O(\text{number of hits in the active window})$.

High-volume optimization

If timestamp resolution is bounded, aggregate into time buckets instead of one entry per hit:

```
bucket timestamp  
bucket count
```

This caps memory near the number of time buckets rather than total event count, at the cost of bucket-level precision and careful boundary handling.

Follow-ups

- What if hits arrive out of order? A simple deque assumes chronological arrival; use ordered storage or event-time/windowing logic otherwise.
- What if the counter is distributed? Per-process counters must be combined or moved to a shared/partitioned store; exact global windows add coordination cost.
- What if approximate counts are acceptable? Approximation/sketching can reduce memory/network overhead at scale.

Python

Source: docs/python.md

Purpose: Python fundamentals, advanced Python, backend coding patterns, concurrency, performance, and coding interview preparation. **Use this file for:** Python technical interviews and backend engineering interviews

Recommended Study Flow

1. Read the **Quick Summary** first.
2. Review the **Key Concepts** and tables.
3. Practice the **Interview Questions & Answers** out loud.
4. Use the code snippets and examples to explain trade-offs clearly.
5. Finish with the **Common Mistakes** and **Revision Checklist** sections.

Quick Summary

This file has been refreshed to keep the original repository topic while merging relevant detailed Q&A from the consolidated topic-wise interview-prep pack. Use the top sections for fast revision and the consolidated section for deeper interview preparation.

Core Topics to Master

- OOP: encapsulation, inheritance, polymorphism, abstraction
- Data structures: list, tuple, set, dict
- Decorators, generators, context managers, lambda functions
- Exception handling and logging
- GIL, threading, multiprocessing, asyncio
- Performance optimization, memory management, profiling
- Testing with Pytest and coding interview patterns

Consolidated Interview Questions & Technical Notes

Python fundamentals, data structures, decorators, generators, exception handling, GIL, threading, multiprocessing, asyncio, performance, and coding exercises.

4. Python Internals & Concurrency

4.1 Python GIL

Interview answer:

The Global Interpreter Lock, or GIL, is a mechanism in CPython that allows only one thread to execute Python bytecode at a time inside a process. It simplifies memory management but limits true parallelism

for CPU-bound multithreaded workloads. For I/O-bound tasks like API calls or database queries, threads or asyncio still work well because they spend most of their time waiting. For CPU-bound work, multiprocessing is usually better.

4.2 Threading vs Multiprocessing vs AsyncIO

Approach	Best For	Notes
Threading	Simple I/O-bound work	Shared memory, affected by GIL for CPU work
Multiprocessing	CPU-bound work	Separate processes, avoids GIL limits
AsyncIO	High-concurrency I/O	Single-threaded event loop, efficient for many network calls

Interview answer:

I choose based on workload. For many API calls or LLM requests, I use asyncio with an async HTTP client and semaphores for rate limits. For CPU-heavy tasks, I use multiprocessing. Threading is useful for simpler I/O tasks, but not ideal for CPU-bound computation because of the GIL.

4.3 AsyncIO with rate limiting

```
import asyncio
import time
import aiohttp

MAX_CONCURRENCY = 10
RATE_LIMIT = 50 # requests per minute

semaphore = asyncio.Semaphore(MAX_CONCURRENCY)

class RateLimiter:
    def __init__(self, rate_per_minute: int):
        self.interval = 60 / rate_per_minute
        self.lock = asyncio.Lock()
        self.last_call = 0.0

    async def acquire(self):
        async with self.lock:
            now = time.monotonic()
            wait_time = max(0, self.interval - (now - self.last_call))
            if wait_time:
                await asyncio.sleep(wait_time)
            self.last_call = time.monotonic()

rate_limiter = RateLimiter(RATE_LIMIT)

async def fetch(session: aiohttp.ClientSession, url: str):
    retries = 0
    start = time.perf_counter()

    while True:
        try:
            await rate_limiter.acquire()

            async with semaphore:
                with session.get(url) as response:
                    if response.status >= 500 and retries < 3:
                        raise RuntimeError("Server error")

                    data = await response.json()
                    latency = time.perf_counter() - start
                    print(f"{url} latency={latency:.2f}s retries={retries}")
                    return data

        except Exception:
            if retries >= 3:
                return None

            await asyncio.sleep(2 ** retries)
            retries += 1

async def fetch_all(urls):
    async with aiohttp.ClientSession() as session:
        tasks = [fetch(session, url) for url in urls]
        return await asyncio.gather(*tasks)
```

4.4 Semaphore

Interview answer:

A semaphore controls how many tasks can run concurrently. In an LLM application, I might allow only 10 simultaneous API requests to avoid hitting provider rate limits or overwhelming downstream services.

4.5 Retry Strategy

For API or LLM failures:

- Retry transient 5xx errors
- Use exponential backoff
- Add jitter
- Set max retry count
- Use timeouts
- Add circuit breaker for repeated failures
- Fallback to another model/service if needed

15. Python Topics

15.1 Why Use Generators in AI Pipelines?

"Generators help process large datasets efficiently by producing one item at a time instead of loading everything into memory."

15.2 Generator Example

```
def read_documents(file_paths):
    for path in file_paths:
        with open(path, "r", encoding="utf-8") as file:
            yield file.read()

for document in read_documents(["doc1.txt", "doc2.txt"]):
    print(document[:100])
```

15.3 AI Pipeline Use Case

In a RAG ingestion pipeline:

```
def generate_chunks(document: str, chunk_size: int = 500):
    words = document.split()

    for i in range(0, len(words), chunk_size):
        yield " ".join(words[i:i + chunk_size])

def process_documents(documents):
    for doc in documents:
        for chunk in generate_chunks(doc):
            embedding = create_embedding(chunk)
            store_embedding(chunk, embedding)
```

15.4 Benefits of Generators

- Memory efficient
- Good for large files
- Useful for streaming data
- Improves scalability
- Works well for ETL pipelines
- Useful for document chunking
- Prevents loading everything into RAM

15.5 Strong Interview Line

"I use generators in AI pipelines to stream large datasets, documents, logs, or chunks one item at a time, which improves memory efficiency and scalability."

4. Python Core Topics

The role expects strong Python fundamentals.

Topics to revise

- Data structures: list, tuple, set, dict.
- List comprehensions and generator expressions.
- Functions, decorators, closures.
- Iterators and generators.
- Exception handling.
- Context managers.
- Async programming with asyncio.
- Type hints.
- Packaging and dependency management.
- Performance profiling.
- Memory management basics.

Example: Type hints

```
from typing import Optional

def calculate_total(price: float, tax_rate: float, discount: Optional[float] = None) -> float:
    total = price + (price * tax_rate)
    if discount:
        total -= discount
    return round(total, 2)
```

Interview explanation:

Type hints improve readability, make large codebases easier to maintain, and allow tools such as mypy or IDEs to detect type-related issues before runtime.

Example: Async programming

```
import asyncio
import httpx

async def fetch_url(url: str) -> str:
    async with httpx.AsyncClient() as client:
        response = await client.get(url)
        response.raise_for_status()
        return response.text

async def main() -> None:
    urls = [
        "https://example.com/a",
        "https://example.com/b",
        "https://example.com/c",
    ]
    results = await asyncio.gather(*(fetch_url(url) for url in urls))
    print(len(results))

## asyncio.run(main())
```

Interview explanation:

asyncio is useful for I/O-bound workloads such as API calls, database operations, or network requests. It is not ideal for CPU-heavy tasks unless combined with multiprocessing or worker systems.

Example: Decorator

```
import time
from functools import wraps
from typing import Callable, Any

def log_duration(func: Callable[..., Any]) -> Callable[..., Any]:
    @wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        start = time.perf_counter()
        result = func(*args, **kwargs)
        duration = time.perf_counter() - start
        print(f"{func.__name__} took {duration:.4f}s")
        return result
    return wrapper

@log_duration
def process_records(records: list[dict]) -> int:
    return len(records)
```

Interview explanation:

A decorator wraps a function to add reusable behavior such as logging, timing, authentication, caching, or retry handling without changing the function's core logic.

1. Python Fundamentals

1.1 What is the difference between a tuple and a set?

Interview Answer

A **tuple** is an ordered, immutable collection that allows duplicates. A **set** is an unordered collection that stores only unique elements.

Feature	Tuple	Set
Ordered	Yes	No guaranteed order
Mutable	No	Yes
Allows duplicates	Yes	No
Indexing	Yes	No
Common use	Fixed records	Unique values, membership checks

Example

```
my_tuple = (3, 1, 2, 2)
my_set = {3, 1, 2, 2}

print(my_tuple)
print(my_set)
```

Possible output:

```
(3, 1, 2, 2)
{1, 2, 3}
```

Key Point

The tuple preserves the order and duplicates. The set removes duplicates and does not guarantee insertion order.

Strong One-Liner

Tuples preserve order and allow duplicates, while sets are unordered collections optimized for uniqueness and fast membership checks.

1.2 Write a list comprehension to find squares from 1 to 5

Code

```
squares = [x ** 2 for x in range(1, 6)]  
print(squares)
```

Output

```
[1, 4, 9, 16, 25]
```

Explanation

- `range(1, 6)` generates numbers from 1 to 5.
- `x ** 2` calculates the square of each number.
- The list comprehension creates a new list in one line.

2. Python Operators & Expressions

2.1 What is the output of `2**3**2`?

Code

```
print(2 ** 3 ** 2)
```

Output

```
512
```

Explanation

Exponentiation in Python is evaluated **right to left**.

So:

```
2 ** 3 ** 2
```

is interpreted as:

```
2 ** (3 ** 2)
```

Step-by-step:

```
3 ** 2 = 9  
2 ** 9 = 512
```

2.2 What is the output of `a | b` when `a = 4` and `b = 11`?

Code

```
a = 4  
b = 11  
  
print(a | b)
```

Output

15

Explanation

| is the **bitwise OR** operator.

Binary representation:

```
4  = 0100
11 = 1011
```

Bitwise OR:

```
0100
1011
----
1111
```

Binary 1111 equals decimal 15.

3. Lists, Tuples, and Sets

3.1 What does `seta ^ setb` mean?

Answer

`^` between two sets means **symmetric difference**.

It returns elements that are present in either set, but **not in both**.

Example

```
seta = {1, 2, 3}
setb = {3, 4, 5}

print(seta ^ setb)
```

Output

```
{1, 2, 4, 5}
```

Explanation

- 3 exists in both sets, so it is removed.
- Unique elements from both sets are returned.

Equivalent expression:

```
(seta - setb) | (setb - seta)
```

3.2 Different ways to join two lists

Method 1: Using +

```
a = [1, 2]
b = [3, 4]

result = a + b

print(result)
```

Output:

```
[1, 2, 3, 4]
```

This creates a new list.

Method 2: Using `extend()`

```
a = [1, 2]
b = [3, 4]

a.extend(b)

print(a)
```

Output:

```
[1, 2, 3, 4]
```

This modifies the original list.

Method 3: Using unpacking

```
a = [1, 2]
b = [3, 4]

result = [*a, *b]

print(result)
```

Output:

```
[1, 2, 3, 4]
```

Method 4: Using `itertools.chain()`

```
from itertools import chain

a = [1, 2]
b = [3, 4]

result = list(chain(a, b))

print(result)
```

Output:

```
[1, 2, 3, 4]
```

Useful for large iterables.

Method 5: Using a loop

```
a = [1, 2]
b = [3, 4]

for item in b:
    a.append(item)

print(a)
```

Output:

```
[1, 2, 3, 4]
```

3.3 Difference between `append()` and `extend()`**`append()`**

Adds the entire object as one element.

```
list1 = [1, 2]
list2 = [3, 4]

list1.append(list2)
```

```
print(list1)
```

Output:

```
[1, 2, [3, 4]]
```

Here, [3, 4] is added as a single object.

extend()

Adds each element individually.

```
list1 = [1, 2]
list2 = [3, 4]

list1.extend(list2)

print(list1)
```

Output:

```
[1, 2, 3, 4]
```

Equivalent to:

```
for item in list2:
    list1.append(item)
```

Strong One-Liner

`append()` adds the whole object as a single element, while `extend()` iterates through the iterable and adds each element individually.

3.4 What happens when we multiply a list by 2?

Example

```
my_list = [1, 2, 3]

print(my_list * 2)
```

Output:

```
[1, 2, 3, 1, 2, 3]
```

Explanation

* 2 repeats the list twice. It does **not** multiply individual elements.

3.5 Character list multiplication

Example

```
chars = ['a', 'b', 'c']

print(chars * 2)
```

Output:

```
['a', 'b', 'c', 'a', 'b', 'c']
```

It does **not** produce:

```
['aa', 'bb', 'cc']
```

Important Caveat

```
arr = [[]] * 3  
  
arr[0].append(1)  
  
print(arr)
```

Output:

```
[[1], [1], [1]]
```

Why? All inner lists refer to the same object in memory.

Correct way:

```
arr = [[] for _ in range(3)]  
  
arr[0].append(1)  
  
print(arr)
```

Output:

```
[[1], [], []]
```

4. Python Functions, Decorators, and Generators

4.1 What are decorators in Python?

Interview Answer

A decorator is a function that modifies or extends the behavior of another function without changing its source code.

Decorators are commonly used for:

- Logging
 - Authentication
 - Authorization
 - Caching
 - Retry logic
 - Performance measurement
 - API route registration
-

Simple Decorator Example

```
def logger(func):  
    def wrapper():  
        print("Function started")  
        func()  
        print("Function ended")  
    return wrapper  
  
@logger  
def greet():  
    print("Hello")  
  
greet()
```

Output:

```
Function started  
Hello
```

Function ended

What does @logger mean?

This:

```
@logger
def greet():
    print("Hello")
```

is equivalent to:

```
def greet():
    print("Hello")

greet = logger(greet)
```

Decorator with Arguments

```
def logger(func):
    def wrapper(*args, **kwargs):
        print("Calling function")
        return func(*args, **kwargs)
    return wrapper
```

```
@logger
def add(a, b):
    return a + b
```

```
print(add(2, 3))
```

Output:

```
Calling function
5
```

FastAPI Decorator Example

```
@app.get("/users")
def get_users():
    return {"users": []}
```

Here, `@app.get("/users")` is a decorator.

It tells FastAPI:

When a GET request comes to `/users`, execute this function.

Strong One-Liner

Decorators allow us to add reusable behavior like logging, authentication, caching, or route registration without modifying the original function.

4.2 What is a generator in Python?

Interview Answer

A generator is a special function that returns values one at a time using `yield`, instead of returning all values at once.

Generators are memory efficient because they produce values lazily.

Normal Function Using return

```
def get_numbers():  
    return [1, 2, 3]  
  
result = get_numbers()  
print(result)
```

Output:

```
[1, 2, 3]
```

The entire list is created in memory.

Generator Function Using yield

```
def get_numbers():  
    yield 1  
    yield 2  
    yield 3  
  
gen = get_numbers()  
  
print(next(gen))  
print(next(gen))  
print(next(gen))
```

Output:

```
1  
2  
3
```

The values are generated one by one.

4.3 Difference between yield and return

return	yield
Ends function execution	Pauses function execution
Returns full result at once	Produces one value at a time
More memory usage for large data	Memory efficient
Function state is lost	Function state is preserved

4.4 When should we use yield and when should we use return?

Use return when:

- The dataset is small.
- You need all results immediately.
- The result should be available at once.
- Memory usage is not a concern.

Example:

```
def get_users():  
    return ["John", "Alice", "Bob"]
```

Use yield when:

- The dataset is large.
- You are streaming data.
- You are reading large files.
- You want memory efficiency.
- You are processing records one by one.

Example:

```
def read_large_file():
    with open("logs.txt") as f:
        for line in f:
            yield line
```

Large Dataset Example

Using return:

```
def numbers():
    return [i for i in range(1, 1000001)]
```

This creates one million numbers in memory.

Using yield:

```
def numbers():
    for i in range(1, 1000001):
        yield i
```

This generates values on demand.

Strong One-Liner

Use return when you need the complete result immediately. Use yield when processing large datasets, streams, files, or values that should be generated one at a time.

5. Exception Handling

5.1 When does the finally block execute?

Answer

The finally block is executed **always**, whether an exception occurs or not.

It runs:

- If no exception occurs.
- If an exception occurs and is handled.
- If an exception occurs and is not handled.
- Even if there is a return statement inside try or except.

Example 1: No Exception

```
try:
    print("Inside try")
except:
    print("Inside except")
finally:
    print("Inside finally")
```

Output:

```
Inside try
Inside finally
```

Example 2: Exception Occurs

```
try:
    10 / 0
except ZeroDivisionError:
    print("Exception handled")
finally:
    print("Inside finally")
```

Output:

```
Exception handled
Inside finally
```

Example 3: return Statement

```
def test():
    try:
        return "try"
    finally:
        print("finally")

print(test())
```

Output:

```
finally
try
```

The finally block executes before the function actually returns.

Real-World Use

finally is commonly used for cleanup:

```
file = open("data.txt")

try:
    data = file.read()
finally:
    file.close()
```

Even if reading fails, the file is closed.

Strong One-Liner

The finally block always executes and is mainly used for cleanup operations like closing files, database connections, or network sockets.

6. Concurrency in Python

6.1 What is the GIL?

Answer

GIL stands for **Global Interpreter Lock**.

It is a mechanism in CPython that allows only one thread to execute Python bytecode at a time within a process.

Why does Python have GIL?

The GIL simplifies:

- Memory management
- Thread safety of Python objects
- Reference counting

Important Effect

Even if we create multiple threads, only one thread can execute Python bytecode at a time in CPython.

6.2 Is multithreading useless because of GIL?

No.

Multithreading is still useful for **I/O-bound tasks**, such as:

- API calls
- Database queries
- File operations
- Network requests
- External LLM calls

While one thread waits for I/O, another thread can run.

6.3 Why is multiprocessing better for CPU-heavy tasks?

For CPU-heavy tasks, threads compete for the GIL.

Examples:

- Image processing
- ML computation
- Mathematical calculations
- Large data transformations

Multiprocessing creates separate processes with separate Python interpreters, so it bypasses the GIL.

6.4 Multithreading vs Multiprocessing

Multithreading	Multiprocessing
Multiple threads in one process	Multiple independent processes
Shared memory	Separate memory
Lightweight	Heavier
Good for I/O-bound tasks	Good for CPU-bound tasks
Affected by GIL	Bypasses GIL

Multithreading Example

```
import threading

def task():
    print("Running thread")

t1 = threading.Thread(target=task)
t1.start()
t1.join()
```

Multiprocessing Example

```
from multiprocessing import Process

def task():
    print("Running process")

p = Process(target=task)
p.start()
p.join()
```

Strong Interview Answer

Multithreading is mainly useful for I/O-bound concurrency, while multiprocessing is better for CPU-bound parallel execution because it bypasses Python's GIL by running separate processes.

13. Coding Interview Practice Topics

These topics were discussed as likely coding interview preparation areas.

13.1 Arrays and strings

Prepare:

- Reverse string
- Palindrome check
- Valid anagram
- Longest common prefix
- Two sum
- Remove duplicates
- Count character frequency

Frequency Counter Example

```
def char_frequency(s):
    freq = {}

    for char in s:
        freq[char] = freq.get(char, 0) + 1

    return freq

print(char_frequency("banana"))
```

Output:

```
{'b': 1, 'a': 3, 'n': 2}
```

13.2 Hash maps

Useful for:

- Two Sum
- Frequency counting
- Group anagrams
- Detect duplicates

Two Sum Example

```
def two_sum(nums, target):
    seen = {}

    for i, num in enumerate(nums):
        diff = target - num

        if diff in seen:
            return [seen[diff], i]

        seen[num] = i

    return []

print(two_sum([2, 7, 11, 15], 9))
```

Output:

```
[0, 1]
```

13.3 Sliding window

Useful for:

- Longest substring without repeating characters
- Maximum sum subarray of size k
- Minimum window substring

Example: Maximum Sum of Size K

```
def max_sum_subarray(nums, k):
    window_sum = sum(nums[:k])
    max_sum = window_sum

    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]
        max_sum = max(max_sum, window_sum)

    return max_sum

print(max_sum_subarray([2, 1, 5, 1, 3, 2], 3))
```

Output:

```
9
```

13.4 Stack

Useful for:

- Valid parentheses
- Browser history
- Expression evaluation
- Monotonic stack problems

Valid Parentheses Example

```
def is_valid(s):
    stack = []
    mapping = {
        ")": "(",
        "}": "{",
        "]": "["
    }

    for char in s:
        if char in mapping.values():
            stack.append(char)
        elif char in mapping:
            if not stack or stack.pop() != mapping[char]:
                return False

    return not stack

print(is_valid("()[]{}"))
```

Output:

True

13.5 Linked list

Prepare:

- Reverse linked list
- Detect cycle
- Merge two sorted lists

Reverse Linked List Pattern

```
def reverse_list(head):
    prev = None
    current = head

    while current:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node

    return prev
```

13.6 Trees

Prepare:

- DFS
- BFS
- Maximum depth
- Validate BST
- Level order traversal

Max Depth Example

```
def max_depth(root):
    if not root:
        return 0

    return 1 + max(max_depth(root.left), max_depth(root.right))
```

13.7 SQL topics

Prepare:

- Joins
- Aggregations
- Group By
- Window functions
- ROW_NUMBER()
- RANK()
- DENSE_RANK()

Example: Top transaction per customer

```
SELECT *
FROM (
    SELECT
        t.*,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY amount DESC
        ) AS rn
    FROM transactions t
) ranked
WHERE rn = 1;
```

10. Python Core Concepts

10.1 Threading, multiprocessing, asyncio, and GIL

GIL

The Global Interpreter Lock allows only one thread to execute Python bytecode at a time in CPython.

Threading

Best for I/O-bound tasks.

```
import threading

def fetch():
    print("I/O task")

thread = threading.Thread(target=fetch)
```



```
thread.start()
thread.join()
```

Multiprocessing

Best for CPU-bound tasks.

```
from multiprocessing import Pool

def square(x):
    return x * x

with Pool() as pool:
    print(pool.map(square, [1, 2, 3]))
```

AsyncIO

Best for high-concurrency I/O with async libraries.

```
import asyncio

async def fetch():
    await asyncio.sleep(1)
    return "done"

asyncio.run(fetch())
```

10.2 Mutable vs immutable objects

Mutable

Can be changed after creation.

Examples:

- list
- dict
- set

Immutable

Cannot be changed after creation.

Examples:

- int
- float
- str
- tuple
- frozenset

10.3 List vs tuple

List

- Mutable.
- Good for dynamic collections.
- Supports append/remove/update.

Tuple

- Immutable.
- Good for fixed records.
- Can be used as dictionary key if all elements are hashable.

```
items = [1, 2, 3]
items.append(4)

point = (10, 20)
```

10.4 Dictionary keys

Dictionary keys must be hashable.

Valid keys

```
my_dict = {
    "name": "Ava",
    1: "one",
    (1, 2): "tuple key"
}
```

Invalid key

```
## TypeError: unhashable type: 'list'
my_dict = {[1, 2]: "bad"}
```

Tuple caveat

```
valid = {(1, 2): "ok"}

## invalid because tuple contains mutable list
## invalid = {[1, 2], 3): "bad"}
```

10.5 Class variables vs instance variables

```
class User:
    role = "member"          # class variable

    def __init__(self, name):
        self.name = name     # instance variable
```

Behavior

- Class variable shared across class unless shadowed by instance.
- Instance variable belongs to a specific object.

```
u1 = User("A")
u2 = User("B")

u1.role = "admin" # creates instance attribute shadowing class variable

print(u1.role) # admin
print(u2.role) # member
print(User.role) # member
```

10.6 Instance method, class method, static method

```
class Example:
    value = 10

    def instance_method(self):
        return self.value

    @classmethod
    def class_method(cls):
        return cls.value

    @staticmethod
    def static_method(x, y):
        return x + y
```

Use cases

- Instance method: needs object state.
 - Class method: needs class state or alternate constructor.
 - Static method: utility function logically belongs to class.
-

10.7 Decorators and custom decorators

Simple decorator

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_call
def greet(name):
    return f"Hello {name}"
```

Flask/FastAPI-like use case

- Authentication.
 - Authorization.
 - Logging.
 - Timing.
 - Validation.
-

10.8 *args and **kwargs

```
def demo(required, *args, **kwargs):
    print(required)
    print(args)
    print(kwargs)

demo(1, 2, 3, name="Ava")
```

Output:

```
1
(2, 3)
{'name': 'Ava'}
```

10.9 == vs is

==

Checks value equality.

is

Checks object identity.

```
a = [1, 2]
b = [1, 2]

print(a == b) # True
print(a is b) # False
```

Use is for None:

```
if value is None:
    pass
```

10.10 Exceptions, multiple except blocks, and finally

```
try:
    result = 10 / x
except ZeroDivisionError:
    print("division by zero")
except TypeError:
    print("wrong type")
finally:
    print("always runs")
```

Important behavior

finally runs even if return occurs in try.

```
def test():
    try:
        return "try"
    finally:
        print("finally still runs")
```

10.11 Mutable default argument bug

Bad

```
def add_item(item, items=[]):
    items.append(item)
    return items
```

Good

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

Why

Default arguments are evaluated once when the function is defined.

10.12 Lists, sets, and dicts for membership/deduplication

List

- Ordered.
- Allows duplicates.
- Membership is $O(n)$.

Set

- Unique values.
- Average $O(1)$ membership.
- Best for deduplication.

Dict

- Key-value mapping.
- Average $O(1)$ key lookup.
- Good for counts/frequencies.

```
seen = set()
unique = []

for item in items:
    if item not in seen:
        seen.add(item)
        unique.append(item)
```

11. Python Coding Problems and Algorithms

11.1 Longest substring without repeating characters

Sliding window solution

```
def longest_substring(s):
    seen = {}
    left = 0
    best_len = 0
    best_start = 0

    for right, char in enumerate(s):
        if char in seen and seen[char] >= left:
            left = seen[char] + 1

        seen[char] = right

        if right - left + 1 > best_len:
            best_len = right - left + 1
            best_start = left

    return s[best_start:best_start + best_len]

print(longest_substring("abcabcbb")) # abc
```

Complexity

- Time: $O(n)$
- Space: $O(k)$ where k is unique chars.

11.2 Minimum window substring without Counter

Problem

Find the smallest substring in s containing all characters of t , including duplicates.

```
def min_window_substring(s: str, t: str) -> str:
    if not s or not t:
        return ""

    target = {}
    for char in t:
        target[char] = target.get(char, 0) + 1

    window = {}
    required = len(target)
    formed = 0
    left = 0
    min_len = float("inf")
    result = ""

    for right in range(len(s)):
        char = s[right]
        window[char] = window.get(char, 0) + 1

        if char in target and window[char] == target[char]:
            formed += 1

        while formed == required:
            if right - left + 1 < min_len:
                min_len = right - left + 1
                result = s[left:right + 1]

            left_char = s[left]
            window[left_char] -= 1

            if left_char in target and window[left_char] < target[left_char]:
                formed -= 1

            left += 1

    return result

print(min_window_substring("ADOBECODEBANC", "ABC")) # BANC
```

11.3 Kth largest element using Quickselect

Idea

Convert kth largest to (n-k)th smallest.

```
import random

def find_kth_largest(nums, k):
    target = len(nums) - k

    def quickselect(left, right):
        pivot_index = random.randint(left, right)
        nums[pivot_index], nums[right] = nums[right], nums[pivot_index]
        pivot = nums[right]

        p = left
        for i in range(left, right):
            if nums[i] <= pivot:
                nums[p], nums[i] = nums[i], nums[p]
                p += 1

        nums[p], nums[right] = nums[right], nums[p]

        if p == target:
            return nums[p]
        if p < target:
            return quickselect(p + 1, right)
        return quickselect(left, p - 1)

    return quickselect(0, len(nums) - 1)

print(find_kth_largest([3, 2, 1, 5, 6, 4], 2)) # 5
```

Complexity

- Average: $O(n)$
 - Worst: $O(n^2)$
-

11.4 Reverse Polish Notation evaluation

```
def eval_rpn(tokens):
    stack = []

    for token in tokens:
        if token in "+-*/*":
            b = stack.pop()
            a = stack.pop()

            if token == "+":
                stack.append(a + b)
            elif token == "-":
                stack.append(a - b)
            elif token == "*":
                stack.append(a * b)
            else:
                stack.append(int(a / b)) # truncate toward zero
        else:
            stack.append(int(token))

    return stack[0]

print(eval_rpn("2 1 + 3 *".split())) # 9
```

Important division note

Use `int(a / b)` instead of `a // b` when truncation toward zero is required.

11.5 Combination calculation with memoization

Problem

Naive recursion recalculates same subproblems.

Improved solution

```
def compute_combinations(n, k, memo=None):
    if memo is None:
        memo = {}

    if k == 0 or k == n:
        return 1

    if (n, k) in memo:
        return memo[(n, k)]

    memo[(n, k)] = (
        compute_combinations(n - 1, k - 1, memo)
        + compute_combinations(n - 1, k, memo)
    )

    return memo[(n, k)]

print(compute_combinations(5, 3)) # 10
```

Complexity

- Naive: exponential.
 - Memoized: $O(n*k)$.
-

11.6 Even number function

```
def is_even(number):  
    return number % 2 == 0
```

11.7 Merge dictionaries

Original issue

Mutating dict1 may cause side effects.

Better solution

```
def merge_dictionaries(dict1, dict2):  
    return {**dict1, **dict2}
```

Or:

```
def merge_dictionaries(dict1, dict2):  
    merged = dict1.copy()  
    merged.update(dict2)  
    return merged
```

Overlapping keys from dict2 overwrite dict1.

11.8 Print queue / priority order without heapq

Problem summary

Jobs are printed by highest priority. If same priority, preserve original order.

```
def print_order(priorities):  
    jobs = [(priority, index) for index, priority in enumerate(priorities)]  
    result = []  
  
    while jobs:  
        max_priority = max(priority for priority, _ in jobs)  
  
        for _ in range(len(jobs)):  
            priority, index = jobs.pop(0)  
  
            if priority == max_priority:  
                result.append(index)  
                break  
            else:  
                jobs.append((priority, index))  
  
    return result  
  
print(print_order([3, 1, 4, 2])) # [2, 0, 3, 1]  
print(print_order([2, 2, 2]))   # [0, 1, 2]
```

Complexity

$O(n^2)$ because of repeated `max()` and queue rotations.

Python

Python is likely to come up because it is central to AI/ML, backend development, data processing, and automation.

Python GIL

Interview Question

What is the Python GIL?

Answer

The **Global Interpreter Lock (GIL)** is a lock in CPython that allows only one thread to execute Python bytecode at a time.

This means Python threads are not ideal for CPU-heavy parallel computation, because multiple threads cannot truly execute Python bytecode in parallel.

When It Matters

Task Type	Best Approach
CPU-bound tasks	Multiprocessing
I/O-bound tasks	Threading or AsyncIO
Network calls	AsyncIO
File/database calls	Threading or AsyncIO
Heavy ML computation	NumPy/PyTorch/TensorFlow native operations or multiprocessing

Sample Answer

The GIL prevents multiple native Python threads from executing Python bytecode simultaneously. For I/O-bound tasks, threading can still help because threads release the GIL while waiting for external operations. But for CPU-bound workloads, multiprocessing is usually better because each process has its own Python interpreter and memory space.

Threading vs Multiprocessing vs AsyncIO

Interview Question

When would you use threading, multiprocessing, or AsyncIO?

Comparison Table

Approach	Best For	Example
Threading	I/O-bound tasks	Reading files, calling APIs
Multiprocessing	CPU-bound tasks	Image processing, heavy computation
AsyncIO	High-concurrency I/O	Thousands of API calls, async web servers

Threading Example

```
import threading
import requests

def fetch_url(url: str) -> None:
    response = requests.get(url, timeout=10)
    print(url, response.status_code)

urls = [
    "https://example.com",
    "https://example.org",
]

threads = []

for url in urls:
    thread = threading.Thread(target=fetch_url, args=(url,))
    threads.append(thread)
    thread.start()

for thread in threads:
    thread.join()
```

Multiprocessing Example

```
from multiprocessing import Pool

def square(number: int) -> int:
    return number * number

if __name__ == "__main__":
    numbers = [1, 2, 3, 4, 5]

    with Pool(processes=4) as pool:
        results = pool.map(square, numbers)

    print(results)
```

AsyncIO Example

```
import asyncio
import aiohttp

async def fetch(session: aiohttp.ClientSession, url: str) -> str:
    async with session.get(url) as response:
        return await response.text()

async def main() -> None:
    urls = [
        "https://example.com",
        "https://example.org",
    ]

    async with aiohttp.ClientSession() as session:
        tasks = [fetch(session, url) for url in urls]
        results = await asyncio.gather(*tasks)

    print(len(results))

asyncio.run(main())
```

Decorators

Interview Question

What is a decorator in Python?

Answer

A decorator is a function that wraps another function to extend or modify its behavior without changing the original function code.

Common Uses

- Logging
- Authentication
- Authorization
- Caching
- Timing
- Retry logic
- Monitoring

Code Example

```
from functools import wraps
from time import perf_counter

def timing_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = perf_counter()
        result = func(*args, **kwargs)
        end = perf_counter()
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result
    return wrapper

@timing_decorator
def process_data():
    return sum(range(1_000_000))

print(process_data())
```

Interview Tip

Mention `functools.wraps` because it preserves the original function's metadata.

Generators

Interview Question

What is a generator in Python?

Answer

A generator is a function that uses `yield` to return values lazily one at a time instead of storing everything in memory.

Why Generators Are Useful

- Memory efficient
- Good for streaming data
- Useful for large files
- Helpful in data pipelines

Code Example

```
def read_large_file(file_path: str):
    with open(file_path, "r", encoding="utf-8") as file:
        for line in file:
            yield line.strip()

for row in read_large_file("data.txt"):
    print(row)
```

yield vs return

Keyword	Behavior
return	Ends the function and returns one value
yield	Pauses the function and resumes later

Try, Except, Finally

Interview Question

When does the finally block execute?

Answer

The finally block executes regardless of whether an exception occurs or not. It is commonly used for cleanup operations.

Code Example

```
def read_file(path: str) -> None:
    file = None

    try:
        file = open(path, "r", encoding="utf-8")
        print(file.read())
    except FileNotFoundError:
        print("File not found")
    finally:
        if file:
            file.close()
        print("Cleanup completed")
```

Important Point

Even if there is a return inside try or except, finally usually still executes before the function exits.

Python Memory and Performance

Interview Question

How would you improve Python performance?

Answer

Possible approaches:

- Use efficient data structures
- Avoid unnecessary loops
- Use generators for large data
- Use NumPy/Pandas vectorization
- Use multiprocessing for CPU-bound tasks
- Cache repeated computations
- Profile before optimizing

Example: Caching

```
from functools import lru_cache

@lru_cache(maxsize=128)
def fibonacci(n: int) -> int:
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(35))
```

4. Python Engineering

Likely Questions

- How strong is your Python experience?
 - What Python frameworks have you used?
 - How do you structure a Python backend project?
 - How do you handle errors in Python services?
 - How do you write maintainable Python code?
 - What are decorators?
 - What are generators?
 - What is the difference between list, tuple, set, and dictionary?
 - What is the difference between shallow copy and deep copy?
 - What is the Python GIL?
 - How do you manage dependencies in Python?
 - How do you handle environment variables and secrets?
-

Python Error Handling Example

```
def get_order(order_id: int) -> dict:
    try:
        order = order_repository.find_by_id(order_id)

        if order is None:
            raise ValueError(f"Order {order_id} not found")

        return order

    except ValueError:
        raise

    except Exception as exc:
        logger.exception("Unexpected error while fetching order", extra={"order_id":
order_id})
        raise RuntimeError("Unable to fetch order right now") from exc
```

Interview Explanation

- Use specific exceptions for known business cases.
 - Log unexpected errors with context.
 - Do not expose internal errors directly to users.
 - Preserve the original exception using `from exc`.
-

Decorator Example

```
import time
from functools import wraps

def measure_time(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        duration = time.time() - start
        print(f"{func.__name__} took {duration:.2f}s")
        return result
    return wrapper

@measure_time
def process_orders():
    # expensive operation
    return "done"
```

When to Use Decorators

Use decorators for cross-cutting concerns such as:

- Logging
- Timing
- Authentication
- Authorization
- Retry logic
- Caching

Generator Example

```
def stream_order_ids(orders):
    for order in orders:
        yield order["id"]

orders = [{"id": 1}, {"id": 2}, {"id": 3}]

for order_id in stream_order_ids(orders):
    print(order_id)
```

Interview Explanation

A generator is useful when:

- Data is large
- You do not want to load everything into memory
- You want lazy evaluation
- You are processing streams or batches

Python GIL

Question

What is the Global Interpreter Lock?

Answer

The GIL is a lock in CPython that allows only one thread to execute Python bytecode at a time. This means CPU-bound Python code does not get true parallel execution with threads.

Practical Impact

- Use **threading** for I/O-bound work.
- Use **multiprocessing** for CPU-bound work.
- Use **asyncio** for high-concurrency I/O workloads.

Threading vs Multiprocessing vs AsyncIO

Approach	Best For	Example
Threading	I/O-bound tasks	Calling multiple APIs
Multiprocessing	CPU-bound tasks	Image processing, heavy computation
AsyncIO	High-concurrency I/O	WebSocket server, async API calls

```
import asyncio
import httpx

async def fetch_url(url: str):
    async with httpx.AsyncClient() as client:
        response = await client.get(url)
        return response.status_code

async def main():
    results = await asyncio.gather(
        fetch_url("https://example.com"),
        fetch_url("https://example.org"),
    )
    print(results)

asyncio.run(main())
```

Fan-Out / Fan-In I/O Aggregation

A common backend/dashboard problem is:

```
request arrives

start several independent I/O operations

wait until the required operations finish

combine results

build one response/dashboard
```

If seven independent calls each take roughly one second, running them sequentially can take roughly seven seconds. Concurrent execution is closer to the duration of the slowest required call, plus aggregation overhead.

Async-compatible I/O: `asyncio.gather()`

```
import asyncio

async def fetch_source(name: str) -> dict[str, str]:
    await asyncio.sleep(1) # stand-in for async network/database I/O
    return {name: f"data_from_{name}"}

async def build_dashboard() -> dict[str, str]:
    tasks = [fetch_source(f"source_{i}") for i in range(1, 8)]
    results = await asyncio.gather(*tasks)

    combined: dict[str, str] = {}
    for result in results:
        combined.update(result)
    return combined
```

Use this when the underlying clients are async-compatible. `asyncio.gather()` is a fan-out/fan-in primitive: schedule independent work, await the group, then aggregate.

If partial results are acceptable, failure policy must be explicit rather than accidental:

```
results = await asyncio.gather(*tasks, return_exceptions=True)
successful = [result for result in results if not isinstance(result, Exception)]
```

For stricter structured concurrency, modern Python also provides `asyncio.TaskGroup`; failure of one child task can cancel sibling tasks according to the task-group semantics.

Blocking I/O: `ThreadPoolExecutor`

When the libraries are synchronous/blocking, a thread pool is often the cleanest adapter:

```
from concurrent.futures import ThreadPoolExecutor, as_completed

def fetch_source(name: str) -> dict[str, str]:
    # stand-in for a blocking API/DB/file call
    return {name: f"data_from_{name}"}

sources = [f"source_{i}" for i in range(1, 8)]
results: list[dict[str, str]] = []

with ThreadPoolExecutor(max_workers=7) as executor:
    futures = [executor.submit(fetch_source, source) for source in sources]
    for future in as_completed(futures):
        results.append(future.result())
```

Interview explanation:

If the operations are independent and I/O-bound, I would fan them out concurrently, wait for the required results, then fan them back in through an aggregation step. I would use `asyncio` when the libraries are async-native and a bounded thread pool when they are blocking. For production I would also define timeouts, partial-failure behavior, concurrency limits, and downstream rate-limit protection.

Important nuance: threads/async tasks provide **concurrency** for I/O. Calling this "parallel" is common conversationally, but CPU-bound Python parallelism generally requires processes or native code that releases the GIL.

Focused Python Semantics and Debugging Drills

These short questions test whether a candidate can reason about Python execution rather than only recall syntax.

Iterator Consumption

An iterator keeps traversal state. Calling `next()` consumes one item, so a later loop starts from the following item.

```
values = iter([10, 20, 30])

print(next(values))
for value in values:
    print(value)
```

Output:

```
10
20
30
```

The loop does not restart the iterator.

Generator Exhaustion

A generator expression is also a one-pass iterator.

```
gen = (number * number for number in range(3))

print(list(gen))
print(list(gen))
```

Output:

```
[0, 1, 4]
[]
```

Create a new generator when the sequence must be traversed again, or materialize it once into a list when the data is small enough.

Missing Dictionary Keys

Direct indexing raises `KeyError` when the key is absent:

```
role = user_roles[username]
```

Choose behavior explicitly:

```
role = user_roles.get(username) # returns None when missing
role = user_roles.get(username, "guest")

if username in user_roles:
    role = user_roles[username]
```

Use indexing when absence is an error. Use `.get()` when a default or optional result is part of the contract.

Stacked Decorator Order

Decorators are applied from the bottom upward but execute from the outer wrapper inward.

```
def first(func):
    def wrapper():
        print("first")
        func()
    return wrapper

def second(func):
    def wrapper():
        print("second")
        func()
    return wrapper
```

```
@first
```

```
@second
def greet():
    print("hello")
```

The definition is equivalent to:

```
greet = first(second(greet))
```

Calling `greet()` prints:

```
first
second
hello
```

Preserving Decorated Function Metadata

Without `functools.wraps`, the decorated function exposes the wrapper's name and documentation.

```
from functools import wraps
```

```
def log_call(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper
```

`@wraps(func)` preserves metadata such as `__name__`, `__doc__`, annotations, and the `__wrapped__` reference used by inspection tools.

Context Manager Lifecycle

A `with` statement calls `__enter__` before the block and `__exit__` when the block finishes.

```
class Resource:
    def __enter__(self):
        print("Open")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print("Close")

with Resource():
    print("Working")
```

Output:

```
Open
Working
Close
```

`__exit__` also runs when the block raises an exception. Returning a truthy value from `__exit__` suppresses that exception; returning `False` or `None` allows it to propagate.

Frozen Dataclasses

A frozen dataclass prevents normal field reassignment after construction.

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
class User:
    name: str
```

```
user = User("Maya")
user.name = "Brian" # dataclasses.FrozenInstanceError
```

Use frozen dataclasses for value objects whose identity should not change after creation.

Metaclass Registration Timing

A metaclass can register subclasses when their class bodies are executed.

```
class Registry(type):
    registered: list[str] = []

    def __new__(mcls, name, bases, namespace):
        cls = super().__new__(mcls, name, bases, namespace)
        if name != "Plugin":
            mcls.registered.append(name)
        return cls

class Plugin(metaclass=Registry):
    pass

class Logger(Plugin):
    pass

class Cache(Plugin):
    pass
```

`Registry.registered` becomes `['Logger', 'Cache']` at class-definition time. No instances are required. This pattern appears in plugin systems, serializers, command registries, and framework extension points.

Shared Mutable Class Attributes

A mutable class attribute is shared by every instance:

```
class User:
    active = []

    def __init__(self, name):
        self.active.append(name)
```

After creating `User("Maya")` and `User("Ethan")`, both instances observe:

```
['Maya', 'Ethan']
```

Use an instance attribute for per-object state:

```
class User:
    def __init__(self, name):
        self.active = [name]
```

Type Hints Do Not Enforce Runtime Types

Python annotations support readers, IDEs, linters, and static type checkers, but Python does not reject mismatched arguments automatically.

```
def add_tax(amount: int) -> int:
    return amount + 5
```

```
print(add_tax(True)) # 6
```

`bool` is a subclass of `int`, so `True` behaves numerically like 1. Use runtime validation when an API contract must reject such values.

Tuple Unpacking Arity

The number of targets must match the number of returned values unless starred unpacking is used.

```
def profile():
    return "Alice", 25, "Chicago"
```

```
name, age = profile() # ValueError: too many values to unpack
```

Valid alternatives:

```
name, age, city = profile()
name, *details = profile()
```

Restricting Attributes with `__slots__`

`__slots__` declares the allowed instance attributes and can reduce per-object memory overhead when many objects are created.

```
class User:
    __slots__ = ("name",)

    def __init__(self, name):
        self.name = name
```

```
user = User("Maya")
user.email = "maya@example.com" # AttributeError
```

Add email to `__slots__` or remove slots when dynamic attributes are required.

Property-Based Validation

A property keeps attribute-style access while centralizing validation.

```
class Account:
    def __init__(self, balance: float):
        self._balance = balance

    @property
    def balance(self) -> float:
        return self._balance

    @balance.setter
    def balance(self, value: float) -> None:
        if value < 0:
            raise ValueError("Balance cannot be negative")
        self._balance = value
```

This is useful when an object's invariants must be enforced without exposing internal storage directly.

Quick Interview Checklist

- State whether an object is reusable or one-shot.
- Distinguish class-level state from instance-level state.
- Explain when framework hooks run: definition time, construction time, or call time.
- Mention the exact exception when the question asks for output or failure behavior.
- Separate static type checking from runtime validation.

Frequency Aggregation, Top-K Selection & Streaming Follow-Ups

A common interview progression is to start with a frequency map, then ask whether sorting every unique value is still appropriate when only a very small top-k result is required.

Baseline: Count Then Sort

For n total events and u unique values:

```
def top_k_by_sort(values, k):
```

```

counts = {}
for value in values:
    counts[value] = counts.get(value, 0) + 1

ranked = sorted(counts, key=lambda value: (-counts[value], value))
return ranked[:k]

```

Complexity:

- Counting: $O(n)$.
- Sorting unique values: $O(u \log u)$.
- Total: $O(n + u \log u)$.
- Frequency storage: $O(u)$.

The important interview distinction is that n , u , and k can be very different orders of magnitude. If millions of events collapse to a few unique values, sorting is cheap. If u is huge and k is tiny, sorting every unique value does unnecessary work.

Small k : Min-Heap of Size k

After counting, keep only the best k candidates in a min-heap. The root represents the weakest retained candidate.

```

count all events       $O(n)$ 
scan  $u$  unique values  $O(u \log k)$ 
final ordering         $O(k \log k)$ 

```

Overall:

$O(n + u \log k)$

This is better than $O(n + u \log u)$ when $k \ll u$.

Why heapq?

A min-heap of size k keeps the weakest current top- k candidate at the root. For each new unique value, I either ignore it or replace that root in $O(\log k)$. That avoids sorting all u unique values when I only need a small prefix.

Special Case: $k = 1$

A heap is unnecessary. After counting, scan the frequency map once and keep the current maximum.

```

 $O(n + u)$  time
 $O(u)$  frequency storage

```

This is a useful follow-up because it shows that the data structure should match the requested result size rather than being applied mechanically.

Dynamic k

A fixed size- k heap discards candidates. If a later query asks for a larger k , those discarded values are no longer available in the heap.

Choose based on the query pattern:

Situation	Better Approach
One query with small fixed k	Frequency map + size- k min-heap
Many queries over the same static data with changing k	Count once, sort all unique values once, return cached prefixes

Situation	Better Approach
Stream of updates with occasional arbitrary k queries	Maintain running counts; build size-k heap at query time
Stream with very frequent top-k reads	Maintain counts plus an ordered ranking index
Unique cardinality is too large for exact in-memory counts	Approximate heavy hitters such as Count-Min Sketch

Streaming Variant

For an unbounded stream, the original input list is unavailable. The minimum exact state is a running frequency map:

```
counts[path] = counts.get(path, 0) + 1
```

At any point, an arbitrary top_k(k) query can scan the current u counts and keep a size-k heap. That gives:

- Update: $O(1)$ average per event.
- Query: $O(u \log k)$.
- Exact state: $O(u)$.

If top-k queries are very frequent, maintain a secondary ordered index keyed by frequency and deterministic tie-break. That makes updates more expensive, typically $O(\log u)$, but can make top-k reads close to $O(k)$.

For distributed streams:

- Partition by a stable hash of the key/path.
- Maintain local counts/state.
- Checkpoint consumer offsets and state.
- Make replay idempotent.
- Define window semantics explicitly: all-time, tumbling, sliding, or session window.
- Merge partition-level candidates or counts at the aggregation layer.

If exact $O(u)$ state is not feasible, use an approximate heavy-hitter algorithm and state the accuracy/memory trade-off explicitly.

Interview answer:

I would separate total events n , unique values u , and requested results k . The simple solution is a frequency map plus sorting, which is $O(n + u \log u)$. If u is large and k is small, I would keep a min-heap of size k , reducing selection to $O(u \log k)$. If k changes dynamically, a fixed heap is not enough because it discarded candidates, so for static data I would sort once and reuse the ranking, while for a stream I would keep running counts and compute or maintain the ranking based on query frequency.

See `coding_questions/top_k_frequent_paths.py` for runnable batch, heap, dynamic-k, and streaming examples.

Loop Progress Invariants in Grouped Simulations

Nested loops over sorted groups are easy to make accidentally non-terminating. This came up in a priority-allocation exercise where records were grouped by bid and processed in timestamp order.

The Core Invariant

Every loop must make measurable progress toward its exit condition.

Bad pattern:

```
group_end = index

while group_end < index and rows[group_end].bid == bid:
    ...
```

`group_end < index` is false immediately because both variables start equal. A different typo can be worse: the loop condition may remain true while the variable used in that condition never changes, causing a timeout.

Safer grouped scan:

```
group_end = group_start

while group_end < len(rows) and rows[group_end].bid == bid:
    group_end += 1

# Outer pointer advances exactly once after the group is complete.
group_start = group_end
```

Keep responsibilities separate:

- `group_start` identifies the beginning of the current group.
- `group_end` discovers the first record after that group.
- The inner loop advances `group_end` only.
- The outer loop advances `group_start` only after the group decision is complete.
- Prefer descriptive state names such as `group_start`, `group_end`, and `remaining_inventory` when several indices or counters coexist; short names are fine only when the scope and role are obvious.

Optimize for the Required Output

If the question asks only which customers received **zero** items, a literal one-item-per-round simulation may be unnecessary. After sorting by priority, group-level arithmetic can often determine who receives at least one item without iterating once per allocated item.

That changes a potentially inventory-dependent simulation into a sort-and-scan solution:

```
O(total_inventory) style simulation
O(n log n) sort + O(n) group scan
```

Interview answer:

When I use multiple pointers, I define the progress invariant explicitly before coding: which pointer each loop owns, what must change on every iteration, and what state transition advances the outer loop. For large-quantity simulations I also ask whether the requested output can be derived at group level so runtime depends on the number of records rather than the number of individual units processed.

See `coding_questions/inventory_bid_allocation.py` for the optimized allocation exercise.

Senior Python Concurrency & Thread-Safety Interview Guide

This section focuses on the follow-up style common in senior coding rounds: implement a correct stateful data structure, then make it thread-safe, discuss contention, and explain how the design changes when moved across processes or machines.

Concurrency vs Parallelism

- **Concurrency:** multiple tasks make progress during overlapping time periods.
- **Parallelism:** multiple tasks literally execute at the same time on different CPU cores/processors.

A Python program can be highly concurrent even when CPython's GIL limits parallel execution of Python bytecode in one process.

What Is the GIL?

CPython's Global Interpreter Lock allows only one thread at a time to execute Python bytecode within a process.

Important consequences:

- CPU-bound pure-Python threads usually do not scale linearly across cores.
- Threads are still useful for I/O-bound work because waiting threads can release control while another thread runs.
- Many blocking I/O operations and C extensions release the GIL.
- The GIL does **not** make arbitrary shared state logically thread-safe.

Strong interview answer:

The GIL limits simultaneous execution of Python bytecode in CPython, but it does not protect multi-step application invariants. A read-modify-write sequence or a dictionary plus linked-list invariant can still be interleaved across threads, so I still need synchronization around shared mutable state.

Threading vs Multiprocessing vs AsyncIO

Model	Best Fit	Key Trade-Off
Threads	Blocking/I/O-bound work, legacy sync libraries	Shared-memory synchronization and GIL for CPU-heavy Python code
Processes	CPU-bound Python work requiring true parallelism	Serialization, IPC, higher memory/process overhead
asyncio	Many concurrent I/O operations with async-compatible libraries	Cooperative model; blocking calls must not run on event loop

Interview line:

I choose the concurrency model from the workload. Threads and asyncio are strong for I/O concurrency, while processes are more appropriate when CPU-bound Python work needs parallel execution across cores.

Race Condition

A race condition occurs when correctness depends on unpredictable operation interleaving.

Conceptually, this is not one indivisible operation:

```
counter += 1

read counter
add 1
write counter
```

Possible interleaving:


```
Thread A reads 5
Thread B reads 5
Thread A writes 6
Thread B writes 6
```

Expected: 7. Actual: 6.

Protect the shared invariant using appropriate synchronization.

threading.Lock

A mutex permits one thread at a time inside the protected critical section.

```
import threading

class Counter:
    def __init__(self):
        self._value = 0
        self._lock = threading.Lock()

    def increment(self):
        with self._lock:
            self._value += 1

    def value(self):
        with self._lock:
            return self._value
```

Start with correctness. Keep the critical section as small as practical, but do not split one logical invariant across multiple unsynchronized operations.

RLock

RLock is reentrant: the same thread can acquire it multiple times before releasing it the same number of times.

Use it when reentrant acquisition is actually part of the design, for example:

```
public synchronized method
    calls another synchronized method
    same thread needs same lock again
```

Strong interview answer:

I prefer a normal Lock unless reentrancy is required. RLock can be useful when synchronized methods call each other, but using it everywhere can hide accidental recursive lock acquisition.

Semaphore

A semaphore permits up to N concurrent holders rather than exactly one.

Examples:

```
allow at most 10 concurrent DB-heavy tasks
allow at most 20 outbound API calls
limit parallel downloads

import asyncio

semaphore = asyncio.Semaphore(10)

async def call_service(client, url):
    async with semaphore:
        return await client.get(url)
```

A semaphore controls concurrency; it does not by itself protect a complex shared-state invariant.

Condition Variable

A condition variable lets threads wait for a state predicate while temporarily releasing a lock.

Typical bounded queue predicates:

```
producer waits while queue is full
consumer waits while queue is empty
```

Always re-check the predicate in a while loop:

```
with condition:
    while not predicate():
        condition.wait()
```

Why while, not if?

- Condition waits can wake spuriously.
 - Another thread can change the state before this thread reacquires the lock.
-

Thread-Safe Queue

Python's `queue.Queue` is the default production answer when the problem simply needs a synchronized producer/consumer queue.

If the interview asks you to implement one, use:

- deque for $O(1)$ append/popleft
- one lock
- `not_empty` condition
- `not_full` condition for bounded capacity

See `coding_questions/concurrent_data_structures.py` for a runnable implementation.

Deadlock

Classic example:

```
Thread 1: acquire A -> waits for B
Thread 2: acquire B -> waits for A
```

Both wait forever.

Practical defenses:

- Consistent global lock ordering
- Avoid unnecessary nested locks
- Keep lock scope small
- Use timeouts where appropriate
- Avoid calling unknown/external code while holding a lock

Interview explanation:

If multiple locks are necessary, I define one acquisition order and enforce it everywhere. That removes the circular-wait pattern that commonly causes deadlocks.

Coarse-Grained vs Fine-Grained Locking

Coarse-grained

One lock protects the complete logical state.

Advantages:

- Simple
- Easier to prove correct
- Lower deadlock risk

Disadvantage:

- More contention

Fine-grained

Different locks protect independent state partitions.

Advantages:

- More concurrency

Disadvantages:

- More complex invariants
- Lock-ordering concerns
- Harder multi-key operations

Senior interview pattern:

I would first protect the whole invariant with one lock. If profiling shows lock contention is material, I would consider partitioning/sharding the state or reducing critical-section work rather than prematurely introducing fine-grained locking.

Lock Sharding

For independent key-based operations:

```
lock = locks[hash(key) % len(locks)]
```

This lets operations on different shards proceed concurrently.

Trade-offs:

- Collisions still serialize unrelated keys.
- Multi-key operations become more complex.
- Global ordering/eviction policies may still need centralized synchronization.

Why LRU get () Is Not Really Read-Only

An LRU cache maintains recency ordering.

```
get(key)
```

```
lookup value
```

```
move node to most-recent position
```

So `get()` mutates shared state and must participate in the synchronization strategy.

The LRU invariant spans both:

```
hash map
+
doubly linked list ordering
```

A strong first implementation protects both under one lock.

TTL / Expiring Map Design

A map may store:

```
key -> (value, expiry)
```

and an ordered expiration structure.

Fixed TTL with insertion-ordered expiry

A deque can work because expiration order matches insertion order.

Arbitrary TTL per key

Use a min-heap:

```
(expiration_time, key)
```

Heap operations cost $O(\log n)$.

Stale expiration metadata

If key A is overwritten:

```
put A with expiry 10
put A with expiry 20
```

the old expiry entry must not delete the new value. Compare the queued expiry/version to the map's current expiry/version before deleting.

This is a useful general pattern: secondary index entries can become stale after an overwrite, so validate them against authoritative current state before acting.

Bounded Blocking Queue

Requirements:

```
put(item) waits when full
take() waits when empty
fixed capacity
```

Core structure:

```
deque
lock
not_empty condition
not_full condition
```

Producer:

```
while full:
    wait on not_full
append item
notify not_empty
```

Consumer:

```
while empty:
    wait on not_empty
popleft item
notify not_full
```

This also demonstrates **backpressure**: producers cannot create unbounded queued work when consumers are slower.

Rate Limiter Evolution

Fixed window

Simple per-key counter keyed by time bucket.

Problem: boundary bursts.

```
100 requests at 12:00:59
100 requests at 12:01:00
```

The client can generate 200 requests in roughly one second despite a nominal 100/minute limit.

Sliding-window log

Store recent timestamps, often in a deque/sorted structure.

- More accurate
- More state per key

Token bucket

Maintain:

```
bucket capacity = allowed burst
refill rate      = sustained rate
```

Token bucket is a strong practical design when controlled bursts are acceptable.

Concurrent limiter

A single lock is the simplest correctness baseline. If contention is high, shard locks/state by key.

Distributed limiter

Local process counters cannot enforce one global limit across many API servers.

Options:

- Central/partitioned atomic counter store
- Redis atomic script/operation
- Allocate local token budgets from a global quota

Trade-off:

Centralized exact state adds a network dependency; local quotas scale better but can become approximate.

Task Scheduler

For:

```
schedule(task, execution_time)
get next due task
```

use a min-heap ordered by execution time.

```
schedule:  $O(\log n)$   
peek next due:  $O(1)$   
pop:  $O(\log n)$ 
```

Concurrency follow-ups:

- A producer inserts an earlier task while a worker sleeps.
- Worker must be notified to recompute its wait duration.
- Duplicate execution after crash/retry.
- Durable scheduling across process restart.

Distributed extension:

```
durable task table  
  
conditional lease claim  
  
worker  
  
heartbeat / completion
```

Optimistic vs Pessimistic Concurrency

Optimistic

Assume conflicts are uncommon. Detect them at write time.

Example:

```
UPDATE job  
SET state = 'RUNNING',  
    version = version + 1  
WHERE id = :id  
    AND state = 'QUEUED'  
    AND version = :expected_version;
```

0 rows affected means another actor changed the state first.

Good for:

- Short transactions
- Low/moderate conflict
- High read concurrency

Pessimistic

Lock before making the change.

Good when:

- Conflict is frequent
- Exclusive ownership is necessary
- The critical operation is short

Trade-offs:

- Blocking
 - Deadlocks
 - Reduced concurrency
-

Testing Concurrent Code

Useful strategies:

- Many threads writing the same key/counter.
- Readers and writers operating simultaneously.
- Independent keys to verify concurrency does not corrupt shared metadata.
- Barriers/events to intentionally align threads around a suspected race.
- Repeated stress runs rather than one execution.
- Assert invariants after all workers finish.
- Validate capacity never exceeds its limit.
- Validate every produced queue item is consumed exactly as the test contract expects.
- Use race/thread-sanitizer tooling where the language/runtime supports it.

Do not rely only on `sleep()` to "make the race happen"; that creates fragile tests.

Quick Concurrency Interview Questions

Does the GIL make Python data structures thread-safe?

No. Some individual CPython operations may appear atomic today, but application-level multi-step invariants still require synchronization. Do not build correctness on implementation accidents.

Why use deque instead of list for FIFO?

`deque.popleft()` is $O(1)$; `list.pop(0)` is $O(n)$ because elements shift.

Why not use one lock per key everywhere?

Lock lifecycle, eviction, multi-key operations, and global invariants become more complicated. Start coarse, measure contention, then partition when justified.

Could LRU `get()` be lock-free?

Not in the straightforward design because it mutates recency ordering. A specialized approximate/concurrent cache could relax this, but that changes semantics and complexity.

What is starvation?

A thread/task waits indefinitely because other contenders repeatedly acquire the required resource even though the system as a whole continues making progress.

What is contention?

Multiple workers frequently compete for the same synchronization/resource, reducing parallel progress and increasing latency.

Senior Concurrency Revision Card

Race condition
Critical section
Lock / mutex
RLock
Semaphore
Condition variable
Thread-safe Queue
Deadlock + lock ordering
Starvation
Contention
Coarse vs fine-grained locking
Lock sharding
GIL limits vs logical thread safety
Threads vs processes vs asyncio
Fan-out/fan-in: gather async I/O, thread pool for blocking I/O
Timeouts + partial-failure policy for aggregated calls
Optimistic vs pessimistic concurrency
Leases/heartbeats for distributed ownership

Runnable examples are available in `coding_questions/concurrent_data_structures.py`.

REST API & Backend APIs

Source: docs/rest_api.md

Purpose: REST, FastAPI, Flask, backend API design, security, validation, idempotency, gRPC, GraphQL, and API scalability. **Use this file for:** backend API interviews and Python web-service roles

Recommended Study Flow

1. Read the **Quick Summary** first.
2. Review the **Key Concepts** and tables.
3. Practice the **Interview Questions & Answers** out loud.
4. Use the code snippets and examples to explain trade-offs clearly.
5. Finish with the **Common Mistakes** and **Revision Checklist** sections.

Quick Summary

This file has been refreshed to keep the original repository topic while merging relevant detailed Q&A from the consolidated topic-wise interview-prep pack. Use the top sections for fast revision and the consolidated section for deeper interview preparation.

Core Topics to Master

- REST fundamentals and HTTP methods
- FastAPI and Flask API implementation
- Request validation with Pydantic
- Status codes and error handling
- Authentication and authorization
- Pagination, filtering, sorting, versioning
- Idempotency keys for safe retries
- gRPC and GraphQL comparisons

Consolidated Interview Questions & Technical Notes

Backend API design, FastAPI, Flask, REST, gRPC, GraphQL, request validation, API security, pagination, scaling APIs, and clean architecture.

5. FastAPI & Backend API Design

5.1 What is FastAPI?

Interview answer:

FastAPI is a modern high-performance Python framework for building REST APIs. It is built on Starlette for ASGI/async request handling and Pydantic for validation. It uses Python type hints to provide automatic request validation, serialization, and OpenAPI/Swagger documentation.

5.2 Why FastAPI?

- Native async support
- High performance with ASGI
- Pydantic validation
- Automatic OpenAPI docs
- Dependency injection
- Easy testing
- Strong fit for AI APIs and microservices

5.3 FastAPI vs Flask

Flask	FastAPI
WSGI	ASGI
Mostly sync	Native async
Manual validation	Pydantic validation
Swagger via extensions	Built-in OpenAPI
Lightweight	Better for modern APIs

5.4 Request flow

Client

Load Balancer / API Gateway

FastAPI

Middleware

Pydantic Validation

Business Logic

Database / Vector DB / LLM API

JSON Response

5.5 Pydantic Example

```
from pydantic import BaseModel
```

```
class UserRequest(BaseModel):  
    name: str  
    age: int
```

If age is not an integer, FastAPI automatically returns a validation error.

5.6 Dependency Injection

Use for:

- Database sessions
- Authentication
- Configuration
- Logging
- Reusable services

```
from fastapi import Depends, FastAPI

app = FastAPI()

def get_db():
    db = "database_session"
    return db

@app.get("/items")
async def get_items(db=Depends(get_db)):
    return {"db": db}
```

5.7 FastAPI for AI Applications

Answer:

FastAPI is a strong fit for AI applications because LLM calls, vector database queries, and external API calls are I/O-bound. Async endpoints allow the service to handle many concurrent requests efficiently. FastAPI also integrates naturally with Python AI libraries, Pydantic validation, Docker, Kubernetes, and CI/CD pipelines.

13. API Integration Topics

13.1 REST vs GraphQL

REST	GraphQL
Multiple endpoints	Single endpoint
Server defines response shape	Client requests exact fields
Simple and widely used	Flexible for complex data
Can over-fetch/under-fetch	Reduces over-fetching

13.2 How to Secure APIs

Mention:

- HTTPS
- JWT / OAuth
- API keys
- Rate limiting
- RBAC
- Input validation
- Request logging
- Secrets management
- CORS configuration
- Error handling without leaking details

13.3 API Integration in AI Agents

An AI agent may call APIs to:

- Fetch customer details
- Retrieve order status
- Create support tickets
- Update records
- Trigger workflows
- Search databases
- Retrieve documents

13.4 Strong Interview Line

"When integrating APIs into an AI agent, I focus on authentication, permissions, input validation, error handling, timeout handling, retries, logging, and making sure the agent only has access to tools it is allowed to use."

14. FastAPI Topics

14.1 What Is FastAPI?

"FastAPI is a modern Python web framework for building APIs. It supports async endpoints, automatic documentation, Pydantic validation, dependency injection, and high performance."

14.2 Dependency Injection in FastAPI

"Dependency injection in FastAPI is used to provide reusable components such as database sessions, authentication, configuration, or service classes to endpoints using Depends."

Example

```
from fastapi import Depends, FastAPI

app = FastAPI()

async def get_current_user():
    return {"username": "alex"}

@app.get("/profile")
async def get_profile(user=Depends(get_current_user)):
    return {"user": user}
```

14.3 Benefits of Dependency Injection

- Loose coupling
- Reusable dependencies
- Cleaner code
- Better testing
- Centralized resource management
- Easier mocking
- Better separation of concerns

14.4 Async Endpoints in FastAPI

"Async endpoints help improve concurrency when the application is waiting on I/O operations like database queries, external APIs, vector database retrievals, or LLM calls."

Example

```
from fastapi import FastAPI

app = FastAPI()

async def call_llm_api(query: str) -> str:
    # Simulated async external API call
    return f"Answer for: {query}"

@app.get("/agent-response")
async def agent_response(query: str):
    result = await call_llm_api(query)
    return {"answer": result}
```

14.5 When to Use Async

Use async for:

- External API calls
- Database calls with async drivers
- LLM provider calls
- Vector DB retrieval
- File/network I/O
- Streaming responses

Do not expect async to improve CPU-heavy tasks by itself.

14.6 FastAPI Troubleshooting

Step-by-Step Approach

1. Check logs and traceback
2. Validate request input
3. Check Pydantic model
4. Test with Swagger UI, Postman, or curl
5. Check dependencies
6. Check authentication/authorization
7. Check async issues such as missing `await`
8. Check database connection/session
9. Check external API failures
10. Check response model mismatch

Common FastAPI Issues

- 422 validation errors
- Missing required fields
- Incorrect data types
- Dependency failure
- Auth failure
- Missing `await`
- Blocking call inside async route
- DB session issue
- External API timeout
- Response model mismatch

Strong Interview Line

"Most FastAPI endpoint issues come from validation errors, dependency failures, missing awaits, database/session problems, external API failures, or response model mismatches."

19. Scaling FastAPI Applications

19.1 Scaling FastAPI from 5 Pods to 100 Pods

"To scale a FastAPI application from 5 pods to 100 pods, I would use Kubernetes Horizontal Pod Autoscaler. First, I would make sure the application is stateless, containerized properly, and has health checks. Then Kubernetes can scale pods based on CPU, memory, or custom request metrics."

19.2 Key Steps

- Make application stateless
- Containerize with Docker
- Deploy to Kubernetes
- Configure HPA
- Add readiness and liveness probes
- Use Kubernetes Service / Load Balancer
- Move sessions to Redis
- Use managed database
- Add connection pooling
- Monitor CPU, memory, latency, errors
- Watch external API rate limits
- Monitor LLM latency and token cost

19.3 Example Kubernetes HPA

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: fastapi-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: fastapi-app
  minReplicas: 5
  maxReplicas: 100
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

19.4 Readiness and Liveness Probe Example

```
readinessProbe:
  httpGet:
    path: /health/ready
    port: 8000
  initialDelaySeconds: 10
  periodSeconds: 5

livenessProbe:
  httpGet:
    path: /health/live
    port: 8000
  initialDelaySeconds: 30
  periodSeconds: 10
```

19.5 Strong Interview Line

"Scaling pods is easy; the real challenge is making sure the database, cache, external APIs, vector database, and LLM provider can handle the increased traffic too."

5. Backend API Development

Topics to revise

- API design principles.
- Request/response lifecycle.
- HTTP methods: GET, POST, PUT, PATCH, DELETE.
- Status codes.
- Validation.
- Authentication and authorization.
- Pagination.
- Rate limiting.
- Idempotency.
- Error handling.
- Versioning.
- API documentation.

Common interview questions

1. How do you design a reliable backend API?
2. What is the difference between PUT and PATCH?
3. How do you handle API errors?
4. How do you secure an API?
5. How do you validate incoming data?
6. How do you design APIs for high traffic?
7. How do you handle backward compatibility?

Example: API error response format

```
{
  "error": {
    "code": "INVALID_INPUT",
    "message": "The provided document_id is invalid.",
    "details": {
      "field": "document_id",
      "reason": "Must be a valid UUID"
    }
  }
}
```

Interview explanation:

Consistent error formats make APIs easier to consume, debug, monitor, and document.

6. FastAPI

FastAPI is a strong match for Python backend and GenAI services because it supports async endpoints, automatic OpenAPI docs, dependency injection, and Pydantic validation.

Topics to revise

- FastAPI routing.
- Pydantic models.
- Dependency injection.
- Request validation.
- Response models.
- Async endpoints.
- Middleware.
- Exception handlers.
- Background tasks.
- Authentication dependencies.
- Testing with TestClient.

Example: FastAPI endpoint

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI(title="Document QA API")

class QuestionRequest(BaseModel):
    document_id: str = Field(..., min_length=1)
    question: str = Field(..., min_length=5)

class AnswerResponse(BaseModel):
    answer: str
    confidence: float

@app.post("/ask", response_model=AnswerResponse)
def ask_question(payload: QuestionRequest) -> AnswerResponse:
    if payload.document_id != "demo-doc":
        raise HTTPException(status_code=404, detail="Document not found")

    return AnswerResponse(
        answer="This is a sample answer from the retrieval pipeline.",
        confidence=0.87,
    )
```

Interview explanation:

Pydantic validates request bodies automatically. FastAPI also generates API documentation and supports typed response models, which improves maintainability.

7. Flask

Flask may be discussed because the role mentioned FastAPI/Flask.

Topics to revise

- Flask app structure.
- Routes.
- Blueprints.
- Middleware-like hooks.
- Request validation.
- Error handling.
- When Flask is suitable vs FastAPI.

FastAPI vs Flask

Area	FastAPI	Flask
Async support	Built-in and modern	Possible, but not the main design focus
Validation	Pydantic-based	Usually manual or with extensions
API docs	Automatic OpenAPI	Requires extensions
Best fit	Modern APIs, typed services, async workloads	Lightweight apps, simple services, legacy apps

Django Request Flow and MVT

Django is a batteries-included Python web framework. A normal request can be explained as:

```
client
middleware
URL router
view
service/business logic
model / Django ORM / database
template or JSON response
```

Django is commonly described with **MVT**:

- **Model:** persistent/domain data and ORM mapping.
- **View:** request handling and application logic.
- **Template:** server-rendered presentation.

Django also includes migrations, authentication, sessions, admin tooling, middleware, forms, and a mature ORM. For API-heavy systems, Django REST Framework is a common addition.

Interview answer:

Django is a higher-level, batteries-included framework. Requests pass through middleware and URL routing into a view, which may use models through the ORM and return either rendered HTML or an API response. I would choose Django when built-in ORM, auth, admin, migrations, and convention are valuable; I would choose a lighter framework when I want a smaller API-first surface.

Flask Application Context vs Request Context

Flask uses context-local proxies so code can access the active app/request without passing those objects through every function.

Context	Typical Objects	Meaning
Application context	current_app, g	The active Flask application and context-local scratch state
Request context	request, session	One specific HTTP request and its request/session data

For a normal HTTP request, Flask pushes the request context and the corresponding application context, then removes them when request processing ends.

```
request starts

application context available
request context available

view executes

request context removed
application context removed
```

A common error outside an active context is:

```
RuntimeError: Working outside of application context
```

For work that needs an application context without an incoming request:

```
with app.app_context():
    # current_app and application-bound extensions are available here
    ...
```

Important nuance: the application context usually does **not** mean "the entire lifetime of the server process." Its lifetime is the active context, which during normal request handling is typically request-scoped.

Flask Application Factory Pattern

Instead of creating and configuring one global application at import time, construct it through a factory:

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

def create_app(config_object: str | None = None) -> Flask:
    app = Flask(__name__)

    if config_object:
        app.config.from_object(config_object)

    db.init_app(app)

    from .routes import api_blueprint
    app.register_blueprint(api_blueprint)

    return app
```

Why it helps:

- Different development/test/production configurations.
- Cleaner tests that create isolated app instances.
- Extensions are created once but bound with `init_app(app)`.
- Reduced circular-import pressure.
- Easier blueprint/module registration.

Interview answer:

The application factory pattern moves Flask construction into `create_app()`. That lets me create independently configured app instances, initialize extensions against a specific app, improve testability, and reduce global coupling/circular imports.

Production Flask Deployment

Do not treat Flask's development server as the production serving layer. A traditional Flask deployment uses a production **WSGI server** such as Gunicorn (or another production-capable WSGI server), often behind a load balancer or reverse proxy.

```
client
load balancer / reverse proxy
Gunicorn worker processes
Flask application created by create_app()
DB / cache / downstream APIs
```

Example factory-style Gunicorn target:

```
gunicorn "app:create_app()"
```

The WSGI server owns worker/process lifecycle and accepts production traffic; Flask still owns routing, contexts, views, and application behavior.

8. REST, gRPC, and API Design

REST

REST is commonly used for web APIs. It uses HTTP methods, resources, and status codes.

Example:

```
GET /documents/{document_id}
POST /documents
DELETE /documents/{document_id}
```

gRPC

gRPC is often used for internal service-to-service communication where performance, strict contracts, and streaming are important.

REST vs gRPC

Area	REST	gRPC
Transport	HTTP/JSON	HTTP/2 + Protocol Buffers
Human readability	High	Lower
Browser support	Easy	Less direct
Performance	Good	Very high
Best for	Public APIs, web clients	Internal microservices, low-latency systems

Common interview question

When would you choose gRPC over REST?

Answer:

Use gRPC when services need strong contracts, low latency, efficient binary serialization, streaming, and internal service-to-service communication. Use REST when simplicity, broad compatibility, and public-facing API access matter more.

9. Clean Architecture & Code Organization

Topics to revise

- Separation of concerns.
- Controllers/routes.
- Services/use cases.
- Repositories/data access.
- Domain models.
- Dependency inversion.
- Testability.

Example folder structure

```
app/  
  api/  
    routes.py  
  core/  
    config.py  
    security.py  
  services/  
    document_service.py  
    rag_service.py  
  repositories/  
    document_repository.py  
  models/  
    document.py  
  schemas/  
    document_schema.py  
  tests/  
    test_documents.py
```

Interview explanation

Clean architecture keeps business logic separate from frameworks and databases. This makes the system easier to test, maintain, and change.

7. REST API Fundamentals

7.1 What is a REST API?

Answer

A REST API is a way for applications to communicate over HTTP using standard methods like GET, POST, PUT, PATCH, and DELETE.

It is commonly used for:

- Frontend to backend communication
- Mobile app to backend communication
- Service-to-service communication
- Third-party integrations

Example

Request:

```
GET /restaurants
```

Response:

```
[
  {
    "name": "Pizza Place",
    "rating": 4.5
  }
]
```

7.2 REST API characteristics

Characteristic	Explanation
Stateless	Each request is independent
Resource-based	Uses URLs like /users, /orders
Uses HTTP methods	GET, POST, PUT, PATCH, DELETE
Usually JSON	Data is commonly exchanged in JSON
Language independent	Any language can consume REST APIs

7.3 Common HTTP methods

Method	Purpose
GET	Fetch data
POST	Create/send/upload data
PUT	Replace entire resource
PATCH	Partially update a resource
DELETE	Remove data

7.4 GET API example

FastAPI Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {
        "id": user_id,
        "name": "Alex"
    }
```

Request:

```
GET /users/1
```

Response:

```
{
  "id": 1,
  "name": "Alex"
}
```

Interview Explanation

GET APIs are used to retrieve data from the server. They should generally be read-only and should not modify server data.

7.5 Query parameter example

```
@app.get("/search")
def search(name: str):
    return {"result": name}
```

Request:

```
GET /search?name=alex
```

Response:

```
{
  "result": "alex"
}
```

7.6 POST API example

POST is used to send or create data on the server.

FastAPI Example

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    email: str

@app.post("/users")
def create_user(user: User):
    return {
        "message": "User created",
        "data": user
    }
```

Request:

```
POST /users
```

Body:

```
{
  "name": "Alex",
  "email": "alex@example.com"
}
```

Response:

```
{
  "message": "User created",
  "data": {
    "name": "Alex",
    "email": "alex@example.com"
  }
}
```

7.7 POST file upload example

POST is commonly used for uploading files such as resumes, images, or documents.

```
from fastapi import FastAPI, UploadFile, File

app = FastAPI()

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    return {"filename": file.filename}
```

Request:

```
POST /upload
```

Content type:

```
multipart/form-data
```

Response:

```
{
  "filename": "resume.pdf"
}
```

7.8 How do you fetch one specific record?

Use a path parameter or query parameter.

Path Parameter

```
GET /cars/5
```

FastAPI:

```
@app.get("/cars/{car_id}")
def get_car(car_id: int):
    return {
        "car_id": car_id,
        "price": 25000
    }
```

Response:

```
{
  "car_id": 5,
  "price": 25000
}
```

Query Parameter

```
GET /cars?price=25000
```


FastAPI:

```
@app.get("/cars")
def get_car(price: int):
    return {"price": price}
```

7.9 GET vs POST

GET	POST
Fetches data	Sends/creates/uploads data
Parameters usually in URL	Data usually in request body
Should not modify server data	Can modify server data
Usually cacheable	Usually not cacheable
Idempotent	Not always idempotent

7.10 Common HTTP status codes

Code	Meaning
200	Success
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
500	Internal Server Error

8. FastAPI and Flask

8.1 Have you worked with Flask and FastAPI?

Strong Answer

Yes, I have worked with both Flask and FastAPI. Flask is a lightweight and flexible Python web framework, while FastAPI is a modern framework designed for high-performance APIs using Python type hints, async support, and automatic documentation.

8.2 Major differences between Flask and FastAPI

Flask	FastAPI
Older, mature framework	Newer, modern API-first framework
Mostly synchronous by default	Native async/await support
Manual validation usually required	Built-in validation using Pydantic
Swagger docs need extra setup	Swagger/OpenAPI docs built in
Very flexible and minimal	More structured for API development
Good for simple apps and custom control	Good for scalable, high-performance APIs

Strong Interview Answer

Flask gives more flexibility and is great for lightweight web apps or when we want full control over structure. FastAPI is better for modern REST APIs because it provides automatic Swagger documentation, request and response validation with Pydantic, native async support, and strong performance. I prefer FastAPI for API-first services, but Flask is still useful for simple services and quick prototypes.

8.3 Why use FastAPI?

FastAPI is useful because it provides:

- Native async support
- Pydantic request validation
- Automatic Swagger documentation
- OpenAPI support
- Type hint-based development
- High performance
- Dependency injection
- Cleaner API development experience

9. Async APIs and Concurrent Requests

9.1 What is async support in FastAPI?

FastAPI supports async and await using ASGI.

This allows non-blocking request handling.

9.2 API without async

```
@app.get("/users")
def get_users():
    data = call_external_api()
    return data
```

Here, the request is blocking.

If `call_external_api()` takes 5 seconds, the worker waits for 5 seconds.

Good for:

- Simple APIs
- CPU-heavy logic
- Libraries that are not async
- Basic CRUD apps

9.3 API with async

```
@app.get("/users")
async def get_users():
    data = await call_external_api_async()
    return data
```

Here, the request is non-blocking.

If the external API takes 5 seconds, FastAPI can handle other requests while waiting.

Good for:

- External API calls
- Database calls
- File operations
- GenAI/LLM calls
- Vector database calls

9.4 Async vs sync interview answer

The main difference is blocking versus non-blocking execution. In a synchronous API, the worker waits until the task finishes. In an async API, when the code is waiting for an I/O operation like a database call or external API call, the event loop can process other requests. Async improves concurrency and scalability for I/O-heavy workloads, especially GenAI APIs where LLM calls may take several seconds.

Important:

Async does not automatically make CPU-heavy code faster. It mainly helps when the API spends time waiting on I/O.

9.5 What happens when two users hit the same API at the same time?

Answer

The backend treats them as two independent HTTP requests.

Each request has its own:

- Request object
- Headers
- Body
- Authentication token
- Response lifecycle

Read-only API scenario

Example:

```
GET /cars/available
```

If both users are checking available cars, both can get the same response.

Response:

```
{
  "car_id": 101,
  "status": "available"
}
```

This is usually fine because no data is being modified.

Write API scenario

Example:

```
POST /cars/101/book
```

If two users try to book the same car at the same time, this can create a **race condition**.

Bad scenario:

1. User A checks car 101 is available.
2. User B also checks car 101 is available.
3. Both try to book it.
4. Without proper controls, both may get success.

That is incorrect.

9.6 How to handle race conditions in APIs

Use:

- Database transactions
- Row-level locking
- Unique constraints
- Optimistic locking
- Idempotency keys
- Proper status checks

Safe Booking Example

```
@app.post("/cars/{car_id}/book")
def book_car(car_id: int, user_id: int):
    car = (
        db.query(Car)
        .filter(Car.id == car_id)
        .with_for_update()
        .first()
    )

    if car.status != "available":
        return {"message": "Car already booked"}

    car.status = "booked"
    car.booked_by = user_id
    db.commit()

    return {"message": "Booking successful"}
```

Expected behavior:

First user:

```
{
  "message": "Booking successful"
}
```

Second user:

```
{
  "message": "Car already booked"
}
```

Better HTTP status for second user:

```
409 Conflict
```

Important Interview Point

Async helps the API handle concurrent requests efficiently, but it does not solve data consistency problems by itself. Race conditions must be handled using database-level consistency controls.

15. Code Review and API Fix Case Topics**15.1 User API endpoint defects identified****Issues covered**

- SELECT * returns unnecessary/sensitive fields.
- password_hash leaked in response.
- Deleted/soft-deleted users still returned.
- SQL injection risk due to string concatenation.
- Missing input validation.
- Invalid filters silently return all users.
- N+1 query pattern for posts per user.
- No pagination on large table.
- In-memory filtering instead of DB filtering.
- No request-level field filtering based on caller type.

15.2 Which issue to fix first?**Highest priority options**

- SQL injection vulnerability.
- Password hash leakage.

Reason

Security issues have the highest blast radius and can expose sensitive data or allow data compromise.

Suggested answer

I would fix the security issue first, especially password hash leakage or SQL injection, because those create immediate risk beyond performance degradation.

15.3 Corrected API implementation pattern

Pseudocode

```
app.get('/api/users', async (req, res) => {
  const allowedFilters = ['active', 'inactive'];
  const filter = req.query.filter;
  const limit = Math.min(parseInt(req.query.limit || '50', 10), 100);
  const cursor = req.query.cursor;

  if (filter && !allowedFilters.includes(filter)) {
    return res.status(400).json({ error: 'Invalid filter' });
  }

  const params = [];
  let where = 'WHERE deleted_at IS NULL';

  if (filter === 'active') {
    where += ' AND last_login IS NOT NULL';
  }

  if (cursor) {
    params.push(cursor);
    where += ` AND id > ${params.length}`;
  }

  params.push(limit);

  const users = await db.query(
    `SELECT id, username, display_name, avatar_url, last_login
     FROM users
     ${where}
     ORDER BY id
     LIMIT ${params.length}`,
    params
  );

  const userIds = users.rows.map(u => u.id);

  const posts = await db.query(
    `SELECT user_id, id, title, created_at
     FROM posts
     WHERE user_id = ANY($1)`,
    [userIds]
  );

  const postsByUser = groupBy(posts.rows, 'user_id');

  const response = users.rows.map(user => ({
    ...user,
    posts: postsByUser[user.id] || []
  }));

  res.json({ data: response, next_cursor: getNextCursor(response) });
});
```

What this fixes

- Prevents SQL injection through parameterized queries.
- Excludes deleted users.
- Removes password hash.
- Validates filters.
- Adds pagination.
- Avoids N+1 posts query.
- Projects only required fields.

15.4 One performance improvement to recommend

Best choices

1. Pagination.
2. Batch query/JJOIN to eliminate N+1.
3. Database indexes on filter columns.
4. Caching for hot users.

Strongest general answer

For a large user table and list endpoint, **pagination** is usually the first stability improvement because it limits response size and DB load.

If the known bottleneck is N+1

Choose **batch query/JJOIN** because it directly removes hundreds/thousands of extra DB queries.

If many requests target same top users

Choose **Redis/in-memory caching for hot users**.

Backend Engineering & FastAPI

FastAPI

Interview Question

Why use FastAPI?

Answer

FastAPI is a modern Python web framework used for building APIs quickly and efficiently.

Benefits

- High performance
- Async support
- Pydantic validation
- Automatic OpenAPI documentation
- Easy dependency injection
- Strong typing support

FastAPI Example

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class PredictionRequest(BaseModel):
    text: str

class PredictionResponse(BaseModel):
    label: str
    confidence: float

@app.post("/predict", response_model=PredictionResponse)
async def predict(request: PredictionRequest):
    return PredictionResponse(
        label="positive",
        confidence=0.92
    )
```

API Security

Interview Question

How can you secure an API?

Answer

API security can be implemented through:

- Authentication
- Authorization
- HTTPS
- JWT tokens
- OAuth2
- API keys
- Rate limiting
- Input validation
- CORS configuration
- Logging and monitoring
- Secrets management

JWT Example Concept

```
Client logs in
-> server verifies credentials
-> server issues JWT
-> client sends JWT in Authorization header
-> API validates token before processing request
```

API Monitoring

Interview Question

How would you track which user calls which API and token usage?

Answer

Track request-level metadata.

What to Log

- User ID
- Endpoint
- Timestamp
- Request ID
- Response status
- Latency
- Token usage
- Cost estimate
- Error message if failed

Example Logging Middleware

```
import time
from fastapi import FastAPI, Request

app = FastAPI()

@app.middleware("http")
async def log_requests(request: Request, call_next):
    start = time.time()

    response = await call_next(request)

    latency_ms = (time.time() - start) * 1000

    print({
        "path": request.url.path,
        "method": request.method,
        "status_code": response.status_code,
        "latency_ms": round(latency_ms, 2)
    })

    return response
```

5. Backend APIs: REST, GraphQL, FastAPI, Flask

Likely Questions

- How do you design REST APIs?
 - What is the difference between REST and GraphQL?
 - What experience do you have with FastAPI?
 - What experience do you have with Flask?
 - How do you handle validation?
 - How do you version APIs?
 - How do you secure APIs?
 - How do you handle pagination?
 - How do you handle idempotency?
 - How do you design APIs that are easy for frontend teams to consume?
-

REST API Example With FastAPI

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class OrderCreate(BaseModel):
    customer_id: int
    total_amount: float
    currency: str = "CAD"

orders = {}

@app.post("/orders", status_code=201)
def create_order(payload: OrderCreate):
    order_id = len(orders) + 1

    orders[order_id] = {
        "id": order_id,
        "customer_id": payload.customer_id,
        "total_amount": payload.total_amount,
        "currency": payload.currency,
        "status": "created",
    }

    return orders[order_id]

@app.get("/orders/{order_id}")
def get_order(order_id: int):
    if order_id not in orders:
        raise HTTPException(status_code=404, detail="Order not found")

    return orders[order_id]
```

Interview Explanation

This demonstrates:

- Request validation with Pydantic
 - Proper status codes
 - Clear resource-oriented design
 - Error handling with HTTP exceptions
-

REST vs GraphQL

Topic	REST	GraphQL
Data fetching	Multiple endpoints	Single endpoint
Response shape	Server-defined	Client-defined
Over-fetching	Common	Reduced
Caching	Easier with HTTP	More complex
Learning curve	Lower	Higher
Best for	Standard CRUD APIs	Flexible UI-driven data needs

Example REST Request

GET /orders/123

Example GraphQL Query

```
query {  
  order(id: 123) {  
    id  
    status  
    customer {  
      name  
      email  
    }  
  }  
}
```

API Pagination Example

```
@app.get("/orders")  
def list_orders(limit: int = 50, offset: int = 0):  
    limit = min(limit, 100)  
  
    all_orders = list(orders.values())  
    return {  
        "items": all_orders[offset : offset + limit],  
        "limit": limit,  
        "offset": offset,  
        "total": len(all_orders),  
    }  
}
```

Interview Explanation

Pagination prevents:

- Huge responses
- Slow queries
- Memory pressure
- Poor frontend performance

API Idempotency

Question

How do you prevent duplicate payment/order creation when a client retries?

Answer

Use an idempotency key.

```
processed_requests = {}
```

```
def create_payment(idempotency_key: str, payload: dict):
    if idempotency_key in processed_requests:
        return processed_requests[idempotency_key]

    payment = charge_customer(payload)
    processed_requests[idempotency_key] = payment
    return payment
```

Interview Explanation

This is important when:

- Clients retry after timeout
 - Payment APIs are involved
 - Order creation must not duplicate records
-

Production-Style AI-Enabled Backend Project Walkthrough

A strong project explanation should make it clear that the application is a backend product with an AI capability, not merely a prompt demo.

Example Architecture

```
Client
|
v
FastAPI Routes
|
+--> Authentication and authorization
+--> Pydantic request validation
|
v
Application Services
|
+--> Resume/profile domain logic
+--> Job-description processing
+--> AI workflow orchestration
|
+--> PostgreSQL repositories
+--> Model-provider adapter
|
v
External LLM / Embedding Provider
```

Backend Responsibilities

A complete implementation can include:

- REST endpoint design and OpenAPI documentation.
- JWT-based authentication and authorization checks.
- Pydantic request and response schemas.
- PostgreSQL persistence and repository abstractions.
- Service-layer business logic.
- Model-provider integration behind an adapter.
- Structured errors and consistent HTTP status codes.
- Unit, integration, and API tests.
- Docker packaging, logging, and deployment configuration.

Keep AI Logic Out of the Route Layer

Routes should handle transport concerns and delegate business behavior:

```
@router.post("/resumes/{resume_id}/tailor", response_model=TailoredResumeResponse)
def tailor_resume(
    resume_id: int,
    payload: TailorResumeRequest,
    service: ResumeService = Depends(get_resume_service),
) -> TailoredResumeResponse:
    return service.tailor_resume(
        resume_id=resume_id,
        job_description=payload.job_description,
    )
```

The service can coordinate storage, prompt construction, model invocation, validation, and response formatting. This keeps the API independent of a specific model vendor.

Suggested Delivery Timeline

A practical phased approach is:

1. Define use cases, API contracts, and database entities.
2. Implement authentication and core CRUD endpoints.
3. Add the application-service and repository layers.
4. Integrate the model through a provider interface.
5. Add validation, error handling, and observability.
6. Write unit and integration tests.
7. Containerize and validate deployment readiness.
8. Treat the agreed feature set as the MVP boundary.

What Makes the MVP Complete

A personal project does not need endless features. It can be considered complete when the planned vertical slice works end to end:

- Authenticated request enters the API.
- Domain data is validated and persisted.
- The AI workflow executes through a replaceable adapter.
- Output is validated and returned through a documented schema.
- Failures are logged and translated into safe API errors.
- Core behavior is covered by automated tests.
- The service can be started consistently through Docker.

Possible later phases include analytics, feedback loops, rate limiting, model fallback, asynchronous jobs, and load-based scaling.

Explaining Recent API Experience Accurately

When recent work has focused more on model evaluation than endpoint ownership, separate the two clearly:

My recent responsibilities concentrated on AI evaluation, prompt workflows, Python automation, and supporting integrations rather than owning a public production API end to end. My earlier backend roles

and personal projects included production-style REST services, database integration, testing, containerization, and deployment. The API fundamentals remain current because I continued writing Python and working across service boundaries.

This answer is stronger than overstating recent API ownership because it identifies both proven experience and the exact ramp-up area.

Technical Ramp-Up Plan for a Backend Platform

The main ramp-up should be project-specific rather than a relearning of backend fundamentals:

- Map the request path from gateway to storage and downstream services.
- Read API contracts, architecture decisions, and operational runbooks.
- Run the service and test suite locally.
- Trace one production-like request through logs and metrics.
- Ship a small bug fix or endpoint change first.
- Review current framework, dependency, security, and deployment conventions.

The transferable foundation is Python, HTTP semantics, data modeling, testing, observability, and production debugging.

Create-Endpoint Debugging: Persistence, Nested Resources & Business Identifiers

A create endpoint can look superficially correct because it parses JSON and returns 201 Created, while still failing its actual contract if nothing is persisted or the response body is incomplete.

Trace the Complete Request Path

For a broken create API, trace the full path rather than editing the first suspicious line:

```
route
authentication / middleware
request parsing and validation
foreign-key / related-object lookups
business identifier generation
persistence
secondary side effects / audit activity
serialization
status code + response body
```

A useful debugging comparison is a nearby working read/update handler because it often shows the repository's established lookup, select_related, error, and serialization patterns.

Validate Before Mutating

For a typical issue/task create endpoint:

1. Require authentication.
2. Validate non-blank title.
3. Resolve teamId; return 404 if the team does not exist.
4. Resolve optional assignee.
5. Resolve optional parent item and validate relationship invariants.
6. Generate the team-local business identifier.
7. Set creator from the authenticated principal.
8. Persist the record.
9. Create audit/activity data if required.
10. Serialize the persisted object and return 201.

Do not return success before step 8.

Use Authenticated Identity for Audit Fields

Do not trust request-body fields such as creator when the authenticated principal already establishes identity.

```
issue = Issue.objects.create(
    ...,
    creator=request.user,
)
```

This prevents callers from spoofing ownership/audit metadata.

Nested Resources Through the Same Create Endpoint

A sub-item can use the same endpoint with an optional parentIssue field rather than requiring a second create route.

Validate:

- Parent exists.
- Parent and child belong to the expected scope/team.
- Parent relationship is persisted.
- The response exposes the parent identifier in the contract's expected shape.

The UI is not an integrity boundary; parent-child rules should be enforced in the backend as well.

Sequential Business Identifiers

A business identifier such as:

```
TEAM-1
TEAM-2
TEAM-3
```

must be independent per team.

Avoid:

```
number = Issue.objects.count() + 1
```

Problems:

- It is global rather than team-local.
- Deletions can cause reused numbers.
- Concurrent requests can compute the same next value.

A production-safe design typically uses a serialized per-team sequence source, for example:

```
transaction.atomic()

lock team-sequence row with SELECT ... FOR UPDATE

read and increment next_number

create issue with TEAM_KEY-number

commit
```

Keep a database uniqueness constraint on the final identifier as a last line of defense. A simpler `MAX(suffix) + 1` query may be enough for a coding exercise but is still not concurrency-safe without serialization/retry.

Error Shape Is Part of the API Contract

These are not equivalent contracts:

```
{}
```

and:

```
{
  "message": "Authentication required"
}
```

Even if both return 401, clients and tests may depend on the documented body shape. Avoid route-specific middleware exceptions unless the API explicitly requires them.

Success Response Should Reflect Persisted State

Prefer returning the canonical serializer representation of the saved entity:

```
return JsonResponse({"issue": issue.to_dict()}, status=201)
```

If relationships are required by the serializer, reload with `select_related(...)` or otherwise ensure the returned object contains the expected data.

Interview answer:

For a broken create endpoint, I trace route, auth, validation, lookups, persistence, side effects, and serialization as one flow. A 201 response is meaningless if the write never happened. I validate required fields and relationships first, derive creator from the authenticated user, allocate any business identifier safely, persist the entity, record audit activity, then return the canonical serialized representation. I also verify error body shape, because status code alone is not the full API contract.

See `coding_questions/issue_creation_workflow.py` for a runnable generalized version of this debugging exercise.

Senior Backend API Interview Addendum

This section consolidates reusable API/FastAPI material that commonly appears in senior backend rounds, especially when the interviewer follows a high-level answer with "what exactly happens in the request?", "how do you persist it?", "how do you test it?", or "what happens under concurrency?"

HTTP Method Semantics

Method	Safe	Idempotent by semantics	Typical use
GET	Yes	Yes	Read a resource/query.
POST	No	No	Create under server-assigned identity, commands/actions.
PUT	No	Yes	Create/replace resource at a known identity.
PATCH	No	Not guaranteed	Partial update.
DELETE	No	Yes in effect	Ensure resource is absent.

DELETE being idempotent does not mean every repeated response must have the same status code; it means repeating the operation should not create additional state changes after the resource is already absent.

200 vs 201 vs 202 vs 204

- 200 OK: successful request with a response representation.
- 201 Created: a resource was created; often include Location.
- 202 Accepted: request accepted for processing, but the promised operation/resource creation is not yet complete.
- 204 No Content: successful request with no response body.

Important long-running-job nuance:

```
POST /jobs
```

```
create durable Job(id=123, state=QUEUED)
```

```
commit DB
```

```
201 Created + Location: /jobs/123
```

```
worker executes asynchronously
```

A long-running job can still return 201 if the job resource itself is durably created synchronously.

Authentication: Do Not Trust `user_id` from the Request Body

Client:

```
GET /v1/feed
```

```
Authorization: Bearer <access-token>
```

Backend flow:

```
request
```

```
auth middleware validates token
```

```
trusted identity / user_id in request context
```

```
authorization
```

```
business logic
```

The caller may submit resource IDs needed for the operation, but the server should derive the caller identity from trusted authentication context when possible.

Authentication answers "who are you?" Authorization answers "are you allowed to do this?"

Cursor Pagination

For fast-changing ordered data, cursor pagination is usually safer than offset pagination.

Example:

```
GET /v1/feed?limit=50&cursor=<opaque-token>
```

Underlying stable ordering might be:

```
(created_at, id)
```

or:

```
(published_at, episode_id)
```

The cursor can encode the last returned ordering tuple. New rows inserted at the top do not shift all later pages the way offsets can.

Optimistic Concurrency with ETag / Version

For update races:

```
GET /v1/resources/123
ETag: "v7"
```

Client update:

```
PUT /v1/resources/123
If-Match: "v7"
```

If the resource is now version 8, reject the stale write rather than silently overwriting a newer update.

At the database layer the same concept may be implemented with a version column and conditional UPDATE.

REST vs gRPC

Use REST/HTTP+JSON when:

- Broad client/browser interoperability matters.
- Public APIs need easy debugging/tooling.
- Human-readable payloads and standard HTTP semantics are valuable.

Use gRPC when:

- Internal service-to-service contracts are strongly typed.
- Efficient binary serialization matters.
- Streaming is important.
- Client/server code generation is useful.

Good interview answer:

I would not choose gRPC simply because it is faster. I would choose it when the service boundary benefits from strongly typed generated contracts or streaming and all consumers can support the protocol. REST remains a strong default for external/public APIs.

Polling vs SSE vs WebSocket

Mechanism	Best fit
Polling	Simple, low-frequency status checks.
Server-Sent Events	Server -> client streaming over HTTP; one-way updates.
WebSocket	Bidirectional low-latency messaging.

Choose the simplest communication model that meets the update/interaction requirement.

FastAPI: Interview-Ready Architecture

Why FastAPI?

Strong answer:

FastAPI is a good fit for Python APIs because type annotations integrate with Pydantic validation, OpenAPI documentation is generated automatically, dependency injection keeps infrastructure concerns explicit, and async endpoints work well for I/O-bound concurrency when the full dependency chain is async-compatible. I still choose sync versus async from the workload rather than assuming async automatically makes the service faster.

Where Async Helps

Use async when the endpoint spends meaningful time waiting for async-capable I/O:

- HTTP calls
- Async database driver calls
- Object storage/network calls
- Multiple independent I/O operations that can overlap

Example:

```
import asyncio

async def build_dashboard(client, urls):
    results = await asyncio.gather(
        *(client.get(url) for url in urls)
    )
    return results
```

This can overlap seven independent I/O waits rather than performing them sequentially.

Do **not** use async to make CPU-heavy Python work magically parallel. CPU-heavy work can block the event loop; use process workers/background compute where appropriate.

Pydantic Models

Use Pydantic to define API contracts and validation boundaries.

```
from pydantic import BaseModel, Field

class CreateUserRequest(BaseModel):
    email: str
    age: int = Field(ge=0, le=130)

class UserResponse(BaseModel):
```

```
id: int
email: str
```

Pydantic helps with:

- Parsing and validation
- Serialization
- Clear request/response schemas
- OpenAPI generation
- Useful validation errors

It does not replace:

- Domain/business validation
- Authorization
- Database uniqueness/foreign-key constraints
- Transactional invariants

SQL Access in FastAPI with SQLAlchemy

A common synchronous pattern is one DB session per request through dependency injection.

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "postgresql+psycopg://user:pass@db/app"

engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Endpoint:

```
from fastapi import Depends, FastAPI, HTTPException
from sqlalchemy.orm import Session

app = FastAPI()

@app.get("/users/{user_id}")
def get_user(user_id: int, db: Session = Depends(get_db)):
    user = db.get(User, user_id)
    if user is None:
        raise HTTPException(status_code=404, detail="User not found")
    return user
```

For write workflows:

```
request
validate
start/use transaction
write all related rows
commit
```

```
return
```

On failure, rollback the transaction rather than leaving partially persisted state.

For an async stack, use SQLAlchemy async sessions with an async driver; do not mix blocking DB calls into an async endpoint under the assumption that `async def` alone makes them non-blocking.

Service Layer Function Example

Keep HTTP-specific concerns thin when business logic is reused/tested independently.

```
def create_subscription(db, user_id: int, podcast_id: int):
    existing = (
        db.query(Subscription)
        .filter_by(user_id=user_id, podcast_id=podcast_id)
        .one_or_none()
    )

    if existing is not None:
        return existing

    subscription = Subscription(
        user_id=user_id,
        podcast_id=podcast_id,
    )
    db.add(subscription)
    db.commit()
    db.refresh(subscription)
    return subscription
```

In a production implementation, a database uniqueness constraint should still enforce `(user_id, podcast_id)` so two concurrent requests cannot create duplicates even if both application checks initially see no row.

Mocking External Services

Suppose the application uses an injected HTTP client abstraction:

```
class WeatherClient:
    def get_weather(self, city: str) -> dict:
        ...
```

Service:

```
def build_forecast(city: str, client: WeatherClient) -> dict:
    data = client.get_weather(city)
    return {"city": city, "temperature": data["temperature"]}
```

Unit test:

```
from unittest.mock import Mock

def test_build_forecast():
    client = Mock()
    client.get_weather.return_value = {"temperature": 22}

    result = build_forecast("Toronto", client)

    assert result == {"city": "Toronto", "temperature": 22}
    client.get_weather.assert_called_once_with("Toronto")
```

Strong interview rule:

Mock unstable or expensive external boundaries, not every internal method. If the behavior being tested depends on SQL constraints or ORM integration, a real test database is often more valuable than mocking the repository layer into meaninglessness.

Unit Testing FastAPI

Basic endpoint test

```
from fastapi.testclient import TestClient

client = TestClient(app)

def test_health():
    response = client.get("/health")

    assert response.status_code == 200
    assert response.json() == {"status": "ok"}
```

Dependency override

For a DB/external client dependency:

```
def override_get_db():
    db = TestSessionLocal()
    try:
        yield db
    finally:
        db.close()

app.dependency_overrides[get_db] = override_get_db
```

This lets the endpoint execute through real routing/validation while controlling infrastructure dependencies.

What to test

- Happy path
- Request validation errors
- Authentication/authorization failures
- Not-found behavior
- Conflict/idempotency behavior
- Database rollback on failure
- External-service timeout/error mapping
- Response schema
- Concurrency-sensitive invariants at an integration level

Request Flow: Explain It Literally

When an interviewer asks what happens when two users call an API, avoid a vague "FastAPI handles concurrency."

A more precise explanation:

```
Client connection
ASGI server accepts request
FastAPI/Starlette routing + middleware
auth/dependencies
endpoint/service code
```

DB / cache / downstream I/O

response serialization

Multiple requests may overlap depending on worker/process/thread/event-loop configuration and whether operations block. Shared mutable state and database invariants still need explicit correctness mechanisms.

API Race Condition Example

Suppose two callers try to reserve the same seat.

Unsafe pattern:

```
SELECT status = AVAILABLE
```

request A sees available

request B sees available

both write reserved

Safer conditional write:

```
UPDATE seat
SET status = 'HELD', held_by = :user_id
WHERE id = :seat_id
  AND status = 'AVAILABLE';
```

Check affected-row count to determine which request won.

The database remains the final authority even if the API also performs a preliminary availability read.

Backward-Compatible API Evolution

When one backend/framework version supports multiple client versions, avoid uncontrolled pairwise special cases.

Useful tools:

- Explicit API versioning where contract changes are intentional.
- Capability flags/feature negotiation where appropriate.
- Compatibility matrix listing supported client/server combinations.
- Contract tests derived from supported combinations.
- Deprecation window and usage monitoring.

A compatibility matrix is especially useful when multiple client and server/framework versions coexist because it turns "which combinations work?" into explicit testable data rather than scattered conditionals.

API Interview Quick Questions

Why FastAPI over Flask?

FastAPI provides stronger built-in typing/validation/OpenAPI and async-friendly ASGI behavior. Flask is intentionally lighter and can be preferable when a minimal synchronous WSGI stack or a mature existing ecosystem/codebase matters.

Why dependency injection?

It makes request-scoped dependencies such as DB sessions, auth context, configuration, and external clients explicit and easier to replace in tests.

How do you secure APIs?

TLS, authentication, authorization, input validation, least-privilege credentials, secrets management, rate limiting, audit logging, and secure error handling. Security depends on the threat model; JWT alone is not "API security."

How do you handle external API failures?

Timeouts, retry only transient/idempotent operations with bounded exponential backoff + jitter, circuit breaking where repeated failure would amplify load, and clear mapping to application-level errors/fallbacks.

How do you version APIs?

Prefer stable contracts and additive changes where possible. When a breaking contract is necessary, expose an explicit version boundary and operate a deprecation/migration window with usage telemetry and compatibility tests.

How do you cancel a long-running job?

Treat cancellation as a state transition, for example:

```
POST /v1/jobs/{job_id}/cancel
```

rather than deleting the job resource if history/audit/status must remain available.

Quick Backend API Revision Card

HTTP semantics
201 vs 202 nuance
Idempotency-Key
Trusted auth context, not body user_id
Cursor pagination
ETag / optimistic concurrency
REST vs gRPC
Polling vs SSE vs WebSocket
FastAPI request flow
Django MVT + middleware/URL/view/ORM flow
Flask app vs request context (`current\_app`/`g` vs `request`/`session`)
Flask application factory + production WSGI/Gunicorn
Pydantic contract vs business/DB validation
SQLAlchemy session per request
Transactions + uniqueness constraints
Mock external boundaries
FastAPI dependency overrides
Race conditions resolved at authoritative store
Backward-compatibility matrix + contract tests

HTTP/2 vs HTTP/3 Quick Note

HTTP/2

- Multiplexes multiple request/response streams over one TCP connection.
- Header compression reduces repeated metadata overhead.
- TCP packet loss can still delay progress for streams sharing the same TCP connection because reliability/order is enforced at the transport layer.

HTTP/3

- Uses QUIC over UDP.
- Provides independent transport streams so packet loss affecting one stream does not create the same cross-stream TCP head-of-line behavior.
- Includes modern connection establishment/migration characteristics.

Interview line:

HTTP/2 improves connection utilization through multiplexing, while HTTP/3 moves HTTP onto QUIC so streams have more independent transport behavior and can avoid TCP-level head-of-line blocking across streams.

About the HTTP QUERY Method

RFC 10008, published in June 2026, defines the HTTP QUERY method for a request whose query description is carried in the request content. QUERY is explicitly **safe** and **idempotent**, unlike a general POST, while avoiding the URI-size, encoding, and logging drawbacks of putting a large query into a GET URI.

```
QUERY /feed HTTP/1.1
Host: example.org
Content-Type: application/json
```

```
{"keywords": ["python", "fastapi"], "limit": 20}
```

Important points:

- The request requires a media type describing the query content.
- A successful 200 OK can contain the results.
- Location can identify a resource representing the query for later repetition.
- Content-Location can identify a resource representing the result.
- Responses are cacheable, but a cache key must incorporate the request content and relevant metadata.
- Accept-Query can advertise supported query media types.
- Because QUERY is not a CORS-safelisted method, browser use requires preflight.
- Ecosystem/framework/proxy support should still be verified before choosing it for a public API.

Interview comparison:

Method	Safe	Idempotent	Request content	Typical use
GET	Yes	Yes	No defined semantics	Retrieve a URI-identified representation
QUERY	Yes	Yes	Expected	Complex read/query whose description needs a body
POST	Not guaranteed	Not guaranteed	Expected	Submit processing or create/change state

Reference: [RFC 10008 - The HTTP QUERY Method](#)

Async FastAPI, SQL, Service Layers, and Test Doubles

This section extends the basic examples with a production-shaped async PostgreSQL flow and layered tests.

Async SQLAlchemy Session per Request

FastAPI does not execute SQL itself. Use a database driver or ORM, obtain a short-lived session from a pool, and inject it into the route/service.

```
from collections.abc import AsyncGenerator

from sqlalchemy.ext.asyncio import (
    AsyncSession,
    async_sessionmaker,
    create_async_engine,
)

DATABASE_URL = "postgresql+asyncpg://user:password@localhost:5432/app"

engine = create_async_engine(
    DATABASE_URL,
    pool_pre_ping=True,
    pool_size=10,
    max_overflow=20,
)

SessionFactory = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False,
)

async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with SessionFactory() as session:
        yield session
```

Production points:

- Size the application pool against pod/worker count and the database connection limit.
- Keep transactions short and always roll back failed writes.
- Use an async driver throughout an async path; blocking database calls still block the event loop.
- Keep credentials in approved secret/configuration management.
- Use Alembic or another controlled migration workflow rather than creating schemas from request paths.

ORM Model vs API Schema

Keep persistence models separate from public API contracts.

```
from decimal import Decimal
from uuid import UUID, uuid4

from pydantic import BaseModel, ConfigDict, Field
from sqlalchemy import Numeric, String
from sqlalchemy.dialects.postgresql import UUID as PostgresUUID
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class Account(Base):
    __tablename__ = "accounts"

    id: Mapped[UUID] = mapped_column(
        PostgresUUID(as_uuid=True),
        primary_key=True,
        default=uuid4,
    )
    owner_name: Mapped[str] = mapped_column(String(100), nullable=False)
```

```
balance: Mapped[Decimal] = mapped_column(Numeric(12, 2), nullable=False)
```

```
class AccountCreate(BaseModel):
    owner_name: str = Field(min_length=1, max_length=100)
    opening_balance: Decimal = Field(ge=0, max_digits=12, decimal_places=2)
```

```
class AccountResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: UUID
    owner_name: str
    balance: Decimal
```

Use Decimal/NUMERIC, not binary floating point, for money.

Repository and Parameterized Query

```
from sqlalchemy import select, text
```

```
async def find_account(db: AsyncSession, account_id: UUID) -> Account | None:
    result = await db.execute(
        select(Account).where(Account.id == account_id)
    )
    return result.scalar_one_or_none()
```

```
async def find_account_raw(db: AsyncSession, account_id: UUID):
    result = await db.execute(
        text(
            """
            SELECT id, owner_name, balance
            FROM accounts
            WHERE id = :account_id
            """
        ),
        {"account_id": account_id},
    )
    return result.mappings().one_or_none()
```

Never interpolate untrusted values into an SQL string. Parameterization prevents input from being interpreted as SQL syntax; authorization remains a separate requirement.

Small Endpoint with a Service-Layer Boundary

```
from decimal import Decimal
from typing import Annotated, Protocol
from uuid import UUID

from fastapi import Depends, FastAPI, Header, HTTPException, status
from pydantic import BaseModel, Field

app = FastAPI()

class TransactionCreate(BaseModel):
    account_id: UUID
    amount: Decimal = Field(gt=0, max_digits=12, decimal_places=2)
    currency: str = Field(min_length=3, max_length=3)

class TransactionRepository(Protocol):
    async def create_if_absent(
        self,
        request: TransactionCreate,
        idempotency_key: str,
    ) -> dict: ...

class TransactionService:
    SUPPORTED_CURRENCIES = {"CAD", "USD"}

    def __init__(self, repository: TransactionRepository):
        self.repository = repository

    async def create(self, request, idempotency_key: str) -> dict:
        currency = request.currency.upper()
        if currency not in self.SUPPORTED_CURRENCIES:
            raise ValueError(f"unsupported currency: {currency}")

        normalized = request.model_copy(update={"currency": currency})
        return await self.repository.create_if_absent(
            normalized,
            idempotency_key,
        )

def get_transaction_service() -> TransactionService:
    raise NotImplementedError

@app.post("/transactions", status_code=status.HTTP_201_CREATED)
async def create_transaction(
    request: TransactionCreate,
    idempotency_key: Annotated[
        str,
        Header(alias="Idempotency-Key", min_length=8),
    ],
    service: Annotated[
        TransactionService,
        Depends(get_transaction_service),
    ],
):
    try:
        return await service.create(request, idempotency_key)
    except ValueError as exc:
        raise HTTPException(status_code=422, detail=str(exc)) from exc
```

The repository operation must enforce the idempotency key atomically, normally with a unique database constraint. A separate SELECT followed by INSERT has a race condition.

Mock an External Async API Through Dependency Injection

Wrap an external API behind an application-owned protocol instead of calling httpx directly from every route.

```
from typing import Protocol

import httpx

class FraudClient(Protocol):
    async def approve(self, account_id: UUID, amount: Decimal) -> bool: ...

class HttpFraudClient:
    def __init__(self, client: httpx.AsyncClient):
        self.client = client

    async def approve(self, account_id: UUID, amount: Decimal) -> bool:
        response = await self.client.post(
            "/fraud/check",
            json={"account_id": str(account_id), "amount": str(amount)},
        )
        response.raise_for_status()
        return response.json()["approved"]

class FraudCheckRequest(BaseModel):
    account_id: UUID
    amount: Decimal = Field(gt=0)

def get_fraud_client() -> FraudClient:
    raise NotImplementedError

@app.post("/fraud-decisions")
async def check_fraud(
    request: FraudCheckRequest,
    fraud_client: Annotated[FraudClient, Depends(get_fraud_client)],
):
    approved = await fraud_client.approve(request.account_id, request.amount)
    return {"approved": approved}
```

Endpoint tests can replace the client dependency without a network call:

```
from unittest.mock import AsyncMock

from fastapi.testclient import TestClient

client = TestClient(app)

def test_transaction_uses_fraud_client():
    fraud_client = AsyncMock()
    fraud_client.approve.return_value = True

    account_id = "7a85a83e-c53e-4ca8-9616-b1de36114e07"

    app.dependency_overrides[get_fraud_client] = lambda: fraud_client
    try:
        response = client.post(
            "/fraud-decisions",
            json={"account_id": account_id, "amount": "100.00"},
        )
        assert response.status_code == 200
        assert response.json() == {"approved": True}
        fraud_client.approve.assert_awaited_once_with(
            UUID(account_id),
            Decimal("100.00"),
        )
    finally:
        app.dependency_overrides.clear()
```

Use `httpx.MockTransport` when testing the HTTP adapter itself, including path, headers, serialization, status mapping, and response parsing:

```
async def test_http_fraud_client():
```

```

account_id = UUID("7a85a83e-c53e-4ca8-9616-b1de36114e07")

def handler(request: httpx.Request) -> httpx.Response:
    assert request.url.path == "/fraud/check"
    return httpx.Response(200, json={"approved": True})

transport = httpx.MockTransport(handler)

async with httpx.AsyncClient(
    base_url="https://fraud-service.test",
    transport=transport,
) as http_client:
    fraud_client = HttpFraudClient(http_client)
    assert await fraud_client.approve(account_id, Decimal("100.00"))

```

FastAPI Unit-Test Layers

Use different tests for different contracts:

Test	Real pieces	Replaced pieces
Service unit test	Business/service object	Repository, clock, external client
Endpoint unit/component test	Routing, Pydantic, HTTP mapping, service call	Service/infrastructure dependencies
Database integration test	Repository, ORM, migrations, real test DB	External APIs
End-to-end test	Deployed request path	Only uncontrollable third parties when needed

Example endpoint test:

```

from decimal import Decimal
from unittest.mock import AsyncMock

def test_create_transaction_success():
    service = AsyncMock()
    service.create.return_value = {
        "transaction_id": "txn-101",
        "status": "created",
    }
    app.dependency_overrides[get_transaction_service] = lambda: service
    account_id = "7a85a83e-c53e-4ca8-9616-b1de36114e07"

    try:
        response = client.post(
            "/transactions",
            headers={"Idempotency-Key": "request-123"},
            json={
                "account_id": account_id,
                "amount": "100.00",
                "currency": "cad",
            },
        )

        assert response.status_code == 201
        assert response.json()["status"] == "created"
        service.create.assert_awaited_once()
    finally:
        app.dependency_overrides.clear()

```

Test at least:

- Successful request and response schema
- Missing, malformed, and boundary values
- Authentication and authorization failures
- Not found and conflict/idempotency behavior
- Database rollback on failure
- External timeout, invalid JSON, and 4xx/5xx mapping
- Correct dependency calls
- Concurrency-sensitive uniqueness/invariants in integration tests

Interview answer:

I keep FastAPI routes thin and inject services, database sessions, authentication, and external clients with Depends. Endpoint unit tests use `TestClient` and `app.dependency_overrides`; async collaborators use `AsyncMock` or fakes. I test business rules directly at the service layer, the HTTP adapter with `MockTransport`, and persistence constraints against a disposable real database. That keeps unit tests fast without mocking away the behavior that only a real boundary can verify.

SQL, Databases, ORM, Caching & Idempotency

Source: docs/sql.md

Purpose: SQL fundamentals, analytical SQL, data modeling, ORM, caching, transactions, idempotency, and database performance. **Use this file for:** SQL interviews, backend database rounds, and data engineering interviews

Recommended Study Flow

1. Read the **Quick Summary** first.
2. Review the **Key Concepts** and tables.
3. Practice the **Interview Questions & Answers** out loud.
4. Use the code snippets and examples to explain trade-offs clearly.
5. Finish with the **Common Mistakes** and **Revision Checklist** sections.

Quick Summary

This file has been refreshed to keep the original repository topic while merging relevant detailed Q&A from the consolidated topic-wise interview-prep pack. Use the top sections for fast revision and the consolidated section for deeper interview preparation.

Core Topics to Master

- DDL, DML, DCL, TCL
- Primary keys, foreign keys, unique constraints
- JOINS, GROUP BY, HAVING, subqueries
- Window functions: ROW_NUMBER, RANK, DENSE_RANK
- Transactions and ACID
- Indexes and query optimization
- ORM performance and N+1 queries
- Caching and idempotency patterns

Consolidated Interview Questions & Technical Notes

SQL patterns, joins, window functions, relational modeling, ORM performance, Django/SQLAlchemy, caching, idempotency, transactions, and data consistency.

11. SQL Interview Patterns

11.1 Top 3 orders per customer

```
SELECT *
FROM (
  SELECT
    o.*,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id
      ORDER BY amount DESC
    ) AS rn
  FROM orders o
) t
WHERE rn <= 3;
```

Pattern: Top N per group using ROW_NUMBER().

11.2 Type 2 SCD as-of join

```
SELECT *
FROM orders o
JOIN customer_address a
  ON o.customer_id = a.customer_id
 AND o.order_date >= a.effective_start
 AND (
   o.order_date < a.effective_end
  OR a.effective_end IS NULL
 );
```

Pattern: Join facts to the dimension version active at the event date.

11.3 Monthly spend increased for 3 consecutive months

```
WITH m AS (
  SELECT
    customer_id,
    DATE_TRUNC('month', order_date) AS month,
    SUM(amount) AS total_amount
  FROM orders
  GROUP BY customer_id, DATE_TRUNC('month', order_date)
),
x AS (
  SELECT
    *,
    CASE
      WHEN total_amount > LAG(total_amount) OVER (
        PARTITION BY customer_id
        ORDER BY month
      ) THEN 1
      ELSE 0
    END AS increased
  FROM m
)
SELECT
  customer_id,
  month,
  total_amount
FROM x
QUALIFY SUM(increased) OVER (
  PARTITION BY customer_id
  ORDER BY month
  ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
) = 3;
```

Pattern: Aggregate monthly, compare with LAG, then use a rolling window.

10. Databases, ORM, and Data Modeling

Topics to revise

- Relational databases.
- SQL basics.
- Indexing.
- Normalization.
- Transactions.
- SQLAlchemy ORM.
- Migrations.
- Connection pooling.
- Data modeling.

Example: SQLAlchemy model

```
from sqlalchemy import Column, Integer, String, DateTime
from sqlalchemy.orm import declarative_base
from datetime import datetime

Base = declarative_base()

class Document(Base):
    __tablename__ = "documents"

    id = Column(Integer, primary_key=True)
    title = Column(String(255), nullable=False)
    storage_path = Column(String(500), nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)
```

Common interview questions

1. What is an ORM?
2. What are the benefits and risks of using an ORM?
3. How do indexes improve query performance?
4. How would you model documents and embeddings?
5. How do you handle migrations?

16. Databases, Caching, and Messaging Systems

Databases

Topics mentioned or implied:

- PostgreSQL.
- MySQL.
- SQLAlchemy.
- Data modeling.
- Transactions.
- Indexes.

Redis

Redis can be used for:

- Caching.
- Rate limiting.
- Session storage.
- Job status tracking.
- Semantic cache metadata.

Kafka / SQS / Queue Systems

Messaging systems are used for asynchronous workloads.

Examples:

- Document ingestion.
- Embedding generation.
- Email notifications.
- Background LLM processing.
- Event-driven pipelines.

Common interview question

When would you use a queue?

Answer:

Use a queue when a task is slow, asynchronous, retryable, or should not block the API request. For example, document parsing and embedding generation can be queued so the API remains responsive while background workers process the document.

3. REST API Design and Idempotency

3.1 Idempotent HTTP methods

Idempotent methods

- GET
- PUT
- DELETE
- HEAD
- OPTIONS

Not idempotent by default

- POST

Why it matters

Idempotency determines whether clients can safely retry after timeouts or network failures.

Example

```
GET /orders/123      -> safe to retry
PUT /profile/123     -> safe if same payload
DELETE /users/123    -> safe final state remains deleted
POST /orders         -> unsafe unless idempotency key is used
```

3.2 Safely retrieable POST endpoint

Problem

A client retries POST /orders after timeout. Without protection, duplicate orders may be created.

Solution

Use an Idempotency-Key.

```
POST /api/v1/orders
Authorization: Bearer <token>
Idempotency-Key: 7f6d2a91-8c4e-4b8b-9c2e-12345
Content-Type: application/json
```

Store

- Client/user ID.
 - Idempotency key.
 - Request hash.
 - Status: PROCESSING, SUCCEEDED, FAILED.
 - Response body.
 - Status code.
 - Safe response headers.
-

3.3 Request/response contract for idempotent create order

First response

```
201 Created
Location: /api/v1/orders/order_789

{
  "order_id": "order_789",
  "status": "created"
}
```

Retry with same key and same body

Return the same stored response.

```
201 Created

{
  "order_id": "order_789",
  "status": "created"
}
```

Same key but different body

```
409 Conflict

{
  "error": "Idempotency key reused with different request payload"
}
```

Original request still processing

```
202 Accepted
Retry-After: 2

{
  "status": "processing"
}
```

3.4 Exactly-once behavior with RDS and DynamoDB

Topic covered

How to avoid duplicate order creation without distributed transactions.

Recommended approach

- Use DynamoDB for fast idempotency lookup/replay.
- Use RDS unique constraint as the final correctness guardrail.

RDS uniqueness

```
UNIQUE (client_id, idempotency_key)
```

Why

Even if DynamoDB and RDS are not atomically committed together, RDS prevents duplicate order rows.

Recovery scenario

If RDS commits but DynamoDB update fails:

- Retry sees idempotency record stuck in PROCESSING.
 - Backend checks RDS by (client_id, idempotency_key).
 - If order exists, rebuild response and mark DynamoDB SUCCEEDED.
-

3.5 Django implementation of idempotency

Model example

```
class IdempotencyKey(models.Model):
    key = models.CharField(max_length=255)
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    request_hash = models.CharField(max_length=64)
    status = models.CharField(max_length=20) # PROCESSING, SUCCEEDED, FAILED
    response_body = models.JSONField(null=True, blank=True)
    response_status_code = models.IntegerField(null=True, blank=True)
    response_headers = models.JSONField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["user", "key"],
                name="unique_idempotency_key_per_user",
            )
        ]
```

Transaction and lock

```
from django.db import transaction, IntegrityError
from rest_framework.response import Response

with transaction.atomic():
    try:
        record = IdempotencyKey.objects.create(
            user=request.user,
            key=idempotency_key,
            request_hash=request_hash,
            status="PROCESSING",
        )
        is_owner = True
    except IntegrityError:
        record = (
            IdempotencyKey.objects
            .select_for_update()
            .get(user=request.user, key=idempotency_key)
        )
        is_owner = False

    if record.request_hash != request_hash:
        return Response(
            {"error": "Idempotency key reused with different request"},
            status=409,
        )

    if not is_owner:
        if record.status == "SUCCEEDED":
            return Response(
                record.response_body,
                status=record.response_status_code,
                headers=record.response_headers or {},
            )
        return Response({"status": "processing"}, status=202)
```

Key idea

Use unique constraints plus row-level locks to ensure two simultaneous requests do not both process.

3.6 REST API versioning

Preferred external API style

```
/api/v1/orders
/api/v2/orders
```

Why URL versioning

- Clear for external clients.
- Easy to document.
- Easy to route and monitor.
- Simple deprecation tracking.

Three decision factors

1. Client simplicity.
2. Operational visibility.
3. Type of change: breaking vs non-breaking.

3.7 Deprecating an old API version

Safe process

1. Announce deprecation.

2. Add headers:

```
Deprecation: true
Sunset: <date>
Link: </api/v2/docs>; rel="successor-version"
```

1. Monitor usage by client/API key.

2. Provide migration guide.

3. Keep old version running during migration window.

4. Sunset only after usage is low and clients are migrated.

3.8 Client SDK design

SDK responsibilities

- Authentication.
- Timeouts.
- Retries.
- Pagination.
- Error mapping.
- API version compatibility.

Example

```
client = UrlShortenerClient(api_key="...", base_url="https://api.example.com")

client.create_url(long_url, expiry=None, custom_alias=None)
client.get_url(short_code)
client.list_urls(limit=50, cursor=None)
client.delete_url(short_code)
```

Custom errors

```
class RateLimitError(Exception): pass
class AuthenticationError(Exception): pass
class ValidationError(Exception): pass
class NotFoundError(Exception): pass
```

4. Django and ORM Performance

4.1 Avoiding N+1 queries in nested serialization

Use `select_related`

For single-valued relationships:

```
queryset = Order.objects.select_related("customer")
```

Use `prefetch_related`

For many-to-many or reverse FK relationships:

```
queryset = Order.objects.prefetch_related("items")
```

Use Prefetch for nested optimization

```
from django.db.models import Prefetch

queryset = Order.objects.prefetch_related(
    Prefetch("items", queryset=OrderItem.objects.select_related("product"))
)
```

Signals of N+1

- Query count grows with row count.
- Same SQL repeated many times.
- Django Debug Toolbar shows repetitive queries.
- APM shows DB time dominating request latency.

4.2 Structuring large list endpoints with optional filters

Recommended pattern

Use a query builder or custom manager instead of putting all logic in the view.

```
def build_order_queryset(params):
    qs = Order.objects.all()
    qs = apply_base_selects(qs)
    qs = apply_filters(qs, params)
    qs = apply_annotations(qs, params)
    qs = apply_sorting(qs, params)
    return qs
```

Efficient annotations

```
from django.db.models import Exists, OuterRef, Count, Q

def apply_annotations(qs, params):
    recent_payment = Payment.objects.filter(
        order_id=OuterRef("pk"),
        created_at__gte=params.get("recent_since"),
    )

    return qs.annotate(
        has_recent_payment=Exists(recent_payment),
        failed_payment_count=Count(
            "payments",
            filter=Q(payments__status="failed"),
        ),
    )
```

Why this helps

- Readable.
- Testable.
- Easier to optimize.
- Avoids one giant view function.

4.3 RESTful filtering and sorting with validation

Query parameter example

```
GET /api/orders?status=paid&customer_id=123&created_after=2026-01-01&sort=-created_at&limit=50
```


DRF validation example

```
class OrderListParamsSerializer(serializers.Serializer):
    status = serializers.ChoiceField(
        choices=["pending", "paid", "failed"],
        required=False,
    )
    customer_id = serializers.IntegerField(required=False)
    created_after = serializers.DateField(required=False)
    sort = serializers.ChoiceField(
        choices=["created_at", "-created_at", "amount", "-amount"],
        required=False,
    )
```

Safe sort mapping

```
SORT_MAP = {
    "created_at": "created_at",
    "-created_at": "-created_at",
    "amount": "total_amount",
    "-amount": "-total_amount",
}

order_by = SORT_MAP[validated_data.get("sort", "-created_at")]
qs = qs.order_by(order_by)
```

Avoid

```
qs.order_by(request.GET["sort"])
```

Raw user input should not be passed directly into query construction.

5. Databases and Data Modeling

5.1 Primary key vs unique constraint

Primary key

- Main identifier for a row.
- Cannot be null.
- One primary key per table.
- Common target for foreign keys.

Unique constraint

- Enforces business uniqueness.
- Multiple allowed per table.
- Null behavior depends on database.

Example

```
CREATE TABLE users (
    id BIGINT PRIMARY KEY,
    email VARCHAR(255) UNIQUE
);
```

5.2 Many-to-many modeling

Join table example

```
CREATE TABLE users (  
    id BIGINT PRIMARY KEY  
);  
  
CREATE TABLE roles (  
    id BIGINT PRIMARY KEY  
);  
  
CREATE TABLE user_roles (  
    user_id BIGINT NOT NULL,  
    role_id BIGINT NOT NULL,  
    PRIMARY KEY (user_id, role_id),  
    FOREIGN KEY (user_id) REFERENCES users(id),  
    FOREIGN KEY (role_id) REFERENCES roles(id)  
);
```

Add metadata if needed

```
CREATE TABLE user_roles (  
    id BIGINT PRIMARY KEY,  
    user_id BIGINT NOT NULL,  
    role_id BIGINT NOT NULL,  
    assigned_by BIGINT,  
    created_at TIMESTAMP NOT NULL,  
    UNIQUE (user_id, role_id)  
);
```

5.3 MongoDB embedding vs referencing

Embed when

- Data is read together.
- Child belongs to parent.
- Data is small and bounded.
- Updates happen together.

Example:

```
{  
  "order_id": "123",  
  "items": [  
    {"sku": "A1", "quantity": 2},  
    {"sku": "B2", "quantity": 1}  
  ]  
}
```

Reference when

- Data is large or unbounded.
- Data is shared.
- Data updates independently.
- Independent querying/indexing is needed.

5.4 INNER JOIN vs LEFT JOIN

INNER JOIN

Returns only rows with matches in both tables.

```
SELECT *  
FROM orders o  
INNER JOIN customers c  
ON o.customer_id = c.id;
```

LEFT JOIN

Returns all left table rows and nulls for missing right-side matches.

```
SELECT *  
FROM orders o  
LEFT JOIN customers c  
ON o.customer_id = c.id;
```

Expected result if no right-side match

- INNER JOIN: row excluded.
- LEFT JOIN: row included with NULL right-side columns.

5.5 Strong consistency vs eventual consistency

Strong consistency when

- Duplicate order creation is unacceptable.
- Money movement is involved.
- Inventory limits matter.
- Permissions/security decisions are involved.
- Stale data violates business rules.

Eventual consistency when

- Analytics counters.
- Notifications.
- Search indexes.
- Activity feeds.
- Reporting dashboards.

Signal

If stale/conflicting data breaks an invariant or causes financial/security risk, use strong consistency. If the system can correct itself later and the user can tolerate delay, eventual consistency is acceptable.

5.6 Preventing duplicate processing in eventual consistency

Techniques

- Idempotency key/event ID.
- Inbox pattern.
- Outbox pattern.
- Unique constraints.
- Idempotent state transitions.

Inbox pattern

Consumer receives event -> inserts event_id into inbox table -> processes only if insert succeeds.

Outbox pattern

Business update + outbox event written in same DB transaction -> background publisher sends event.

Transaction handling

```
Begin transaction
  Insert inbox event
  Apply business update
  Mark inbox event processed
Commit
```

If transaction fails, retry starts cleanly.

5.7 Optimizing equality queries without an index

Techniques

- Check execution plan.
- Filter early.
- Avoid functions around filtered columns.
- Select only needed columns.
- Partition/cluster/sort data around common filters.
- Precompute smaller tables for frequent queries.

Example

```
EXPLAIN SELECT id, status, created_at
FROM orders
WHERE customer_id = 123;
```

DuckDB-specific note

Sorting or clustering data can help zone-map pruning skip row groups even without a traditional index.

3. SQL Interview Questions

3.1 Successful Revenue by Region

Interview Question

"How do you calculate successful revenue by region?"

```
SELECT
  region,
  SUM(amount) AS total_revenue,
  COUNT(*) AS successful_transactions
FROM transactions
WHERE status = 'SUCCESS'
GROUP BY region
ORDER BY total_revenue DESC;
```

Explanation

- WHERE status = 'SUCCESS' filters only successful transactions.
- GROUP BY region groups revenue by region.
- SUM(amount) calculates revenue.
- COUNT(*) counts successful transactions.

3.2 Top One Transaction Per Customer

Interview Question

"For each customer, find the top one transaction."

If **top** means highest transaction amount:

```
SELECT
    transaction_id,
    customer_id,
    amount,
    transaction_date
FROM (
    SELECT
        transaction_id,
        customer_id,
        amount,
        transaction_date,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY amount DESC
        ) AS rn
    FROM transactions
) t
WHERE rn = 1;
```

Explanation

SQL Part	Meaning
PARTITION BY customer_id	Creates separate ranking per customer
ORDER BY amount DESC	Highest transaction comes first
ROW_NUMBER()	Assigns unique rank within each customer
WHERE rn = 1	Selects top transaction per customer

Interview Answer

"I would use ROW_NUMBER() as a window function. I partition by customer_id, order by amount descending, and select only rows where row number is 1. This gives exactly one top transaction per customer."

3.3 Top Transaction With Ties

Follow-Up Question

"What if two transactions have the same highest amount?"

Use RANK() instead of ROW_NUMBER():

```
SELECT
    transaction_id,
    customer_id,
    amount
FROM (
    SELECT
        transaction_id,
```

```

        customer_id,
        amount,
        RANK() OVER (
            PARTITION BY customer_id
            ORDER BY amount DESC
        ) AS rnk
    FROM transactions
) t
WHERE rnk = 1;

```

Difference

Function	Behavior
ROW_NUMBER()	Returns exactly one row per customer
RANK()	Returns all tied top rows, but leaves rank gaps
DENSE_RANK()	Returns all tied top rows, no rank gaps

3.4 Latest Transaction Per Customer

If **top** means most recent transaction:

```

SELECT
    transaction_id,
    customer_id,
    transaction_date,
    amount
FROM (
    SELECT
        transaction_id,
        customer_id,
        transaction_date,
        amount,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY transaction_date DESC
        ) AS rn
    FROM transactions
) t
WHERE rn = 1;

```

3.5 Customers With More Than One Paid Transaction

Interview Question

"Find customers with more than one paid transaction only."

```

SELECT
    customer_id,
    COUNT(*) AS paid_transaction_count
FROM transactions
WHERE status = 'PAID'
GROUP BY customer_id
HAVING COUNT(*) > 1;

```

If the dataset uses SUCCESS instead of PAID:

```

SELECT
    customer_id,
    COUNT(*) AS successful_transaction_count
FROM transactions
WHERE status = 'SUCCESS'
GROUP BY customer_id
HAVING COUNT(*) > 1;

```

Why HAVING?

- WHERE filters rows before aggregation.
- HAVING filters grouped/aggregated results.

If They Want Full Transaction Details

```
SELECT *
FROM transactions
WHERE status = 'PAID'
  AND customer_id IN (
    SELECT customer_id
    FROM transactions
    WHERE status = 'PAID'
    GROUP BY customer_id
    HAVING COUNT(*) > 1
  );
```

Interview Answer

"I would first filter paid transactions using WHERE status = 'PAID', then group by customer and use HAVING COUNT(*) > 1 because HAVING is used to filter aggregate results."

6. Databases, ORMs & PostgreSQL

Likely Questions

- What is relational data modeling?
 - How do you design tables for orders/customers/payments?
 - What is normalization?
 - What are indexes?
 - What is a foreign key?
 - How do you use SQLAlchemy or Django ORM?
 - How do you avoid N+1 queries?
 - How do transactions work?
 - How do you handle migrations?
 - How do you optimize slow queries?
-

Basic Relational Model Example

```
CREATE TABLE customers (  
    id SERIAL PRIMARY KEY,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    name VARCHAR(255) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE orders (  
    id SERIAL PRIMARY KEY,  
    customer_id INTEGER NOT NULL REFERENCES customers(id),  
    status VARCHAR(50) NOT NULL,  
    total_amount NUMERIC(10, 2) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE payments (  
    id SERIAL PRIMARY KEY,  
    order_id INTEGER NOT NULL REFERENCES orders(id),  
    amount NUMERIC(10, 2) NOT NULL,  
    status VARCHAR(50) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Interview Explanation

- customers owns customer data.
- orders references customers.
- payments references orders.
- Foreign keys enforce data integrity.
- Separate tables reduce duplication and improve consistency.

SQLAlchemy Model Example

```
from sqlalchemy import Column, Integer, String, Numeric, ForeignKey  
from sqlalchemy.orm import relationship, declarative_base  
  
Base = declarative_base()  
  
class Customer(Base):  
    __tablename__ = "customers"  
  
    id = Column(Integer, primary_key=True)  
    email = Column(String, unique=True, nullable=False)  
    name = Column(String, nullable=False)  
  
    orders = relationship("Order", back_populates="customer")  
  
class Order(Base):  
    __tablename__ = "orders"  
  
    id = Column(Integer, primary_key=True)  
    customer_id = Column(Integer, ForeignKey("customers.id"), nullable=False)  
    status = Column(String, nullable=False)  
    total_amount = Column(Numeric(10, 2), nullable=False)  
  
    customer = relationship("Customer", back_populates="orders")
```

Avoiding N+1 Queries

Problem

If you load 100 orders and then separately query the customer for each order, you may execute 101 queries.

SQLAlchemy Fix

```
from sqlalchemy.orm import joinedload

orders = (
    session.query(Order)
    .options(joinedload(Order.customer))
    .all()
)
```

Interview Explanation

Use eager loading when related data is needed upfront.

Index Example

```
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_orders_status_created_at ON orders(status, created_at);
```

When to Add Indexes

Add indexes on:

- Foreign keys
- Frequently filtered columns
- Frequently sorted columns
- Columns used in joins

Tradeoff

Indexes speed up reads but slow down writes and take storage.

Transaction Example

```
def create_order_with_payment(session, order_data, payment_data):
    try:
        order = Order(**order_data)
        session.add(order)
        session.flush()

        payment = Payment(order_id=order.id, **payment_data)
        session.add(payment)

        session.commit()
        return order

    except Exception:
        session.rollback()
        raise
```

Interview Explanation

Use transactions when multiple database changes must either all succeed or all fail.

Relational Versus NoSQL Databases and ACID

Relational Databases

Relational databases such as PostgreSQL and MySQL are a strong fit when the application needs structured relationships, transactions, joins, constraints, and strong consistency.

Typical strengths:

- Primary and foreign keys.
- Referential integrity.
- Transactions.
- Complex joins and reporting.
- Mature indexing and query optimization.

NoSQL Databases

NoSQL is a broad category rather than one database model. Common forms include document stores, key-value stores, wide-column databases, and graph databases.

Use a NoSQL database when its access pattern or data model provides a concrete advantage, for example:

- Flexible document-shaped data.
- Extremely high-scale key-based access.
- Specialized graph relationships.
- Simple horizontally distributed lookups.

The choice should be based on access patterns, consistency needs, relationships, scale, and operational requirements rather than choosing SQL or NoSQL by default.

ACID Transactions

Property	Meaning
Atomicity	All operations in a transaction succeed or all are rolled back.
Consistency	A committed transaction preserves database rules and invariants.
Isolation	Concurrent transactions do not expose unsafe intermediate state to one another.
Durability	Once committed, data survives process or machine failures according to the database durability guarantees.

Example transfer:

```
BEGIN;
```

```
UPDATE accounts  
SET balance = balance - 100  
WHERE id = 1;
```

```
UPDATE accounts  
SET balance = balance + 100  
WHERE id = 2;
```

```
COMMIT;
```

If the second update cannot complete, the transaction should roll back rather than leaving only the debit applied.

Database Ownership in Microservices

Prefer each service to own its persistence boundary. Other services should use an API, event, or replicated read model instead of directly querying another service's private tables. This reduces coupling but introduces eventual-consistency and distributed-workflow considerations.

Interview answer:

I prefer a relational database when relationships, transactions, constraints, and reporting are central to the workload. I consider NoSQL when a document, key-value, graph, or horizontally distributed access pattern fits better. In microservices I also try to keep clear data ownership so one service does not depend directly on another service's internal schema.

Backend Persistence & Concurrency Interview Patterns

One Database Session / Transaction Boundary per Request

For typical request/response APIs, use one scoped database session/transaction context per request rather than one global mutable session shared across concurrent requests.

Conceptual flow:

```
request
open/request DB session
read + validate
perform atomic writes
commit
close session
```

On failure:

```
exception
rollback
return mapped error
```

Long-running external work should generally not hold a database transaction open for the entire network call or user interaction.

Constraints Are Part of Correctness

Application checks improve error messages but do not replace database constraints under concurrency.

Example subscription uniqueness:

```
CREATE UNIQUE INDEX uq_subscription_user_podcast
ON subscription(user_id, podcast_id);
```

Even if two concurrent API requests both execute:

```
SELECT -> no existing subscription
```

only one should be allowed to create the unique pair.

Useful constraints:

- Primary keys
- Unique keys
- Foreign keys
- NOT NULL
- Check constraints

Strong interview line:

I use application validation for usability and domain rules, but I keep invariants that must survive concurrency enforced at the authoritative database boundary as well.

Composite Indexes from Access Patterns

Example feed query:

```
SELECT *
FROM episode
WHERE podcast_id = :podcast_id
ORDER BY published_at DESC, episode_id DESC
LIMIT 50;
```

Useful index:

```
CREATE INDEX idx_episode_podcast_published
ON episode(podcast_id, published_at DESC, episode_id DESC);
```

Why column order matters:

The leading columns should match the high-value filtering/order pattern. A composite index is not equivalent to having independent single-column indexes for every query.

Trade-off:

- Faster qualifying reads
- More storage
- More write/update maintenance

Optimistic Concurrency with a Version Column

Schema:

```
id
state
version
```

Conditional update:

```
UPDATE job
SET state = 'RUNNING',
    version = version + 1
WHERE id = :job_id
    AND state = 'QUEUED'
    AND version = :expected_version;
```

Result:

```
1 row updated -> caller won
0 rows updated -> state/version changed; handle conflict
```

This is useful when conflicts are possible but not common enough to justify holding pessimistic locks across normal reads.

Conditional Allocation for Scarce Resources

Do not implement scarce-resource ownership as:

```
SELECT available  
then later UPDATE
```

without a concurrency guard.

Prefer one authoritative conditional write or a short transaction/lock:

```
UPDATE seat  
SET status = 'HELD', held_by = :user_id  
WHERE seat_id = :seat_id  
  AND status = 'AVAILABLE';
```

Affected-row count becomes the concurrency result.

This same pattern applies to:

- Inventory
- Driver assignment
- Job leasing
- Quota claims
- One-time tokens

Idempotent Ingestion with Source IDs

When a scraper/partner can retry the same logical event, store a stable source identity:

```
(podcast_id, source_episode_id)
```

and enforce uniqueness:

```
UNIQUE(podcast_id, source_episode_id)
```

A retry can then safely return/reuse the existing record rather than creating a duplicate.

The same principle applies to payment-provider event IDs, webhook IDs, and external order IDs.

Transactional Outbox Schema

Example tables:

```
business_table  
-----  
id  
state  
...  
  
outbox  
-----  
event_id  
aggregate_type  
aggregate_id  
event_type  
payload  
created_at  
published_at nullable
```

Write in one DB transaction:

```
BEGIN;  
  
INSERT/UPDATE business_table ...;  
INSERT INTO outbox(...);
```

```
COMMIT;
```

A separate publisher reads unpublished outbox rows and sends them to the broker. Marking an event published must tolerate retries, and downstream consumers should be idempotent.

Read Replicas and Read-After-Write

In a single-primary topology:

```
writes -> primary  
eligible reads -> replicas
```

Asynchronous replication can produce:

```
write committed on primary  
  
immediate read on replica  
  
old value
```

For workflows requiring read-after-write consistency:

- Read from primary for a short session/window.
- Route based on a replication position/token where supported.
- Keep critical ownership/permission checks on the authoritative path.

Do not assume every query needs strong consistency; define it per workflow.

SQL vs NoSQL Interview Decision

Ask:

```
What are the relationships?  
What transactions/constraints are required?  
What are the dominant access patterns?  
How stable is the schema?  
What write/read scale is expected?  
What consistency is required?
```

Relational databases are often strong for:

- Orders/payments
- Subscriptions
- User-resource ownership
- Workflow state
- Control-plane metadata

Specialized NoSQL stores can be strong for:

- Very high-scale simple key access
- Flexible documents
- Time-series/telemetry
- Geo/index-oriented access

A production architecture may use both, but every store adds operational and consistency complexity.

Database Interview Quick Questions

Why PostgreSQL for a subscription/feed control plane?

Because users, subscriptions, podcasts, and episodes have clear relationships, uniqueness constraints, and transactional invariants. Read-heavy scale can first be addressed with indexes, caching, and replicas before assuming a different database model is necessary.

Why not cache the final allocation decision?

Caches can be stale and usually do not provide the same transactional invariant as the authoritative database. Use cache for discovery/read acceleration; use authoritative conditional state for ownership.

What is an N+1 query problem?

One parent query triggers one additional query per returned record, for example one query for 100 users plus 100 separate queries for each user's data. Fix with eager loading, batching, joins, or a deliberately designed aggregate query/read model depending on ownership boundaries.

What does a connection pool solve?

It reuses a bounded set of database connections instead of creating a new physical connection for every request. Pool size must respect database capacity; scaling app instances without adjusting aggregate connection demand can exhaust the database.

How do you debug a slow query?

- Identify the exact SQL and frequency.
- Examine execution plan (EXPLAIN/EXPLAIN ANALYZE).
- Check scans, join strategy, row estimates, sorts, lock waits.
- Verify indexes match predicates/order.
- Reduce unnecessary selected data.
- Check N+1 and repeated queries.
- Measure before/after rather than adding indexes blindly.

Quick Persistence Revision Card

Session/transaction scoped to request
Rollback on failure
DB constraints enforce concurrency invariants
Composite index follows access pattern
Optimistic version/conditional UPDATE
Unique source IDs for idempotent ingestion
Transactional outbox
Single-primary replicas + lag/read-after-write
Connection-pool capacity
SQL vs NoSQL from relationships/invariants/access patterns

System Design, Microservices & Distributed Systems

Source: docs/system_design.md

Purpose: Scalability, microservices, distributed architecture, service communication, queues, caching, feature flags, and production trade-offs. **Use this file for:** system design interviews and senior backend engineering rounds

Recommended Study Flow

1. Read the **Quick Summary** first.
2. Review the **Key Concepts** and tables.
3. Practice the **Interview Questions & Answers** out loud.
4. Use the code snippets and examples to explain trade-offs clearly.
5. Finish with the **Common Mistakes** and **Revision Checklist** sections.

Quick Summary

This file has been refreshed to keep the original repository topic while merging relevant detailed Q&A from the consolidated topic-wise interview-prep pack. Use the top sections for fast revision and the consolidated section for deeper interview preparation.

Core Topics to Master

- Horizontal vs vertical scaling
- Load balancing and caching
- Database replication and sharding
- Message queues and async processing
- Microservices and database-per-service
- Feature flags and safe rollouts
- Reliability, observability, and incident response
- URL shortener, chat system, rate limiter, and other design patterns

Consolidated Interview Questions & Technical Notes

Microservices, distributed architecture, service communication, queues, Celery, Redis, RabbitMQ, feature flags, architecture trade-offs, and domain system design.

2. Backend and Microservices Architecture

2.1 Pros and cons of microservices

Pros

- Independent deployment.
- Independent scaling.
- Clear service ownership.
- Technology flexibility.
- Fault isolation when designed well.

Cons

- Distributed system complexity.
- More network latency.
- Harder debugging and tracing.
- Data consistency challenges.
- Deployment and observability overhead.

Interview wording

Microservices help with scale and independent ownership, but the trade-off is operational complexity. You need strong observability, API contracts, CI/CD, and clear ownership to make them work well.

2.2 Database ownership in microservices

Topic covered

How services handle data when one service needs user information while another owns transactions.

Best practice

Each service should own its database. Other services should access data through APIs/events/read models rather than directly joining across databases.

Patterns

- Service API call for real-time lookup.
- Cached read model.
- Event-driven replication.
- API composition/BFF layer.
- Avoid shared database coupling.

Example

Transaction Service -> needs user risk tier

User Service -> owns user profile

Solution -> Transaction Service calls User Service API or reads a replicated user summary table.

2.3 Service communication patterns

Synchronous communication

Use when immediate response is required.

Examples:

- Authorization check.
- Payment validation.
- User lookup needed immediately.

Asynchronous communication

Use when work can happen later.

Examples:

- Notifications.
- Analytics.
- Audit events.
- Email dispatch.

Interview wording

I keep critical request-path operations synchronous and move non-critical operations async to reduce latency and improve resilience.

2.4 API Gateway and BFF/API composition

Question covered

How to expose one public API without coupling clients to each microservice.

Recommended design

Client -> API Gateway -> BFF/API Composition Layer -> Internal Microservices

API Gateway handles

- Authentication.
- TLS termination.
- Rate limiting.
- Routing.
- Request limits.
- Basic observability.

BFF/composition layer handles

- Aggregating multiple service calls.
- Shaping responses for web/mobile/partners.
- Hiding internal service boundaries.

Interview wording

I use API Gateway for edge concerns and a BFF/composition layer for orchestration. That keeps clients decoupled from internal microservice changes.

6. Caching, Rate Limiting, Abuse Prevention, and Feature Flags

6.1 Cache consistency when target URL changes

Approach

- Treat DB as source of truth.
- Update DB first.
- Invalidate or update cache immediately after commit.
- Use versioned cache keys or short TTLs for safety.

Example

Update destination URL in DB -> delete Redis key short_code:abc -> next redirect reloads fresh value.

6.2 Preventing cache stampede

Techniques

- Distributed lock using Redis SETNX.
- Stale-while-revalidate.
- Jittered TTL.
- Request coalescing.

Interview wording

Only one request should rebuild the cache; others should wait briefly or serve stale data rather than all hitting the database.

6.3 Graceful degradation if cache is unavailable

Design

- Cache is optimization, not source of truth.
- Short cache timeout.
- Fallback to DB.
- Circuit breaker for cache failures.
- Protect DB with backpressure/rate limiting.
- Local in-memory LRU cache for very hot keys.

6.4 Rate limiting URL/order creation endpoints

Best approach

Use Redis-backed distributed rate limiting at gateway/middleware.

Dimensions

- Per IP.
- Per user.
- Per API key.
- Per tenant/org.

Algorithms

- Token bucket.
- Sliding window.

Response

429 Too Many Requests
Retry-After: 60

6.5 Protecting against automated abuse beyond rate limits

Techniques

- API keys/authentication.
- Bot detection.
- CAPTCHA/challenge for suspicious traffic.
- URL reputation checks.
- Malware/phishing domain scanning.
- Behavior-based anomaly detection.
- Progressive enforcement.
- Async moderation pipeline.

6.6 Feature flag system for safe releases

Design principles

- Separate deployment from release.
- Centralized flag config.
- Local in-memory evaluation.
- Background refresh.
- Percentage rollout.
- Tenant/user/region targeting.
- Sticky assignment using hashing.
- Audit all changes.
- Safe default if flag service fails.

Example rollout

internal users -> 1% -> 5% -> 25% -> 50% -> 100%

6.7 Fast and reliable runtime flag checks

Best practice

Evaluate flags locally from memory, not by calling the flag service on every request.

Why

- Lower latency.
 - Higher reliability.
 - No network dependency in hot path.
-

7. Distributed Systems: Celery, Redis, RabbitMQ

Likely Questions

- What are background jobs?
 - Why use Celery?
 - What is Redis used for?
 - What is RabbitMQ used for?
 - How do you handle retries?
 - How do you avoid duplicate job execution?
 - How do you design async processing for slow tasks?
 - What happens if a worker crashes?
 - How do you monitor background jobs?
-

Why Use Background Jobs?

Use background jobs for work that should not block the API response:

- Sending emails
 - Generating reports
 - Syncing with third-party APIs
 - Processing uploads
 - Running scheduled tasks
 - Retrying failed integrations
-

Celery Example

```
from celery import Celery

celery_app = Celery(
    "tasks",
    broker="redis://localhost:6379/0",
    backend="redis://localhost:6379/1",
)

@celery_app.task(bind=True, max_retries=3)
def sync_order_to_partner(self, order_id: int):
    try:
        response = external_partner_api.sync_order(order_id)
        return response

    except TemporaryAPIError as exc:
        raise self.retry(exc=exc, countdown=60)
```

Interview Explanation

This shows:

- Redis as broker/backend
- Async background execution
- Retry handling
- Protection from temporary external API failures

Redis Use Cases

Redis can be used for:

- Caching
- Rate limiting
- Distributed locks
- Session storage
- Celery broker
- Temporary counters
- Idempotency keys

Redis Cache Example

```
import json
import redis

cache = redis.Redis(host="localhost", port=6379, decode_responses=True)

def get_customer_profile(customer_id: int):
    key = f"customer:{customer_id}"

    cached = cache.get(key)
    if cached:
        return json.loads(cached)

    profile = database.fetch_customer(customer_id)
    cache.setex(key, 300, json.dumps(profile))

    return profile
```

Distributed Lock Example

```
def process_order(order_id: int):
    lock_key = f"lock:order:{order_id}"

    lock_acquired = cache.set(lock_key, "1", nx=True, ex=60)

    if not lock_acquired:
        return "Order is already being processed"

    try:
        # process order safely
        return "processed"
    finally:
        cache.delete(lock_key)
```

Interview Explanation

Distributed locks prevent multiple workers from processing the same item at the same time.

16. Code Reviews & Architecture Discussions

Likely Questions

- How do you approach code reviews?
 - How do you give feedback?
 - How do you receive feedback?
 - How do you make architecture tradeoffs?
 - How do you handle disagreement?
 - How do you improve legacy code safely?
-

Code Review Checklist

Check for:

- Correctness
 - Readability
 - Simplicity
 - Tests
 - Error handling
 - Security
 - Performance
 - Database impact
 - Backward compatibility
 - Observability
 - Maintainability
-

Good Code Review Comment Example

This works, but I wonder if we should move this validation closer to the service layer. That would keep the API handler thinner and make the rule easier to reuse from background jobs.

Why This Is Good

- Specific
- Respectful
- Explains reasoning
- Suggests improvement
- Does not attack the person

Refactoring Legacy Code Safely

Good strategy:

1. Add tests around current behavior
2. Identify risky areas
3. Refactor in small steps
4. Keep external behavior stable
5. Use feature flags if needed
6. Monitor after release

17. E-Commerce / Logistics / Fintech Domain Topics

These are general domain concepts that may come up in product engineering interviews for commerce, logistics, payments, or ERP-style platforms.

Likely Questions

- How would you design an order management workflow?
- How do you handle inventory updates?
- How do you prevent duplicate payments?
- How do you handle third-party shipping integrations?
- How do you reconcile financial transactions?
- How do you handle webhook retries?
- How do you design for eventual consistency?

Order Lifecycle Example

created

paid

confirmed

packed

shipped

delivered

Possible exception states:

cancelled

refunded


```
payment_failed
shipment_failed
returned
```

Webhook Handling Example

```
@app.post("/webhooks/shipping")
def handle_shipping_webhook(payload: dict, idempotency_key: str):
    if webhook_already_processed(idempotency_key):
        return {"status": "already_processed"}

    validate_signature(payload)
    save_webhook_event(payload, idempotency_key)
    process_shipping_update.delay(payload)

    return {"status": "accepted"}
```

Interview Explanation

Webhook handlers should:

- Validate signatures
- Be idempotent
- Respond quickly
- Process heavy work asynchronously
- Store raw event for audit/debugging
- Retry safely

Inventory Race Condition Example

Problem

Two users buy the last item at the same time.

Solution Ideas

- Database row locking
- Atomic update
- Reservation system
- Idempotent order processing

```
UPDATE inventory
SET quantity = quantity - 1
WHERE sku = 'ABC123'
  AND quantity > 0;
```

Then check affected rows. If zero rows were updated, inventory was unavailable.

Payment Idempotency

Payment systems should prevent duplicate charges when clients retry.

Client sends payment request with idempotency key

Server checks if key already exists

If yes, return previous result

If no, process payment and store result

AI-Enabled Backend Versus Agentic Architecture

AI integration does not automatically make a product agentic. The distinction affects architecture, testing, cost, and operational risk.

Architecture	Control Flow	Typical Use Case
AI-enabled	Application code chooses when and how to call AI	Summarization, extraction, classification
Workflow-based AI	Predetermined sequence of model and service steps	RAG, document processing, content generation
Agentic AI	Model dynamically chooses actions or tools	Multi-step investigation and task completion

AI-Enabled Product Flow

```
API request
-> deterministic validation
-> domain service
-> optional model call
-> output validation
-> persistence
-> API response
```

The application remains responsible for control flow. The model provides a bounded capability inside a normal backend system.

Agentic Product Flow

```
User goal
-> planner/model
-> select tool
-> execute tool
-> observe result
-> repeat or finish
```

This requires stronger controls around tool permissions, iteration limits, state, audit trails, and recovery from partial execution.

Design Principle: Preserve a Stable Backend Core

Keep domain rules independent of the model:

```
Routes -> Application Services -> Domain Rules
                                   -> Model Adapter
                                   -> Repositories
                                   -> External Services
```

Benefits:

- Model vendors can be replaced without rewriting routes.
- Deterministic business rules remain testable without an LLM.
- Model failures can use fallback or degraded behavior.
- Cost, latency, and token usage can be measured at one boundary.
- AI can be added gradually rather than redesigning the whole platform.

Backend Engineering Skills That Transfer Across AI Systems

The domain may change, but the production concerns remain familiar:

- API contracts and backward compatibility.
- Authentication, authorization, and data protection.
- Database consistency and idempotency.
- Retries, timeouts, queues, and failure isolation.
- Horizontal scaling and stateless service design.
- Structured logs, metrics, traces, and alerts.
- Deployment safety and rollback.

The added AI concerns are model quality, prompt/version management, retrieval quality, nondeterminism, token cost, and hallucination risk.

Architecture Tradeoff Answer

I would start with an AI-enabled deterministic workflow unless the use case genuinely requires dynamic planning. It is easier to test, operate, secure, and explain. I would introduce agentic behavior only where flexible tool selection creates measurable value, and I would bound it with explicit permissions, step limits, validation, and audit logging.

Monolith Versus Microservices: Decision Framework

A monolith packages most business capabilities into one deployable application. Microservices split capabilities into smaller services that can be deployed and scaled independently. Neither is automatically better; the right choice depends on team size, domain boundaries, scaling needs, and operational maturity.

Area	Monolith / Modular Monolith	Microservices
Deployment	One deployable unit	Independent service deployments
Scaling	Scale the whole application	Scale only the constrained service
Communication	In-process calls	Network calls, events, or messaging
Data	Often one database/schema	Prefer service-owned data
Debugging	Easier local tracing	Distributed tracing is important
Failure model	Larger shared blast radius	Better isolation when designed well
Team overhead	Lower	Higher operational and ownership overhead
Best fit	Small/medium team, evolving domain	Clear boundaries, independent scale/release needs

When to Start With a Modular Monolith

Prefer a modular monolith when:

- The product or domain is still changing rapidly.
- One team owns most of the application.
- Independent scaling is not yet necessary.
- Operational simplicity is more valuable than deployment independence.
- Clear internal modules can preserve boundaries without network calls.

When Microservices Become Justified

Microservices become more attractive when:

- Different capabilities have materially different scaling requirements.
- Multiple teams need independent ownership and release cycles.
- A failure in one capability should be isolated from the rest.
- The domain has stable service boundaries.
- The organization can support service discovery, observability, CI/CD, and distributed failure handling.

Interview answer:

I would not choose microservices by default. A modular monolith is often the better starting point because it is simpler to build, test, deploy, and debug. I would introduce microservices when there are clear business boundaries, independent scaling or release requirements, or multiple teams that need separate ownership. The benefit is deployment and scaling flexibility, but the trade-off is distributed-system complexity around networking, data consistency, observability, and operations.

Async I/O Versus Background Processing

async/await and background processing solve different problems.

Async I/O Inside a Request

Use async/await when the request still needs the result, but the work spends time waiting on I/O such as:

- External APIs.
- Databases.
- LLM providers.
- Vector databases.
- Object storage.

```
Request
-> async API handler
-> await external service
-> event loop handles other requests while waiting
-> return response
```

This improves concurrency, but the original HTTP request is still open.

Background Processing Outside the Request

Use a queue and worker when the work is long-running, retryable, or does not need to finish before the HTTP response.

```
Client
-> POST /reports
-> API creates job and returns 202 + job_id
-> queue
-> background worker
-> result store / database
-> client polls or receives notification
```

Typical use cases:

- Large document processing.
- Embedding generation.
- Email and notification delivery.
- Report generation.
- Batch data processing.
- Long-running AI workflows.

Production Concerns for Background Jobs

- Idempotency so retries do not duplicate side effects.
- Bounded retries with exponential backoff.
- Dead-letter queue for repeatedly failing jobs.
- Explicit states such as queued, running, completed, and failed.
- Timeouts and cancellation.
- Queue-depth and worker-latency monitoring.
- Handling at-least-once delivery and possible duplicate messages.

Interview answer:

Async I/O helps a service handle many concurrent I/O-bound requests while each request remains active. Background processing moves work out of the request-response cycle entirely. For a quick database or LLM call I may use `async/await`; for a multi-minute report, ingestion pipeline, or retryable workflow I would return 202 Accepted, enqueue a job, and process it with a worker.

Capacity Planning, Load Testing & Scaling Under Unexpected Demand

Capacity planning should model the work the system performs, not only the number of requests entering the API. One customer action may fan out into several service calls, database queries, cache lookups, or background jobs, so top-level request count can underestimate the effective workload.

Build the Capacity Model from the End-to-End Request Path

Start with business demand and translate it into technical load:

expected users / transactions

peak concurrency and requests per second

request fan-out and downstream operations

CPU, memory, DB connections, I/O, queue work

required capacity + growth/failure headroom

Important inputs include:

- Sustained peak traffic, not only daily averages.
- Burstiness and concurrency.
- Per-request CPU and memory cost.
- Database operations and connection usage.
- Calls to downstream services.
- Retry amplification during partial failures.
- Background work generated by foreground requests.
- Expected growth.
- Capacity needed when an instance or availability zone is unavailable.

Detecting an Underestimated Workload

A production-like load test should increase traffic progressively and compare incoming traffic with internal resource consumption.

Monitor:

- Throughput/RPS.
- p50, p95, and p99 latency.
- Error and timeout rates.
- CPU and memory saturation.
- Pod/instance count and autoscaling events.
- Database query latency and connection-pool usage.
- Queue depth and worker lag.
- Downstream call volume and latency.
- Cache hit ratio.

If effective work grows much faster than incoming RPS, trace representative requests to identify request amplification, fan-out, retries, or unexpectedly expensive data access.

Interview answer:

I would start from expected peak business volume and convert it into concurrency, RPS, database work, and downstream calls. Then I would validate those assumptions with progressive load tests. If the system consumes roughly twice the expected resources, I would compare top-level traffic with traces and downstream operation counts to find amplification, update the model, and retest with growth and failure headroom.

Customer Experience Is Part of the Capacity Test

A system can remain technically "up" while the user experience has already failed. Track the important customer journey, not just aggregate infrastructure health.

For a transactional workflow, measure stage-level latency and success rate for operations such as:

- Loading data required to begin the workflow.
- Availability or eligibility lookup.
- Validation.
- Transaction submission.
- Confirmation.

A common degradation pattern is:

healthy latency
increasing concurrency
p95/p99 latency rises
timeouts and retries appear
retries add more load
error rate or outage

Vertical vs Horizontal Scaling

Strategy	Best Use	Trade-Off
Vertical scaling / scale up	Immediate headroom, stateful systems, short-term recovery	Has hardware limits, larger failure unit, can be costly
Horizontal scaling / scale out	Stateless services and sustained growth	Requires load balancing, distributed coordination, good observability

A practical incident may use both: scale vertically first to restore headroom, then move to a sustainable horizontally scaled design.

Horizontal Scaling Requires Stateless Services

Horizontal scaling works best when any healthy instance can handle any request. Keep shared state in external systems such as databases, Redis, object storage, or durable queues rather than process memory.

Typical architecture:

Clients
Load Balancer
Stateless API replicas <--> Redis/cache
Database / downstream services

Autoscaling should trigger before the service reaches saturation. Useful signals can include CPU, memory, request count, queue depth, and latency, depending on the workload.

Add Caching Selectively

Caching reduces repeated expensive reads, but correctness determines what may be cached.

Good candidates:

- Reference/configuration data.
- Product/catalog-style reads.
- Expensive query results with acceptable staleness.
- Read-heavy external lookups.

Use short TTLs or explicit invalidation where appropriate. Avoid treating the cache as the source of truth for rapidly changing or transaction-critical state unless the consistency model explicitly supports it.

Do Not Move the Bottleneck Downstream

Adding application replicas can overload the database or another dependency. Before scaling the application aggressively, inspect:

- Query plans and indexes.
- Slow or repeated queries.
- Database CPU/I/O.
- Connection-pool limits.
- Per-instance connection counts.
- Lock contention.
- Dependency rate limits.
- Retry behavior.

Useful protections include connection pooling/proxies, per-instance limits, backpressure, read replicas for suitable workloads, asynchronous handling of non-critical work, and graceful degradation.

Choosing the Ideal Long-Term Scaling Strategy

A sound decision sequence is:

1. Measure the actual bottleneck before adding capacity.
2. Use a low-risk short-term mitigation if customers are actively affected.
3. Optimize clearly inefficient work.
4. Keep the application stateless where practical.
5. Prefer horizontal scaling for sustained application-tier growth.
6. Add caching only where staleness is acceptable.
7. Protect databases and downstream dependencies from amplified load.
8. Plan for peak demand plus growth and failure headroom.
9. Retest at expected peak and beyond it.
10. Confirm graceful degradation and recovery, not just maximum throughput.

Interview answer:

My preferred long-term approach is not simply to add CPU. I would identify the actual bottleneck, keep the application stateless, scale the application tier horizontally behind a load balancer, use selective caching for safe read-heavy data, optimize the database and connection pools, and plan enough headroom to survive both peak traffic and a component failure. I would validate the design with progressive load tests and p95/p99 latency, throughput, errors, saturation, database connections, and cache effectiveness.

Design Sequencing, Storage Fit & Messaging Resilience

Several system-design questions become easier when the answer is framed around **what stage of design we are in** and **what access pattern the workload actually needs**.

Functional Requirements vs Non-Functional Requirements

Functional requirements describe **what the system must do**. Non-functional requirements (NFRs) describe **how well or under what constraints it must do it**.

Example:

Functional:
process a payment

Non-functional:
p95 latency < 500 ms
99.99% availability
no loss of acknowledged transactions
10,000 transactions/second
strong authorization/auditability

Common NFR dimensions:

- Scale and throughput.
- Latency (p50/p95/p99 as appropriate).
- Availability and fault tolerance.
- Durability and acceptable data loss.
- Consistency/read-after-write requirements.
- Security and compliance.
- Cost/operational constraints.

A strong interview opening is:

Before choosing components, I want to clarify the functional requirements and the NFRs that will shape the architecture: expected scale, read/write pattern, latency target, availability, durability/consistency, security, and cost.

Learning-platform onboarding example

For an onboarding/learning platform, functional requirements might include signup, interest selection, joining courses/channels, progress tracking, and recommendations. NFRs might include fast content delivery, secure authentication, scalable media delivery, reliable progress persistence, and bounded recommendation latency. The example is useful because the same feature list can lead to very different designs depending on the NFRs.

What Does System Overload Mean?

A system is overloaded when incoming work exceeds the sustainable capacity of one or more resources: application CPU/memory, worker slots, database connections, queue consumers, cache capacity, or a downstream dependency.

incoming load

API tier

DB / queue / downstream service

```
if arrival rate > sustainable service rate
-> queues grow
-> latency rises
-> timeouts/retries amplify load
-> failures cascade
```

A senior response should not be only "add more servers." First identify the bottleneck and protect dependencies.

Useful overload controls:

- Horizontal scaling of stateless application instances.
- Admission control/rate limiting.
- Backpressure and bounded queues.
- Caching where staleness is acceptable.
- Async queues for non-critical/offline work.
- Timeouts and bounded retries with jitter.
- Circuit breaking/fallbacks for unhealthy dependencies.
- Database connection limits, indexing, query optimization, and read scaling where valid.
- Graceful degradation instead of total failure.

Interview answer:

I would first determine which resource is saturated rather than assuming the API tier is the bottleneck. Then I would scale the correct tier and add protection such as rate limiting, backpressure, bounded queues, timeouts, and graceful degradation so overload does not propagate into the database or downstream services.

Sequence Architecture Work by Project Stage

Do not treat every engineering activity as equally urgent on day one.

For a new system whose requirements are still being established:

1. Clarify customer/business requirements and non-functional constraints.
2. Define core use cases and failure cases.
3. Sketch the high-level architecture and critical data flow.
4. Identify the highest-risk assumptions.
5. Build a focused proof of concept only for uncertain/high-risk constructs.
6. Review the proposed architecture with senior engineers and affected teams.
7. Finalize detailed schemas/contracts and implementation plan.
8. Add production dashboards, alarms, runbooks, and operational documentation before launch.

Operational readiness is essential before production, but creating dashboards before the workload, architecture, and service boundaries are understood is usually premature.

Interview answer:

I start by clarifying the problem and non-functional requirements, then sketch the end-to-end architecture and identify the riskiest assumptions. I use a proof of concept to validate those risks rather than building arbitrary production code early. Once the design is coherent, I review it with the relevant engineers, then complete implementation and operational readiness including metrics, alarms, runbooks, and load testing.

Choosing a Data Store by Access Pattern

Consider a short, extremely bursty write window where millions of clients submit and may update a value, and the service must remain available while totals are refreshed frequently.

Technology	Fit	Why
Distributed key-value store	Very strong	Direct keyed reads/writes, horizontal scale, predictable access path
Document database	Strong when each record has evolving document-shaped metadata	Flexible schema but more capability than a simple key lookup may need
Relational database	Viable with careful partitioning/scaling	Strong constraints/transactions, but burst pattern must be designed
Ledger database	Useful for audit history, not necessarily the primary hot path	Append-only traceability is different from low-latency mutable access
Graph database	Poor fit for simple vote/state lookup	Relationship traversal is not the main access pattern
Batch/distributed processor	Analytics/aggregation layer, not transactional source of truth	Processing engine rather than primary request-time storage

A common key design is one record per user/entity combination so an update replaces the previous value idempotently rather than creating duplicate votes/actions.

Images and Thumbnails

Binary images should normally live in object/cloud file storage, not inside the primary relational or search database.

client

CDN / object storage URL

thumbnail object

metadata DB

object key, dimensions, owner, checksum, content type

Why object storage is the default:

- Cheap durable blob storage.
- Horizontal scale.
- CDN integration.
- Lifecycle/retention policies.
- Avoids bloating transactional database pages and backups.

A local filesystem may be acceptable for a prototype or single-host tool, but it is a weak SaaS production choice because instances are replaceable and storage is not naturally shared. Elasticsearch is for search/indexing, not primary binary storage.

High Availability: What Actually Prevents Outage

Availability mechanisms should reduce dependency on manual intervention and remove single-instance capacity limits.

High-value actions include:

- Run multiple stateless instances across failure domains.
- Support horizontal scaling/autoscaling as demand changes.
- Load test expected peak and failure scenarios before launch.
- Use health checks and automatic replacement/failover.
- Externalize regional/environment configuration so instances are replaceable and consistent.
- Add dependency timeouts, backpressure, and bounded retries.

Important distinction:

Backups improve recoverability and durability. They are important, but a daily backup does not by itself keep the live service available when an instance or database endpoint fails.

Similarly, debug-level logging can help diagnosis but does not make the service highly available and can increase cost/noise if enabled globally in production.

Designing a Versioned Message Envelope

For queued messages produced by heterogeneous devices or services, define a stable envelope before optimizing individual payloads.

Useful envelope fields include:

```
message_id
message_type / purpose
schema_version
producer_id
created_at / event_timestamp
correlation_id or trace_id
content_type / encoding
payload or payload_reference
checksum
```

Priorities during design:

1. List the distinct purposes/messages that must be represented.
2. Research/adopt an established serialization/message format where it fits rather than inventing one unnecessarily.
3. Define versioning and backward/forward compatibility rules.
4. Define binary serialization for component payloads where size/latency matters.
5. Define timestamps, checksums, identifiers, and validation rules.
6. Understand queue size limits, delivery semantics, retry behavior, and poison-message handling.
7. Add dashboards after the contract and processing path are clear.

Large Payloads: Keep the Queue Small

Message brokers are optimized for messages, not arbitrary large blobs. A strong default is:

```
producer
  write large payload to object storage
  queue small message containing metadata + object reference + checksum
consumer downloads payload
```

This separates transport metadata from large binary data and avoids broker size limits.

Alternative approaches have trade-offs:

Approach	When It Helps	Trade-Off
Fragment into many messages	Broker-only environments	Requires ordering, completeness detection, retry/reassembly, deduplication
Streaming protocol	Continuous/very large transfer	More protocol/session complexity and backpressure handling
Send during low-volume time	Non-urgent bandwidth-heavy work	Does not solve hard broker message-size limits and adds scheduling latency
Physical/manual transfer	Exceptional offline migration	Not a scalable online service design

Queue Resilience After Dropped Messages

For an at-least-once queueing system, resilience comes from preserving failed work and making retries safe.

High-value controls:

- Review retry count, backoff, and visibility timeout/lease configuration.
- Add a dead-letter queue for messages that repeatedly fail.
- Alert on DLQ depth, age of oldest message, retry rate, and consumer lag.
- Retain source messages long enough to survive downstream outages and permit replay.
- Make consumers idempotent so retries do not duplicate side effects.
- Store enough metadata to diagnose why a message failed.

A separate queue of already successful messages is usually not the first resilience mechanism; durable source retention/audit storage is more useful when replay or forensic history is required.

Time-to-live for successfully processed messages can be useful for retention control, but it does not replace retry/DLQ handling for messages that never completed successfully.

Interview answer:

For dropped messages I would first verify delivery semantics, retry/backoff, visibility timeout, and consumer idempotency. Repeated failures should move to a dead-letter queue with alerts rather than being lost or retried forever. I would keep enough source retention for replay and monitor queue depth, age, retry rate, DLQ volume, and consumer lag. For very large payloads, I would store the blob in object storage and queue only a small reference plus metadata and checksum.

Senior 60-Minute System Design Interview Framework

Goal: Derive the architecture live instead of presenting a prebuilt diagram. Every major component should be justified by a requirement, bottleneck, failure mode, or ownership/scaling boundary.

This section consolidates a reusable backend/API system-design approach for a 60-minute senior interview. It deliberately emphasizes the gaps that commonly appear when candidates know individual concepts but do not clearly connect them into one executable request path.

Core Interview Opening

A strong opening is:

I'll stay at the high level initially. I'll clarify the functional and non-functional requirements, define the major APIs and data model, and start with the simplest architecture that satisfies those requirements. As

we encounter scaling or reliability concerns, I'll evolve the design, explain why each component is needed, how it works, its trade-offs and failure modes, and we can zoom into any component you'd like.

This signals:

requirements

APIs/resources

simplest HLD

access patterns/data model

measured bottleneck

justified evolution

critical deep dive

reliability/security/observability

60-Minute Pacing

Time	Focus
0-3 min	Frame the discussion and core functional scope.
3-8 min	Non-functional requirements and scale assumptions.
8-13 min	API/resource design and authentication assumptions.
13-18 min	Simplest architecture and service/module boundaries.
18-25 min	Scaling, statelessness, routing, data model, indexes.
25-32 min	Cache/CDN where relevant and database read scaling.
32-47 min	One or two system-defining deep dives.
47-54 min	Async processing, failure handling, consistency, sharding if justified.
54-58 min	Security, observability, cost, operational concerns.
58-60 min	Final evolved diagram and concise recap.

Do not attempt to deep-dive every possible subsystem. Breadth establishes the architecture; depth should concentrate on the defining problem.

1. Functional vs Non-Functional Requirements

Functional requirements answer:

What must the product do?

Examples:

- Create/read/update a resource.
- Subscribe/unsubscribe.
- Book/cancel.
- Upload/download.
- Match users/resources.
- View an ordered feed.

Non-functional requirements answer:

How well must it do those things?

Always resolve these five questions early:

1. How many users/requests/entities are we designing for, and how quickly are they growing?
2. Is the workload read-heavy or write-heavy?
3. Which data can never be lost, and which data can be reconstructed?
4. What latency is acceptable for the critical workflows?
5. What are the cost constraints?

Also clarify where relevant:

- Availability target
- Consistency expectations
- Geographic scope
- Peak/burst traffic
- Retention period
- Security/multi-tenancy
- RPO/RTO

A component introduced later should trace back to one of these constraints.

2. Put APIs Early in the Design

For backend/API interviews, concrete endpoints expose the resources and operations the architecture must support.

Example resource API:

```
POST /v1/jobs
GET /v1/jobs/{job_id}
POST /v1/jobs/{job_id}/cancel
GET /v1/jobs?limit=50&cursor=...
```

Discuss where relevant:

- HTTP method semantics
- 200 vs 201 vs 202 vs 204
- Idempotency keys
- Authentication and authorization
- Cursor pagination
- Optimistic concurrency
- Versioning
- Structured errors
- Async job semantics

Important 201 vs 202 nuance

A long-running operation does **not** automatically require 202 Accepted.

If the API synchronously creates a durable job resource:

```
POST /jobs

DB commits Job(id=123, state=QUEUED)

201 Created
Location: /jobs/123
```

Execution can continue asynchronously afterward.

Use 202 when the request has been accepted but the final resource/operation has not yet been completed or durably created in the way the API contract promises.

Idempotency for unsafe retries

For retry-sensitive POST workflows:

```
POST /v1/payments
Idempotency-Key: 6f...
```

Store a uniqueness scope such as:

```
(tenant_id, idempotency_key)
```

plus a normalized request hash and the created resource/response. Reusing the same key with a different payload should fail explicitly rather than silently returning the wrong result.

3. Server, Service, Module, Instance, Gateway, Load Balancer

These terms should never be blurred together.

Server / instance

A server instance is **running compute** executing application code.

Examples:

- Process
- VM
- Container
- Kubernetes pod

Service

A service is a **logical business/platform capability** exposed through an interface/API.

Examples:

```
Feed Service
Subscription Service
Payment Service
Inventory Service
```

One service may run on many instances:

```
Feed Service
|
+-- Feed instance #1
+-- Feed instance #2
+-- Feed instance #3
```

Module

A module is a logical code boundary **inside the same deployable application**.


```

App Instance
-----
Auth Module
Feed Module
Subscription Module
Podcast Module

```

A module call within one process does not require a network hop.

API Gateway / router

The gateway answers:

Which logical service owns this request?

Example:

```

/feed/*      -> Feed Service
/podcasts/*  -> Podcast Service
/subscriptions/* -> Subscription Service

```

Load balancer

A load balancer answers:

Which equivalent running instance should handle this request?

Example:

```

Feed Service LB
|
+-- Feed #1
+-- Feed #2
+-- Feed #3

```

The two routing decisions are different:

```

Client
|
v
API Gateway
| chooses logical service
v
Service Load Balancer
| chooses equivalent instance
v
Running service instance

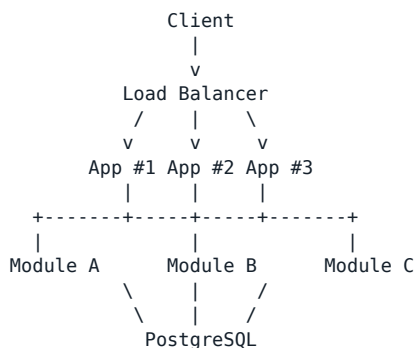
```

Do not draw one generic load balancer randomly selecting between unrelated services unless it is explicitly doing route-aware L7 proxying and that responsibility is explained.

4. Start with a Modular Monolith When It Is Enough

Do not begin with microservices only because the interview is called "system design."

A strong first version is often:



Every app instance contains all modules, so any endpoint can be routed to any healthy app instance.

Why this can be better initially

- Simple deployments
- Easier local development/debugging
- Transactions remain local
- Fewer network failure modes
- Less observability/operational overhead

When microservices become justified

Split a module when there is a meaningful reason such as:

- Very different scaling characteristics
- Strong failure-isolation requirement
- Independent deployment cadence
- Clear ownership boundary
- Independent data lifecycle
- Technology/runtime needs that materially differ

A good interview statement:

I would keep the logical boundaries clean from the start, but I would not pay distributed-system costs until independent scaling, deployment, ownership, or isolation justifies them.

5. Explain What the Server Actually Does

If an interviewer asks "what does the server do?", answer literally.

For GET /v1/feed:

```
App instance receives HTTP request
|
+-- parse request
+-- authenticate token
+-- derive trusted user identity
+-- authorize operation
+-- execute feed business logic
+-- query cache/database/downstream services
+-- merge/rank/filter results
+-- serialize JSON
+-- return HTTP response
```

The server is not an unnecessary forwarding box. It is the compute executing the service/module code.

6. Mandatory End-to-End Request Trace

For the most important workflow, always walk one request from client to response.

Use this checklist:

What does the user do?

What HTTP request is created?

How is the caller authenticated?

Where does the request first arrive?

How is it routed?

Which running instance executes it?

What business code runs?

What data does it read?

From which datastore/cache/service?

What computation/transformation occurs?

What response is created?

How does the client use it?

Avoid vague statements such as:

The feed service handles it.

Replace them with the exact data flow and transformation.

7. Scaling: Vertical First, Horizontal When Justified

Vertical scaling:

same instance

more CPU / memory

Advantages:

- Simple
- Minimal application changes
- Useful short-term capacity increase

Limitations:

- Hardware ceiling
- Still one failure domain

Horizontal scaling:

```
    +-- App #1
Client--LB--+-- App #2
    +-- App #3
```

Use when:

- Capacity exceeds one host
- Availability requires multiple replicas
- Failure isolation matters
- Workloads need independent scale

High availability can justify horizontal replicas **before** the vertical capacity ceiling is reached.

8. Stateless Application Instances

Critical cross-request state should not live only in one process.

Bad:

```
User session created on App #1
next request reaches App #3
App #3 has no session
```

Better:

```
App #1 --+
App #2 --+--> shared session/state store
App #3 --+
```

or use an appropriate signed token model where the required session claims are carried by the client and verified by the backend.

Sticky sessions

Sticky sessions can preserve affinity, but they are a trade-off rather than a complete solution:

- Can produce uneven load
- Complicate failover
- Reduce routing flexibility
- Do not remove the need for load balancing

9. Derive the Data Model from Access Patterns

Before creating tables, ask:

What will the application repeatedly need to answer?

Example:

```
Which podcasts does user 123 follow?
Does user 123 follow podcast 42?
What are podcast 42's latest episodes?
What are the newest eligible items for user 123?
```

Then derive tables/indexes.

This is stronger than inventing a generic schema and hoping it supports the workload.

10. SQL vs NoSQL Decision Framework

Do not say "NoSQL because scale."

Evaluate:

- Relationships and joins
- ACID transactions
- Constraints/uniqueness
- Consistency requirements
- Query flexibility
- Access-pattern stability
- Write throughput
- Schema flexibility
- Operational experience

A relational database is often a strong default for control-plane/business state that has meaningful invariants. A specialized store can be introduced later for a distinct read/write pattern.

11. Indexing: Query First, Index Second

Example query:

```
SELECT *
FROM episode
WHERE podcast_id = ?
ORDER BY published_at DESC, episode_id DESC
LIMIT 50;
```

Candidate index:

```
(podcast_id, published_at DESC, episode_id DESC)
```

Explain the trade-off:

The index reduces read work for this access pattern, but it consumes storage and increases write/update work.

Do not add indexes without naming the query they optimize.

12. Cache Only After Identifying a Hot Read

Basic flow:

```
request
  |
  v
cache
 /  \
hit  miss
 |    |
return DB
      |
      v
    populate
      |
      v
    return
```

TTL vs invalidation

- TTL is simple but allows bounded staleness.
- Active invalidation/update reduces staleness but adds coordination.
- A combination is often practical: invalidate on known writes and keep a TTL as a safety net.

Cache stampede

A hot key expires:

```
10,000 requests
  |
  v
MISS
  |
  v
DB
```

Defenses:

- Single-flight / request coalescing
- Stale-while-refresh
- Jittered TTLs

Cache failure

The cache should usually be an optimization rather than the only source of correctness.

cache unavailable

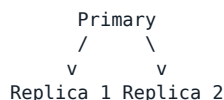
short timeout

fallback to authoritative store

rate-limit/backpressure to protect it

13. Replication and Read-After-Write

Single-primary model:



In this architecture:

writes -> primary

eligible stale-tolerant reads -> replicas

Replication lag

If a user writes and immediately reads from a replica, the new state may be missing.

Options for a workflow requiring read-after-write:

- Route the immediate read to primary.
- Use a consistency/session token.
- Wait for a confirmed replication position where supported.

Normal analytics/feed reads may tolerate small lag while permission/inventory/ownership checks may not.

Failover

Primary fails

protect/pause ambiguous writes

promote sufficiently current replica

routing moves to new primary

Discuss synchronous vs asynchronous replication if RPO/write latency matters.

14. Queue Choice and Async Work

Introduce a queue when work:

- Is long-running
- Can complete after the request
- Arrives in bursts
- Benefits from buffering/backpressure
- Should be decoupled from the producer

Different semantics matter:

Need	Better abstraction
One task should be processed by one worker	Work queue
Many independent consumers need the event	Pub/sub
Consumers need replay/order/history	Replayable log/stream

Do not reflexively choose Kafka when a simple work queue is sufficient.

15. At-Least-Once Delivery + Idempotent Consumer

Avoid casually promising "exactly once."

Practical design:

message delivered

worker processes

ack lost / worker crashes

message redelivered

Therefore the consumer must tolerate duplicates.

Use:

- Stable event/message IDs
- Unique database constraints
- Conditional state transitions
- Idempotency records

A strong answer:

I prefer at-least-once delivery with idempotent consumers and durable state transitions rather than relying on a vague exactly-once claim.

16. Transactional Outbox

Problem:

DB commit
publish event
process crash

The business state exists, but downstream consumers never hear about it.

Outbox pattern:

```
BEGIN
    write business row
    write outbox row
COMMIT

outbox publisher

message broker
```

The business update and intent-to-publish are committed atomically. The publisher can retry independently. Consumers still need idempotency because the event may be published more than once.

17. Retries, Timeouts, Circuit Breakers, Backpressure

Retry strategy

Retry only failures that are plausibly transient and retry-safe.

Use:

- Timeout
- Bounded retries
- Exponential backoff
- Jitter

Do not retry permanent validation failures.

Why jitter?

Without jitter, thousands of clients that fail together can retry on the same schedule and create another synchronized traffic spike.

Circuit breaker

When a dependency is persistently failing, stop sending it full request volume for a period and fail/fallback quickly. This protects both the caller and the unhealthy dependency.

Backpressure

If downstream capacity is lower than incoming demand, prevent unbounded work growth through:

- Bounded queues
- Admission control
- Rate limits
- Producer slowing
- Load shedding

18. Concurrency and Authoritative State Transitions

For scarce resources such as seats, inventory, driver assignments, quotas, or job ownership, do not make the final allocation decision from stale cache/read-replica state.

Example conditional update:

```
UPDATE resource
SET status = 'ALLOCATED'
WHERE id = :id
    AND status = 'AVAILABLE';
```


Interpret result:

```
1 row changed -> success
0 rows changed -> conflict/lost race
```

Other tools:

- Optimistic version columns
- Transactions
- Row locks
- Leases
- Heartbeats
- Reconciliation

Choose based on conflict probability and workflow duration.

19. Leases and Worker Failure

For long-running work, a worker should not own a job forever merely because it dequeued it once.

Example model:

```
job_id
state
lease_owner
lease_expires_at
attempt
```

Worker flow:

```
claim job with conditional update
set lease expiry
heartbeat/extend lease while running
complete -> final state
```

If the worker dies, the lease expires and a reconciler/new worker can safely reclaim the job.

20. Sharding Comes Late

Prefer a progression such as:

```
query optimization
indexes
vertical scaling
caching
read replicas
partitioning/archival
sharding when necessary
```

If sharding is required, discuss:

- Shard key
- Routing
- Hot shards
- Rebalancing
- Cross-shard queries
- Cross-shard transactions
- Operational complexity

Important:

Sharding does not automatically solve a single hot key or one celebrity/event receiving most of the traffic.

21. Security and Multi-Tenancy

Cover where relevant:

- TLS
- Authentication
- Authorization
- Service-to-service identity
- Secrets management
- Tenant scoping
- Rate limiting
- Input validation
- Audit logging
- Signed URLs
- PII handling

Keep authentication and authorization distinct:

Authentication -> who are you?

Authorization -> are you allowed to perform this action?

Do not trust `user_id` from request body as identity when a verified token/context already establishes the caller.

22. Observability

API/service metrics:

- Request rate
- Error rate
- p50/p95/p99 latency
- Saturation

Database:

- Query latency
- Connection usage
- Lock waits
- Replication lag

Cache:

- Hit/miss rate
- Evictions
- Rebuild latency
- Fallback-to-DB volume

Queue:

- Depth
- **Age of oldest message**
- Consumer lag
- Retry rate
- Dead-letter volume

The age of the oldest queued item can be more actionable than depth alone: a small queue containing one task stuck for hours can be more serious than a large queue draining normally.

Trace with identifiers such as:

```
request_id  
trace_id  
user_id / tenant_id  
resource_id  
job_id
```

23. Cost

Tie cost to the architecture rather than saying "cost matters."

Examples:

- Media-heavy product -> object storage and egress
- Personalized feed -> database reads and cache cardinality
- Realtime service -> connection count and fan-out
- Logging platform -> ingestion and retention
- Performance-test platform -> executor compute

A design can be technically scalable but economically poor.

Complete Example: Podcast Subscription and Latest-Episode Feed

This example is intentionally explicit about the complete request path, what the server does, what the feed logic does, and how the architecture evolves.

Requirements

Core workflows:

1. Subscribe/unsubscribe to podcasts.
2. View the latest episodes from subscribed podcasts.
3. Register/update podcast and episode metadata through a separate scraper/ingestion process.
4. Actual audio/video remains hosted by the publisher.

Assumptions for discussion:

10M users
hundreds of thousands of podcasts
read-heavy
subscriptions + metadata durable
feed/cache reconstructible
few-hundred-ms feed target
cost matters

APIs

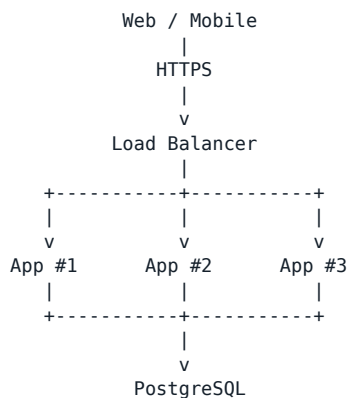
```
PUT /v1/subscriptions/{podcast_id}
DELETE /v1/subscriptions/{podcast_id}
GET /v1/subscriptions?limit=50&cursor=...
GET /v1/feed?limit=50&cursor=...
GET /v1/podcasts/{podcast_id}
GET /v1/episodes/{episode_id}
```

Internal ingestion:

```
POST /v1/internal/podcasts
POST /v1/internal/podcasts/{podcast_id}/episodes
```

PUT is a good subscription operation because the logical resource identity is naturally (authenticated_user_id, podcast_id) and repeated calls can be idempotent.

Initial HLD: Modular Backend



Scraper
|
v
internal authenticated endpoint

Each app instance contains:

```
Auth Module
Subscription Module
Podcast Module
Episode Module
Feed Module
Ingestion Module
```

No network hop exists between these modules inside the same application process.

Data Model

```
User
-----
user_id
email
created_at

Podcast
-----
podcast_id
title
feed_url
website_url
created_at
updated_at

Episode
-----
episode_id
podcast_id
source_episode_id
title
published_at
media_url
created_at

Subscription
-----
user_id
podcast_id
created_at
```

Constraints/indexes:

```
UNIQUE(user_id, podcast_id)
UNIQUE(podcast_id, source_episode_id)
INDEX subscription(user_id, podcast_id)
INDEX episode(podcast_id, published_at DESC, episode_id DESC)
```

Exact Feed Request Flow

Client sends:

```
GET /v1/feed?limit=50
Authorization: Bearer <token>
```

End-to-end:

```
Web / Mobile
|
| GET /v1/feed
| Authorization token
v
DNS / Edge
|
v
Load Balancer
|
| choose healthy app instance
v
App #2
|
|-- authentication middleware validates token
|
|   trusted user_id = 123
|
|-- Feed module
|
|   |-- query subscriptions for user 123
|   |
```

```

      |   [pod_A, pod_C, pod_F]
      |
      +-- fetch eligible recent episodes
      |
      +-- merge/order by published_at DESC
      |
      +-- take 50 + create cursor
      v
JSON response
  |
  v
Client

```

The subscription graph determines **eligibility**. `published_at` determines the basic **ordering** when the product requirement says "latest." A recommendation model is unnecessary unless personalized ranking becomes a separate requirement.

Simple SQL Feed

```

SELECT e.*
FROM subscription AS s
JOIN episode AS e
  ON e.podcast_id = s.podcast_id
WHERE s.user_id = :user_id
ORDER BY e.published_at DESC, e.episode_id DESC
LIMIT 50;

```

Start here if it meets the latency/scale requirement.

Large Subscription Set: K-Way Merge

If a user follows many podcasts, retrieve a small newest-first stream per podcast and merge only enough heads to produce the requested page.

```

Podcast A: 12:30, 11:50, 10:20
Podcast C: 12:25, 11:40
Podcast F: 12:10, 11:30
      |
      v
  heap of stream heads
      |
      v
  newest top 50

```

With P streams and N returned items, the merge work is approximately:

$O(N \log P)$

rather than sorting all historical episodes from every subscribed podcast.

Cursor Pagination

Use an opaque cursor representing ordering position, for example the last:

`(published_at, episode_id)`

This is more stable than offsets when new episodes are inserted at the top between page requests.

Caching

Prefer reusable podcast-level building blocks before millions of personalized full-feed entries:

```

podcast:A:latest
podcast:C:latest
podcast:F:latest

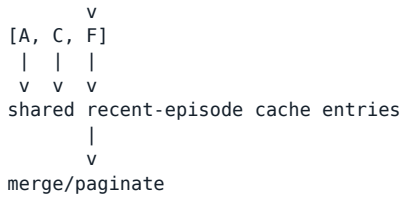
```

Then:

```

user subscriptions
  |

```



A full cache key such as `feed:user_123` can be useful later if measurement shows merge/query cost dominates, but it creates high cardinality and harder invalidation when popular podcasts publish.

Scraper Ingestion

```
POST /v1/internal/podcasts/pod_42/episodes
```

Payload includes a publisher/source episode identifier. The unique constraint:

```
UNIQUE(podcast_id, source_episode_id)
```

makes retries safe against duplicate insertion.

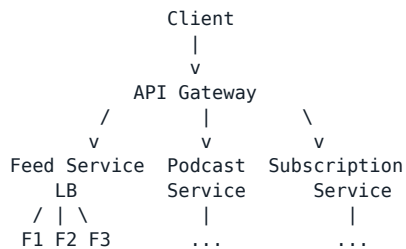
For reliable downstream cache invalidation/projections:

```
BEGIN
  INSERT episode
  INSERT outbox_event
COMMIT
```

The outbox publisher later emits `EPISODE_CREATED` and idempotent consumers invalidate/update read models.

When to Split Services

If feed generation develops different scaling/failure characteristics:



Then `/feed` routing is:

API Gateway chooses Feed Service

Feed Service LB chooses F1/F2/F3

running Feed Service instance executes the request

Avoid an $N+1$ service fan-out such as one network call per subscribed podcast. Use batched internal APIs, shared read models, or direct ownership-appropriate read stores.

System-Specific Deep-Dive Patterns

Ticketing / Seat Reservation

Core insight:

Browsing availability may tolerate staleness; final allocation cannot.

Calendar availability read path

A calendar or event-list page usually needs a fast read model such as availability counts by event/date:

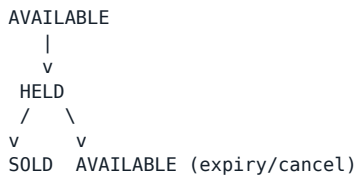
```
GET /events/{event_id}/availability?date=2026-08-22
```

availability/read model

remaining counts / sections / coarse seat state

Those counts may be cached or slightly stale for browsing. Once the user selects a specific seat/ticket, the authoritative inventory store must perform an atomic conditional transition such as AVAILABLE -> HELD. The UI read does not reserve inventory.

State machine:



Use an authoritative transaction/conditional update for allocation. Do not hold a database lock for the entire external payment flow.

Flash crowd controls:

- Waiting room/admission control
- Per-event capacity protection
- Rate limiting
- Short holds
- Idempotent payment confirmation
- Reconciliation after uncertain payment states

Sharding by event can distribute normal load but does not eliminate one extremely hot event.

Ride Matching

Core insight:

Geospatial search finds candidate drivers; authoritative trip/driver state decides who actually owns the assignment.

Separate data characteristics:

Driver location:
high write rate
loss of a few updates tolerable
geo-optimized

Trip assignment/payment:
stronger consistency
transactional state

Use a conditional transition such as AVAILABLE -> ASSIGNED to prevent two riders from owning the same driver.

Performance-Test Execution Platform

Typical flow:

Clients/CI

Gateway/Auth

Run API

Postgres + Outbox

Scheduler

Work Queue

Executors / Load Generators

Target Systems

Use:

- Durable TestRun state machine
- Idempotency key for submission
- Tenant quotas/fair scheduling
- Leases/heartbeats for worker ownership
- Object storage for large raw logs/results
- Queue depth **and age** monitoring
- Cancellation as a state transition rather than deleting history

Photo / Media Application

Keep binary objects outside the relational metadata database:

Client

signed upload

Object Storage

image-processing queue/workers

thumbnails/variants

CDN

Store metadata, ownership, permissions, and object keys in the database.

For social feeds, start with fan-out-on-read and evolve toward fan-out-on-write/hybrid only when measured read cost requires it. Celebrities/high-fanout publishers often need a hybrid strategy.

High-Probability System Design Follow-Up Questions

Practice answering these directly:

1. What exactly does this server do?
2. What is the difference between the service and the server/container/pod?
3. Why is this component a separate service rather than a module?
4. Which component receives this URL first?
5. How does the API know which user is making the request?
6. Why is `user_id` not accepted from the body as identity?
7. How does the gateway know which service owns the route?
8. How does the load balancer select an instance?
9. What happens if one application instance dies?
10. What happens if the load balancer/gateway fails?
11. Why do we need more than one server?
12. Why not just vertically scale?
13. Where is session state stored?
14. Do sticky sessions solve the problem?
15. Why SQL instead of NoSQL?
16. Which exact query does this index optimize?
17. What happens when the cache is stale?
18. What happens when the cache is completely unavailable?
19. How do you prevent a cache stampede?
20. What happens immediately after a write if a read goes to a lagging replica?
21. What if the primary fails?
22. Why do we need a queue?
23. Why this queue abstraction instead of Kafka/pub-sub/a work queue?
24. What happens if a worker crashes after performing the side effect but before acking?
25. How do you make retries safe?
26. What happens if DB commit succeeds but event publication fails?
27. What happens if two users try to allocate the same resource concurrently?
28. Why not use a cache to decide ownership/inventory?
29. What is the shard key?
30. What happens to one hot key/event after sharding?
31. How do you observe that the queue is stuck?
32. What metrics identify the actual latency bottleneck?
33. What is the largest cost driver in this architecture?
34. What happens at 10x traffic?
35. What data can be rebuilt after failure, and what must be durably protected?

Common System Design Mistakes

- Starting with many microservices before establishing a reason for them.
 - Drawing "server -> service" as if the server is merely forwarding to code running nowhere.
 - Using one load balancer to choose among unrelated services without explaining L7 routing.
 - Saying "the service handles it" without explaining the request/data flow.
 - Introducing Redis/Kafka/sharding because they are common interview buzzwords rather than because a bottleneck exists.
 - Choosing NoSQL only because the workload is "large."
 - Treating a cache as authoritative for a scarce resource allocation.
 - Assuming replicas are instantly consistent.
 - Claiming read replicas universally cannot accept writes rather than describing the proposed replication topology.
 - Promising exactly-once delivery without defining failure semantics.
 - Retrying non-idempotent operations without an idempotency design.
 - Using offsets for fast-changing feeds without discussing insertion drift.
 - Saying 202 is always required for long-running work.
 - Adding sharding before indexes/cache/replication are considered.
 - Ignoring the full client -> routing -> compute -> data -> response path.
-

Quick Interview Revision Card

1. Functional requirements
 2. Five NFR questions: scale, read/write, durability, latency, cost
 3. Overload: identify the saturated resource; protect dependencies with rate limits/backpressure/timeouts/degradation
 4. APIs/resources/auth
 5. Start Client -> App -> DB
 6. Define server/service/module precisely
 7. Trace one request end to end
 8. Scale stateless app behind LB if justified
 9. Derive data model from access patterns
 10. SQL/NoSQL based on invariants and queries
 11. Index the critical query
 12. Cache only hot reads; cover miss/stale/stampede/failure
 13. Replicas; cover lag/read-after-write/failover
 14. Queue only asynchronous/bursty work
 15. Outbox + at-least-once + idempotency
 16. Strong consistency for ownership/allocation
 17. Shard late and discuss hot keys
 18. Security + observability + cost
 19. Final evolved diagram + trade-off summary
-

Additional Senior System-Design Deep Dives

Load Balancer Reliability and Algorithms

A load balancer should not become the new single point of failure.

In production, use a managed/redundant load-balancing tier or multiple proxy instances with failover rather than one manually managed process.

Common routing strategies:

- Round robin: distribute requests cyclically across healthy instances.
- Least connections: prefer the instance with fewer active connections.
- Weighted routing: send more traffic to larger/faster instances or during staged rollout.
- Consistent hashing: useful when affinity by key is intentionally required, but it should not be introduced casually.

Health checks determine which instances remain eligible for routing.

Strong interview line:

The logical diagram shows one load-balancing tier, but I would deploy that tier redundantly or use a managed HA load balancer so it is not itself a single point of failure.

Multi-Tenant Performance-Test Platform: Resource and API Details

Useful resources:

```
TestConfiguration
TestRun
ResultSummary
Artifact
TenantQuota
```

Possible APIs:

```
POST /v1/test-runs
Idempotency-Key: <key>

GET /v1/test-runs/{run_id}
POST /v1/test-runs/{run_id}/cancel
GET /v1/test-runs?limit=50&cursor=...
GET /v1/test-runs/{run_id}/results
```

If `POST /test-runs` durably creates:

```
TestRun(id=R1, state=QUEUED)
```

then `201 Created` is valid even though execution continues later.

Control plane:

```
Client / CI
  |
  v
Gateway + Auth
  |
  v
Run API
  |
  v
PostgreSQL
  |
  +-- transactional outbox
```

Execution plane:

```
Scheduler
  |
  v
Work Queue
  |
  +-- Executor 1
  +-- Executor 2
  +-- Executor N
  |
```

```
      v
target systems
```

Large raw test logs/artifacts:

Executor

Object Storage

DB stores metadata/object key + summary

Fair scheduling

A simple global FIFO can let one tenant consume all executors.

Use tenant-aware quotas/fair scheduling, for example:

```
Tenant A queue --+
Tenant B queue --+--> fair scheduler --> executor capacity
Tenant C queue --+
```

Possible controls:

- Per-tenant concurrent-run limit
- Weighted fair queues
- Priority with starvation protection
- Admission control when platform is saturated

Cancellation

Cancellation is not equivalent to deleting history.

```
QUEUED -> CANCELLED
RUNNING -> CANCELLING -> CANCELLED
```

Workers should periodically observe cancellation or receive a control signal. Final state transitions must remain idempotent.

Ticketing: Payment and Hold State Details

A seat hold should be short-lived and represented durably:

```
seat_id
state
hold_id
held_by
hold_expires_at
version
```

Allocation:

```
UPDATE seat
SET state = 'HELD',
    hold_id = :hold_id,
    held_by = :user_id,
    hold_expires_at = :expiry
WHERE seat_id = :seat_id
AND state = 'AVAILABLE';
```

External payment should not keep the database row locked for the full payment interaction.

Payment state can be modeled separately:

```
CREATED

PROCESSING
 /      \
v        v
```

```
SUCCEEDED FAILED
  |
  uncertain callback/timeout
  v
RECONCILING
```

Use provider/idempotency IDs and reconciliation for cases where the payment provider may have succeeded but the local response was lost.

Ride Matching: Candidate Search vs Ownership

Do not confuse "closest driver" with "assigned driver."

```
Geo store

returns candidate drivers near rider

Trip/Driver authoritative store

conditional AVAILABLE -> ASSIGNED

winner owns trip
```

This separation is powerful because the geo index can tolerate rapidly changing/occasionally stale location signals while assignment requires a stronger invariant.

If multiple candidate drivers are attempted:

- Use bounded fan-out rather than contacting every nearby driver.
 - Apply timeout to each offer.
 - Avoid assigning one driver to two trips via conditional state.
 - Release/reconcile stale reservations if the matching worker fails.
-

Feed Generation: Fan-Out-on-Read vs Fan-Out-on-Write

Fan-out-on-read

At request time:

```
user

subscriptions/followees

recent items per source

merge/rank

feed response
```

Pros:

- Simple writes
- No need to precompute feeds for inactive users
- Fresh relationship changes are easy to reflect

Cons:

- Expensive read-time fan-out for users following many sources

Fan-out-on-write

When publisher creates an item:

new item

find followers

write item ID into follower feed projections

Pros:

- Fast feed reads

Cons:

- Write amplification
- Expensive celebrity/high-follower publishers
- More invalidation/replay complexity

Hybrid

Precompute for ordinary publishers; merge celebrity/high-fanout sources at read time.

A strong interview answer starts with fan-out-on-read if scale allows and introduces precomputation only after feed-read cost is shown to be the bottleneck.

Reusable Mock-Interview Prompt Template

Use this template when practicing a new design:

Act as me, the candidate, in a real 60-minute senior backend/API system-design interview. Start by clarifying functional and non-functional requirements, including scale, read/write ratio, durability, latency, and cost. Define concrete APIs and authentication assumptions early. Start with the simplest architecture and evolve it only when a requirement creates a bottleneck. At every major step show the current cumulative HLD and the relevant LLD/data flow. Be precise about server/instance vs service vs module, API gateway vs service load balancer, and trace at least one critical request end to end from client through routing, compute, data access, transformation, and response. Cover data model, SQL/NoSQL, indexes, caching, replication/read-after-write, queues/outbox/idempotency, concurrency/consistency, failure modes, security, observability, and cost where relevant. Spend most deep-dive time on the one or two system-defining problems. Keep the entire answer realistic for 60 minutes and finish with the final evolved architecture and a 60-90 second summary.

Software Design Patterns for Python and Backend Interviews

Source: docs/software_design_patterns.md

Purpose: Provide a practical, interview-focused reference for the 23 Gang of Four patterns and modern backend patterns. **Use this file for:** pattern selection, Python examples, low-level design, comparisons, trade-offs, and quick revision.

Recommended Study Flow

1. Identify whether the changing force is **creation**, **structure**, or **interaction**.
2. Explain a pattern through intent, participants, one concrete example, and one trade-off.
3. Implement only the collaboration that matters; avoid ceremonial abstractions.
4. Compare neighboring patterns because interviews often test selection rather than recall.
5. Combine patterns in one small LLD, then explain why every pattern earns its complexity.

Foundations

A design pattern is a reusable arrangement of responsibilities for a recurring design problem. It is not finished code and not an algorithm.

Concept	Question answered	Examples
Principle	What quality should guide the decision?	SOLID, DRY, KISS, YAGNI
Design pattern	How can recurring responsibilities be arranged?	Strategy, Adapter, Observer
Architecture	How do major components and data flows fit?	Layered, event-driven, hexagonal
Algorithm	Which computational steps solve the input problem?	Binary search, merge sort

Selection Workflow

Ask:

1. What changes independently?
2. Where is the conditional or coupling pressure?
3. Is the change about object creation, object assembly, or object interaction?
4. Can a function, protocol, composition, or dependency injection solve it more simply?
5. What new indirection, state, or operational cost will the pattern add?

Strong interview line:

I use a pattern when it isolates a real source of change or protects an invariant. I do not introduce one merely to name it in the design.

Creational Patterns

1. Singleton

Intent: Ensure one instance and provide controlled access to it.

Backend use: Process-local configuration registry or metrics collector.

Trade-off: Hidden global state makes tests, concurrency, and multi-process deployment harder. In FastAPI, application lifespan plus dependency injection is usually clearer.

```
class Settings:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

One instance per Python process is not one instance across containers, pods, or machines.

2. Factory Method

Intent: Define an object-creation interface while allowing a subclass or creator function to choose the concrete type.

```
from typing import Protocol

class Notifier(Protocol):
    def send(self, message: str) -> None: ...

def create_notifier(channel: str) -> Notifier:
    if channel == "email":
        return EmailNotifier()
    if channel == "sms":
        return SmsNotifier()
    raise ValueError(f"unsupported channel: {channel}")
```

Use when callers should depend on a stable interface rather than constructors and selection logic.

3. Abstract Factory

Intent: Create a compatible family of related objects without naming concrete classes.

Example: A cloud factory creates storage, queue, and secret-store adapters for AWS or Azure. The selected family remains internally compatible.

Trade-off: Adding a new product type to every family can be expensive.

4. Builder

Intent: Construct a complex object step by step while separating construction from representation.

```
class QueryBuilder:
    def __init__(self):
        self._filters: list[str] = []
        self._limit = 100

    def where(self, expression: str) -> "QueryBuilder":
        self._filters.append(expression)
        return self

    def limit(self, value: int) -> "QueryBuilder":
        self._limit = value
        return self

    def build(self) -> dict:
```

```
return {"filters": tuple(self._filters), "limit": self._limit}
```

Prefer a dataclass/Pydantic model when construction is not genuinely complex.

5. Prototype

Intent: Create an object by cloning a configured prototype.

Example: Copy a baseline workflow configuration and customize a few fields.

Trade-off: Shallow copies share nested mutable objects; deep copies can be expensive or invalid for sessions, sockets, locks, and other resources.

Structural Patterns

6. Adapter

Intent: Convert one interface into another expected by the client.

```
class LegacyRiskApi:
    def score(self, cents: int) -> int:
        ...

class RiskClientAdapter:
    def __init__(self, legacy: LegacyRiskApi):
        self.legacy = legacy

    def risk_score(self, amount: float) -> float:
        return self.legacy.score(round(amount * 100)) / 100
```

Useful at API-version, vendor, or legacy-system boundaries.

7. Bridge

Intent: Separate an abstraction from its implementation so both dimensions can vary independently.

Example: Report types vary independently from delivery channels. Report delegates delivery to an injected `DeliveryChannel`.

Trade-off: Adds value only when both dimensions genuinely vary.

8. Composite

Intent: Treat individual objects and object groups uniformly through a tree structure.

Examples: File/folder trees, nested policy rules, UI components, organization hierarchies.

```
class Rule:
    def evaluate(self, context: dict) -> bool:
        raise NotImplementedError

class AllRules(Rule):
    def __init__(self, children: list[Rule]):
        self.children = children

    def evaluate(self, context: dict) -> bool:
        return all(child.evaluate(context) for child in self.children)
```

9. Decorator

Intent: Add behavior by wrapping an object without modifying the wrapped class.

```
class TracedPaymentGateway:
    def __init__(self, wrapped, tracer):
        self.wrapped = wrapped
        self.tracer = tracer
```

```
async def authorize(self, payment):
    with self.tracer.start_as_current_span("authorize-payment"):
        return await self.wrapped.authorize(payment)
```

Use for tracing, caching, retry, authorization, and metrics when wrapper order and error semantics remain clear.

10. Facade

Intent: Expose a simplified interface over a complicated subsystem.

Example: CheckoutFacade.checkout() coordinates inventory, payment, order persistence, and notification behind one application-facing operation.

Trade-off: A facade can become a god object if it absorbs business rules instead of coordinating them.

11. Flyweight

Intent: Share immutable intrinsic state among many logical objects to reduce memory use.

Example: Reuse parsed rule definitions while keeping request-specific evaluation context separate.

Trade-off: Separating intrinsic and extrinsic state increases complexity and may not be worthwhile until profiling proves a memory problem.

12. Proxy

Intent: Control access to another object while preserving its interface.

Examples: Authorization proxy, remote proxy, lazy-loading proxy, caching proxy.

Unlike an Adapter, a Proxy preserves the conceptual interface; unlike a Decorator, its primary purpose is access/control rather than adding optional behavior.

Behavioral Patterns

13. Observer

Intent: Notify registered dependents when an in-process subject changes.

Example: A domain object notifies local observers that a state transition occurred.

Trade-off: Notification order, failure isolation, memory leaks, and re-entrant updates need explicit policy. For distributed systems, durable publish-subscribe is normally a different architectural mechanism.

14. Strategy

Intent: Encapsulate interchangeable algorithms behind one interface.

```
from typing import Protocol

class PricingStrategy(Protocol):
    def price(self, subtotal: int) -> int: ...

class StandardPricing:
    def price(self, subtotal: int) -> int:
        return subtotal

class PromotionalPricing:
    def price(self, subtotal: int) -> int:
        return max(0, subtotal - 500)
```

Use when a conditional selects an algorithm and new variants should not modify the caller.

15. Command

Intent: Represent a request as an object so it can be queued, retried, logged, authorized, or undone.

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
class TransferCommand:
    operation_id: str
    source_account: str
    destination_account: str
    amount_cents: int
```

Distributed commands need idempotency, durable state, authorization, and observable outcomes.

16. Chain of Responsibility

Intent: Pass a request through an ordered chain of handlers until one handles or rejects it.

Examples: Middleware, validation pipeline, fraud rules, support escalation.

Trade-off: Handler order becomes behavior. Trace which handler made the decision.

17. Template Method

Intent: Define an algorithm skeleton in a base class while allowing selected steps to vary.

```
class ImportPipeline:
    def run(self, source):
        rows = self.extract(source)
        valid = self.validate(rows)
        return self.persist(valid)

    def extract(self, source):
        raise NotImplementedError

    def validate(self, rows):
        return rows

    def persist(self, rows):
        raise NotImplementedError
```

Prefer Strategy/composition when inheritance would couple implementations too tightly.

18. Iterator

Intent: Traverse a collection without exposing its representation.

Python iterators and generators provide language-native support.

```
def batches(rows, size):
    batch = []
    for row in rows:
        batch.append(row)
        if len(batch) == size:
            yield batch
            batch = []
    if batch:
        yield batch
```

19. State

Intent: Change behavior when an object's internal state changes.

Examples: Job lifecycle, payment state, order state, agent workflow.

```
CREATED -> PROCESSING -> SUCCEEDED
          \-> FAILED -> RETRYING
```

Every transition should define allowed source states, authorization, side effects, idempotency, and audit behavior.

20. Mediator

Intent: Centralize interactions among components so they do not depend directly on one another.

Example: An application mediator routes commands to handlers and publishes resulting domain events.

Trade-off: The mediator can become an opaque god object unless message contracts and handler ownership remain clear.

21. Memento

Intent: Capture and restore object state without exposing internals.

Examples: Undo, workflow checkpoints, draft recovery.

Trade-off: Snapshots can be expensive and may contain sensitive data. Define encryption, retention, schema compatibility, and replay semantics.

22. Visitor

Intent: Add operations across a stable object structure without modifying every element class.

Example: Generate audit, validation, and export representations across a stable policy AST.

Trade-off: Adding new element types requires updating visitors; therefore Visitor is strongest when element types are stable and operations change frequently.

23. Interpreter

Intent: Represent the grammar and evaluator for a small language.

Examples: Search filters, policy DSLs, rule expressions.

Security: Never pass an untrusted DSL directly to eval, SQL, or shell execution. Parse into an allow-listed AST, validate complexity, and enforce resource limits.

Modern Backend Patterns

Dependency Injection

Supply dependencies from the outside rather than constructing them in business logic.

```
class PaymentService:
    def __init__(self, repository, gateway):
        self.repository = repository
        self.gateway = gateway
```

Benefits include testability, explicit lifetimes, replaceable implementations, and centralized wiring.

Repository

Place persistence operations behind a domain-oriented interface.

```
class AccountRepository:
    async def get(self, account_id: str): ...
    async def save(self, account): ...
```

Do not hide every database capability behind generic CRUD if the application needs optimized queries, transactions, locking, or bulk operations.

Object Pool

Reuse expensive resources such as database connections.

Important controls:

- Maximum pool size and overflow
- Acquisition timeout
- Health validation
- Return/cleanup on every path
- Backpressure rather than unbounded connection creation

Lazy Initialization

Delay expensive creation until first use. It reduces startup work but moves failure and latency into a request path unless warmed and observed deliberately.

MVC / Layered Architecture

Separate transport/presentation, application/service logic, and persistence/domain concerns. FastAPI route handlers should normally coordinate validation and HTTP mapping rather than contain all business logic.

Event-Driven Architecture

Producers publish facts; consumers react asynchronously.

Production requirements include schema evolution, ordering scope, retries, dead-letter handling, idempotent consumers, observability, and transactional outbox when a database update and event must remain consistent.

CQRS

Separate command/write models from query/read models when read and write concerns genuinely differ.

CQRS does not require separate databases, event sourcing, or microservices. Start with logical separation and add infrastructure only when justified.

Event Sourcing

Store the sequence of domain events as the source of truth and rebuild state by replay.

Advantages:

- Complete change history
- Temporal reconstruction
- New projections from old events

Costs:

- Event schema evolution
- Replay and projection recovery
- Eventual consistency
- Privacy/deletion requirements
- Operational complexity

CQRS and event sourcing are independent patterns even though they are often combined.

High-Value Comparisons

Factory Method vs Abstract Factory vs Builder

Pattern	Select when
Factory Method	One product/interface varies
Abstract Factory	A compatible family of products varies together
Builder	One complex result is assembled through controlled steps

Adapter vs Facade vs Proxy vs Decorator

Pattern	Primary intent
Adapter	Change the interface
Facade	Simplify a subsystem
Proxy	Control access while preserving the interface
Decorator	Add composable behavior while preserving the interface

Strategy vs State

- **Strategy:** The caller/configuration selects an interchangeable algorithm.
- **State:** Internal lifecycle state drives behavior and allowed transitions.

The class diagrams may look similar; the intent and ownership of selection differ.

Observer vs Publish-Subscribe

- Observer is commonly an in-process subject-to-observer relationship.
- Publish-subscribe usually introduces a broker/topic and decouples producers from consumers across process boundaries.
- Distributed delivery requires durability, replay, ordering, deduplication, and failure semantics that the GoF Observer description does not provide.

Template Method vs Strategy

- Template Method varies steps through inheritance.
- Strategy varies behavior through composition.
- Prefer composition when algorithms must be changed independently at runtime or inheritance would create rigid coupling.

LLD Case Study: Extensible Transaction Processing

Requirements

- Create a transaction through an API.
- Validate business rules.
- Select a risk/pricing policy.
- Persist atomically and prevent duplicate requests.
- Publish an event after commit.
- Integrate with a replaceable external provider.
- Add tracing without embedding it in core logic.

Pattern Map

Force	Pattern/tool	Reason
Different risk algorithms	Strategy	Replace policy without changing the service
External provider has incompatible API	Adapter	Normalize vendor contract
Persistence boundary	Repository	Keep service independent of ORM details
Request as auditable work	Command	Stable operation ID and explicit input
Transaction lifecycle	State	Validate allowed transitions
Cross-cutting tracing	Decorator	Wrap behavior without changing domain logic
DB commit plus event publication	Transactional outbox	Avoid dual-write inconsistency
Dependency wiring/test replacement	Dependency injection	Explicit construction and lifetimes

```

FastAPI route
|
v
TransactionService
|-- Pricing/Risk Strategy
|-- Provider Adapter
|-- Repository
|-- Outbox Repository
|
v
single DB transaction
|
+--> transaction row
+--> outbox event
    |
    v
    publisher worker

```

Interview Trade-Offs

- A Strategy is justified when policy variants exist; otherwise a function is enough.
- Repository methods should reflect domain operations and important query semantics.
- The outbox solves database/event atomicity but adds a publisher, lag, retries, and monitoring.
- A Decorator is useful for tracing only if wrapper order and exception behavior stay understandable.
- Idempotency still needs a unique database constraint; an application lookup alone races.

Common Red Flags

- Pattern names without the changing force they solve.
- Singleton used as a substitute for dependency management.
- Factory for a class with one stable implementation.
- Repository that hides required transactions or creates N+1 queries.
- Observer used as if it guarantees distributed delivery.
- Event sourcing selected only because audit history is required.
- Strategy and State treated as interchangeable because their diagrams look similar.
- A pattern-heavy design with no simpler baseline.

60-Second Answer Template

The pattern's intent is __. The changing force is __, so I would separate __ from __. The main participants are __. In this system, an example is __. The benefit is __, while the cost is __. If that source of change does not exist yet, I would use the simpler design and introduce the pattern only when needed.

Quick Revision Checklist

Creation: Singleton, Factory Method, Abstract Factory, Builder, Prototype
Structure: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
Interaction: Observer, Strategy, Command, Chain, Template Method, Iterator, State, Mediator, Memento, Visitor, Interpreter

Modern backend: DI, Repository, Object Pool, Lazy Initialization, MVC, Event-Driven, CQRS, Event Sourcing, Transactional Outbox

Always state: intent -> participants -> example -> trade-off -> simpler alternative

Further Reading

- [Refactoring.Guru design patterns](#)
- [Python design-pattern examples](#)
- [Software Design Patterns Interview Guide PDF](#)

GenAI, LLMs, Prompting, Agents, RAG & Evaluation

Source: docs/genai_llm_rag.md

Purpose: Generative AI concepts, prompt engineering, RAG, embeddings, vector databases, AI agents, tool calling, hallucination reduction, guardrails, and LLM evaluation. **Use this file for:** AI engineer, GenAI engineer, AI platform engineer, and LLM application interviews

Recommended Study Flow

1. Read the **Quick Summary** first.
2. Review the **Key Concepts** and tables.
3. Practice the **Interview Questions & Answers** out loud.
4. Use the code snippets and examples to explain trade-offs clearly.
5. Finish with the **Common Mistakes** and **Revision Checklist** sections.

Quick Summary

This is a new topic file created because the attached repository files did not have a dedicated Markdown page for this subject. It is merged from the organized topic-wise interview-prep pack and follows the same repository style as the existing notes.

Consolidated Interview Questions & Technical Notes

Generative AI, LLMs, prompt engineering, AI agents, tool calling, RAG, embeddings, vector databases, hallucination, guardrails, model comparison, and LLM evaluation.

1. Core GenAI & Agentic AI

1.1 What is Generative AI?

Interview answer:

Generative AI refers to AI systems that generate new content such as text, code, images, summaries, or responses based on a prompt. In the LLM context, the model predicts the most likely next tokens to generate useful language outputs.

Examples:

- Summarizing a document
- Writing code
- Generating a cover letter
- Creating a chatbot response
- Explaining a security alert

1.2 What is Agentic AI?

Interview answer:

Agentic AI builds on Generative AI by enabling an LLM-powered system to reason, plan, use external tools, maintain state, and execute multi-step workflows toward a goal. Unlike a regular chatbot that gives one response, an agent can decide what action to take next, call APIs, retrieve data, validate outputs, and iterate until the task is complete.

Agent flow

User Goal

Planner / Reasoning Layer

Tool Selection

External APIs / DB / Search / Vector DB

Observation

Next Action or Final Answer

1.3 Agentic AI vs Generative AI

Generative AI	Agentic AI
Generates content	Completes tasks
Single prompt -> response	Multi-step workflow
Mostly reactive	Goal-driven
Limited tool use	Uses APIs, databases, search, code execution
Limited memory	Maintains state/context
Good for summarization and writing	Good for workflow automation and investigation

60-second answer:

Generative AI focuses on generating content such as text, code, images, or summaries. Agentic AI extends that by giving the system the ability to reason, plan, use tools, maintain context, and execute multi-step workflows. For example, a Generative AI model can summarize a report, while an AI agent can retrieve reports, compare them, call APIs, generate an analysis, and send the result for approval. In production, agents require stronger guardrails, monitoring, access control, and failure handling.

1.4 What makes an AI agent production-grade?

A demo agent may simply call an LLM and return an answer. A production-grade agent needs:

- Authentication and authorization
- Tool access control
- Human-in-the-loop approval for sensitive actions
- Retry and fallback logic
- Observability and audit logging
- Cost and token tracking
- Evaluation datasets
- Prompt/version control
- Guardrails and validation
- Failure handling
- Rollback strategy

Interview line:

Production AI agents are not just prompts. They are distributed systems with LLM reasoning, tools, orchestration, observability, evaluation, guardrails, and operational controls.

1.5 Tool Calling

Interview answer:

Tool calling allows an LLM or agent to invoke external functions, APIs, databases, search systems, or internal services instead of relying only on its own generated text. The model decides which tool to call, passes structured parameters, receives the result, and uses that result to continue reasoning or produce the final answer.

Example flow

User: "Show my recent orders"

LLM decides to call `get_orders(user_id)`

Backend API queries database

Tool returns order list

LLM summarizes result

Tool-calling concerns

- Validate tool arguments
- Restrict allowed tools
- Avoid exposing sensitive data
- Add retries/timeouts
- Log tool calls
- Require human approval for risky actions

1.6 Prompt Engineering

Interview answer:

Prompt engineering is the process of designing instructions, context, examples, constraints, and output formats so that the LLM produces consistent, accurate, and useful responses. In production systems, prompt engineering is closely tied to context engineering, RAG, evaluation, and guardrails.

Good prompt components

Role: You are a support assistant.
Task: Answer the user's question using only the provided context.
Context: <retrieved documents>
Rules:
- Do not invent facts.
- Cite sources.
- If context is insufficient, say you don't know.
Output format: JSON with answer and sources.

1.7 RLHF

Interview answer:

RLHF stands for Reinforcement Learning from Human Feedback. It improves LLM behavior using human preference data. Human reviewers compare or rate model outputs, those preferences help train a reward model, and the LLM is optimized to produce responses that better align with human expectations for helpfulness, correctness, and safety.

Your experience positioning

I contributed to LLM training through RLHF by evaluating model outputs, generating high-quality preference data, identifying failure modes, and improving model behavior across coding, reasoning, safety, and instruction-following tasks.

2. RAG, Retrieval, Chunking & Vector Search

2.1 What is RAG?

Interview answer:

Retrieval-Augmented Generation, or RAG, combines information retrieval with LLM generation. Instead of relying only on the model's training data, the system retrieves relevant documents or chunks from a knowledge base and provides that context to the LLM before generating an answer. This improves factual accuracy, reduces hallucinations, and allows the system to use private or frequently changing enterprise data without retraining the model.

RAG flow

User Query
Query Embedding
Vector Search
Top-k Relevant Chunks
Prompt Construction
LLM
Grounded Answer + Sources

2.2 RAG vs Fine-Tuning

RAG	Fine-tuning
Best for changing knowledge	Best for changing behavior/style
Uses latest documents	Knowledge can become stale
Cheaper to update	Requires training pipeline
Good for enterprise documents	Good for domain adaptation
Can cite sources	Does not guarantee factual grounding

Answer:

If the knowledge changes frequently, I prefer RAG because documents can be updated and re-indexed without retraining the model. Fine-tuning is better when we want to change the model's style, format, behavior, or specialized task performance.

2.3 Embeddings

Interview answer:

Embeddings are dense numerical vector representations of text that capture semantic meaning. Similar concepts have vectors close to each other in vector space, which enables semantic search.

Example:

```
"car" and "vehicle" -> close vectors  
"car" and "banana" -> far apart vectors
```

Used for:

- Semantic search
- RAG
- Similarity matching
- Recommendations
- Clustering

2.4 Vector Databases

Vector databases store embeddings and support similarity search.

Examples:

- FAISS
- Chroma
- Pinecone
- Weaviate
- OpenSearch Vector Search

Interview answer:

A vector database stores document or chunk embeddings and allows nearest-neighbor search against a query embedding. In a RAG system, it helps retrieve the most semantically relevant chunks to ground the LLM response.

2.5 Vector Indexing Strategies

Flat index

- Compares against every vector
- Highest accuracy
- Slow for large datasets
- Good for testing or small data

HNSW

- Graph-based approximate nearest neighbor index
- Very fast search
- High recall
- Higher memory usage
- Common for production RAG

IVF

- Clusters vectors and searches selected clusters
- Good for large datasets
- Lower memory than HNSW
- Recall depends on tuning

Product Quantization (PQ)

- Compresses vectors
- Saves memory
- Slight loss in accuracy
- Useful for very large-scale retrieval

Index	Speed	Recall	Memory	Best Use
Flat	Slow	Excellent	High	Small datasets/testing
HNSW	Very fast	Excellent	High	Production RAG
IVF	Fast	Good	Medium	Large datasets
PQ	Fast	Good	Low	Billion-scale storage

60-second answer:

For production RAG, I usually prefer HNSW because it provides strong recall and low latency. For very large datasets with memory constraints, I would consider IVF or IVF with Product Quantization. The final choice depends on the trade-off between recall, latency, memory, and indexing cost.

2.6 Different Chunking Strategies

1. Fixed-size chunking

Split by fixed token or word count.

Chunk 1: tokens 1-500
Chunk 2: tokens 401-900
Chunk 3: tokens 801-1300

Good because it is simple, but it may split related content.

2. Recursive chunking

Split in this order:

Section -> Paragraph -> Sentence -> Words

Best for preserving meaning while staying within chunk size.

3. Sentence-based chunking

Keeps sentences intact. Useful for articles and policies.

4. Paragraph-based chunking

Keeps full paragraphs together. Useful for documentation, manuals, and policies.

5. Semantic chunking

Splits when the topic changes. Higher quality but more expensive.

6. Sliding window / overlapping chunks

Uses overlap so context is not lost at boundaries.

Typical values:

- Chunk size: **300-500 tokens**
 - Overlap: **50-100 tokens**
-

7. Metadata-based chunking

Preserves fields such as:

- Document name
- Page number
- Section heading
- Department
- Date/version

Interview answer:

For enterprise RAG, I usually prefer recursive chunking with overlap and metadata. A common starting point is 300-500 tokens with 50-100 token overlap, but I tune it based on retrieval quality, document structure, latency, and answer correctness.

2.7 RAG Endpoint Design

A production RAG backend can expose:

Endpoint	Purpose
POST /upload	Upload document
POST /parse	Extract text from uploaded document
POST /index	Chunk, embed, and store vectors
POST /query	Ask a question
POST /chat	Conversational RAG with memory
GET /documents	List indexed documents
DELETE /documents/{id}	Delete document and embeddings
POST /reindex/{id}	Re-index updated document
GET /health	Health check
GET /metrics	Monitoring metrics

FastAPI skeleton

```
from fastapi import FastAPI, UploadFile

app = FastAPI()

@app.post("/upload")
async def upload_document(file: UploadFile):
    return {"status": "uploaded", "filename": file.filename}

@app.post("/query")
async def query_rag(question: str):
    # embed query -> retrieve chunks -> build prompt -> call LLM
    return {"answer": "...", "sources": []}
```

2.8 Building a RAG System from PDFs

Pipeline

PDFs

Text extraction using pypdf/PyMuPDF

OCR fallback if scanned

Clean text

Chunk text

Generate embeddings

Store in vector DB

Retrieve top-k chunks

Build grounded prompt

LLM response with citations

PDF extraction example

```
from pypdf import PdfReader

def extract_pdf_text(path: str):
    reader = PdfReader(path)
    pages = []

    for i, page in enumerate(reader.pages):
        text = page.extract_text() or ""
        pages.append({
            "page": i + 1,
            "text": text
        })

    return pages
```

Simple chunking example

```
def chunk_text(text: str, chunk_size: int = 500, overlap: int = 100):
    words = text.split()
    chunks = []

    start = 0
    while start < len(words):
        end = start + chunk_size
        chunks.append(" ".join(words[start:end]))
        start += chunk_size - overlap

    return chunks
```

Key point:

RAG quality depends heavily on parsing, chunking, metadata, retrieval, and prompt construction. A stronger model cannot fully compensate for poor context.

3. LLM Evaluation, Hallucination & Model Comparison

3.1 What is LLM Evaluation?

Interview answer:

LLM evaluation is the process of systematically measuring a language model's performance across correctness, factual accuracy, reasoning, instruction following, safety, groundedness, latency, cost, and user satisfaction. The goal is to ensure the model behaves reliably before and after deployment.

3.2 What do you evaluate?

- Correctness
 - Faithfulness / groundedness
 - Instruction following
 - Reasoning quality
 - Safety
 - Toxicity
 - PII leakage
 - Tool usage accuracy
 - Latency
 - Token usage
 - Cost
 - User satisfaction
-

3.3 Offline vs Online Evaluation

Offline Evaluation	Online Evaluation
Before deployment	In production
Uses benchmark/golden datasets	Uses real user traffic
Easier to reproduce	Captures real behavior
Helps compare models/prompts	Helps detect drift/regression

3.4 Human vs Automated Evaluation

Human evaluation

Good for:

- Reasoning quality
- Helpfulness
- Safety review
- Preference comparison

Automated evaluation

Examples:

- LLM-as-a-judge
 - RAGAS
 - DeepEval
 - LangSmith
 - Custom scoring scripts
 - BLEU/ROUGE/BERTScore for certain NLP tasks
-

3.5 How to Evaluate a RAG System as a Whole

Evaluate three layers:

1. Retrieval quality

Metrics:

- Precision@K
- Recall@K
- Hit Rate
- MRR
- Context precision
- Context recall

2. Generation quality

Metrics:

- Correctness
- Faithfulness
- Relevance
- Completeness
- Citation accuracy
- Hallucination rate
- Instruction following

3. System performance

Metrics:

- Latency
- Throughput
- Token usage
- Cost per request
- Error rate
- Availability
- User feedback

60-second answer:

I evaluate a RAG system in layers. First, I measure whether the retrieval system finds the right chunks using Precision@K, Recall@K, Hit Rate, and MRR. Second, I evaluate the generated answer for correctness, faithfulness, relevance, citation accuracy, and hallucination rate. Third, I monitor production metrics like latency, cost, token usage, errors, and user feedback. This helps isolate whether failures come from retrieval, prompt construction, or the LLM.

3.6 Hallucination

Interview answer:

Hallucination occurs when an LLM generates information that is incorrect, fabricated, or unsupported while presenting it confidently as true. LLMs can hallucinate because they predict likely text rather than verifying facts against a trusted source.

Causes

- Missing context
- Outdated training data
- Ambiguous prompt
- Poor retrieval
- High temperature
- Weak output validation

Reduction strategies

- RAG with trusted documents
- Better context engineering
- Prompt constraints
- Citations
- Output validation
- Confidence thresholds
- Human review
- Retrieval evaluation
- Lower temperature for factual tasks

Interview line:

Hallucinations cannot be eliminated completely, but they can be significantly reduced with grounding, validation, evaluation, and monitoring.

3.7 BLEU and ROUGE

BLEU

- Precision-oriented
- Often used in machine translation
- Measures word/phrase overlap with reference output

ROUGE

- Recall-oriented
- Often used in summarization
- Measures how much reference content is captured

BLEU	ROUGE
Precision-oriented	Recall-oriented
Translation	Summarization
Exact/near word overlap	Reference content coverage

Limitation:

BLEU and ROUGE rely heavily on lexical overlap, so they may score a semantically correct LLM answer poorly if it uses different wording. For modern LLMs, semantic evaluation, human review, groundedness, faithfulness, and LLM-as-a-judge are often more useful.

3.8 How to say which model is better

Strong answer:

I would not say one model is better based only on benchmark scores. I would define the application requirements and compare models using representative evaluation data. I would evaluate factual accuracy, reasoning quality, instruction following, faithfulness, safety, latency, cost, consistency, and tool-use performance. The best model is the one that meets the business requirements with the right balance of quality, reliability, latency, and cost.

Model comparison dimensions

Dimension	Why it matters
Accuracy	Correctness of answer
Faithfulness	Whether answer is grounded in context
Reasoning	Logical consistency
Safety	Avoids harmful or sensitive output
Latency	User experience
Cost	Production feasibility
Consistency	Stable behavior across runs
Tool success	Agent reliability

3.9 Accuracy, Precision, Recall, F1

If 2 out of 5 predictions are correct

That is usually **accuracy**:

Accuracy = correct predictions / total predictions
Accuracy = 2 / 5 = 40%

Precision

Precision = TP / (TP + FP)

Of everything predicted positive, how many were actually positive?

Recall

Recall = TP / (TP + FN)

Of all actual positives, how many did we catch?

F1

$$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

Useful for imbalanced datasets.

15. System Design for AI Assistants

15.1 Design a production AI assistant

Architecture

User

Frontend / Chat UI

API Gateway / Load Balancer

FastAPI Backend

Auth + Rate Limiting

Agent Orchestrator

Vector DB LLM API External Tools

Monitoring + Logging + Audit

Response + Citations

15.2 Core components

- Frontend UI
 - FastAPI backend
 - Authentication and RBAC
 - Agent orchestrator
 - RAG pipeline
 - Vector database
 - LLM provider/model layer
 - Tool calling layer
 - Cache
 - Database
 - Logging and monitoring
 - Evaluation pipeline
 - Guardrails
-

15.3 Production concerns

- Access control
- Data privacy
- Prompt injection
- PII redaction
- Tool allowlisting
- Audit logging
- Token and cost control
- Rate limits
- Human approval
- Retry/fallback
- Evaluation and monitoring

15.4 Security-focused AI assistant example

Security Logs / Events

Streaming / Batch Ingestion

Context Pipeline

Vector DB + Search Index

Agent Orchestrator

LLM + Security Tools

Analyst Review / Human Approval

Incident Summary / Recommended Action

Use cases:

- Threat summarization
- Log analysis
- Alert correlation
- Incident response assistance
- Playbook generation
- Root cause explanation

4. AI Agents

4.1 What Is an AI Agent?

"An AI agent is a software system that uses an LLM along with tools, memory, external APIs, decision logic, and workflows to complete tasks. Unlike a simple chatbot, an agent can take actions, call APIs, retrieve data, create tickets, check order status, and complete multi-step workflows."

4.2 Chatbot vs AI Agent

Chatbot	AI Agent
Mostly answers questions	Can perform actions
Usually conversation-based	Workflow/task-based
Limited external access	Can call APIs/tools
Often static	Can reason through steps
Usually reactive	Can be goal-oriented

4.3 Components of an AI Agent

- LLM
- Prompt / system instructions
- Tools / API functions
- Memory or conversation history
- Knowledge base
- Retrieval system
- Authentication and permissions
- Logging and monitoring
- Evaluation framework
- Human escalation path

4.4 Example: Customer Support AI Agent

A customer support AI agent may:

1. Understand the user query
2. Retrieve relevant policy or support documentation
3. Check user account status through an API
4. Create or update a support ticket
5. Escalate to a human if confidence is low
6. Log the conversation for monitoring and evaluation

User

React/TypeScript UI

Backend API

Agent Orchestration

Tools / APIs / Knowledge Base

LLM Response

User

4.5 Agent Evaluation Metrics

- Task completion rate
- Conversation success rate
- Escalation rate
- Hallucination rate
- API success/failure rate
- Latency
- Token cost
- User satisfaction
- Safety issue rate

5. RAG Systems

5.1 What Is RAG?

"RAG stands for Retrieval-Augmented Generation. It allows an LLM to retrieve relevant information from an external knowledge source before generating an answer. This helps make responses more accurate, grounded, and up to date."

5.2 Simple RAG Flow

Documents

Chunking

Embeddings

Vector Database

User Query

Query Embedding

Similarity Search

Relevant Chunks

LLM

Final Answer

5.3 Interview Answer: RAG System You Worked On

"A typical RAG system I have worked with or evaluated involves document ingestion, chunking, embedding generation, vector storage, retrieval, and LLM response generation. When the user asks a question, the system retrieves the most relevant chunks and passes them to the LLM as context. This improves accuracy and reduces hallucinations compared to relying only on the model's internal knowledge."

5.4 Why Use RAG?

- Reduces hallucinations
- Improves factual grounding
- Allows domain-specific answers
- Uses up-to-date company knowledge
- Helps with compliance and traceability
- Supports citations or source references

5.5 How to Improve a RAG System

"I would improve a RAG system by tuning chunking, improving embeddings, using better retrieval strategies, adding reranking, improving prompts, and evaluating both retrieval quality and answer quality."

Improvement Areas

Area	Improvement
Chunking	Adjust chunk size and overlap
Embeddings	Use domain-suitable embedding model
Retrieval	Use hybrid search or metadata filters
Ranking	Add reranking model
Prompt	Make instructions clearer
Evaluation	Measure retrieval and answer quality

5.6 RAG Quality Metrics

Split evaluation into two parts:

Retrieval Quality

- Are the correct documents retrieved?
- Are retrieved chunks relevant?
- Is important context missing?

Metrics:

- Precision@K
- Recall@K
- Context relevance
- Context coverage
- Retrieval accuracy

Response Quality

- Is the answer correct?
- Is it grounded in retrieved context?
- Is it complete?
- Is it relevant?
- Does it hallucinate?

Metrics:

- Accuracy
- Relevance
- Completeness
- Groundedness
- Hallucination rate
- User satisfaction

5.7 Strong RAG Interview Line

"Even if the LLM is strong, poor retrieval will lead to poor answers. That is why I evaluate both retrieval quality and response quality."

6. LLM Evaluation & Metadata

6.1 Simple Explanation

"When evaluating an LLM or AI agent, I look at whether the answer is correct, useful, safe, and grounded. I also track metadata such as latency, token usage, model version, prompt version, retrieved documents, user feedback, and conversation outcome."

6.2 What Metadata Would You Track?

Prompt Metadata

- Prompt version
- System instructions
- User query
- Few-shot examples used
- Prompt template

Model Metadata

- Model name/version
- Temperature
- Token usage
- Latency
- Cost per request

Retrieval Metadata

- Retrieved document IDs
- Similarity scores
- Number of chunks retrieved
- Source documents used
- Retrieval latency

User Interaction Metadata

- User feedback
- Escalation status
- Conversation success/failure
- Number of turns
- Drop-off point

6.3 Simple Interview Answer

"I track whether the model gave the correct answer, how fast it responded, how many tokens it used, whether it hallucinated, whether users were satisfied, and whether the right documents were retrieved in RAG-based flows."

6.4 Metrics for LLM / Agent Evaluation

- Accuracy
- Relevance
- Groundedness
- Hallucination rate
- Latency
- Token usage
- Cost
- User satisfaction
- Conversation success rate
- Escalation rate
- Safety issue rate

6.5 How Do You Know an Agent Is Improving?

"I compare metrics before and after changes. If hallucinations decrease, conversation success increases, latency stays acceptable, and user satisfaction improves, then the agent is improving."

7. Prompt Engineering

7.1 What Is Prompt Engineering?

"Prompt engineering is the process of designing instructions, examples, constraints, and output formats to guide an LLM toward reliable and consistent responses."

7.2 Prompt Engineering Techniques

- Zero-shot prompting
- Few-shot prompting
- Role-based prompting
- Structured output prompting
- Tool-calling instructions
- Guardrail instructions
- Context injection
- Output formatting
- Chain-of-thought style reasoning guidance, where appropriate

7.3 Example Prompt Template

You are a customer support AI assistant.

Your goal:

- Answer only using the provided company policy context.
- If the answer is not available in the context, say you do not have enough information.
- Keep the response concise and helpful.
- Escalate to a human if the issue involves billing disputes or legal complaints.

User question:

{user_question}

Retrieved context:

{retrieved_context}

Answer:

7.4 What Is Temperature?

"Temperature controls randomness in model responses. Lower temperature makes answers more deterministic and consistent. Higher temperature makes answers more creative but less predictable."

Temperature	Behavior
0.0 - 0.2	Deterministic, consistent
0.3 - 0.7	Balanced
0.8+	Creative, more variable

7.5 Interview Line

"For customer support or enterprise agents, I usually prefer lower temperature because consistency and reliability matter more than creativity."

8. Productionizing AI Agents

8.1 What Does It Mean to Take an Agent to Production?

"Taking an AI agent into production means moving it from a prototype or demo into a reliable system that real users can use safely. It includes API integrations, authentication, testing, monitoring, logging, deployment, fallback handling, and continuous evaluation."

8.2 Production Checklist

- Clear scope and supported workflows
- API/database integrations
- Authentication and authorization
- Secure secret management
- Simulation tests
- Edge-case testing
- Monitoring and logging
- Human escalation path
- Rate limiting
- Error handling
- Cost monitoring
- Feedback loop
- Version control
- CI/CD pipeline
- Rollback strategy

8.3 Strong Interview Answer

"First, I would define the agent's scope and supported workflows. Then I would integrate the required APIs or databases, add authentication and permissions, and test common and edge-case user journeys. Before release, I would run simulation tests and evaluate accuracy, hallucinations, latency, and failure cases. After deployment, I would monitor live conversations, review negative feedback, create issues for failures, and continuously improve prompts, retrieval, and API handling."

8.4 Common Production Failure Cases

- Hallucinated response
- Wrong API called
- Missing context
- User asks unsupported question
- External API timeout
- Permission issue
- Sensitive data leakage
- High latency
- High token cost
- Retrieval failure
- Poor escalation handling

10. Current GenAI Role Explanation

10.1 Simple Explanation

"My current GenAI work focuses on LLM evaluation, prompt engineering, AI quality assessment, response analysis, and model behavior improvement. I review AI responses, identify hallucinations or weak reasoning, compare outputs, and provide structured feedback to improve accuracy, consistency, safety, and user experience."

10.2 How to Connect It to AI Engineering

"This experience helps me understand how GenAI systems behave in real-world scenarios. I have worked with evaluation criteria, edge cases, prompt strategies, and conversation flows. Now I want to apply that knowledge more directly in building and deploying AI agents end-to-end."

10.3 Key Skills to Mention

- LLM evaluation
- Prompt engineering
- Response quality assessment
- Hallucination detection
- Model behavior analysis
- AI workflow assessment
- Agentic workflow evaluation
- Data quality
- Structured feedback
- Production readiness

17. LLM / GenAI Application Development

Topics to revise

- Hosted LLM APIs.
- Azure OpenAI.
- OpenAI.
- Anthropic.
- Prompt design.
- Context construction.
- Chaining/orchestration.
- Tool/function calling.
- RAG systems.
- Summarization.
- Classification.
- Code assistants.
- Evaluation.
- Guardrails.
- Cost and latency management.

Common interview questions

- 1.** How have you worked with LLMs?
 - 2.** What is RAG?
 - 3.** How do you reduce hallucinations?
 - 4.** How do you evaluate an LLM application?
 - 5.** How do you manage token cost?
 - 6.** What are guardrails?
 - 7.** What is prompt engineering?
 - 8.** What is context construction?
 - 9.** How do you monitor LLM applications in production?
-

Example: Simple LLM wrapper pattern

```
from dataclasses import dataclass

@dataclass
class LLMResponse:
    text: str
    model: str
    prompt_tokens: int
    completion_tokens: int

class LLMClient:
    def __init__(self, provider: str, model: str) -> None:
        self.provider = provider
        self.model = model

    def generate(self, prompt: str) -> LLMResponse:
        # In real code, call provider SDK/API here.
        return LLMResponse(
            text="Sample generated answer",
            model=self.model,
            prompt_tokens=len(prompt.split()),
            completion_tokens=10,
        )
```

Interview explanation:

Wrapping LLM providers behind an internal interface makes it easier to switch providers, add retries, track cost, implement fallback routing, and standardize logging.

18. Prompt Engineering & Context Construction

Prompt engineering topics

- Clear instructions.
- Role/context setting.
- Constraints.
- Examples/few-shot prompting.
- Output format control.
- Chain-of-thought alternatives such as concise reasoning summaries.
- Prompt versioning.
- Prompt testing.

Context construction topics

- Selecting relevant retrieved chunks.
- Avoiding irrelevant context.
- Managing token limits.
- Ordering documents by relevance.
- Adding metadata.
- Separating system, developer, and user instructions.

Example prompt template

You are a technical assistant. Answer the user's question using only the provided context.

Context:
{retrieved_context}

Question:
{question}

Rules:
- If the context does not contain the answer, say you do not know.
- Do not invent facts.
- Cite the document section used.
- Keep the answer concise.

Common interview question

How do you reduce hallucinations using prompt design?

Answer:

I would ground the model with retrieved context, explicitly instruct it to answer only from the provided context, require it to say when information is missing, add citations or source references, and evaluate outputs against expected answers. Prompting helps, but it should be combined with retrieval quality, guardrails, and monitoring.

19. RAG: Retrieval-Augmented Generation

RAG is one of the most important technical areas for this role.

RAG pipeline

Documents

Ingestion

Parsing / Cleaning

Chunking

Embedding Generation

Vector Database

Retriever

Prompt Construction

LLM Generation

Evaluation / Monitoring

Topics to revise

- Document ingestion.
- Chunking strategies.
- Embeddings.
- Vector search.
- Hybrid search.
- Metadata filtering.
- Reranking.
- Prompt construction.
- Citation generation.
- Evaluation.
- Monitoring.

Example: Simple RAG pseudocode

```
class RAGService:
    def __init__(self, vector_store, llm_client) -> None:
        self.vector_store = vector_store
        self.llm_client = llm_client

    def answer(self, question: str) -> str:
        chunks = self.vector_store.search(query=question, top_k=5)
        context = "\n\n".join(chunk.text for chunk in chunks)

        prompt = f"""
        Answer the question using only this context.

        Context:
        {context}

        Question:
        {question}
        """

        return self.llm_client.generate(prompt).text
```

Chunking strategies

Strategy	Use case
Fixed-size chunks	Simple documents, fast baseline
Sliding window	Preserves context across chunk boundaries
Semantic chunking	Better for topic-based documents
Structure-aware chunking	PDFs, Markdown, HTML, legal/financial docs
Hierarchical chunking	Long documents where section-level retrieval helps

Common interview question

How would you build a RAG system?

Answer:

I would design an ingestion pipeline to parse and clean documents, split them into meaningful chunks, generate embeddings, store them in a vector database with metadata, retrieve relevant chunks at query time, optionally rerank the results, construct a grounded prompt, call the LLM, return the answer with

citations, and monitor retrieval quality, hallucination rate, latency, and cost.

20. Embeddings & Vector Databases

Embeddings

Embeddings are numerical representations of text that capture semantic meaning. Similar text should have similar vectors.

Vector databases mentioned or relevant

- FAISS.
- pgvector.
- Pinecone.
- Other managed vector stores.

Topics to revise

- Cosine similarity.
- Approximate nearest neighbor search.
- Embedding model selection.
- Metadata filtering.
- Hybrid search.
- Reranking.
- Vector index refresh.
- Embedding drift.

Example: Cosine similarity concept

```
import numpy as np

def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))
```

Common interview questions

1. What are embeddings?
2. What is a vector database?
3. What is semantic search?
4. What is hybrid search?
5. How do you choose chunk size?
6. How do you evaluate retrieval quality?

21. LLM Evaluation, Guardrails, and Safety

Evaluation topics

- Accuracy.
- Relevance.
- Completeness.
- Faithfulness/groundedness.
- Hallucination rate.
- Toxicity/safety.
- Instruction following.
- Human-in-the-loop review.
- Automated evaluation.
- Regression testing for prompts.

Guardrails topics

- Input validation.
- Output validation.
- Policy filters.
- PII redaction.
- Prompt injection detection.
- Grounding requirements.
- Safe fallback responses.
- Human escalation.

Example LLM evaluation checklist

For each generated answer:

1. Is it grounded in retrieved context?
2. Does it answer the actual question?
3. Is any unsupported claim present?
4. Is sensitive information exposed?
5. Is the tone appropriate?
6. Is the output format correct?
7. Does it include citations where required?

Common interview question

How would you evaluate a RAG system?

Answer:

I would evaluate both retrieval and generation. For retrieval, I would measure whether the correct documents or chunks appear in the top-k results. For generation, I would measure groundedness, factual accuracy, answer relevance, completeness, hallucination rate, and user satisfaction. I would use a mix of automated metrics, curated test sets, and human review.

23. Model Selection, Fine-Tuning, LoRA/QLoRA

Model selection topics

- Quality.
- Latency.
- Cost.
- Context window.
- Compliance requirements.
- Data residency.
- Provider reliability.
- Task type.

Fine-tuning topics

- When fine-tuning is useful.
- When RAG is better.
- Dataset quality.
- Evaluation before/after tuning.
- Overfitting risk.

LoRA / QLoRA

LoRA and QLoRA are parameter-efficient fine-tuning techniques used to adapt models without updating all model parameters.

Common interview question

When would you use RAG vs fine-tuning?

Answer:

I would use RAG when the model needs access to changing or private knowledge, such as internal documents or policies. I would consider fine-tuning when the model needs to learn a specific style, format, domain behavior, or task pattern that cannot be solved well through prompting and retrieval alone.

24. Multimodal AI Familiarity

The role mentions familiarity with models that can handle text, images, or structured data.

Topics to revise

- Text + image models.
- OCR/document understanding.
- Table extraction.
- Structured data reasoning.
- Multimodal prompt design.
- Safety concerns for images/documents.

Possible interview question

How would multimodal AI be useful in business applications?

Answer:

Multimodal AI can help process documents, screenshots, forms, invoices, charts, and images. For example, it can extract information from PDFs, summarize visual reports, classify document types, or support workflows where users upload screenshots or scanned files.

11. GenAI, ChatGPT API, and Prompt Engineering

11.1 What is the ChatGPT API?

Answer

The ChatGPT API allows developers to integrate large language model capabilities into applications.

Instead of using ChatGPT through a website, applications can send prompts programmatically from:

- Web apps
- Mobile apps
- Backend services
- Automation tools
- Enterprise systems

The API receives input, processes it using a language model, and returns a generated response.

11.2 Simple ChatGPT API flow

User/Application

Backend API

ChatGPT / LLM API

Generated Response

Application UI

11.3 Common use cases of ChatGPT API

- Chatbots
 - Resume analysis
 - Cover letter generation
 - Document summarization
 - Code assistance
 - Customer support automation
 - Search assistants
 - Content generation
 - Workflow automation
-

11.4 Pros of ChatGPT API

Advantage	Explanation
Easy integration	Can be used from backend services
Powerful language understanding	Handles natural language well
Flexible	Useful across many domains
Saves development time	No need to train a model from scratch
Scalable	Can support many AI features
Useful for automation	Helps automate repetitive text tasks

11.5 Cons of ChatGPT API

Limitation	Explanation
Hallucinations	Model can generate confident but incorrect answers
Cost	Token usage can become expensive
Latency	LLM responses can take seconds
Data privacy concerns	Sensitive data must be handled carefully
Output inconsistency	Same prompt can produce slightly different answers
Dependency risk	Application depends on external model/provider
Context limits	Large documents may exceed context size

11.6 What is prompt engineering?

Answer

Prompt engineering is the process of designing prompts to get accurate, relevant, structured, and consistent outputs from an AI model.

It includes:

- Clear instructions
- Context
- Examples
- Output format
- Constraints
- Tone/style guidance
- Validation rules

11.7 How do you know if a prompt is good?

Strong Interview Answer

A good prompt is one that consistently produces accurate, relevant, and structured outputs aligned with the intended objective. I do not judge a prompt by a single response; I judge it based on repeatability, clarity, and output quality across multiple test cases.

11.8 Criteria for a good prompt

1. Clarity

Bad prompt:

Write about Python.

Good prompt:

Explain Python multithreading in less than 200 words with one real-world example.

2. Specificity

Bad prompt:

Improve my resume.

Good prompt:

Optimize my resume for a Senior Python Developer role focusing on FastAPI, AWS, and GenAI while maintaining a 2-page limit.

3. Output structure

Good prompt:

Return the answer as JSON with fields:

- score
 - strengths
 - weaknesses
 - recommendations
-

4. Hallucination control

Good prompt:

Use only the information provided in the resume and job description.
Do not invent experience, skills, companies, or projects.

5. Consistency

A good prompt should produce similar quality outputs across multiple runs and multiple test cases.

6. Business alignment

A prompt is good if it solves the intended business problem.

Examples:

- Resume AI: better resume-job alignment
 - Chatbot: accurate customer answers
 - RAG system: grounded responses
 - Support bot: fewer escalations
-

13. LLMs, RAG, Prompt Engineering, and GenAI Evaluation

13.1 LLM model selection

Models discussed generally

- GPT-style models.
- Claude-style models.
- LLaMA/open-source models.
- Gemini-style models.
- Smaller/mini models.
- Embedding models.

Selection criteria

- Accuracy/reasoning.
- Latency.
- Cost.
- Context window.
- Data privacy.
- Deployment control.
- Fine-tuning/customization.
- Ecosystem/integration.

Interview wording

I choose the model based on use case constraints, not popularity. The main trade-offs are accuracy, latency, cost, privacy, and deployment control.

13.2 Claude-style model use cases

Strengths

- Long-context document analysis.
- Safer/controlled outputs.
- Structured summarization.
- Enterprise workflows.

When to use

- Large document processing.
- Compliance-heavy tasks.
- Internal knowledge assistants.

13.3 Why use RAG?

RAG solves

- Hallucinations.
- Need for private enterprise data.
- Need for up-to-date information.
- Need for traceability/auditability.
- Avoiding retraining for every knowledge update.

One-line answer

RAG grounds the model in retrieved trusted context before generation.

Banking/regulated style answer

In regulated domains, RAG is important because answers must be grounded, traceable, and auditable instead of relying on pure model memory.

13.4 RAG evaluation

Evaluate retrieval separately

- Did we retrieve the right chunks?
- Top-k relevance.
- Context precision/recall.

Evaluate generation separately

- Correctness.
- Groundedness.
- Completeness.
- Hallucination rate.
- Citation/source accuracy.

13.5 Prompt engineering

Effective prompt structure

1. Clear task.
2. Relevant context.
3. Constraints.
4. Output format.
5. Examples/few-shot if needed.
6. Safety instructions.
7. Evaluation/iteration.

Example

You are a risk analyst.
Analyze the following transaction data.
Use only the provided data.
If information is missing, say "Insufficient information".
Return JSON with risk_level and reason.

13.6 Prompt versioning

Production principle

Treat prompts like code: versioned, tested, deployed, monitored, and rollbackable.

Track

- Prompt version.
- Model version.
- Retrieval config.
- Temperature/top_p.
- Dataset used.
- Evaluation metrics.
- Latency/cost.

13.7 MLflow for prompt/model experiments

What to log

- Prompt version.
- Model version.
- Parameters.
- Evaluation metrics.
- Latency.
- Token usage/cost.
- Sample outputs.
- Error cases.

Best practice

Use Git/config for source control and MLflow for experiment tracking.

13.8 LLM evaluation and factual accuracy

Criteria

- Factual correctness.
- Groundedness.
- Relevance.
- Completeness.
- Consistency.
- Safety/compliance.
- Latency/cost.

Consistency and objectivity

- Fixed golden dataset.
 - Structured scoring rubrics.
 - Blind comparisons when possible.
 - Automated + human evaluation.
 - Versioned prompts/models.
-

13.9 Stabilizing AI systems with regression tests

Golden evaluation set

Include:

- Common prompts.
- Edge cases.
- Known failures.
- Risky compliance/finance cases.
- Expected behaviors.

Release gate

Block deployment if:

- Correctness drops.
 - Hallucination rises.
 - Groundedness falls.
 - Latency/cost exceeds threshold.
-

LLMs & Generative AI

What Is a Large Language Model?

Interview Question

What is an LLM?

Answer

A Large Language Model is a deep learning model trained on large amounts of text to understand and generate human-like language. Most modern LLMs are based on the Transformer architecture.

Common Capabilities

- Text generation
 - Summarization
 - Question answering
 - Code generation
 - Classification
 - Reasoning assistance
 - Information extraction
-

Transformers

Interview Question

Why are Transformers important in modern AI?

Answer

Transformers use self-attention to understand relationships between tokens in a sequence. This allows them to process context more effectively than older architectures like RNNs.

Key Concepts

- Tokenization
 - Embeddings
 - Self-attention
 - Multi-head attention
 - Positional encoding
 - Feed-forward layers
-

RLHF

Interview Question

Explain RLHF.

Answer

RLHF stands for **Reinforcement Learning from Human Feedback**. It is used to align model outputs with human preferences.

Typical RLHF Pipeline

1. Pretrain a base model
2. Collect human preference data
3. Train a reward model
4. Fine-tune the model using reinforcement learning
5. Evaluate output quality and safety

Strong Sample Answer

RLHF is a technique used to improve the behavior of language models using human feedback. Instead of only training on next-token prediction, the model is further optimized based on human preferences, such as helpfulness, correctness, clarity, and safety.

Prompt Engineering

Interview Question

What makes a good prompt?

Answer

A good prompt is clear, specific, contextual, and testable.

Good Prompt Characteristics

Characteristic	Meaning
Clear goal	The model knows what to do
Context	Provides necessary background
Constraints	Defines boundaries
Output format	Specifies structure
Examples	Shows expected behavior
Evaluation criteria	Makes quality measurable

Example

Weak prompt:

Summarize this.

Better prompt:

Summarize the following technical incident report in 5 bullet points. Focus on root cause, customer impact, mitigation, long-term fix, and unresolved risks. Avoid adding assumptions not supported by the text.

Prompt Orchestration

Interview Question

What is prompt orchestration?

Answer

Prompt orchestration is the process of coordinating multiple prompts, tools, memory, retrieved context, and intermediate reasoning steps to complete a complex task.

Example Workflow

```
User query
-> intent classification
-> retrieval
-> prompt construction
-> tool call if needed
-> response generation
-> validation
-> final answer
```

RAG: Retrieval-Augmented Generation

What Is RAG?

Interview Question

Explain RAG.

Answer

RAG stands for **Retrieval-Augmented Generation**. It combines search/retrieval with LLM generation.

Instead of relying only on the model's internal knowledge, a RAG system retrieves relevant external context and gives that context to the model before generation.

RAG Pipeline

```
Documents
-> chunking
-> embeddings
-> vector database
-> user query
-> query embedding
-> similarity search
-> retrieve top-k chunks
-> prompt with context
-> LLM response
```

Why Use RAG?

Benefits

- Reduces hallucination
 - Adds current knowledge
 - Supports enterprise/private data
 - Avoids expensive full model fine-tuning
 - Improves traceability through sources
-

RAG Code Example

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

documents = [
    "Python is commonly used for machine learning.",
    "FastAPI is a modern Python web framework.",
    "Vector databases store embeddings for similarity search."
]

model = SentenceTransformer("all-MiniLM-L6-v2")

embeddings = model.encode(documents)
embeddings = np.array(embeddings).astype("float32")

dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(embeddings)

query = "What is used for similarity search?"
query_embedding = model.encode([query]).astype("float32")

distances, indices = index.search(query_embedding, k=2)

for idx in indices[0]:
    print(documents[idx])
```

Chunking Strategies

Interview Question

What are different ways to chunk documents for RAG?

Types of Chunking

Strategy	Description	Best For
Fixed-size chunking	Split by token/character count	Simple documents
Sliding window	Overlapping chunks	Preserving context
Sentence-based	Split by sentence boundaries	Clean text
Paragraph-based	Split by paragraphs	Reports/articles
Semantic chunking	Split by topic meaning	Complex documents
Structure-aware	Split by headings/sections	PDFs, manuals, docs

Example

```
def chunk_text(text: str, chunk_size: int = 500, overlap: int = 100):
    chunks = []
    start = 0

    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start += chunk_size - overlap

    return chunks

sample_text = "A" * 2000
chunks = chunk_text(sample_text)

print(len(chunks))
```

RAG Evaluation

Interview Question

How do you evaluate a RAG system as a whole?

Evaluation Areas

Area	Question
Retrieval quality	Did we retrieve the right context?
Generation quality	Did the answer use the context correctly?
Faithfulness	Is the answer grounded in retrieved documents?
Relevance	Does the answer address the user query?
Completeness	Is anything important missing?
Latency	Is the system fast enough?
Cost	Are token and infrastructure costs acceptable?

Useful Metrics

- Recall@K
 - Precision@K
 - MRR
 - NDCG
 - Faithfulness score
 - Answer relevance
 - Context relevance
 - Hallucination rate
-

AI Agents & Tool Calling

What Is an AI Agent?

Interview Question

What is an AI agent?

Answer

An AI agent is an LLM-powered system that can plan steps, use tools, observe results, and iterate toward completing a goal.

Agent Workflow

```
User goal
-> planning
-> tool selection
-> tool execution
-> observation
-> reasoning
-> final response
```

Agent vs Generative AI

Interview Question

What is the difference between Agentic AI and Generative AI?

Answer

Generative AI	Agentic AI
Generates content	Takes actions toward a goal
Usually single-turn or prompt-based	Multi-step workflow
Produces text/images/code	Uses tools/APIs/systems
Example: summarize document	Example: investigate incident and create report

Strong Answer

Generative AI produces outputs such as text, code, or summaries. Agentic AI goes further by planning, calling tools, checking results, and iterating until it completes a task.

Tool Calling

Interview Question

What is tool calling in LLM systems?

Answer

Tool calling allows an LLM to call external functions, APIs, databases, or services when it needs information or actions beyond text generation.

Example Tool Calling Flow

```
User: What is the status of order 123?  
LLM identifies required tool: get_order_status(order_id=123)  
Tool returns data  
LLM summarizes answer to user
```

Python-Style Tool Example

```
def get_order_status(order_id: str) -> dict:  
    return {  
        "order_id": order_id,  
        "status": "shipped",  
        "estimated_delivery": "2026-07-05"  
    }  
  
def agent_response(order_id: str) -> str:  
    status = get_order_status(order_id)  
    return (  
        f"Order {status['order_id']} is {status['status']} "  
        f"and is expected by {status['estimated_delivery']}."  
    )  
  
print(agent_response("123"))
```

Embeddings, Vector Databases & Indexing

Embeddings

Interview Question

What is an embedding?

Answer

An embedding is a numerical vector representation of data such as text, images, or audio. It captures semantic meaning so similar items are close together in vector space.

Example

```
"car" and "vehicle" should have similar embeddings.  
"car" and "banana" should be farther apart.
```

Vector Databases

Interview Question

Why use a vector database?

Answer

A vector database stores embeddings and supports similarity search. It is commonly used in RAG systems and recommendation systems.

Examples

- FAISS
- ChromaDB
- Pinecone
- Weaviate
- Milvus
- Qdrant

Similarity Search

Common Similarity Metrics

Metric	Use
Cosine similarity	Measures angle/direction
Dot product	Common in embedding retrieval
Euclidean distance	Measures direct distance

Simple Cosine Similarity Example

```
import numpy as np

def cosine_similarity(a, b):
    a = np.array(a)
    b = np.array(b)

    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

vector_1 = [1, 2, 3]
vector_2 = [1, 2, 4]

print(cosine_similarity(vector_1, vector_2))
```

Indexing Strategies

Interview Question

What are indexing strategies for vector search?

Answer

Indexing strategies improve retrieval speed and scalability.

Strategy	Description
Flat index	Exact search, slower at scale
HNSW	Graph-based approximate nearest neighbor search
IVF	Clusters vectors before search
PQ	Compresses vectors to reduce memory
Hybrid search	Combines keyword + vector search

Strong Answer

For small datasets, exact search may be enough. For large-scale systems, approximate nearest neighbor indexes like HNSW or IVF are better because they reduce latency while maintaining high recall.

LLM Evaluation & Hallucination Reduction

What Is Hallucination?

Interview Question

What is hallucination in LLMs?

Answer

Hallucination happens when an LLM generates information that sounds confident but is false, unsupported, or not grounded in the provided context.

How to Reduce Hallucinations

Techniques

- Use RAG with trusted sources
- Require citations or source references
- Add instruction to avoid unsupported claims
- Use validation checks
- Use structured outputs
- Use retrieval confidence thresholds
- Use human review for high-risk cases
- Evaluate outputs against ground truth

Example Prompt Instruction

Answer only using the provided context. If the context does not contain the answer, say: "I do not have enough information to answer."

LLM Evaluation

Interview Question

How do you evaluate an LLM system?

Evaluation Dimensions

Dimension	Meaning
Accuracy	Is the answer correct?
Relevance	Does it answer the question?
Faithfulness	Is it grounded in context?
Completeness	Does it cover necessary details?
Safety	Does it avoid harmful output?
Consistency	Similar inputs produce stable outputs
Latency	Response time
Cost	Token and infrastructure cost

Evaluation Methods

- Human evaluation
 - Golden test sets
 - LLM-as-judge
 - Automated metrics
 - A/B testing
 - Regression testing
-

BLEU and ROUGE

Interview Question

What are BLEU and ROUGE?

Answer

BLEU and ROUGE are text evaluation metrics.

Metric	Common Use
BLEU	Machine translation
ROUGE	Summarization

BLEU

BLEU measures n-gram overlap between generated text and reference text.

ROUGE

ROUGE measures overlap, recall, and similarity between generated summaries and reference summaries.

Important Limitation

BLEU and ROUGE can be useful, but they do not always capture semantic correctness, factuality, or usefulness. For LLM applications, human evaluation and task-specific metrics are often needed.

8. AI / LLM / GenAI Integration

Likely Questions

- What is your experience with LLMs?
- Have you worked with OpenAI or Claude?
- How do you integrate AI into a production workflow?
- What is prompt engineering?
- What is hallucination?
- How do you reduce hallucinations?
- What is RAG?
- What is structured output?
- How do you evaluate LLM responses?
- How do you use AI tools while maintaining code quality?
- Difference between ML research and AI product integration?
- How do you secure AI features?

How to Explain Your AI Experience

I have worked with LLM evaluation, prompt-based workflows, model output analysis, and AI quality assessment. My interest is not only in evaluating models, but in building software systems that integrate AI into practical workflows.

Prompt Engineering

Good Prompt Structure

You are an assistant helping summarize customer orders.

Task:

Summarize the order issue in 3 bullet points.

Rules:

- Do not invent missing information.
- If the customer did not provide a shipping address, say "shipping address not provided".
- Keep the summary under 80 words.

Input:

{customer_message}

Why This Is Good

- Defines role
 - Gives a specific task
 - Adds constraints
 - Defines output style
 - Reduces hallucination
-

Structured Output Example

```
from pydantic import BaseModel
from typing import Literal

class OrderIssue(BaseModel):
    issue_type: Literal["shipping_delay", "payment_issue", "return_request", "other"]
    urgency: Literal["low", "medium", "high"]
    summary: str
```

Interview Explanation

Structured outputs help because:

- Backend services need predictable formats.
- JSON is easier to validate.
- Pydantic can reject invalid responses.
- Downstream systems can safely consume the result.

LLM Integration Pseudocode

```
def classify_customer_message(message: str) -> OrderIssue:
    prompt = build_prompt(message)

    response = llm_client.generate(
        prompt=prompt,
        response_format="json"
    )

    return OrderIssue.model_validate_json(response)
```

Production Considerations

- Validate model output
- Add retries and timeouts
- Log prompt/version metadata
- Avoid sending sensitive data unnecessarily
- Have fallback behavior
- Monitor cost and latency
- Evaluate quality over time

Hallucination Reduction

Use:

- Clear instructions
- Source-grounded context
- RAG
- Structured outputs
- Confidence thresholds
- Human review for high-risk actions
- Refusal rules
- Post-generation validation

Example Guardrail

```
def safe_answer(question: str, retrieved_docs: list[str]) -> str:
    if not retrieved_docs:
        return "I do not have enough information to answer that."

    prompt = (
        "Answer only using the provided context. "
        "If the answer is not in the context, say you do not know.\n\n"
        f"Context:\n{retrieved_docs}\n\n"
        f"Question:\n{question}"
    )

    return llm_client.generate(prompt)
```

RAG High-Level Flow

User question

Embed question

Search vector database

Retrieve relevant documents

Send context + question to LLM

Generate grounded answer

Validate / cite / log response

Interview Explanation

RAG is useful when the model needs company-specific, document-specific, or fresh information that is not reliably stored in its training data.

AI Evaluation Criteria

Evaluate outputs for:

- Correctness
- Relevance
- Completeness
- Faithfulness to source
- Safety
- Tone
- Format compliance
- Latency
- Cost
- User usefulness

Using AI Tools as an Engineer

Good Answer

I use AI tools to move faster with drafting, debugging ideas, boilerplate, test generation, and code review support. But I treat AI output as a suggestion, not as final code. I still review logic, security, edge cases, tests, and production impact before shipping anything.

AI Evaluation Work Versus AI Product Engineering

AI-focused roles can contain both model-quality work and software engineering. Interview answers should distinguish the activities rather than describing all AI work as prompt writing.

Evaluation and Prompting Responsibilities

- Design representative evaluation cases.
- Compare model outputs against rubrics or reference behavior.
- Analyze hallucination, relevance, safety, and instruction-following failures.
- Refine prompts and structured output requirements.
- Validate retrieval quality and grounding.
- Review tool selection and tool arguments.
- Track regressions across model or prompt versions.

Engineering Responsibilities Around Evaluation

- Build Python automation for repeatable evaluations.
- Normalize datasets and model responses.
- Create scoring and aggregation pipelines.
- Integrate model, retrieval, and tool APIs.
- Persist experiment results and metadata.
- Add retries, rate limits, concurrency controls, and error handling.
- Produce logs, reports, and dashboards for failure analysis.

A credible explanation can state an approximate split only when it reflects the real role. The more important point is to identify which deliverables were code and which were human evaluation.

Production API Ownership Versus Supporting Integrations

These are different levels of ownership:

Level	Typical Responsibility
Model evaluation	Rubrics, test cases, output analysis, regression findings
Evaluation engineering	Python harnesses, datasets, scoring, automation, reporting
AI integration	Calling model/retrieval services from an application workflow
Production API ownership	Contracts, auth, persistence, deployment, SLOs, on-call, backward compatibility

Do not claim end-to-end production API ownership when the recent role mainly covered the first two levels. Instead, connect that work to earlier backend experience and current personal implementations.

Transitioning from Evaluation to Backend Platform Work

The transition is not from nontechnical work to engineering. It is a shift in where the engineering effort is concentrated.

Transferable skills include:

- Python development and debugging.
- Data transformation and schema validation.
- External API integration.
- Test design and regression analysis.
- Reliability controls for model calls.
- Clear separation between deterministic and probabilistic behavior.

The platform-specific refresh should focus on the team's current web framework, service conventions, database patterns, deployment pipeline, and operational tooling.

AI as an Additional Capability

A useful positioning statement is:

My backend foundation remains the core. Recent AI work added experience with model behavior, evaluation, prompt workflows, retrieval, and AI reliability. I can apply those capabilities inside a conventional production platform without assuming every workflow should become agentic.

Safe Tool-Using Agents: Orchestration, State, Cost, and Evaluation

A production agent should not let model-generated text directly execute arbitrary code or business actions. A safer design separates model decisions from trusted orchestration and tool execution.

Production Agent Architecture

Client

- > API / authentication layer
- > agent orchestrator / task state
- > policy and authorization engine
- > model gateway -----> retrieval service
- > validated tool registry
- > domain services / external systems
- > audit logs, traces, metrics, evaluations

Core rule: the model proposes what to do, while trusted application code decides whether that action is valid and authorized. This keeps authentication, business invariants, financial rules, and tenant isolation outside nondeterministic model output.

ReAct-Style Agent Loop From Scratch

A simple ReAct-style engineering loop is:

User goal

- > model decides: final answer or tool call
- > orchestrator validates decision
- > authorized tool executes
- > structured observation is returned to model
- > repeat until complete or bounded limit is reached

The important part is not exposing hidden reasoning. The important engineering behavior is the repeated **decision -> action -> observation** cycle with explicit validation and termination.

Pseudo-code:

```
for step in range(MAX_TOOL_STEPS):
    decision = model.decide(messages, tool_schemas)

    if decision.type == "final":
        return decision.answer

    tool = registry.get(decision.tool_name)
    validate_tool_call(tool, decision.arguments)
    enforce_permissions(tool, context)
    result = tool.execute(**decision.arguments)
    messages.append(structured_observation(result))

return escalate_to_human("Tool-call limit reached")
```

Structured Tool Contracts

A strong tool contract should define:

- A precise, action-oriented name.
- A description explaining when the tool should be used.
- Structured input parameters and required fields.
- Predictable success and error output.
- Read-only versus state-changing classification.
- Authentication and authorization requirements.
- Idempotency expectations.
- Timeout and retry behavior.
- Audit metadata.

Example OpenAI-compatible schema:

```
CHECK_STOCK_TOOL = {
```

```
"type": "function",
"function": {
  "name": "check_realtime_stock",
  "description": "Check current warehouse quantity for a known SKU.",
  "parameters": {
    "type": "object",
    "properties": {
      "sku": {"type": "string"},
    },
    "required": ["sku"],
    "additionalProperties": False,
  },
},
},
}
```

Purpose-built business tools are usually safer than exposing every low-level API endpoint to a model. They reduce ambiguity, centralize domain validation, and make authorization and evaluation easier.

Bounded Execution and Cost Ceilings

Agent loops must have hard limits. Useful limits include:

- Maximum tool-call iterations.
- Maximum wall-clock duration.
- Token budget.
- Per-tool timeout.
- Per-user or per-tenant quotas.

A simple cost ceiling is:

```
maximum tool cost per task
= maximum tool calls per task
x average or worst-case cost per tool call
```

For example, if a workflow allows at most five tool-call iterations, the orchestrator has a predictable upper bound even if the model keeps asking for more actions. When the limit is reached, stop safely and hand off rather than looping indefinitely.

Avoiding Duplicate Calls Without Using Stale State

Within one active task, repeated read-only lookups can be cached. A useful cache key is:

```
(order_id, intent)
```

This distinguishes different tasks for the same entity:

```
(123, "tracking") != (123, "refund_eligibility")
```

Clear task-local cached state when any of these change:

- Order or entity ID.
- Intent.
- Authentication state.
- User cancels or contradicts the request.
- A final answer or human handoff completes the task.

The key rule is: **cache only within the active task, and never let cost optimization override correctness.**

Fresh Revalidation Before High-Impact Actions

Time-sensitive or financial decisions should bypass stale cached eligibility. Immediately before a protected action:

1. Re-check authentication/authorization.
2. Re-read the latest authoritative state.
3. Re-run eligibility or invariant checks.
4. Enforce approval thresholds.
5. Require explicit user confirmation where appropriate.
6. Use an idempotency key for the write.

This protects against state-state bugs such as a second refund occurring between the original eligibility check and the final action.

Human-in-the-Loop Boundaries

A useful action classification is:

Risk level	Example	Typical policy
Read-only	Search catalog, retrieve status	Allow after required authorization
Reversible low-risk write	Create a draft or internal note	Validate, log, possibly confirm
High-impact write	Send money, change address, cancel order, change permissions	Human approval or human-only execution

The model may recommend an action, but trusted application code should enforce the policy independently of the prompt.

Guardrails for Infrastructure and Operations Agents

Infrastructure agents can have a much larger blast radius than ordinary conversational agents, so permissions and execution paths should be designed around failure rather than assuming the model will always choose correctly.

A practical risk policy is:

Action class	Example	Default policy
Read-only analysis	Inspect metrics, logs, deployment status, or Terraform plan	Allow within scoped permissions and audit every call
Reversible low-risk action	Restart one non-critical workload or create a draft change	Allow only inside tight policy bounds with validation and rollback
High-impact production write	Delete infrastructure, change IAM, major scale action, irreversible DB change	Require explicit human approval or keep the action human-only

Additional controls should include:

- Task-scoped and least-privilege credentials rather than broad cloud administrator access.
- Tool allowlists and resource allowlists.
- Quotas and concurrency limits.
- Dry-run/plan modes before mutation where the underlying tool supports them.
- Policy-as-code validation independent of the LLM.
- Structured change requests instead of free-form shell execution.
- Full audit logs containing actor, tool, arguments, target resource, result, and approval identity.
- Post-change health validation.
- Deterministic rollback or recovery procedures.
- Automatic stop conditions when telemetry becomes ambiguous or the requested action exceeds policy.

Interview answer:

I would not give an infrastructure agent broad credentials and trust the prompt to keep it safe. Read-only analysis can often run automatically, low-risk reversible actions can execute inside narrow policy boundaries, and high-impact actions such as privilege changes, production deletion, major scaling, or irreversible database operations should require explicit approval. Trusted code should enforce permissions, quotas, policy checks, audit logging, validation, and rollback independently of the model.

Robust Structured-Output Parsing

Models sometimes return structured data surrounded by prose or Markdown fences. The orchestrator should not blindly call `json.loads()` on the entire response and should not use `eval()`.

A safer approach:

1. Prefer native structured-output/tool-call APIs when available.
2. Otherwise strip code fences and scan for candidate JSON objects.
3. Parse with `json.JSONDecoder().raw_decode()` or an equivalent strict parser.
4. Validate the parsed object against the expected schema.
5. Reject malformed or ambiguous output rather than guessing.

```
import json

def first_json_object(text: str) -> dict | None:
    decoder = json.JSONDecoder()

    for index, char in enumerate(text):
        if char != "{":
            continue

        try:
            value, _ = decoder.raw_decode(text[index:])
        except json.JSONDecodeError:
            continue

        if isinstance(value, dict):
            return value

    return None
```


Model-Provider Abstraction

Wrap provider SDKs behind a small internal interface so orchestration code does not depend on one vendor's response format.

Useful responsibilities for the adapter layer:

- Normalize messages and structured outputs.
- Map provider-specific errors.
- Apply retries and timeouts.
- Track latency, tokens, and cost.
- Support fallback or routing policies.
- Record model and prompt versions.

This makes it easier to switch models without rewriting domain services or tool authorization.

MCP and Tool Surfaces

Model Context Protocol-style tool exposure is most useful when the tool layer remains a deliberate business interface rather than a raw mirror of internal APIs. The same production rules still apply:

- Authenticate the caller independently of model text.
- Apply tenant and object-level authorization at execution time.
- Validate every argument.
- Keep high-risk actions behind approval boundaries.
- Emit audit events for reads and writes.

Agent Evaluation Beyond Answer Quality

Evaluate an agent at several layers:

Dimension	Question
Task success	Did the workflow achieve the requested business outcome?
Tool selection	Did the agent choose the correct tool?
Argument correctness	Were tool arguments complete and valid?
Groundedness	Is the final response supported by tool/retrieval evidence?
Safety	Were permissions, escalation, and protected actions respected?
Efficiency	Did it avoid unnecessary calls and loops?
Recovery	Did it handle timeouts, malformed output, and tool failures?
Cost/latency	Was the workflow operationally acceptable?

A strong evaluation suite includes golden scenarios, adversarial inputs, malformed tool outputs, permission failures, stale-state scenarios, prompt-injection attempts, and regression tests over complete tool traces.

High-Stakes RAG and Financial Data

For financial or other high-stakes workflows:

- Enforce tenant permissions before retrieval, not after generation.
- Preserve source IDs and citations.
- Separate retrieval quality from generation quality in evaluation.
- Prefer deterministic services/queries for balances, totals, dates, and eligibility calculations.
- Use the LLM for interpretation and communication rather than asking it to invent authoritative numeric state.
- Abstain or escalate when evidence is insufficient.

Interview answer:

I treat an agent as a distributed application with an LLM inside it, not as a prompt that can directly perform actions. The model can choose a proposed tool call, but trusted code validates the schema, checks permissions, executes the tool, records the result, and enforces step and cost limits. I cache safe read results only within the active (entity_id, intent) task, clear that state when intent changes, and always revalidate authoritative state before financial or other high-impact actions.

Retrieval Top-K, LangGraph, and MCP Protocol Deep Dive

What Are Top-K Chunks in a RAG Pipeline?

Top-k chunks are the k highest-ranked document segments returned by the retrieval stage for a query. They are evidence candidates, not automatically correct context.

```
documents
  parse / clean / classify
chunks + metadata
  embeddings + lexical index
query
  authorization and metadata filters
candidate retrieval
  reranking / deduplication
best k chunks
  prompt/context assembly
LLM answer + citations
```

If $k = 3$, a vector search might return:

Rank	Chunk	Similarity	Meaning
1	18	0.91	Strongest semantic match
2	42	0.87	Additional supporting evidence
3	7	0.82	Third context candidate

Why k Matters

If k is too small:

- Relevant evidence can be missed.
- Multi-section answers become incomplete.
- Retrieval recall falls.

If k is too large:

- Irrelevant or conflicting context enters the prompt.
- Token cost and latency increase.
- Important evidence can be diluted or lost in the middle.
- The model has more opportunities to attach a claim to the wrong source.

Candidate Retrieval vs Final Context

A strong production design commonly retrieves more candidates than it finally sends:

```
hybrid retrieval: 20 candidates  
  
cross-encoder / learned reranker  
  
deduplicate + diversify  
  
best 3-6 chunks in final context
```

Initial retrieval optimizes recall. Reranking and context selection optimize precision.

```
candidate_chunks = vector_store.similarity_search(  
    query=user_question,  
    k=20,  
)  
  
ranked_chunks = reranker.rank(  
    query=user_question,  
    documents=candidate_chunks,  
)  
  
context_chunks = deduplicate(ranked_chunks)[:5]
```

How to Tune Top-K

Use a representative evaluation set and measure:

- Recall@k / whether required evidence appears
- Context precision / proportion of supplied chunks that are useful
- Mean reciprocal rank or nDCG when ranking quality matters
- Answer groundedness, correctness, completeness, and citation accuracy
- Latency, prompt tokens, and cost
- Performance by query type rather than one global average

A reasonable starting experiment might retrieve 10-20 candidates and send 3-6 after reranking, but the correct values depend on chunk size, corpus, query complexity, filters, and context budget.

Interview answer:

Top-k chunks are the k highest-ranked document segments returned for a query. A small k can miss evidence while a large k introduces noise and cost. I normally retrieve a broader authorized candidate set, rerank and deduplicate it, and place only the strongest few chunks into the model context. I tune both candidate and final k using retrieval recall, context precision, groundedness, citation quality, latency, and token cost.

LangGraph as an Orchestration Runtime

LangGraph is a low-level orchestration runtime for long-running, stateful agents. Its key value is explicit state and control flow rather than a particular prompt or model provider.

Core ideas:

- **State:** Typed shared data carried between steps.
- **Nodes:** Deterministic functions, model calls, tools, validators, or approval steps.
- **Edges:** Fixed or conditional transitions.
- **Persistence/checkpointing:** Durable execution and recovery from an intermediate state.
- **Human-in-the-loop:** Interrupt, inspect, approve, edit, and resume.
- **Streaming:** Surface tokens, events, or state updates during execution.
- **Memory:** Short-term workflow state and, when designed explicitly, longer-term cross-session memory.

```
from typing import TypedDict

from langgraph.graph import END, START, StateGraph

class ReviewState(TypedDict):
    request: str
    risk: str
    approved: bool

def classify(state: ReviewState):
    return {"risk": classify_risk(state["request"])}

def route_after_classification(state: ReviewState):
    return "approval" if state["risk"] == "high" else "execute"

graph = StateGraph(ReviewState)
graph.add_node("classify", classify)
graph.add_node("approval", request_human_approval)
graph.add_node("execute", execute_authorized_action)
graph.add_edge(START, "classify")
graph.add_conditional_edges("classify", route_after_classification)
graph.add_edge("approval", "execute")
graph.add_edge("execute", END)
compiled = graph.compile(checkpointer=checkpointer)
```

Workflow vs Agent

- A deterministic workflow follows application-defined transitions.
- An agent can let a model select a route or tool within bounded policy.
- A production graph can combine both: deterministic authorization and approval around model-driven planning.

Checkpointer vs Long-Term Store

- A checkpointer persists graph execution state for a thread/run so it can resume safely.
- A long-term store holds intentionally selected information across sessions/users.
- Neither should become an uncontrolled dump of prompts, secrets, or sensitive data.

Failure and Recovery

Design each node for:

- Stable operation IDs and idempotency
- Timeouts and bounded retries
- Typed output validation
- Checkpoint placement before/after side effects
- Compensation or reconciliation for uncertain external results
- Maximum step, time, and cost limits
- Traceable transitions and model/tool versions

Reference: [LangGraph overview](#)

Model Context Protocol (MCP) - 2026-07-28 Notes

MCP standardizes how an AI host communicates with servers that expose context and capabilities. It is a protocol boundary, not an authorization system or automatic safety layer.

Architecture

```
Host application
|
+-- MCP client A <--> MCP server A
|
+-- MCP client B <--> MCP server B
```

- **Host:** Coordinates AI integration, creates clients, controls connection permissions, enforces consent/security policy, and aggregates context.
- **Client:** Maintains a one-to-one relationship with one server and attaches protocol version/capabilities to requests.
- **Server:** Exposes focused capabilities and context through protocol primitives.

Server Primitives

- **Tools:** Callable operations such as querying a system, invoking an API, or performing a computation.
- **Resources:** Context/data that a host can read or subscribe to according to server capabilities.
- **Prompts:** Reusable parameterized prompt templates exposed by a server.

Treat every tool description and external result as untrusted input until validated by host policy.

Transports in the 2026-07-28 Revision

MCP uses JSON-RPC semantics over transport bindings:

- **stdio:** Newline-delimited messages over a client-launched local subprocess.
- **Streamable HTTP:** Each request/notification is an HTTP POST to one MCP endpoint; a response is a JSON object or a request-scoped SSE stream.

For the 2026-07-28 Streamable HTTP revision:

- GET and DELETE to the MCP endpoint return 405 Method Not Allowed.
- Protocol-level sessions and the GET stream endpoint were removed.
- A request-scoped SSE response is not a resumable global session stream.
- Older protocol revisions require explicit backward-compatibility behavior/version fallback.

Do not memorize a transport detail without naming the specification revision because MCP has evolved.

Capability Negotiation and Extensions

Protocol version and supported capabilities are negotiated/attached according to the revision. Optional extensions require explicit support by both sides. The 2026-07-28 specification describes opt-in extensions such as long-running Tasks; an application should not assume an extension exists without negotiation.

Secure MCP Design

model proposes tool call

host validates schema

authenticate actor from trusted context

authorize actor + tenant + resource + operation

approval for sensitive/irreversible action

execute with least-privilege, short-lived credential

validate result + audit + return bounded output

Controls:

- User consent and clear visibility into exposed/invoked tools
- Least-privilege credentials scoped to actor, tenant, resource, and action
- Input/output schema validation and size limits
- Prompt-injection separation: tool/resource content cannot override host policy
- Human approval for sensitive or irreversible operations
- Idempotency and fresh-state revalidation before writes
- Timeouts, rate limits, bounded retries, and circuit protection
- Redacted audit events and trace correlation
- SSRF/egress restrictions for remote endpoints
- No secrets placed in model-visible context

MCP vs Direct Function Calling

Direct function calling	MCP
Tool contract is embedded in one app/provider integration	Standardized client/server protocol boundary
Simpler for a small local tool set	Useful for discoverable, reusable integrations
App owns transport and discovery	Protocol defines primitives, messages, and transports

MCP is valuable when standardized discovery and reuse justify the additional protocol, lifecycle, compatibility, and security surface.

Interview answer:

MCP defines a standard boundary between a host, one client per connected server, and servers exposing tools, resources, and prompts. The protocol makes integrations discoverable and reusable, but it does not make them trusted. The host still authenticates the user, authorizes each resource/action, validates schemas, requests approval for high-impact operations, executes with least privilege, and audits the result. I would also name the protocol revision when discussing transport behavior because Streamable

HTTP changed in the 2026-07-28 specification.

References:

- [MCP specification 2026-07-28](#)
- [MCP architecture](#)
- [MCP transport overview](#)
- [MCP tools](#)

Cloud, DevOps, Docker, Kubernetes & CI/CD

Source: docs/cloud_devops.md

Purpose: AWS, Azure, Terraform, Docker, Kubernetes, Git, Jenkins, GitHub Actions, CI/CD, deployment, and cloud architecture. **Use this file for:** DevOps, backend platform, cloud engineering, and production deployment interviews

Recommended Study Flow

1. Read the **Quick Summary** first.
2. Review the **Key Concepts** and tables.
3. Practice the **Interview Questions & Answers** out loud.
4. Use the code snippets and examples to explain trade-offs clearly.
5. Finish with the **Common Mistakes** and **Revision Checklist** sections.

Quick Summary

This is a new topic file created because the attached repository files did not have a dedicated Markdown page for this subject. It is merged from the organized topic-wise interview-prep pack and follows the same repository style as the existing notes.

Consolidated Interview Questions & Technical Notes

AWS, Azure, Terraform, Docker, Kubernetes, Git, CI/CD pipelines, deployment patterns, infrastructure, containers, and cloud architecture.

7. AWS, CI/CD, Docker & Kubernetes

7.1 AWS deployment methods

Service	Use Case
EC2	Full control over VM deployment
ECS	Managed Docker containers
EKS	Managed Kubernetes
Lambda	Serverless event-driven functions
Elastic Beanstalk	Simple app deployment
App Runner	Simple containerized app deployment
SageMaker	ML training and model endpoints

7.2 FastAPI deployment on AWS

Developer
GitHub
CI/CD Pipeline
Run tests
Build Docker image
Push to Amazon ECR
Deploy to EC2/ECS/EKS
Load Balancer
Users

Interview answer:

I would containerize the FastAPI application using Docker, push the image to Amazon ECR, and deploy it on EC2, ECS, or EKS depending on scale. I would use Jenkins or GitHub Actions for CI/CD, place an Application Load Balancer in front, store secrets in AWS Secrets Manager, and monitor using CloudWatch or Prometheus/Grafana.

7.3 CI/CD

Interview answer:

CI/CD automates software delivery. Continuous Integration ensures each code change is built, tested, and validated before merge. Continuous Deployment automates releasing validated changes to environments. A good pipeline includes linting, unit tests, integration tests, security scans, Docker image builds, deployment, smoke tests, and rollback capability.

CI/CD flow

Git Push
Lint + Unit Tests
Integration Tests
Security Scan
Build Docker Image
Push Image
Deploy
Smoke Test
Monitor

7.4 Docker

Interview answer:

Docker packages an application and all dependencies into a portable container so it runs consistently across development, testing, and production environments.

Simple Dockerfile for FastAPI

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

7.5 Kubernetes

Interview answer:

Kubernetes orchestrates containers by handling scheduling, scaling, rolling updates, self-healing, service discovery, and load balancing. It is useful for running microservices reliably at scale.

Key concepts:

- Pod
- Deployment
- Service
- ConfigMap
- Secret
- Ingress
- Horizontal Pod Autoscaler
- Liveness/readiness probes

16. Git / Version Control

16.1 Typical Git Workflow

```
git checkout -b feature/agent-api
git add .
git commit -m "Add agent response endpoint"
git push origin feature/agent-api
## Create pull request
## Code review
## Merge after approval
```

16.2 Merge vs Rebase

Merge	Rebase
Preserves branch history	Creates cleaner linear history
Adds merge commit	Rewrites commits
Safer for shared branches	Good for local cleanup

16.3 Interview Line

"I use Git for feature branches, pull requests, code reviews, version control, and collaboration. For shared branches I prefer merge, while for cleaning up local commits before a PR, rebase can be useful."

17. Cloud / AWS Topics

17.1 EC2 vs Lambda

When to Use EC2

Use EC2 when you need:

- Long-running services
- Full control over environment
- Custom dependencies
- Persistent connections
- Background workers
- WebSockets
- Containerized services
- More networking control

When to Use Lambda

Use Lambda when you need:

- Short-lived event-driven tasks
- Serverless execution
- Auto-scaling without server management
- S3 triggers
- Queue processing
- Lightweight background jobs

17.2 Interview Answer: Why EC2 Instead of Lambda?

"I would choose EC2 when the application needs long-running processes, custom runtime setup, persistent connections, background workers, or more control over CPU, memory, networking, and installed dependencies. Lambda is great for short-lived event-driven tasks, but EC2 is better for continuously running services like a FastAPI backend, AI agent server, vector database worker, or LLM integration service."

17.3 AWS Preparations for Scaling

For scaling a backend/AI application:

- Use EKS or ECS for containers
- Place service behind Application Load Balancer
- Configure autoscaling
- Use RDS for managed database
- Add read replicas if needed
- Use connection pooling
- Use ElastiCache/Redis for caching
- Store documents/files in S3
- Use CloudWatch for logs and metrics
- Use Secrets Manager for secrets
- Configure IAM roles properly
- Configure security groups and VPC networking
- Monitor external API limits and latency

17.4 GenAI-Specific AWS Considerations

- LLM provider rate limits
- Embedding API throughput
- Vector database scaling
- Token cost monitoring
- Response streaming
- Async processing
- Background evaluation jobs
- Queue-based workloads
- Logging and audit trails

18. Kubernetes / Docker Topics

18.1 What Is Kubernetes?

"Kubernetes is a container orchestration platform used to deploy, manage, scale, and monitor containerized applications. It automates deployment, scaling, load balancing, self-healing, and rolling updates."

18.2 Why Use Kubernetes?

- Auto scaling
- Load balancing
- Self-healing
- Rolling updates
- High availability
- Service discovery
- Better container management

18.3 Core Kubernetes Components

Component	Meaning
Pod	Smallest deployable unit
Deployment	Manages desired number of pods
Service	Stable endpoint to reach pods
Ingress	Routes external traffic
ConfigMap	Non-secret configuration
Secret	Sensitive configuration
HPA	Horizontal Pod Autoscaler

18.4 Docker vs Kubernetes

Docker and Kubernetes solve complementary problems rather than competing directly.

Docker	Kubernetes
Builds portable OCI-compatible images	Orchestrates containers created from those images
Runs containers on a host	Schedules workloads across a cluster
Provides packaging and runtime isolation	Provides scaling, self-healing, service discovery, and rollout control
Common for local development and CI	Common for production workloads that need orchestration

Docker Compose is useful for running several related containers on one machine, such as an API, PostgreSQL, and Redis during local development. Kubernetes is justified when the workload needs cluster-level availability, horizontal scaling, controlled rollouts, service discovery, or standardized operations across many services.

Kubernetes does not require the Docker Engine runtime. Modern clusters commonly use runtimes such as containerd, while still running OCI-compatible images that may have been built with Docker.

18.5 Interview Line

"Docker packages an application and its dependencies into a portable container image. Kubernetes operates those containerized workloads across a cluster by handling scheduling, scaling, service discovery, health management, and rolling updates. I use Docker for packaging and Kubernetes when the operational requirements justify orchestration."

18.6 What Happens When a Pod Crashes?

"Kubernetes continuously monitors pod health. If a pod crashes, the deployment controller creates a replacement pod to maintain the desired state."

20. Azure / Terraform Deployment Question

20.1 Client Question

"I need to know Azure/Terraform experience - how did you deploy?"

20.2 Safe Answer Strategy

Be truthful and avoid overstating experience.

A strong answer:

"My primary hands-on production deployment experience has been with cloud deployments using Docker, Kubernetes, CI/CD, and infrastructure automation concepts. I have worked with Azure Database as part of backend application development and understand Terraform's Infrastructure-as-Code workflow, including declarative infrastructure definitions, state management, variables, modules, and automated provisioning. Most of my production deployment experience has been on AWS-based environments, but Terraform principles are cloud-agnostic. The same workflow can be adapted to Azure resources such as resource groups, app services, container apps, Azure SQL, storage accounts, virtual networks, and Key Vault. I am comfortable working with Terraform-based Azure deployments and can quickly contribute using my cloud engineering and IaC background."

20.3 Example Terraform Flow for Azure

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

20.4 Example Terraform Structure

```
terraform/
  main.tf
  variables.tf
  outputs.tf
  providers.tf
  environments/
    dev.tfvars
    prod.tfvars
```

20.5 Example Azure Terraform Snippet

```
provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "app_rg" {
  name     = "rg-fastapi-dev"
  location = "Canada Central"
}

resource "azurerm_storage_account" "app_storage" {
  name                        = "fastapistorageacct"
  resource_group_name        = azurerm_resource_group.app_rg.name
  location                   = azurerm_resource_group.app_rg.location
  account_tier               = "Standard"
  account_replication_type   = "LRS"
}
```

20.6 Interview Line

"The way I think about deployment is: define infrastructure using Terraform, validate the plan, provision the resources, deploy the application through CI/CD, store secrets securely, monitor through cloud logs/metrics, and maintain changes through version control."

12. CI/CD, Git, and Engineering Workflow

Topics to revise

- Git branching.
- Pull requests.
- Code reviews.
- CI pipelines.
- Build/test/lint/type-check stages.
- CD deployment pipelines.
- Rollbacks.
- Environment variables.
- Secrets handling.

Example CI pipeline stages

1. Install dependencies
2. Run linting
3. Run type checks
4. Run unit tests
5. Run integration tests
6. Build Docker image
7. Scan image/security dependencies
8. Deploy to staging
9. Run smoke tests
10. Promote to production

Common interview question

What should a good CI/CD pipeline include?

Answer:

A good CI/CD pipeline should automatically run linting, formatting checks, type checks, unit tests, integration tests, security scans, Docker builds, and deployment steps. It should also support rollback, environment-specific configuration, and release visibility.

13. Docker, Containers, Kubernetes, and Cloud

Docker topics

- Dockerfile basics.
- Images vs containers.
- Multi-stage builds.
- Environment variables.
- Volumes.
- Container networking.
- Docker Compose.

Example Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Kubernetes topics

- Pods.
- Deployments.
- Services.
- ConfigMaps.
- Secrets.
- Ingress.
- Horizontal scaling.
- Health checks.
- Rolling updates.

Cloud topics

- AWS, Azure, or GCP basics.
- Serverless vs containers.
- Managed databases.
- Object storage.
- IAM/least privilege.
- Logs and metrics.
- Infrastructure as code.

Common interview questions

1. Why use Docker?
2. What problem does Kubernetes solve?
3. How do you scale a Python API?
4. How do you handle secrets in cloud applications?
5. What is the difference between a container and a VM?

7. Cloud, AWS, Kubernetes, and GenAI Deployment

7.1 Deploying GenAI applications on cloud/Kubernetes

Typical architecture

Client -> API service -> Retrieval layer/vector DB -> LLM endpoint -> Response

Components

- FastAPI/Flask backend.
- LLM orchestration layer.
- RAG ingestion workers.
- Vector database.
- Redis cache.
- Object storage.
- Secrets manager.
- Observability stack.
- CI/CD.

7.2 Kubernetes deployment components

Common manifests

- Deployment.
- Service.
- Ingress.
- ConfigMap.
- Secret.
- HPA.
- Readiness/liveness probes.

Production concerns

- Resource requests/limits.
- Autoscaling.
- Rolling/canary deployments.
- Secrets handling.
- Observability.
- Graceful shutdown.

7.3 AWS stack for backend/GenAI apps

Common services

- EKS/ECS for compute.
- ECR for images.
- S3 for document/object storage.
- RDS/Aurora for relational data.
- DynamoDB for key-value/idempotency store.
- ElastiCache Redis for caching.
- Secrets Manager for credentials.
- CloudWatch/Prometheus/Grafana for monitoring.
- API Gateway/ALB for ingress.

7.4 Choosing DynamoDB vs RDS for idempotency

DynamoDB advantages

- Low-latency key-value access.
- Conditional writes.
- TTL cleanup.
- High availability.
- Scales well for high retry traffic.

RDS advantage

- Strong relational transactions.
- Simpler if idempotency and business operation must commit together.

Hybrid answer

Use DynamoDB as a fast idempotency/replay layer, but enforce final uniqueness in RDS when order data lives there.

7.5 Securing AWS order-create endpoint

Security layers

- OAuth2/OIDC JWT validation.
- Cached JWKS validation.
- Scopes such as `orders:create`.
- IAM roles for AWS resources.
- Secrets Manager with local TTL cache.
- Private subnets.
- Security groups.
- RDS Proxy/connection pooling.
- Tenant-aware authorization.

7.6 Autoscaling compute while protecting RDS

Compute scaling metrics

- CPU.
- Memory.
- Request count.
- Queue depth.
- p95/p99 latency.

RDS protection

- RDS Proxy or PgBouncer.
- Per-pod connection limits.
- Short transactions.
- Backpressure when DB is saturated.
- Read replicas for read-heavy workloads.
- Async queues for non-critical work.

8. Docker, Git, and CI/CD

8.1 Starting and creating Docker containers

Basic command

```
docker run <image_name>
```

Detached mode

```
docker run -d --name my-container my-image
```

Port mapping

```
docker run -p 8000:8000 my-image
```

Container vs image

- Image: blueprint/template.
- Container: running instance of an image.

A container is created from one image, but it may communicate with containers created from other images.

8.2 Checking Docker image size

```
docker images
```

Specific image inspection:

```
docker image inspect <image_name> --format='{{.Size}}'
```

8.3 Debugging a crashing Docker container

Commands

```
docker ps -a
docker logs <container_id>
docker inspect <container_id>
docker stats <container_id>
docker exec -it <container_id> sh
```

Things to check

- Exit code.
 - OOM kill.
 - Restart count.
 - Environment variables.
 - Mounted volumes.
 - Network connectivity.
 - Healthcheck status.
 - Resource usage.
-

8.4 Dockerized CI/CD pipeline

Pipeline flow

```
lint/test -> build image -> scan image -> push registry -> deploy -> monitor
```

Best practices

- Pin dependencies.
- Use lock files.
- Run vulnerability scans.
- Store images in registry.
- Use same image across environments.
- Inject config through env vars/secrets.
- Use rolling/canary deployment.
- Keep rollback image tags.

Kubernetes rollback

```
kubectl rollout undo deployment/app
```

8.5 Git: remove file from last commit before push

Keep file locally but remove from commit

```
git rm --cached backup.zip
git commit --amend
```

Add to .gitignore

```
echo "*.zip" >> .gitignore
git add .gitignore
git commit --amend
```

8.6 Git: remove experimental files from commit history

If only last commit

```
git reset --soft HEAD~1
git restore --staged experimental_file.py
git commit -m "Clean commit without experimental files"
```

If across multiple commits

```
git rebase -i HEAD~N
```

Then edit commits and remove unwanted files.

8.7 Git: reconciling long-lived branches after restructuring

Safe process

1. Create backup branch.
2. Inspect commit graph.
3. Rebase/merge one branch at a time.
4. Resolve conflicts carefully.
5. Run tests after each major resolution.
6. Use interactive rebase to clean history.
7. Cherry-pick stable commits if merging entire branch is risky.

Commands

```
git log --oneline --graph --all
git diff branch1..branch2
git rebase main
git rebase -i HEAD~N
git cherry-pick <commit_hash>
```

Cloud, DevOps & Monitoring

AWS Services

Interview Question

What AWS services have you worked with or would use?

Common Services

Service	Use
EC2	Virtual machines
S3	Object storage
Lambda	Serverless functions
IAM	Access control
CloudWatch	Logs and monitoring
ECS/EKS	Container orchestration
ECR	Container registry
RDS	Managed relational database
SageMaker	ML training/deployment

EC2 vs Lambda

Interview Question

What is the difference between EC2 and Lambda?

Answer

EC2	Lambda
Server-based	Serverless
You manage instance	AWS manages runtime
Good for long-running workloads	Good for event-driven workloads
More control	Less operational overhead

Docker

Interview Question

What is Docker and why use it?

Answer

Docker packages an application and its dependencies into a container so it runs consistently across environments.

Dockerfile Example

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Kubernetes

Interview Question

What is Kubernetes used for?

Answer

Kubernetes is used to orchestrate containers at scale.

It Handles

- Deployment
 - Scaling
 - Service discovery
 - Load balancing
 - Self-healing
 - Rolling updates
 - Configuration management
-

CI/CD

Interview Question

What is CI/CD?

Answer

CI/CD automates software testing and deployment.

Term	Meaning
CI	Continuous Integration
CD	Continuous Delivery/Deployment

Pipeline Example

```
Code push
-> lint
-> unit tests
-> integration tests
-> build Docker image
-> security scan
-> deploy to staging
-> deploy to production
```

Production Monitoring

Interview Question

What would you monitor in production?

Answer

Monitor both application and infrastructure health.

Metrics

- Request latency
- Error rate
- Throughput
- CPU/memory usage
- Disk usage
- API failures
- Queue length
- Model performance
- Drift
- Token usage
- Cost

Tools

- Prometheus
- Grafana
- CloudWatch
- Datadog
- ELK Stack
- OpenTelemetry

10. Docker & Containerization

Likely Questions

- What is Docker?
 - Why use containers?
 - What is a Dockerfile?
 - What is Docker Compose?
 - How do you reduce Docker image size?
 - How do containers help in local development?
 - Difference between image and container?
 - How do you pass environment variables?
 - How do you debug a container?
-

Dockerfile Example

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Interview Explanation

- Base image provides Python runtime.
- WORKDIR sets app directory.
- Requirements are copied first for better Docker layer caching.
- App code is copied after dependencies.
- Uvicorn runs the FastAPI app.

Docker Compose Example

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://postgres:postgres@db:5432/app
      REDIS_URL: redis://redis:6379/0
    depends_on:
      - db
      - redis

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: app
    ports:
      - "5432:5432"

  redis:
    image: redis:7
    ports:
      - "6379:6379"
```

Image vs Container

Concept	Meaning
Image	Blueprint/package containing app and dependencies
Container	Running instance of an image

11. Kubernetes & Deployment Basics

Likely Questions

- What is Kubernetes?
- What is a pod?
- What is a deployment?
- What is a service?
- What is a config map?
- What is a secret?
- How does Kubernetes help with scaling?
- How do rolling deployments work?
- How do health checks work?

Kubernetes Deployment Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: orders-api
  template:
    metadata:
      labels:
        app: orders-api
    spec:
      containers:
        - name: orders-api
          image: orders-api:latest
          ports:
            - containerPort: 8000
          readinessProbe:
            httpGet:
              path: /health
              port: 8000
          livenessProbe:
            httpGet:
              path: /health
              port: 8000
```

Interview Explanation

- Deployment manages replicas.
- Pods run containers.
- Readiness probe decides if pod can receive traffic.
- Liveness probe restarts unhealthy containers.

Kubernetes Service Example

```
apiVersion: v1
kind: Service
metadata:
  name: orders-api-service
spec:
  selector:
    app: orders-api
  ports:
    - port: 80
      targetPort: 8000
```

12. CI/CD

Likely Questions

- What is CI/CD?
- What steps should a CI pipeline include?
- How do you deploy safely?
- What is a rollback?
- How do you manage environment-specific configs?
- How do you ensure tests run before deployment?
- How do you handle database migrations in deployment?

Typical CI Pipeline

Pull request opened

Install dependencies

Run linting

Run unit tests

Run integration tests

Build Docker image

Security checks

Deploy to staging

Manual/automatic promotion to production

GitHub Actions Example

```
name: Python CI

on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: test_db
        ports:
          - 5432:5432

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt

      - name: Run tests
        run: |
          pytest
```

Safe Deployment Practices

- Run automated tests before deploy
- Use staging environment
- Use feature flags for risky features
- Use rolling deployment
- Monitor errors after deployment
- Keep rollback plan ready
- Avoid manual production changes

13. Cloud Platforms

Likely Questions

- What cloud services have you used?
 - How do you deploy a Python API?
 - How do you store secrets?
 - How do you monitor a cloud app?
 - Difference between IaaS, PaaS, and serverless?
 - How do you scale backend services?
 - How do you handle environment variables?
-

Cloud Deployment Concepts

Concept	Explanation
Compute	Runs application workloads
Database	Stores persistent data
Object storage	Stores files, logs, documents, images
Load balancer	Distributes traffic
Secrets manager	Stores sensitive config
Monitoring	Tracks health, logs, metrics
Auto-scaling	Adds/removes capacity based on load

Environment Variables Example

```
import os

DATABASE_URL = os.environ["DATABASE_URL"]
REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379/0")
DEBUG = os.getenv("DEBUG", "false").lower() == "true"
```

Interview Explanation

Use environment variables for config that changes by environment. Do not hardcode secrets in source code.

Platform Engineering, Golden Paths & Developer Experience

Platform engineering treats the internal engineering platform as a product whose users are application developers. The goal is not to hide every infrastructure detail; it is to make the secure, observable, repeatable path the easiest path.

Golden Paths and Paved Roads

A golden path is a supported, opinionated workflow for a common engineering task. A production service template might provide:

- Repository and application skeleton.
- Dockerfile and image-build conventions.
- Reusable Terraform modules.
- CI/CD pipeline defaults.
- Secrets integration.
- Authentication and authorization hooks.
- Security scanning and policy checks.
- Health checks and readiness/liveness probes.
- Logs, metrics, traces, dashboards, and alerts.
- Safe deployment and rollback behavior.
- Runbook and ownership metadata.

Developers should provide only the information genuinely unique to the service. The platform should encode common operational decisions once instead of making every product team rediscover them.

Interview answer:

I think of platform engineering as treating developers as internal customers. A good platform provides supported golden paths for provisioning, deployment, observability, and operations so teams can move quickly without repeatedly making the same infrastructure and security decisions. The standard path should be opinionated, but there should be documented escape hatches for legitimate exceptions.

Secure and Stable by Default

A self-service platform should make safe defaults automatic:

- Least-privilege IAM/RBAC.
- Encryption in transit and at rest.
- Managed secrets instead of secrets in source control.
- Restricted network exposure.
- Required tests and security scans before deployment.
- Standard observability and audit logging.
- Resource requests/limits and health checks.
- Controlled promotion and rollback.
- Policy-as-code for non-negotiable controls.

This reduces cognitive load and prevents safety from depending on every developer remembering every requirement manually.

Measuring Whether the Platform Is Working

Useful platform and developer-experience metrics include:

- Time to create a new service or environment.
- Deployment frequency and deployment lead time.
- Change failure rate and mean time to recovery.
- Golden-path adoption rate.
- Number of manual infrastructure/support tickets.
- Exception or escape-hatch rate.
- Developer satisfaction and qualitative feedback.

A technically elegant platform that engineers avoid is not successful.

Modernizing Legacy Infrastructure Incrementally

Avoid a rewrite-first approach. A safer modernization sequence is:

1. Map the current architecture, dependencies, manual changes, and major failure modes.
2. Add enough observability to establish a baseline.
3. Bring configuration and infrastructure changes under version control.
4. Convert repeatable infrastructure into reusable IaC modules.
5. Introduce safe deployment patterns and automated validation.
6. Migrate high-risk or high-tail paths incrementally.
7. Preserve compatibility and rollback paths while old and new systems coexist.

Interview answer:

I would modernize incrementally rather than replace everything at once. I would first identify the highest-risk manual processes and observability gaps, establish a baseline, bring configuration under version control, introduce reusable IaC and deployment standards, and migrate in small reversible steps. The objective is to leave the system more stable and secure after every change.

Terraform in a Production Platform

Important production Terraform practices include:

- Remote, encrypted state with restricted access.
- State locking to prevent concurrent modification.
- Small reusable modules with clear inputs and outputs.
- Versioned module releases rather than copying infrastructure definitions.
- `fmt`, `validate`, security/policy checks, and `plan` in CI.
- Human or policy approval for high-impact production changes.
- Drift detection and a preference for code-reviewed changes over manual console edits.
- Separate environment configuration without duplicating the whole codebase.
- Recovery planning for state and provider failures.

A safe pipeline is often:

```
change -> fmt/validate -> security/policy checks -> plan -> review/approval -> apply ->
post-change validation
```

Eliminating Toil Rather Than Automating It Faster

Toil is repetitive, manual, automatable operational work that grows with system scale. The stronger platform response is to remove the recurring category of work:

Repeated manual deployment ticket

Identify common inputs and policy requirements

Encode them in a supported self-service workflow

Automated validation + audit trail + rollback

Product team deploys safely without a platform-team ticket

Interview answer:

I look for recurring manual work that should stop existing. If the platform team repeatedly performs the same safe provisioning or deployment task, I would turn the proven workflow into a self-service capability with guardrails rather than simply creating a faster manual checklist.

Senior CI/CD, Containers & Deployment Interview Addendum

Jenkins Pipeline Stages

A strong default pipeline explanation is:

Checkout

Dependency install / build

Lint / static analysis

Unit tests + coverage

Security scan

Package / Docker image build

Integration tests

Push artifact/image

Deploy to environment

Smoke / health checks

Promotion or rollback

The exact stages depend on the application, but the ordering should fail cheap and deterministic checks before expensive packaging/deployment work.

Why CI/CD is more than automation

A useful pipeline must also provide:

- Fast, actionable failure feedback
- Reproducibility
- Clear promotion rules
- Artifact traceability
- Secrets handled outside source code
- Rollback strategy
- Deployment health verification
- Auditability

Strong interview answer:

I think of CI/CD as a repeatable quality and delivery control, not just a Jenkinsfile. The pipeline should make failures understandable to developers, preserve artifact/version traceability, and reduce manual release risk.

CI vs Continuous Delivery vs Continuous Deployment

- **Continuous Integration:** merge frequently with automated build/test/quality checks.
- **Continuous Delivery:** every passing change is kept releasable; production may still require an approval.
- **Continuous Deployment:** every passing change automatically reaches production.

Do not use "CI/CD" as if all three models are identical.

Security Scanning Rollout Pattern

When introducing a new static security scanner into an existing codebase, do not immediately fail every build on every finding.

A practical rollout:

Identify security gap

Build POC on representative repositories

Review true positives / false positives

Set severity + confidence policy

Agree suppressions / baselines

Run informationally in CI

Make high-value findings blocking

Document local reproduction + remediation

Tighten policy over time

This balances security quality with developer velocity and adoption.

For Python, Bandit is a common static-analysis tool for security-oriented source checks. The reusable lesson is the **incremental enforcement strategy**, not a specific scanner brand.

Docker: What It Solves

Docker packages:

application
runtime
system dependencies
configuration interface

into a repeatable container image.

Benefits:

- Consistent environments
- Reproducible deployment artifact
- Isolation
- Easier CI/CD promotion

Docker does **not** itself provide multi-host scheduling, autoscaling, service discovery, or cluster failover.

Kubernetes: What It Adds

Kubernetes orchestrates containers across a cluster and manages desired state.

Core responsibilities:

- Scheduling pods
- Replica management
- Service discovery/networking
- Rolling deployment primitives
- Restart/replacement of failed pods
- Config/secret integration
- Autoscaling support
- Health/readiness integration

Good interview answer:

Docker gives me a portable container image. Kubernetes manages many running instances of those images across machines and continuously reconciles the actual state toward the desired state.

Pod vs Service vs Application Service

Avoid terminology confusion:

Application service
= logical business capability

Kubernetes Pod
= one deployable/running group of containers

Kubernetes Service object
= stable networking abstraction/load-balancing endpoint for pods

These are related but not interchangeable meanings of "service."

Liveness vs Readiness

Liveness

Question:

Is this process alive or irrecoverably wedged?

Failure can trigger restart.

Readiness

Question:

Can this instance safely receive traffic right now?

Failure should remove it from routing without necessarily restarting it.

Example:

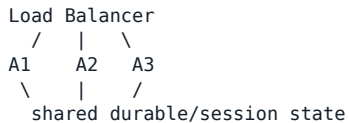
```
startup/migration/warm cache not complete
-> not ready yet
-> do not route traffic

process deadlocked
-> liveness fails
-> restart
```

Do not make liveness depend on every downstream service being healthy; otherwise one dependency outage can cause a restart storm.

Horizontal Scaling Requires Statelessness

If five API instances all need the same user/session/job state, do not keep the authoritative copy only in one process memory.



This allows any healthy instance to serve the next request.

Autoscaling: What Signal Should Drive It?

CPU can be useful, but not every workload is CPU-bound.

Possible scaling signals:

- CPU/memory utilization
- Request rate
- Request latency
- In-flight requests
- Queue depth
- Queue **age**
- Worker utilization
- Custom business workload metric

Strong answer:

I choose an autoscaling signal that correlates with the actual bottleneck. A queue worker may need scaling based on backlog age rather than API CPU utilization.

Secure CI/CD Authentication to AWS and EC2

Separate two questions:

1. How does CI/CD authenticate to AWS APIs?
2. How does the deployment mechanism reach/manage an EC2 host?

CI/CD -> AWS: prefer temporary role credentials

Preferred pattern for hosted CI systems such as GitHub Actions:

```
CI job
  OIDC identity token
AWS IAM trust policy
  AssumeRoleWithWebIdentity
AWS STS temporary credentials
```

ECR / EC2 / S3 / SSM / deployment APIs

This avoids long-lived access keys in repository secrets.

Example GitHub Actions shape:

```
permissions:
  id-token: write
  contents: read

steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::123456789012:role/deploy-role
      aws-region: us-east-1

  - run: aws sts get-caller-identity
```

The IAM role should have only the deployment permissions the pipeline actually needs.

For Jenkins, strong options are:

- Run Jenkins on AWS compute with an attached IAM role/instance profile where appropriate.
- Use an identity-federation/role-assumption setup that produces temporary STS credentials.
- If static credentials are unavoidable in an older environment, keep them in Jenkins Credentials/secret storage, rotate them, scope them narrowly, and treat migration to short-lived credentials as the preferred direction.

CI/CD -> EC2 host

Prefer AWS Systems Manager (SSM) for host management when the environment supports it:

CI/CD with IAM permission

AWS Systems Manager

EC2 instance with SSM agent + instance role

This can avoid opening SSH broadly or distributing private keys. If SSH is required, store the private key in a secure credential store, restrict network access/security groups, verify host keys, and rotate credentials.

EC2 workload -> AWS services

The application running on EC2 should normally use an **instance profile/IAM role**, not embedded `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` values.

```
EC2 application
  instance metadata / role credentials
STS-managed temporary credentials
```

S3 / SQS / Secrets Manager / other allowed AWS APIs

Interview answer:

I separate pipeline identity from instance identity. The CI/CD job assumes a least-privilege IAM role and receives temporary STS credentials-OIDC is preferred for hosted CI. For managing EC2 I prefer Systems Manager when possible rather than distributing SSH keys. The EC2 workload itself gets an instance profile so application code does not contain long-lived AWS credentials.

Deployment to AWS: Reusable Flow

One common containerized deployment pattern:

```
Git commit
CI pipeline
tests / scans
build image
push image to registry
deploy/update workload
load balancer routes only ready instances
metrics / logs / smoke tests
```

Possible AWS building blocks depend on architecture:

- EC2 for VM-hosted workloads
- ECS/EKS for container orchestration
- ECR for container images
- ALB/NLB for traffic distribution
- RDS for relational databases
- S3 for artifacts/object storage
- CloudWatch for AWS-native metrics/logs
- IAM roles for workload identity

Do not claim all of these are required together. Name only what the design actually uses.

Safe Release Across Multiple Products/Services

When one shared change affects several consuming products, treat compatibility as a release-management problem rather than only a code-change problem.

Useful controls:

- Compatibility matrix
- Contract/integration tests
- Versioned artifacts
- Staged rollout
- Release notes
- Consumer upgrade order when required
- Feature flags where behavior can be decoupled from deployment
- Monitoring after promotion
- Rollback plan

A compatibility matrix can represent which producer/framework/server versions support which consumers rather than maintaining many undocumented one-to-one assumptions.

Backward Compatibility Matrix Example

	Client v1	Client v2	Client v3
Server v1			
Server v2			
Server v3			

The matrix can directly drive automated contract tests:

```
supported combination
-> expected success behavior

unsupported combination
-> explicit predictable error / upgrade requirement
```

This reduces scattered special-case logic and makes release risk visible.

Deployment Failure Questions

What if a new release is unhealthy?

- Stop promotion.
- Remove unhealthy instances through readiness/health checks.
- Roll back to known-good artifact if necessary.
- Preserve logs/metrics/traces for diagnosis.
- Avoid changing database schema in ways that make immediate rollback impossible without a compatibility plan.

Blue/green vs rolling deployment?

- **Rolling:** gradually replace old instances; lower duplicate-capacity cost, but versions coexist during rollout.
- **Blue/green:** maintain old/new environments and switch traffic; fast rollback, but higher temporary capacity/cost and data/schema compatibility still matters.

Canary deployment?

Send a small percentage of traffic/users to the new version, evaluate health/business signals, then expand.

Quick DevOps Revision Card

CI vs delivery vs deployment

Pipeline: checkout -> build -> lint -> unit -> security -> package -> integration -> deploy
-> smoke

Static security POC -> tune -> informational -> blocking

Docker packages runtime; Kubernetes orchestrates replicas

Pod vs K8s Service vs logical application service

Liveness vs readiness

Stateless horizontal scaling

Autoscale on real bottleneck signal

CI/CD -> AWS: OIDC/role assumption/STS temporary credentials

EC2 management: prefer SSM; EC2 workload uses instance profile

Compatibility matrix for multi-product releases

Staged/canary/rollback strategy

Testing, QA, Security, Observability & Reliability

Source: docs/testing_security_observability.md

Purpose: Pytest, code quality, QA, AI QA, security, privacy, PII handling, monitoring, logging, metrics, incidents, runbooks, and reliability. **Use this file for:** backend, AI platform, production support, and SRE-adjacent interviews

Recommended Study Flow

1. Read the **Quick Summary** first.
2. Review the **Key Concepts** and tables.
3. Practice the **Interview Questions & Answers** out loud.
4. Use the code snippets and examples to explain trade-offs clearly.
5. Finish with the **Common Mistakes** and **Revision Checklist** sections.

Quick Summary

This is a new topic file created because the attached repository files did not have a dedicated Markdown page for this subject. It is merged from the organized topic-wise interview-prep pack and follows the same repository style as the existing notes.

Consolidated Interview Questions & Technical Notes

Pytest, code quality, QA strategy, AI QA, monitoring, logging, metrics, production reliability, incidents, runbooks, security, privacy, PII, and access control.

6. Production Monitoring, Usage Tracking & Reliability

6.1 Traditional production metrics

- Latency
- Throughput
- Error rate
- CPU
- Memory
- Disk
- Availability
- Request volume

6.2 AI-specific production metrics

- Token usage
- Cost per request
- Prompt tokens
- Completion tokens
- Hallucination rate
- Retrieval quality
- Tool success rate
- Prompt failure rate
- User feedback
- Model/version performance
- Citation accuracy

6.3 Tracking which user calls which API and token usage

Usage table

```
api_usage
-----
request_id
user_id
endpoint
model_name
prompt_tokens
completion_tokens
total_tokens
latency_ms
cost_usd
status_code
created_at
```

FastAPI middleware example

```
import time
import uuid
import logging
from fastapi import FastAPI, Request

app = FastAPI()
logger = logging.getLogger(__name__)

@app.middleware("http")
async def track_request_usage(request: Request, call_next):
    start = time.time()
    request_id = str(uuid.uuid4())
    user_id = request.headers.get("x-user-id", "anonymous")

    response = await call_next(request)
    latency_ms = (time.time() - start) * 1000

    logger.info({
        "request_id": request_id,
        "user_id": user_id,
        "endpoint": request.url.path,
        "method": request.method,
        "status_code": response.status_code,
        "latency_ms": latency_ms,
    })

    return response
```

LLM token tracking

```
result = llm_client.chat.completions.create(...)
usage = result.usage

prompt_tokens = usage.prompt_tokens
completion_tokens = usage.completion_tokens
total_tokens = usage.total_tokens
```

Interview answer:

I would track token usage at the application layer, not only at the LLM provider dashboard. Each request should have user ID, endpoint, model, prompt tokens, completion tokens, total tokens, latency, cost, status, and timestamp. This allows cost visibility, quota enforcement, abuse detection, debugging, and capacity planning.

6.4 Reliability patterns

- Retries with backoff
- Timeouts
- Circuit breakers
- Fallback models
- Rate limiting
- Queues for long-running jobs
- Health checks
- Rollbacks
- Feature flags
- Human-in-the-loop approval
- Audit logs

13. Testing, Quality Engineering & AI QA

13.1 Testing Strategy

Interview answer:

I follow a layered testing strategy. I start with unit tests, then integration tests, API tests, end-to-end tests, regression tests, performance tests, and security tests. For AI systems, I also test prompts, RAG retrieval, hallucination behavior, tool-calling accuracy, and agent workflows.

13.2 Testing Pyramid

E2E Tests
Integration Tests
Unit Tests

Most tests should be unit tests because they are fast and reliable.

13.3 Software testing types

Test Type	Purpose
Unit	Test individual functions/classes
Integration	Test component interactions
API	Validate endpoints, schemas, status codes
E2E	Simulate full user workflow
Regression	Ensure existing features still work
Performance	Check latency, throughput, load
Security	Auth, injection, data leakage, access control

13.4 AI Testing

For AI systems, test:

- Prompt consistency
- Retrieval quality
- Faithfulness
- Hallucination rate
- Citation accuracy
- Tool selection
- Tool arguments
- Agent step sequence
- Fallback behavior
- Safety and policy compliance
- Prompt injection resistance

13.5 Quality Engineering Standards

Interview answer:

I define quality criteria before testing: correctness, factuality, instruction following, safety, latency, reliability, and user impact. I create representative test cases including normal scenarios, edge cases, and negative cases. I use both automated and human evaluation, integrate tests into CI/CD, and monitor production behavior continuously.

9. Security for AI Applications

9.1 Why Security Matters in AI Agent Development

"Security is very important because AI agents may connect to APIs, databases, customer records, internal tools, or sensitive business systems. I would not treat it like just a chatbot; I would secure the full flow."

9.2 Security Areas to Mention

Authentication

- JWT
- OAuth
- SSO
- API keys

Authorization

- Role-based access control
- Permission checks
- Least privilege access

Data Protection

- Masking sensitive data
- Avoiding unnecessary logging of PII
- Encryption in transit and at rest

Secrets Management

- Environment variables
- AWS Secrets Manager
- Azure Key Vault
- HashiCorp Vault

API Security

- Rate limiting
- Input validation
- Request size limits
- HTTPS
- Secure headers

AI-Specific Security

- Prompt injection protection
- Tool-use restrictions
- Output filtering
- Guardrails
- Human escalation
- Audit logs

9.3 Strong Interview Answer

"For AI agents, I would secure both the traditional application layer and the AI-specific layer. That means authentication, authorization, secure API access, secret management, input validation, prompt-injection protection, restricted tool access, logging, monitoring, and human escalation for risky workflows."

21. Testing, Monitoring & Continuous Improvement

21.1 Testing AI Agents

Types of tests:

- Unit tests
- Integration tests
- End-to-end tests
- Prompt regression tests
- Simulation tests
- Retrieval tests
- API tool-calling tests
- Safety tests
- Edge-case tests

21.2 Simulation Tests

"Simulation tests are scripted user journeys used to check how the agent behaves across expected and edge-case scenarios before going live."

Examples:

- User asks a supported question
- User asks unsupported question
- User provides missing information
- API returns timeout
- Retrieved context is empty
- User asks for sensitive information
- Agent needs to escalate to human

21.3 Reviewing Live Conversations

Steps:

1. Review failed or negative conversations
2. Identify issue type
3. Categorize failure
4. Create bug/issue
5. Improve prompt, retrieval, or API handling
6. Re-test
7. Deploy fix
8. Monitor whether the issue decreases

21.4 Production Metrics

- Conversation success rate
- Resolution rate
- Escalation rate
- Hallucination rate
- Latency
- Token usage
- User satisfaction
- API failure rate
- Retrieval accuracy
- Cost per conversation
- Error rate

21.5 Strong Interview Line

"For production AI systems, improvement is continuous. I would review live conversations, analyze failure patterns, create issues, improve prompts or retrieval, test again, and monitor metrics after deployment."

11. Testing & Code Quality

The role emphasizes high-quality tested code.

Topics to revise

- Unit testing.
- Integration testing.
- Test fixtures.
- Mocking.
- Pytest.
- Linting.
- Formatting.
- Type checking.
- Code reviews.
- Test coverage.

Example: Pytest unit test

```
from app.services.pricing import calculate_total

def test_calculate_total_with_tax() -> None:
    result = calculate_total(price=100.0, tax_rate=0.13)
    assert result == 113.0
```

Example: Mocking an external LLM call

```
from unittest.mock import Mock

def test_answer_generation_uses_context() -> None:
    mock_llm = Mock()
    mock_llm.generate.return_value = "The policy allows refunds within 30 days."

    context = "Refunds are allowed within 30 days of purchase."
    question = "Can I get a refund?"

    answer = mock_llm.generate(question=question, context=context)

    assert "30 days" in answer
    mock_llm.generate.assert_called_once()
```

Interview explanation:

For LLM systems, tests should not only check code paths but also validate prompt formatting, retrieval behavior, fallback logic, and safety handling.

14. Observability & Production Monitoring

Topics to revise

- Structured logging.
- Metrics.
- Distributed tracing.
- Dashboards.
- Alerting.
- Error rates.
- Latency percentiles.
- Token usage monitoring for LLM apps.
- Cost dashboards.
- Incident response.
- Post-incident learning.

Example structured log

```
import logging

logger = logging.getLogger(__name__)

def log_llm_request(user_id: str, model: str, tokens: int, latency_ms: float) -> None:
    logger.info(
        "llm_request_completed",
        extra={
            "user_id": user_id,
            "model": model,
            "tokens": tokens,
            "latency_ms": latency_ms,
        },
    )
```

Useful production metrics

Area	Metrics
API	request count, error rate, p95 latency, p99 latency
Database	slow queries, connection pool usage, lock waits
LLM	token count, cost, latency, model errors, fallback rate
RAG	retrieval precision, empty retrieval rate, citation accuracy
System	CPU, memory, pod restarts, queue depth

15. Security, Privacy, and PII Handling

Topics to revise

- Secrets management.
- Least privilege access.
- Input validation.
- Authentication.
- Authorization.
- Encryption at rest.
- Encryption in transit.
- Secure coding.
- PII handling.
- Data retention.
- Audit logs.
- Prompt injection risks.
- Safe output handling.

Common interview questions

1. How do you protect secrets?
2. How do you handle PII in an LLM system?
3. What is least privilege?
4. What is prompt injection?
5. How do you validate user input?
6. How do you prevent sensitive data leakage?

Example answer: Handling PII in GenAI systems

I would avoid sending unnecessary PII to the model, mask or redact sensitive fields where possible, enforce access controls, log only safe metadata, encrypt data at rest and in transit, and define clear retention policies. I would also add guardrails to prevent the model from exposing sensitive information in outputs.

22. LLM Cost, Latency, and Reliability Optimization

Topics to revise

- Token usage tracking.
- Prompt compression.
- Context pruning.
- Caching.
- Semantic caching.
- Batching.
- Streaming responses.
- Smaller/faster model routing.
- Fallback models.
- Retry logic.
- Timeout handling.
- Circuit breakers.

Example: Cost tracking idea

```
MODEL_PRICES = {
    "example-model": {
        "input_per_1k": 0.001,
        "output_per_1k": 0.002,
    }
}

def estimate_cost(model: str, input_tokens: int, output_tokens: int) -> float:
    price = MODEL_PRICES[model]
    return (
        (input_tokens / 1000) * price["input_per_1k"]
        + (output_tokens / 1000) * price["output_per_1k"]
    )
```

Common interview question

How do you reduce LLM cost and latency?

Answer:

I would reduce unnecessary context, use efficient chunk retrieval, cache repeated responses, choose the smallest model that meets quality requirements, batch requests when appropriate, stream responses for user experience, monitor token usage, and route complex requests to stronger models only when necessary.

10. API Security

10.1 How can we secure an API?

Strong Answer

To secure an API, I would apply security at multiple layers: authentication, authorization, input validation, transport security, rate limiting, proper CORS, secrets management, and monitoring.

10.2 Common API security methods

Area	How to secure
Authentication	JWT, OAuth2, API keys
Authorization	RBAC, permissions, scopes
Transport security	HTTPS/TLS
Input validation	Pydantic/schema validation
Rate limiting	Prevent abuse and DDoS
CORS	Allow only trusted frontend domains
Secrets	Store in env vars or secret manager
SQL injection prevention	ORM or parameterized queries
Logging/monitoring	Track suspicious behavior
Token expiry	Short-lived access tokens + refresh tokens

10.3 FastAPI security-style example

```
from fastapi import Depends, FastAPI, HTTPException, status

app = FastAPI()

def verify_token(token: str):
    if token != "valid-token":
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid token"
        )
    return True

@app.get("/secure-data")
def secure_data(is_auth: bool = Depends(verify_token)):
    return {"message": "Secure API response"}
```

10.4 Strong one-liner

API security is not just authentication; it includes authentication, authorization, validation, encryption, rate limiting, CORS, secrets management, and observability.

12. AI Production Challenges

12.1 What is the most challenging problem you faced while working with AI?

Recommended Answer: Hallucinations and response reliability

One of the biggest challenges I encountered while working with AI systems was ensuring response reliability and reducing hallucinations. Large language models can sometimes generate confident but incorrect information, which becomes a serious issue when the output is used for resumes, recommendations, job applications, or business decisions. To address this, I used structured prompts, validation layers, output schemas, and context grounding. For example, in resume optimization workflows, I restricted the model to work only with information already present in the resume and job description instead of allowing it to invent new experience or skills. The key lesson I learned is that building a production AI system is often more about controlling and validating model outputs than simply calling an LLM API.

12.2 Other AI challenges to mention

Latency and scalability

Traditional APIs may respond in milliseconds, but LLM calls can take seconds.

Ways to improve:

- Async FastAPI endpoints
- Caching
- Prompt optimization
- Background workers
- Request queues
- Streaming responses

Context management

Too little context leads to poor answers. Too much context increases cost, latency, and token usage.

Solutions:

- Chunking
- Embeddings
- Vector search
- Reranking
- Context window management

Evaluation difficulty

AI outputs are harder to test than deterministic software.

Possible evaluation methods:

- Golden datasets
- Human review
- Automated scoring
- Exact match for structured tasks
- Semantic similarity
- Hallucination checks
- Regression testing

12.3 What is RAG?

RAG stands for **Retrieval-Augmented Generation**.

It combines information retrieval with LLM generation.

Flow

User query

Create embedding

Search vector database

Retrieve relevant context

Send context + query to LLM

Generate grounded answer

12.4 Why use RAG?

RAG helps:

- Reduce hallucinations
 - Use private/company documents
 - Improve factual grounding
 - Keep answers up to date with external knowledge
 - Avoid putting everything into the prompt manually
-

12.5 What are embeddings?

Embeddings convert text into numerical vectors that capture semantic meaning.

They are used for:

- Semantic search
 - Similarity matching
 - Clustering
 - Recommendations
 - RAG pipelines
-

12.6 Vector databases

Common examples:

- FAISS
- ChromaDB
- Pinecone
- Weaviate

They store embeddings and support similarity search.

9. Production Reliability, Observability, Incidents, and Runbooks

9.1 Diagnosing latency and incorrect predictions in ML API

Step-by-step approach

1. Treat as production incident.
 2. Assess user/business impact.
 3. Stabilize with rollback/fallback if needed.
 4. Compare p50/p95/p99 latency against baseline.
 5. Correlate incorrect predictions with latency windows.
 6. Check recent deployments/config changes.
 7. Validate model inputs and feature freshness.
 8. Replay golden test cases.
 9. Apply targeted fix.
 10. Add missing observability.
-

9.2 Distinguishing infra vs data pipeline vs model degradation

Infrastructure signals

- CPU/GPU/memory saturation.
- Pod restarts.
- Autoscaling events.
- DB/cache latency.
- Traffic spikes.

Data pipeline signals

- Schema changes.
- Null/default spikes.
- Feature freshness issues.
- Drift.
- Failed upstream jobs.

Model degradation signals

- Model/config version changed.
 - Golden test performance dropped.
 - Prediction confidence distribution shifted.
 - Same clean input produces worse output.
-

9.3 Handling contradictory monitoring data

Decision principle

Do not force data to fit the first hypothesis. Choose the lowest-risk reversible action that protects users.

Safe actions

- Rollback.
 - Route to fallback.
 - Disable new feature flag.
 - Increase human review for high-risk outputs.
 - Preserve logs/metrics for later RCA.
-

9.4 Alerting without alert fatigue

Principles

- Alert on symptoms, not every metric.
- Use severity levels.
- Use time windows.
- Group related alerts.
- Alert only when action is required.
- Every alert should have a runbook.

Example

Bad:

Single pod restarted once

Good:

p99 redirect latency > SLA for 5 minutes and error rate > threshold

9.5 Game day / incident readiness testing

Scenarios

- Cache outage.
- DB replica lag.
- Region failure.
- Traffic spike.
- Bad deployment.
- Dependency timeout.

Measure

- Time to detect.
 - Time to mitigate.
 - Alert quality.
 - Runbook usefulness.
 - Escalation path.
 - Customer impact.
-

9.6 On-call runbooks

Runbook structure

- Service overview.
- Critical dependencies.
- Common alerts.
- Investigation steps.
- Recovery actions.
- Safety notes.
- Escalation path.
- Validation checks.
- Post-incident actions.

Measuring effectiveness

- MTTR.
- Escalation frequency.
- Game day performance.
- On-call feedback.
- Repeat incident recovery time.

9.7 Disaster recovery

Key terms

- **RTO**: how fast service must recover.
- **RPO**: acceptable data loss.

DR design

- Multi-AZ deployment.
- Automated backups.
- Point-in-time recovery.
- Cross-region replication for critical systems.
- Stateless services.
- DNS/traffic failover.
- Regular restore testing.

Validating DR

- Failover drills.
- Backup restoration tests.
- Data consistency checks.
- Chaos/game day exercises.
- Measure actual RTO/RPO.

14. Security, Privacy, PII, and Access Control

14.1 Handling PII in GenAI systems

Main principle

Sensitive data should not be sent to the model unless absolutely necessary, and should be masked/tokenized whenever possible.

Steps

1. Detect PII.
2. Minimize data sent.
3. Mask/tokenize sensitive values.
4. Keep raw PII in secure backend services.
5. Redact logs/traces.
6. Filter outputs for leakage.
7. Use compliant/private model endpoints.
8. Define retention and audit policy.

Example masking

John Smith -> [CUSTOMER_NAME]
9876543210 -> [PHONE_NUMBER]
123 Main Street -> [ADDRESS]

14.2 Authentication and authorization for public APIs

First-party web/mobile

- OAuth2/OIDC.
- Short-lived JWTs.
- Refresh tokens.
- HttpOnly cookies for browser clients.
- Secure storage for mobile.

External partners

- OAuth2 client credentials.
- Client ID/secret.
- Scopes.
- Optional mTLS for sensitive partners.

Authorization

- RBAC/ABAC.
- Tenant isolation.
- Scope checks.
- Audit logs.

14.3 Credential and signing key rotation without downtime

JWT signing keys

- Use kid header.
- Publish JWKS.
- Add new key before using it.
- Start signing with new key.
- Accept old key until tokens expire.
- Remove old key after safe window.

Client secrets

- Support multiple active secrets.
- Add new secret.
- Partner migrates.
- Monitor usage.
- Revoke old secret.

14.4 Internal component access control

Design

- Each service gets identity.
- Least privilege.
- mTLS or signed JWT for service-to-service calls.
- Policy enforcement.
- Secrets isolation.
- Audit sensitive actions.

Example

Redirect service -> read URL mappings only
Creation service -> create/update mappings
Analytics service -> write events only
Admin service -> restricted elevated access

14.5 Auditing permissions over time

Practices

- Central inventory of service identities and permissions.
 - Automated policy checks.
 - Detect wildcard permissions.
 - Detect unused permissions.
 - Periodic access reviews.
 - Drift detection between IaC and live environment.
 - Audit trail of permission changes.
-

14.6 Tamper-resistant audit logs

Requirements

- Append-only logs.
- Strict access control.
- Actor/action/resource/timestamp/IP/request ID/reason.
- Immutable storage such as WORM/S3 Object Lock.
- Hash each log entry.
- Optional hash chaining.

Example event

```
{  
  "actor": "admin_123",  
  "action": "DISABLE_SHORT_LINK",  
  "resource": "short_abc",  
  "timestamp": "2026-01-01T12:00:00Z",  
  "request_id": "req_789",  
  "reason": "phishing report"  
}
```

13. Monitoring, Logging, Reliability & Alerts

What to Monitor

Area	Example Metric
Pipeline success/failure	Job status
Freshness	Latest transaction date
Volume	Number of records processed
Data quality	Failed validation count
Duplicates	Duplicate transaction count
Latency	Time from S3 arrival to report availability
Cost	Compute and storage cost
Error records	Quarantine count

Reliability Patterns

Pattern	Purpose
Retries	Handle transient failures
Dead-letter queue	Store failed events
Quarantine folder	Store invalid records
Checkpointing	Resume processing safely
Watermarking	Track incremental progress
Idempotent writes	Prevent duplicate data
Alerts	Notify failures quickly

Example Monitoring Flow

ETL Job
Logs -> CloudWatch / Logging System
Metrics -> Dashboard
Failures -> Alert via Email/SNS/Slack
Bad Records -> Quarantine Folder

Interview Answer

"I would monitor pipeline health, record counts, data freshness, validation failures, duplicate rate, and job latency. I would also add retries, alerting, quarantine handling, and idempotent writes to make the pipeline reliable."

9. Testing With Pytest

Likely Questions

- How do you write tests in Python?
 - What is pytest?
 - Difference between unit, integration, and end-to-end tests?
 - How do you test APIs?
 - How do you mock external services?
 - How do you test background jobs?
 - How do you test AI features?
 - What should be included in CI?
-

Unit Test Example

```
def calculate_order_total(items):
    return sum(item["price"] * item["quantity"] for item in items)

def test_calculate_order_total():
    items = [
        {"price": 10.0, "quantity": 2},
        {"price": 5.0, "quantity": 1},
    ]

    assert calculate_order_total(items) == 25.0
```

FastAPI Test Example

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_create_order():
    response = client.post(
        "/orders",
        json={
            "customer_id": 1,
            "total_amount": 99.99,
            "currency": "CAD",
        },
    )

    assert response.status_code == 201
    assert response.json()["status"] == "created"
```

Mocking External API Example

```
from unittest.mock import Mock

def test_sync_order_success():
    fake_client = Mock()
    fake_client.sync_order.return_value = {"status": "ok"}

    result = sync_order(order_id=123, client=fake_client)

    assert result["status"] == "ok"
    fake_client.sync_order.assert_called_once_with(123)
```

Testing AI Features

For AI features, test:

- Prompt construction
- Input validation
- Output schema validation
- Fallback behavior
- Empty context behavior
- Safety checks
- Logging metadata

```
def test_invalid_llm_output_is_rejected():
    invalid_response = '{"issue_type": "unknown", "urgency": "extreme"}'

    with pytest.raises(ValidationError):
        OrderIssue.model_validate_json(invalid_response)
```

14. Monitoring, Logging & Production Support

Likely Questions

- How do you handle production issues?
- How do you debug a production incident?
- What metrics do you monitor?
- What logs are useful?
- What is alert fatigue?
- How do you write useful logs?
- What is a postmortem?
- How do you decide whether to rollback?

Production Incident Answer Format

Use this structure:

1. Assess user/business impact
2. Communicate status clearly
3. Check recent deployments
4. Review logs, metrics, and traces
5. Mitigate quickly
6. Rollback if needed
7. Fix root cause
8. Add tests/monitoring
9. Write postmortem if serious

Sample Answer

When production issues happen, I first assess the impact and severity. Then I check logs, metrics, recent deployments, dependency health, and database behavior. If the issue is actively affecting users, I focus on mitigation first, such as rollback, feature flag disablement, or temporary workaround. After stabilizing, I investigate root cause and add tests, monitoring, or process improvements to prevent recurrence.

Structured Logging Example

```
import logging

logger = logging.getLogger(__name__)

def process_payment(order_id: int, amount: float):
    logger.info(
        "Processing payment",
        extra={
            "order_id": order_id,
            "amount": amount,
            "component": "payments",
        },
    )

    try:
        result = payment_gateway.charge(order_id, amount)
        logger.info("Payment successful", extra={"order_id": order_id})
        return result

    except Exception:
        logger.exception("Payment failed", extra={"order_id": order_id})
        raise
```

Useful Metrics

Monitor:

- Request latency
 - Error rate
 - Throughput
 - Queue depth
 - Job failure rate
 - Database query latency
 - CPU/memory
 - API dependency failures
 - LLM cost/latency/error rate
 - Payment/order failure rate
-

Postmortem Template

Incident:
Impact:
Timeline:
Root Cause:
Detection:
Resolution:
What went well:
What went wrong:
Action items:
Owner:
Due date:

Reliability Lessons from an Incident Alert Router

Small control-flow mistakes can create serious operational errors in incident systems.

Do Not Return Failure Before the Search Is Complete

This implementation is incorrect:

```
for alert in alerts:
    if alert.service == service:
        return alert.resolved_at is None
```

A resolved alert may appear before a later open alert for the same service. The correct predicate searches for any matching open alert:

```
return any(
    alert.service == service and alert.resolved_at is None
    for alert in alerts
)
```

Retry Backoff Validation

For each alert, sort delivery attempts by timestamp and compare adjacent attempts. A pair violates policy only when:

```
current.attempted_at - previous.attempted_at < cooldown
```

A gap exactly equal to the cooldown is compliant.

High-Value Tests

- Resolved alert followed by an open alert for the same service.
- Service with no alerts.
- Unknown alert ID when recording an attempt.
- Alert with no attempts, which must be omitted from count output.
- Attempts inserted out of order.
- Gap one unit below the cooldown.
- Gap exactly equal to the cooldown.
- Several violating pairs for one alert, which should produce one alert ID.

Production Considerations

A list scan is acceptable for an interview exercise. A production router would usually add:

- An index keyed by alert ID for constant-time lookup.
 - An index or query on (service, resolved_at) for open-alert checks.
 - Durable storage and transactional updates.
 - Idempotency keys for repeated delivery-attempt events.
 - Metrics for attempts, failures, policy violations, and notification latency.
 - Audit logs showing who was contacted and when.
-

Security and Escalation Guardrails for Customer-Support Agents

Customer-support agents often combine public information with sensitive account data. The safest design is to define a hard verification boundary and enforce it in application code rather than relying only on prompt instructions.

Verification Boundary

For an order-specific workflow:

1. Customer provides an order identifier plus the email address or phone number already on file.
2. The system returns the same generic response whether those details match or not, so it does not confirm account existence.
3. Send a one-time code only to the contact already stored on the account/order.
4. Rate-limit verification attempts.
5. Only after successful verification may order-specific tools be used.

Never ask for:

- Passwords.
- Full payment-card numbers.
- Security answers that the support system does not genuinely require.
- A new email or phone number as the destination for the verification code.

Zero Order-Specific Disclosure Before Verification

Before verification, do not reveal or confirm:

- Whether the order exists.
- Order status.
- Products or quantities.
- Prices or totals.
- Tracking number or carrier tied to that order.
- Shipping or billing address.
- Refund history or eligibility.

Generic policies, product descriptions, and public catalog information can remain available without account verification.

Minimize Disclosure After Verification

Successful verification does not mean every field should be shown. Return only what is necessary for the active task and mask sensitive fields where possible.

High-Risk Actions and Warm Handoffs

Some workflows should remain human-controlled even after authentication. Examples include:

- Shipping-address changes.
- Order cancellation.
- Executing a refund.
- Policy exceptions for final-sale items.
- Fraud or account-takeover concerns.

The agent can still reduce support effort by collecting the relevant order number, requested change, reason, supporting evidence, and current state, then creating a warm handoff so the human does not need to repeat discovery.

Damage, Defect, or Wrong-Item Reports

A support agent may request photos through a secure upload flow when they help a human reviewer. Photos should be optional supporting evidence rather than a reason to deny escalation when the customer cannot provide them.

Useful handoff fields:

- Affected item.
- Issue type: damaged, defective, or wrong item.
- Description.
- Photos when available.
- Preferred resolution.
- Verified order context.

Escalation Triggers

Immediately route to a human when the conversation involves:

- Legal threats or requests for legal interpretation.
- Chargebacks or regulator complaints.
- Fraud or account-takeover concerns.
- Safety issues.
- Explicit request for a human.
- Severe anger or abuse where continued automation is unlikely to help.

The bot should remain calm, avoid arguing, avoid making legal admissions, and preserve the relevant conversation context for the reviewer.

Multi-Tenant AI Data Isolation

For SaaS or financial systems with multiple organizations:

- Derive tenant identity from the authenticated session/token, never from model-generated arguments.
- Apply tenant and object-level authorization again when each tool executes.
- Filter vector/keyword retrieval by tenant before documents are returned to the LLM.
- Use least-privilege service credentials for external tools.
- Prevent one tenant's cache entries, embeddings, traces, or conversation state from being visible to another.
- Redact unnecessary PII and secrets from logs and model context.
- Keep auditable records of sensitive reads, writes, approvals, and escalations.

Prompt instructions are not an authorization mechanism. Even if a prompt-injection attempt asks the model to ignore tenant boundaries, the data and tool layers should make cross-tenant access impossible.

Fresh-State and Audit Requirements

For any financial eligibility check or other time-sensitive decision:

- Re-read the authoritative record before handoff/action.
- Log who/what initiated the check.
- Record the policy result and source data version/timestamp where practical.
- Do not reuse cached eligibility from a different intent or earlier task.
- Clear task-local state when the order, intent, or authentication state changes.

High-Value Security Tests

Test at least these cases:

1. Valid order ID but wrong email/phone does not reveal whether the order exists.
2. OTP is sent only to the stored contact destination.
3. Order tools are blocked before verification.
4. Authentication for one order cannot unlock a different order.
5. Intent change clears task-local sensitive cache.
6. Address change, cancellation, and refund execution always escalate when configured as human-only.
7. Final-sale exception requests route to a human.
8. Legal/fraud/chargeback language triggers immediate escalation.
9. Repeated OTP attempts are rate-limited.
10. Logs do not contain full OTPs, card numbers, or unnecessary PII.

Interview answer:

I separate public support from account-specific support with a hard authentication boundary. Before verification, the agent cannot even confirm that an order exists. After order-plus-contact verification and an OTP sent only to the stored contact, it can access only the data needed for the active task. High-risk actions remain human-controlled, and legal, fraud, chargeback, or safety concerns escalate immediately with a context-rich handoff.

Reliability Lessons from Load-Related Application Crashes

When an overloaded application is actively failing for customers, separate **incident mitigation** from **long-term remediation**.

Mitigation vs Remediation

Phase	Goal	Examples
Mitigation	Restore acceptable service with the lowest-risk move	Rollback, restart unhealthy instances, add temporary capacity, shed non-critical load
Remediation	Remove the underlying failure mode	Horizontal scaling, caching, query optimization, autoscaling changes, architecture fixes

Do not attempt a large redesign while the service is unavailable. First stabilize the customer-facing path, preserve evidence, and reduce the blast radius.

Immediate Response to a Load-Related Crash

A safe sequence is:

1. Confirm the scope and customer impact.
2. Check CPU, memory, OOM events, restarts, latency, error rate, DB connections, queue depth, and dependency health.
3. Check recent deployments/configuration changes so a bad release is not mistaken for pure capacity pressure.
4. Apply the smallest reversible mitigation likely to restore service.
5. Preserve logs, traces, metrics, and timestamps for root-cause analysis.
6. Monitor closely while the service recovers.
7. Implement and validate the durable fix after stability is restored.

Possible short-term mitigations, depending on the system, include:

- Restart or replace unhealthy instances.
- Temporarily increase CPU/memory capacity.
- Add replicas if dependencies can safely absorb the load.
- Disable or defer non-essential background work.
- Rate-limit or shed expensive non-critical traffic.
- Disable a problematic feature with a feature flag.
- Roll back a recent change if it is contributing to the failure.

The correct action depends on evidence. Adding replicas blindly can worsen an already saturated database.

Interview answer:

During an active incident, I separate mitigation from remediation. My first goal is to restore the core customer journey with the lowest-risk reversible action, while preserving enough telemetry for diagnosis. That may mean replacing unhealthy instances, adding temporary capacity, shedding non-critical work, or rolling back. Once the system is stable, I address the root cause through scaling, caching, query or connection-pool optimization, autoscaling changes, and stronger load testing.

Observe the User Journey, Not Only Infrastructure

For overload incidents, correlate infrastructure metrics with user-facing service-level indicators:

- Request success rate.
- p95/p99 latency.
- Timeout and retry rate.
- Transaction completion rate.
- Stage-level latency for critical workflow steps.
- CPU/memory saturation.
- DB query latency and connection-pool exhaustion.
- Cache hit/miss ratio.
- Autoscaling activity.

A service that returns slowly enough to cause customer timeouts is functionally degraded even if every process is still running.

Preventing the Repeat Incident

Post-incident actions should convert the lesson into engineering controls:

- Update capacity assumptions using measured peak behavior.
- Add missing load/stress tests.
- Define saturation indicators and alert thresholds.
- Tune autoscaling before saturation rather than after failure.
- Add dependency backpressure and rate limits.
- Reduce retry storms with bounded retries and jitter.
- Optimize slow queries and connection pools.
- Add cache where correctness permits.
- Update runbooks with tested recovery actions.
- Validate the fix above expected peak load and during dependency degradation.

A recurring incident should be treated as a prompt to remove the failure mode, not simply to make the on-call response faster.

Repository Debugging Under Time Pressure

When a repository-level assessment has several failing tests, the fastest reliable path is usually to turn the tests into an executable specification before changing code.

Start With the First Failing Test

Run the narrowest useful command first:

```
pytest -q test/test_app.py -x
```

-x stops on the first failure so one root cause is not hidden behind a long list of downstream failures. After fixing that failure class, run the full target file and finally the broader suite.

Extract the Contract From Tests

For each failing test, write down:

- Endpoint and HTTP method.
- Authentication precondition.
- Required request fields.
- Expected status code.
- Required response keys and nested fields.
- Expected persistence side effects.
- Expected relationship or identifier behavior.

A failure such as:

```
assert "issue" in {}
```

is more informative than just "test failed": it says the route returned a success status with the wrong response shape, so trace how the handler can reach an early success return.

Trace by Layer

Use a consistent order:

1. URL/route mapping
2. authentication decorator or middleware
3. handler/view
4. model/schema fields
5. serializer/to_dict response shape
6. related side effects such as activity/audit rows

This prevents random editing across the repository.

Compare Broken and Working Paths

If `create_*` is broken but `update_*` or `get_*` works, compare them for established patterns:

- `DoesNotExist` handling.
- Foreign-key assignment.
- `select_related(...)` before serialization.
- Activity/audit creation.
- Error response conventions.
- Field naming differences between request JSON and model attributes.

The goal is not to copy the function blindly; it is to reuse repository conventions already proven by nearby code.

High-Signal Failure Patterns

Common repository-repair failures include:

- Early return before `save()/objects.create()`.
- Parsed fields that are never used.
- Request field name differs from model field name.
- Auth wrapper returns a route-specific response body that violates tests.
- Required relation is never looked up.
- Business identifier is derived from the wrong scope.
- Broad `except Exception` converts a precise validation error into a generic 500.
- A response is generated from stale/unpopulated relationships.

Use AI as a Diagnostic Copilot, Not a Source of Unverified Patches

When an assessment permits an assistant for explanation but not direct implementation, use it to accelerate repository navigation and hypothesis testing.

Good diagnostic prompts are narrow and evidence-based:

From these tests, summarize the exact POST contract: required fields, status codes, response keys, identifier rules, and parent-child behavior. Cite the relevant test lines.

Trace POST `/api/issues` from this URL file to the handler. Tell me which middleware runs first, where persistence should happen, and any early return that can bypass it.

Compare the broken create path with the working update path. Identify reusable patterns for lookups, serialization, assignee handling, and activity creation. Do not modify unrelated files.

Then verify every suggestion against source and tests. The assistant is useful for search and comparison; the repository remains the source of truth.

Final Validation Sequence

1. Focused first failure with `-x`.
2. Entire affected test module.
3. Static/syntax validation.
4. Full relevant suite.
5. Inspect changed files for accidental unrelated edits.
6. If there is a UI reproduction path, verify the customer-visible workflow after backend tests pass.

Interview answer:

In an unfamiliar repository I start from the failing tests, extract the exact contract, and trace one request through route, auth, handler, model, and serializer. I compare the broken path with nearby working code rather than redesigning the repository. I fix one root cause at a time, rerun the narrow test, then expand to the full suite. That keeps the debugging loop evidence-driven and minimizes unrelated changes.

Senior Testing, Security & Reliability Interview Addendum

Test Coverage: What a Percentage Does and Does Not Mean

A coverage figure such as 87% is useful as a signal that most instrumented code paths are exercised, but it is not evidence that the system is 87% correct.

Coverage can be measured in different ways:

- Line coverage
- Statement coverage
- Branch coverage
- Function/method coverage

Strong interview answer:

I use coverage as a gap-finding tool rather than a quality target by itself. I care more about whether tests exercise business invariants, failure paths, concurrency boundaries, and important integrations than whether the percentage reaches an arbitrary number.

A healthy strategy is:

critical domain paths

unit tests

edge/failure paths

integration tests around real boundaries

coverage report identifies untested risk

Unit vs Integration vs End-to-End

Unit test

Tests one unit of logic in isolation.

Characteristics:

- Fast
- Deterministic
- Narrow failure signal
- External boundaries often mocked/faked

Integration test

Tests real interaction between components.

Examples:

- FastAPI endpoint + test database
- ORM model + database constraint
- Service + message broker emulator/test instance
- API client adapter + local stub server

End-to-end

Exercises a user/business workflow through most of the deployed stack.

Trade-off:

- Highest realism
- Slowest/most fragile
- Harder failure diagnosis

Use a test pyramid/portfolio rather than trying to make every test end-to-end.

What Should Be Mocked?

Good mock boundaries:

- External HTTP APIs
- Cloud service clients in narrow unit tests
- Payment/partner APIs
- Email/SMS providers
- Time/randomness when deterministic behavior matters

Be cautious about mocking:

- Your own domain/service logic
- ORM behavior you actually need to verify
- Database constraints
- Serialization contracts that are part of the integration

Strong rule:

Mock unstable or expensive boundaries, not internal implementation details merely to make the test pass.

FastAPI Test Strategy

Test layers:

pure service/domain function tests

API route tests with dependency overrides

API + real test DB integration

selected end-to-end smoke tests

Important cases:

- Happy path
- Validation errors
- Authentication failure
- Authorization failure
- Not-found
- Conflict/duplicate request
- Idempotent retry
- Database rollback
- External-service timeout
- Response schema

Security Scanner Rollout from POC to CI Gate

A static security tool should be introduced as an engineering change with adoption risk, not simply installed and made blocking.

Reusable rollout:

1. Identify why static security analysis is needed.
2. Run a proof of concept on representative code.
3. Classify true positives vs noise.
4. Review severity/confidence thresholds.
5. Define suppression/baseline rules with justification.
6. Expose findings in CI without blocking initially.
7. Make high-confidence/high-impact classes blocking.
8. Document how developers reproduce the scan locally.
9. Track false positives and tune the policy.
10. Tighten enforcement as the codebase becomes clean.

For Python, Bandit is one example. The broader interview principle is **incremental security enforcement that protects quality without creating avoidable developer friction.**

Security vs Developer Velocity

Bad extremes:

block every scanner finding immediately
-> false-positive frustration / release disruption

or:

never block anything
-> security becomes advisory noise

Balanced approach:

- Define high-risk non-negotiable findings.
- Start informational for noisy/legacy categories.
- Provide local reproduction/remediation guidance.
- Use baselines only with ownership and review.
- Tighten over time.

Testing Backward Compatibility

If multiple client/server/framework versions coexist, derive test coverage from a compatibility matrix.

Example:

	Client v1	Client v2	Client v3
Server v1			
Server v2			
Server v3			

Test strategy:

- Positive contract test for every supported combination.
- Explicit failure behavior for unsupported combinations.
- Regression tests around boundary versions.
- API/client integration tests for high-risk transitions.
- Track actual version usage before deprecating old combinations.

This is more systematic than manually maintaining pairwise special cases.

Concurrency Testing

Concurrency bugs are often timing-sensitive and may pass ordinary unit tests.

Useful techniques:

- Start many threads/tasks on the same key/state.
- Use barriers/events to force competing operations to begin together.
- Mix readers and writers.
- Test independent keys to detect accidental global corruption.
- Repeat stress tests many times.
- Assert final invariants, not only absence of exceptions.
- Verify queue capacity never exceeds limit.
- Verify one scarce resource is allocated at most once.
- Verify stale TTL metadata cannot delete a newer value.

Avoid relying on arbitrary sleeps as the only race trigger.

How to Debug a Slow API

Use an evidence-first sequence:

1. Confirm request rate and latency distribution (p50/p95/p99).
2. Trace one slow request end to end.
3. Split time among app CPU, DB, cache, network, downstream calls.
4. Inspect slow queries/execution plans.
5. Check connection-pool saturation.
6. Check N+1 downstream/DB calls.
7. Check payload serialization/size.
8. Check CPU, memory, event-loop/thread-pool saturation.
9. Optimize the measured bottleneck.
10. Verify before/after metrics.

Do not jump directly to "add Redis."

Intermittent 500 Errors: Investigation Order

- Error rate and affected endpoints/tenants/inputs
- Stack traces and structured logs
- Correlation with latest deployment/config change
- Dependency errors/timeouts
- Database connection exhaustion/locks
- Container/pod restarts or OOM kills
- Retry storms
- CPU/memory saturation
- Specific request payload patterns
- Trace IDs across services

If a recent release is clearly implicated and impact is significant, restoring service through rollback can take priority over perfect root-cause analysis.

Retry Safety

Retries should be:

- For transient failures
- Bounded
- Exponentially backed off
- Jittered
- Idempotent or protected with an idempotency mechanism

Do not retry:

- Validation errors
- Authorization failures
- Permanent business-rule conflicts

A timeout without a retry policy is incomplete; a retry policy without a timeout can also hang indefinitely.

Queue Reliability Metrics

Monitor more than queue depth:

queue depth
age of oldest message
consumer throughput
consumer lag
retry rate
DLQ depth
DLQ age
processing latency

Why queue age matters:

A queue of 100 items draining in seconds may be healthy. A queue of five items where the oldest has waited two hours can indicate a stuck partition/consumer or poison message.

Liveness vs Readiness from a Reliability Perspective

- **Readiness:** remove an instance from traffic when it cannot safely serve requests.
- **Liveness:** restart when the process is irrecoverably unhealthy/wedged.

Do not make liveness fail just because an external database/API is temporarily down; that can cause every instance to restart simultaneously during a dependency outage.

Observability: Logs, Metrics, Traces

Logs

Best for discrete events and detailed context.

Include structured fields such as:

```
request_id
trace_id
user/tenant_id where appropriate
resource_id
error_type
version/build
```

Avoid secrets, credentials, and unnecessary PII.

Metrics

Best for trends, alerting, and aggregate health.

API golden signals:

- Rate
- Errors
- Latency
- Saturation

Traces

Best for latency/failure propagation across distributed request paths.

A trace can expose:

```
API 20 ms
DB 15 ms
Downstream A 450 ms
Downstream B 30 ms
```

and identify the real bottleneck without guessing.

Useful Production Metrics by Component

API

- Requests/sec
- 4xx/5xx rate
- p50/p95/p99 latency
- In-flight requests

Database

- Query latency
- Active/idle connections
- Pool wait time
- Lock waits/deadlocks
- Replication lag

Cache

- Hit/miss ratio
- Eviction rate
- Rebuild latency
- DB fallback volume

Worker/queue

- Queue depth
- Oldest-message age
- Throughput
- Retry/DLQ rate
- Worker utilization

Deployment

- Error/latency comparison by version
- Restart rate
- Readiness failures
- Rollback/canary health

Internal Developers as Customers

In platform/developer-productivity systems, the "customer" may be another engineer.

Quality includes:

- Reliable CI pipeline
- Fast feedback
- Actionable errors
- Clear documentation
- Stable contracts
- Predictable release/deprecation behavior
- Low-friction local reproduction

A tool that technically works but produces unclear failures or frequent false positives can still be a poor developer product.

Quick Testing/Security/Observability Revision Card

Coverage != correctness
Unit vs integration vs E2E
Mock external boundaries, not every internal detail
FastAPI dependency override + real test DB where useful
Security scanner POC -> tune -> staged enforcement
Compatibility matrix drives contract tests
Concurrency tests use barriers/invariants, not only sleep
Slow API: measure -> trace -> isolate bottleneck -> optimize
Retries: transient + timeout + bounded backoff + jitter + idempotency
Queue depth + oldest-message age + lag + DLQ
Readiness removes traffic; liveness restarts wedged process
Logs + metrics + traces have different roles
Internal developers are customers of platform tooling

Production Scenario Rapid-Fire Answers

An API suddenly becomes slow

1. Confirm scope and user impact by endpoint, tenant/region, error rate, and p50/p95/p99 latency.
2. Correlate the start time with deploys, traffic, database health, dependency incidents, and configuration changes.
3. Follow a correlation ID through traces and split time into application, database, cache, queue, and downstream calls.
4. Check CPU/memory, worker or event-loop saturation, connection pools, slow queries, locks, and dependency timeouts.
5. Roll back or reduce traffic if a recent change is strongly correlated and mitigation is safe.
6. Fix, load test, deploy gradually, and add the missing alert/regression test.

A downstream service is unavailable

Use explicit timeouts, bounded retries only for transient/idempotent operations, exponential backoff with jitter, a circuit breaker, and bulkhead/concurrency limits. Degrade, cache, queue, or fail fast only when the business contract permits. Monitor dependency latency, error rate, circuit state, retry volume, and queue age. An unbounded retry policy amplifies the outage.

A Kafka consumer receives the same event twice

Assume at-least-once delivery. Give every event or business operation a stable ID and atomically record that ID with the business update, normally through a unique constraint/inbox table. If the ID already exists, acknowledge without repeating the effect. Commit the offset only after successful processing. Use an outbox when a database state change and produced event must remain consistent.

A new API field must not break old clients

Make the change additive and optional or provide a safe server default. Preserve existing names, types, and meanings. Keep translation at the version boundary, update the compatibility matrix, and add contract tests for old request/response shapes and new behavior. Release server compatibility first, monitor adoption, and deprecate only after an explicit migration window.

Bandit reports a high-severity finding during release

The required gate stops promotion. Verify the rule, severity, confidence, reachability, and whether it is new. Fix a real issue and rerun the scanner/tests. For a false positive, use the narrowest documented suppression with required review; never disable the rule globally. Exceptional risk acceptance belongs to authorized security/business owners, not one developer.

Several I/O calls must run concurrently

Use async clients, one coroutine per independent operation, explicit timeouts, cancellation handling, and a semaphore to protect the dependency. Decide whether one failure cancels the group or partial results are acceptable. Do not run blocking clients or CPU-heavy work on the event loop.

```
import asyncio

async def fetch_all(client, requests):
    limit = asyncio.Semaphore(8)

    async def one(request):
        async with limit:
            return await asyncio.wait_for(client.fetch(request), timeout=2)

    return await asyncio.gather(*(one(request) for request in requests))
```

A query slows down as data grows

Capture the exact query/parameters and inspect a real execution plan. Look for scans, cardinality-estimation errors, expensive joins/sorts, spills, lock waits, stale statistics, poor pagination, and N+1 access. Choose an index, rewrite, keyset pagination, precomputation, partitioning, or retention change based on evidence, then validate with realistic volume and monitor write/plan impact.

A deployment introduces errors

Pause rollout, compare the new and previous versions, and restore the last known-good immutable artifact or shift traffic when rollback is safe. Database migrations must support rollback through expand-migrate-contract. Preserve logs, traces, image/commit/configuration identifiers, and failed requests. Reproduce, add a regression/deployment check, and redeploy gradually with success criteria.

An AI agent requests a sensitive tool operation

The model cannot authorize itself. Trusted orchestration checks the authenticated actor, tenant/resource, allow-listed tool, action, schema, policy, and data classification. Use short-lived least-privilege credentials. Require human confirmation for sensitive/irreversible actions, revalidate fresh state before execution, make the operation idempotent, redact the audit trail, and retain a kill switch.

Monitor a FastAPI microservice

Track request rate, error rate, p50/p95/p99 latency, CPU, memory, restarts, worker/event-loop saturation, connection-pool usage, dependency/database timing, and queue lag where applicable. Use structured logs with correlation ID and deployment version, distributed traces across dependencies, separate readiness/liveness, and alerts tied to user-impacting SLOs rather than every fluctuation. Avoid high-cardinality user IDs in metric labels.